B.E. COMPUTER SCIENCE AND ENGINEERING · SEMESTER V / VI

KLE Technological University

Deep Learning

Architectures, Contexts and Applications



From crop yield prediction to pest network propagation — one project, seven architectures, one framework.

3 Credits · 50 Contact Hours · 26 Lectures + 24 Labs

Covers: MLP · CNN · LSTM · Transformer · Autoencoder · Generative Models · GNN

Project AgriVision

150 farms · 7 capabilities · 8 chapters

Preface

Deep learning has transformed what computers can do with images, language, sensor streams, and relational data. The architectures behind these capabilities — convolutional networks, transformers, recurrent networks, graph neural networks — are learnable by any undergraduate who has taken a first course in linear algebra and Python. This book builds each of those architectures from the ground up: from the concrete problem that first made it necessary, through the mathematics that defines it, to working code that implements it.

Most introductory treatments, however, present an architecture before the problem that motivated it. A student who finishes a CNN chapter able to calculate output dimensions may still be uncertain why they would choose a CNN over a large feedforward network, or what specifically breaks when they try the simpler model first.

This book reverses the order. Every chapter opens with a concrete problem that the tools from the previous chapter cannot solve. The limitation is shown precisely: what breaks, why it breaks, and what property of the data the current model fails to exploit. Only then is the new architecture introduced. The CNN is not presented as 'the architecture for images.' It is presented as the answer to a specific question: our MLP has 38 million parameters, achieves 58% accuracy on leaf disease photographs, and cannot detect the same lesion pattern in different parts of the image — what architectural change would fix all three of these at once?

When an architecture arrives with that question already on the table, its design choices make immediate sense. The filter's local receptive field addresses the spatial redundancy problem. Weight sharing addresses the parameter explosion. Pooling addresses the position sensitivity. The student does not merely know how the CNN works — they understand why each of its components exists.

The AgriVision Thread

Running through all eight chapters is a single story: Project AgriVision, a three-person agricultural technology startup working with 150 smallholder farms in northern Karnataka. The startup needs to predict crop yields, detect leaf diseases from photographs, forecast irrigation needs, answer advisory queries from field notes, flag anomalous sensor readings, synthesise training images for rare disease classes, and predict how a pest outbreak at one farm is likely to spread to its neighbours.

Each capability requires a different architecture. The MLP can predict yield from soil measurements but fails when the input is a photograph. The CNN can classify disease from photographs but has no memory of yesterday's sensor readings. The LSTM can process sensor sequences but cannot handle the long-range dependencies in natural language. And no architecture from Chapters 1 through 6 can represent the relational structure of 150 connected farms.

The AgriVision spine is not decoration. It is the pedagogical engine of the book. Each chapter's opening question — 'can we solve this problem with what we already have?' — and the honest answer 'no, and here is precisely why not' — is how each new architecture earns its introduction.

The Four Recurring Questions

Every architecture is introduced through the same four questions:

What problem does this architecture solve? Not in general, but specifically: what failure of the previous approach motivated building this one?

What inductive bias does it encode? What assumption about data structure is baked into the architecture's design? Spatial locality for images, temporal ordering for sequences, neighbourhood aggregation for graphs.

When does it fail? Every architecture has characteristic failure modes. Knowing them before deployment is more useful than discovering them after.

When would you choose something else? An architecture is a tool, not an identity. The goal is always to match the data's structure to the model's inductive bias — not to use the most sophisticated model available.

These questions appear at the start of every chapter and are answered again at the end. By Chapter 8, students can answer them for all seven architectures without looking anything up. That is the test of whether the course has achieved its aim.

How to Use This Book

The chapters are cumulative. Each one closes with a bridge to the next, showing exactly what the current tools cannot do and why the next chapter's architecture is needed. Reading them out of order is possible but will cost some of the motivation.

Textbook chapters (1–8). Each chapter covers one architecture family in approximately 4–9 contact hours. Sections are numbered as X.N, and sub-sections as X.N.M throughout.

Laboratory manual. Each chapter has an accompanying laboratory manual with 2–3 hands-on labs. Chapter 1 labs use NumPy and implement every equation from scratch. Chapters 2–8 labs use PyTorch and HuggingFace. Labs are designed for Google Colab and complete on a free T4 GPU within the time shown on each lab header. All datasets are either provided with the book or auto-generated inside the notebook — no external downloads are required.

Chapter 8. The final chapter synthesises the entire course: the Architecture Decision Map, a structured selection framework, systematic debugging guides, evaluation beyond accuracy, model compression for deployment, and responsible development practices. It is designed to be revisited throughout the course, not just at the end.

A Note on Prerequisites and Notation

The book assumes familiarity with linear algebra (matrix multiplication, dot products, eigenvalues at the level of a first undergraduate course), basic probability (conditional probability, Gaussian distribution), and Python programming at the level of first- and second-year CS courses. Calculus is used in Chapters 1–3; the chain rule is explained from scratch in Section 1.5.3.

Mathematical notation is introduced in plain English before the symbol is given. The convention throughout is: concept before notation, analogy before abstraction, AgriVision before the general case. Where a symbol appears in an equation, its meaning in the agricultural context is always stated alongside its general definition.

A Note on the Field

Deep learning moves quickly. The architectures in this book — the MLP, CNN, LSTM, Transformer, Autoencoder, VAE, GAN, Diffusion model, and GNN — are not the last word. Some have already been partially displaced for certain tasks; others will be. What will not change is the underlying framework: understand your data's structure, identify the inductive bias it requires, and select the architecture that encodes that bias. That reasoning process is what this course is training.

> *The architectures are the examples. The framework is the skill.*

Deep Learning

Architectures, Contexts and Applications

B.E. Computer Science and Engineering

Semester V / VI · 3 Credits · 50 Contact Hours

TABLE OF CONTENTS

CHAPTER 1 Foundations and Representation Learning

1.0 The Architecture Landscape: Why So Many Architectures Exist
1.1 Why Deep Learning? The Representation Problem
1.2 The Artificial Neuron and the Multilayer Perceptron
1.3 Activation Functions
1.4 Loss Functions as Task Objectives
1.5 How Networks Learn: Forward and Backward Propagation
1.6 Optimisation: Making Gradient Descent Work
1.7 Generalisation, Overfitting, and Regularisation
1.8 Reading the Training Signal: Debugging Deep Networks
1.9 When to Use Deep Learning — and When Not To
1.10 Chapter Resolution and Bridge

CHAPTER 2 Learning from Images — CNNs and Visual Understanding

2.0 The Inductive Bias for Image Data
2.1 Image Data as a Deep Learning Context
2.2 Why Fully Connected Networks Struggle with Images
2.3 The Convolution Operation
2.4 Pooling and Spatial Downsampling
2.5 CNN Architecture: Assembling the Components
2.6 Architecture Evolution as Problem–Response
2.7 Visual Tasks Beyond Classification
2.8 Transfer Learning in Computer Vision
2.9 When CNNs Fail
2.10 Chapter Resolution and Bridge

CHAPTER 3 Learning from Sequences — RNNs, LSTMs, and Temporal Patterns

3.0 The Inductive Bias for Sequential Data
3.1 Time-Series Data as a Deep Learning Context
3.2 Why Feedforward Networks Struggle with Sequences
3.3 The Recurrent Neural Network
3.4 Long Short-Term Memory
3.5 Gated Recurrent Units
3.6 1D Convolutional Networks for Sequences
3.7 Sequence Architecture Patterns and Application Contexts
3.8 When Sequence Models Fail
3.9 Chapter Resolution and Bridge

CHAPTER 4 Learning from Text — Attention, Transformers, and Language Models

4.0 The Inductive Bias for Text Data
4.1 Text as Data
4.2 Why Sequential Models Are Insufficient for Text
4.3 The Attention Mechanism
4.4 Self-Attention and Multi-Head Attention
4.5 The Transformer Architecture
4.6 Pre-training and the Foundation Model Paradigm
4.7 Fine-Tuning, Adaptation, and Efficient Transfer
4.8 Transformers Beyond Text
4.9 Hallucination, Grounding, and Responsible Use
4.10 NLP Application Contexts and Chapter Resolution

CHAPTER 5 Learning Without Labels — Autoencoders and Self-Supervised Learning

5.0 The Inductive Bias for Unlabelled Data
5.1 Limits of Supervised Learning
5.2 Autoencoder Architecture
5.3 Latent Space and Applications
5.4 Self-Supervised Learning
5.5 Application Contexts, Limitations, and Bridge

CHAPTER 6 Generative Deep Learning — From Prediction to Synthesis

6.0 The Inductive Bias for Generation
6.1 Why Generative Deep Learning?
6.2 Variational Autoencoders
6.3 Generative Adversarial Networks
6.4 Diffusion Models
6.5 Application Contexts and Responsible Use

CHAPTER 7 Relational and Multimodal Data — GNNs and Beyond

7.0 The Inductive Bias for Relational Data
7.1 Graph-Structured Data
7.2 Graph Neural Networks
7.3 Multimodal Deep Learning
7.4 Deep Reinforcement Learning: A Conceptual Survey
7.5 Chapter Resolution and Bridge

CHAPTER 8 Architecture Selection, Evaluation, and Responsible Deployment

8.0 The Architecture Decision Map
8.1 Architecture Selection Framework
8.2 Systematic Debugging
8.3 Evaluation Beyond Accuracy
8.4 Transfer Learning and Model Adaptation
8.5 Model Compression and Deployment
8.6 Responsible Deep Learning
8.7 The Deep Learning Project Lifecycle and Course Resolution

CHAPTER 1

Foundations, Representation Learning, and the Inductive Bias Framework

1.0 The Architecture Landscape: Why So Many Architectures Exist?

1.0.1 The AgriVision Problem

> ✦ THE AGRIVISION CONTEXT
>
> Imagine you are one of three engineers at a startup called AgriVision, working out of a co-working space in Bengaluru. Your company has just signed its first real contract: a pilot with a cooperative of 150 farmers in northern Karnataka who grow tomatoes, cotton, and groundnuts. Crop losses in the region average 25% per season — from leaf disease that spreads before it becomes visible, from irrigation timed by intuition rather than data, from a pest that moves between adjacent farms in ways no individual farmer can predict.
>
> Your mandate: build an AI system that helps. In return, you have been given five years of field data.

The data takes four radically different forms. Sensor readings: 275,000 rows of tabular data — daily soil moisture, temperature, humidity, rainfall, and fertiliser measurements from 150 farms over five years. Photographs: 8,000 leaf images taken by farmers on mobile phones showing healthy and diseased crops, varying widely in lighting and angle. Maintenance logs: 2,400 text entries written by agricultural extension officers — a typical entry reads: "Field 47, 14 March. Observed early-stage powdery mildew on tomato leaves in rows 3–7. Advised copper-based fungicide. Farmer has not irrigated in 11 days." Pest spread records: a graph of which farms reported infestations and their geographic proximity — Farm 47 is 800 m from Farm 52; Farm 52 is 200 m from Farm 61.

You sit down with your laptop and ask: where do I begin?

1.0.2 The Toolkit Problem: Four Data Types, Four Different Challenges

Your first instinct — reasonable if you have studied machine learning — is to build a yield prediction model from the tabular sensor data. You load the CSV, engineer rolling-average moisture features, and train a gradient boosting model. It reaches 8% mean absolute error on held-out farms. A solid first result. You feel confident.

Then a farmer holds up a photograph of a diseased leaf and asks: "Is this leaf sick?" Your gradient boosting model — which works beautifully on the sensor spreadsheet — has nothing to say about a 224×224 pixel image. You could flatten it into 150,528 numbers and try, but you sense this will not work. The number 147 at pixel position (112, 80) means nothing in isolation. Disease appears in different positions across different photographs. The signal is in the spatial relationships between groups of nearby pixels — relationships a tabular model cannot see.

Then your colleague asks whether the system can read the extension officers' maintenance logs to identify which farms need follow-up today. A sentence is not a row in a spreadsheet. And when your team asks which farms will be caught by the pest outbreak next week, the answer depends not on any single farm's sensor readings but on the relational graph of which farms are near which. In four problems from a single project, you have encountered four completely different kinds of data — each with a different internal structure. This is the reason the deep learning landscape looks the way it does.

1.0.3 The Central Insight: Inductive Bias

Every learning algorithm makes assumptions. Without assumptions, learning is impossible: if you have seen a hundred healthy leaves and a hundred diseased ones, you can only generalise to the next leaf if you assume that leaves with similar visual characteristics behave similarly. The structural assumptions a learning algorithm builds into its design are called its inductive bias.

> ◆ KEY CONCEPT
>
> **Inductive bias**
>
> The set of assumptions a learning algorithm encodes about the structure of data, which determine which hypotheses it prefers when fitting training examples.
>
> The foundational insight of modern deep learning: the most appropriate inductive bias for a learning problem depends on the structure of the data. Different data structures require different biases, and different neural network architectures encode different biases. This is the unifying explanation for the entire architecture landscape.

For AgriVision's four data types, the required biases are distinct:

Images have spatial locality and translation invariance. A disease spot looks the same whether it appears in the top-left or bottom-right of the photograph. A good architecture encodes this by processing local neighbourhoods and sharing the same pattern-detectors across all positions. The CNN encodes exactly this bias.

Time-series have temporal ordering and causality. Soil moisture today is explained by yesterday's moisture and last week's rainfall, not by a reading from three months earlier. The relevant structure is sequential and causal. RNNs, LSTMs, and 1D CNNs encode this bias.

Natural language has long-range context. In "The farmer who had ignored low soil moisture for eleven days finally irrigated today," the word "irrigated" is understood by connecting it to "farmer," "soil moisture," and "eleven days" — scattered across the sentence. Transformers encode this bias through self-attention, allowing every word to attend directly to every other word.

Graph data has relational, permutation-invariant structure. Whether Farm 52's neighbours are listed as {47, 61, 83} or {83, 47, 61} should make no difference to a prediction. The relationships matter; their ordering does not. Graph Neural Networks encode this bias through neighbourhood aggregation.

1.0.4 The Architecture Decision Map (First Pass)

The mapping from data type to inductive bias to architecture is the conceptual core of this book. We call it the Architecture Decision Map. Its initial form is shown in Table 1.1; we will add nuance, exceptions, and additional rows as each chapter unfolds. By Chapter 8 you will use it not as a lookup table but as a reasoning framework.

Data type	Structural property	Required inductive bias	Architecture
Images, video	Spatial locality, translation patterns	Local filters, weight sharing across positions	**CNN**
Time-series, sensors	Temporal ordering, local memory	Sequential processing, gated memory	**RNN, LSTM, 1D CNN**
Natural language	Long-range context between words	Global attention over all token pairs	**Transformer**
Unlabelled signals	Self-consistency, reconstruction	Compression and reconstruction loss	**Autoencoder**
Generative tasks	Full data distribution	Probabilistic generation	**VAE, GAN, Diffusion**
Networks, molecules	Relational, permutation-invariant	Neighbourhood aggregation	**GNN**
Sequential decisions	Delayed reward under uncertainty	Policy optimisation	**Deep RL**

Table 1.1 *The Architecture Decision Map. Each chapter adds detail to one or more rows of this table.*

1.0.5 Four Questions to Carry Through This Book

Every time we encounter a new architecture in the chapters ahead, we apply four questions. Memorise them now so they become instinctive.

> **? FOUR QUESTIONS TO CARRY THROUGH THIS BOOK**
>
> **Question 1: What problem does this architecture solve?**
>
> What limitation of a simpler approach motivated its design? No architecture was invented without a concrete failure it was built to fix.
>
> **Question 2: What inductive bias does it encode?**
>
> What assumption about the structure of the data is built into the architecture? This is the question that connects data type to design choice.
>
> **Question 3: When does this architecture fail?**
>
> Every inductive bias is also a constraint. An architecture that assumes spatial locality will fail when the relevant patterns are not local.

Question 4: When would you choose something else?

This is the engineering judgement question — the one that distinguishes a practitioner from a student who has memorised architecture names.

1.1 Why Deep Learning? The Representation Problem

> ⚠ **IMPORTANT NOTE**
>
> Deep learning is not always the right answer. The gradient boosting model AgriVision built for crop yield prediction from tabular sensor data is likely a better choice than a neural network for that specific task. One of the most important competencies this course develops is knowing when deep learning is and is not appropriate. We return to this question at the end of every chapter.

1.1.1 The Feature Engineering Bottleneck

Classical machine learning algorithms — linear regression, support vector machines, random forests, gradient boosting — are extraordinarily effective at learning from structured numerical data where the right features have already been identified. The yield prediction model works because agronomists understand which measurements matter: rolling-average soil moisture, moisture deficit on peak-growth days, temperature variance during fruit set. The algorithm learns how to weight them; the human decides what they are.

But consider the disease detection problem. You have a 224×224 photograph. The model needs numbers. Which numbers from this photograph are informative? You could extract: the ratio of yellow pixels to green pixels, the variance of texture across image regions, the approximate area of any visible lesion. Each choice requires decisions: what threshold defines "yellow"? How do you segment a lesion from the background? What texture measure captures the right discriminative signal across 150 different phone cameras in 150 different lighting conditions?

Computer vision researchers spent decades designing such features by hand — edge detectors, histogram-of-gradient descriptors, colour histograms. These worked to a degree on specific datasets. But they had two fundamental weaknesses. First, human intuitions about which visual patterns matter may be wrong — the feature that best discriminates early powdery mildew may be a subtle pattern no agronomist would have thought to specify. Second, and more deeply, the relevant information in a photograph is in the spatial relationships between groups of pixels, not in individual pixel values. Writing rules that capture all meaningful spatial relationships across all conditions is not merely difficult. It is, in practice, impossible.

1.1.2 What Deep Learning Does Differently: Layered Representation Learning

Deep learning solves this by making feature extraction part of the learning process. Instead of asking a human to design features, a deep network learns which features are useful — directly from the training data, as part of the same optimisation that learns the final prediction.

The mechanism is layered representation learning. Each layer transforms the data into a representation at a higher level of abstraction. For AgriVision's leaf images, the learned hierarchy consistently looks like this: early layers detect oriented edges and colour gradients — the most primitive visual primitives. Middle layers combine edges into textures and shapes: a patch of discolouration, a circular boundary. Deep layers assemble these into disease-specific patterns: the

characteristic white powder and radial spread of powdery mildew. The final layer makes the classification decision — relatively straightforward given what the layers below have built.

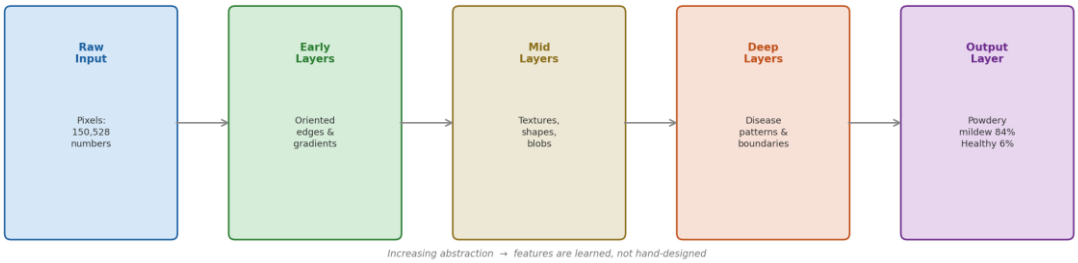


Figure 1.2 *Layered representation learning applied to leaf disease detection. Each layer transforms raw pixels into progressively abstract features — edges, textures, disease structures. The features are not hand-designed; they are learned because they make the final classification tractable.*

1.1.3 The Three Ingredients of the Deep Learning Era

The representational power of layered networks was understood theoretically for decades before it became practical. Three things changed around 2012, roughly simultaneously.

Data. Learning a reliable hierarchy of representations requires many examples. A disease model trained on 200 leaf photographs will memorise them rather than generalise. The same architecture trained on 200,000 images learns something robust. The ImageNet dataset (14 million labelled images), Wikipedia, and countless domain datasets provided the raw material that representation learning requires.

Compute. Each training step of a deep network requires millions of matrix multiplications. Graphics Processing Units — designed for rendering video games — proved exceptionally well-suited to these operations. A training run that takes months on a CPU takes hours on a GPU cluster. This made experimentation practical for the first time.

Algorithms. Targeted improvements in training methodology proved decisive: the ReLU activation (which largely eliminates the vanishing gradient problem, Section 1.3), dropout regularisation (which reduces overfitting, Section 1.7), batch normalisation (which stabilises very deep training, Section 1.7), and residual connections (which allow networks of hundreds of layers, Chapter 2).

The crystallising moment was the 2012 ImageNet Large Scale Visual Recognition Challenge. A team from the University of Toronto entered a deep CNN called AlexNet. The previous year's best system reached 25.8% top-5 error. AlexNet reached 15.3% — an improvement that would have taken the prior paradigm perhaps a decade to accumulate. Within two years, deep networks were the dominant approach to image classification; within five, to speech recognition, translation, and language modelling as well.

1.1.4 When Deep Learning Is and Is Not Appropriate

This question recurs at the end of every chapter. The initial answer:

Deep learning tends to outperform classical machine learning when:

- The input has complex internal structure whose useful features cannot be hand-designed — images, audio, natural language, raw sensor waveforms.

- Sufficient training data is available to learn representations from scratch, or a pretrained model can be fine-tuned with fewer examples.
- The task requires high accuracy across widely varying input conditions — different lighting, phrasing, noise levels.

Classical machine learning is often the better choice when:

- The data is tabular and well-structured, and a domain expert can specify the relevant features.
- Training examples are scarce — fewer than a few thousand.
- The prediction must be interpretable: a farmer, doctor, or regulator needs to understand why a specific decision was made.
- Compute is severely constrained, or training speed matters more than maximum accuracy.

For AgriVision: the yield prediction model from tabular sensor data is best served by gradient boosting. The leaf disease detector needs a CNN. The maintenance log system needs a Transformer. The pest propagation model needs a graph neural network. There is no universal tool; there is only the right tool for each problem's data structure.

1.2 The Artificial Neuron and the Multilayer Perceptron

1.2.1 The Biological Analogy and Its Limits

The story of neural networks conventionally begins with the human brain. A biological neuron receives electrical signals from other neurons through dendrites, accumulates them in its cell body, and fires — sending a pulse along its axon — when the accumulated signal crosses a threshold. Connection strengths (synaptic weights) change with experience. Warren McCulloch and Walter Pitts formalised this mechanism as a mathematical operation in 1943, and the analogy has shaped the vocabulary ever since: neuron, weight, layer, activation.

But we should establish the analogy's limits immediately. A deep neural network is better understood as a composition of mathematical functions than as a model of the brain. Modern networks perform operations biological neurons do not; biological neurons perform operations networks do not. The history is inspiring; the mathematics is what matters for engineering.

1.2.2 The Artificial Neuron: Inputs, Weights, Bias, and Activation

DEFINITION

Artificial neuron

A computational unit that receives n numerical inputs $x_1, x_2, \ldots, x_n$, computes a weighted sum plus a bias (called the pre-activation or logit z), and passes the result through a nonlinear activation function f to produce its output a:

$$z = w_1x_1 + w_2x_2 + \ldots + w_nx_n + b = w^Tx + b$$

$$a = f(z)$$

Weights w and bias b are learnable parameters adjusted during training. The activation function f introduces nonlinearity, without which a network of any depth reduces to a single linear transformation.

In the AgriVision yield prediction network, each input neuron carries one normalised sensor reading: soil moisture fraction, soil temperature, ambient temperature, relative humidity, daily rainfall, cumulative fertiliser applied, and days since last irrigation. The weights determine how strongly each reading influences the neuron's output. A large positive weight on soil moisture means this neuron activates more strongly when the soil is wet. A near-zero weight on fertiliser means this particular neuron treats fertiliser as uninformative. These weights are not set by an agronomist — they are learned by gradient descent from five seasons of yield data.

WORKED EXAMPLE

One neuron at a tomato farm in Mandya district

Inputs (normalised to [0,1]): $x_1 = 0.65$ (soil moisture), $x_2 = 0.83$ (temperature), $x_3 = 0.80$ (days since last rain)

Weights learned during training: $w_1 = +0.92$, $w_2 = -0.47$, $w_3 = +0.61$, $b = -0.18$

Pre-activation: $z = (0.92)(0.65) + (-0.47)(0.83) + (0.61)(0.80) + (-0.18)$

$z = 0.598 - 0.390 + 0.488 - 0.180 = 0.516$

Sigmoid activation (Section 1.3): $a = 1/(1 + e^{-0.516}) \approx 0.626$

What the weights reveal: the neuron responds strongly and positively to moisture ($w_1 = +0.92$), negatively to high temperature ($w_2 = -0.47$), and moderately positively to extended dry periods ($w_3 = +0.61$). These are relationships gradient descent discovered from five seasons of yield data — not agronomic rules that were programmed.

1.2.3 From a Single Neuron to a Network: Layers and Depth

A single neuron can only compute a single nonlinear boundary through its input space. To learn the complex, multi-part relationships in real data, we connect neurons into layers and stack layers into a multilayer perceptron (MLP), also called a feedforward or fully connected network.

Each layer computes, in matrix form:

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}$$

$$a^{(l)} = f^{(l)}(z^{(l)})$$

$W^{(l)}$ is a weight matrix of dimensions $n_l \times n_{l-1}$ — one row per neuron in the current layer, one column per neuron (or input) in the previous layer. $b^{(l)}$ is a bias vector of length n_l. The activation function $f^{(l)}$ is applied element-wise: every neuron applies the same function to its own pre-activation independently.

Parameter count — a worked example. The AgriVision yield network has $n_0 = 7$ inputs, $n_1 = 16$ hidden neurons, $n_2 = 8$ hidden neurons, $n_3 = 1$ output neuron (predicted yield in kg/acre).

- Layer 1: $W^{(1)}$ is $16 \times 7 = 112$ weights, plus 16 biases $\rightarrow$ 128 parameters
- Layer 2: $W^{(2)}$ is $8 \times 16 = 128$ weights, plus 8 biases $\rightarrow$ 136 parameters
- Layer 3: $W^{(3)}$ is $1 \times 8 = 8$ weights, plus 1 bias $\rightarrow$ 9 parameters
- Network total: $128 + 136 + 9 = 273$ trainable parameters

By modern standards this is tiny — large language models have hundreds of billions of parameters. But for a tabular regression problem with 150 farms and five seasons of data, 273 parameters is a reasonable starting capacity, sufficient to learn the nonlinear interactions between moisture, temperature, and yield that a linear model would miss.

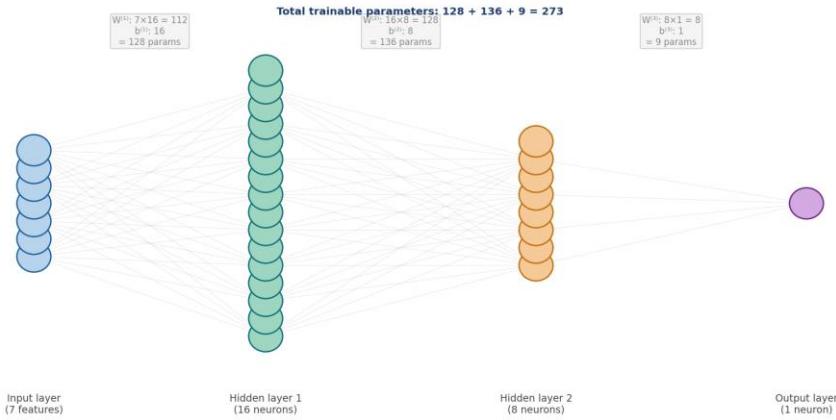


Figure 1.3 *Architecture of the 7-16-8-1 AgriVision yield prediction network. Connections between all layers are shown; parameter counts at each layer are annotated above. Total trainable parameters: 273.*

1.2.4 The Universal Approximation Intuition

> ◆ **KEY CONCEPT**
>
> **Universal Approximation Theorem (Cybenko, 1989)**
>
> A feedforward neural network with at least one hidden layer, sufficiently many neurons, and a nonlinear activation function can approximate any continuous function on a bounded domain to any desired degree of accuracy.
>
> This is a theoretical guarantee that the function class is rich enough — not a guarantee that gradient descent will find the right weights, or that the required network will be tractably small.

The theorem has three important limitations. First, "sufficiently many neurons" can mean exponentially many for some functions — the required network may be too large to train. Second, the theorem says nothing about whether gradient descent on a finite training set will find the weights that achieve the approximation. Third, the theorem applies to shallow networks; it does not explain why depth is so much more practical than extreme width.

1.2.5 Why Depth Matters More Than Width

Consider two networks for the leaf disease problem, with similar parameter counts:

- Network A: one hidden layer with 1,000 neurons
- Network B: five hidden layers with 45 neurons each

On image data, Network B will almost always outperform Network A — often dramatically. The reason is compositional representation. Real-world functions are not arbitrary; they are built from simpler functions composed together. The function mapping a pixel array to "this tomato leaf has early powdery mildew" has a natural hierarchy: detect colour gradients → identify characteristic texture patches → locate discolouration boundaries → determine whether the shape and spatial distribution match a disease template → classify type and severity.

A deep network can learn each step in a separate layer, where each layer builds on the representations of the one below. A shallow network must discover this entire hierarchy in a single layer — and while the Universal Approximation Theorem guarantees it can in principle, the required

width grows exponentially and the optimisation landscape becomes far harder to navigate. Deep networks are easier to train to high accuracy, not just more expressive in principle. This is the architectural intuition that underlies every architecture in this book.

KEY TERMS — THIS SECTION

Term	Definition
Artificial neuron	Computes $z = w^T x + b$, then $a = f(z)$; w and b are learned during training
Weight	A learnable scalar determining how strongly one neuron influences another
Bias	A learnable constant that shifts a neuron's activation threshold
Pre-activation	The weighted sum before the activation function: $z = w^T x + b$
Activation	The neuron's output after the activation function: $a = f(z)$
Hidden layer	An intermediate layer of neurons not directly observed during training
Multilayer perceptron	A feedforward network of neurons organised into input, hidden, and output layers
Universal Approximation	A sufficiently wide one-hidden-layer network can approximate any continuous function
Compositional repr.	Learning complex functions as a hierarchy of progressively abstract simpler functions

✎ CHECK YOUR UNDERSTANDING

1. A neuron has 4 inputs and no bias term. How many parameters does it have? What geometric constraint does the absence of a bias impose on the functions this neuron can represent?

2. A network has $n_0 = 12$ inputs, $n_1 = 64$ neurons, $n_2 = 32$ neurons, $n_3 = 5$ output neurons. How many total trainable parameters does it have? Show the calculation for each layer.

3. In the Worked Example above, soil moisture drops to $x_1 = 0.28$ (severely dry) while temperature and rainfall remain unchanged. Recompute z and a. Does the output increase or decrease? What agronomic interpretation might you assign to this change?

4. A student argues: "The Universal Approximation Theorem guarantees my network can learn any function, so I do not need to worry about architecture design." Identify two distinct flaws in this argument.

5. [Challenge] Suppose f^1 and f^2 in a two-layer network are both identity functions, $f(z) = z$. Show algebraically that the two-layer network $a^{(2)} = W^{(2)}(W^{(1)}x + b^{(1)}) + b^{(2)}$ is equivalent to a single-layer network $Ax + c$ where $A = W^{(2)}W^{(1)}$ and $c = W^{(2)}b^{(1)} + b^{(2)}$. What does this imply about the necessity of nonlinear activations?

1.3 Activation Functions

1.3.1 Why Nonlinearity Is Non-Negotiable

Challenge Question 5 in the previous section demonstrated algebraically that a network with identity (linear) activations collapses to a single linear function regardless of its depth. This is not a quirk of a specific network — it follows from the mathematics of matrix multiplication. Multiplying two matrices yields another matrix. A composition of linear transformations is always a linear transformation.

The activation function breaks this collapse. By introducing a nonlinearity at each layer, we ensure the composition of layers produces a genuinely richer function than any single layer could express. Choosing an activation is therefore not stylistic — it determines whether the network can learn anything meaningful, how fast it learns, and whether training is stable. We also encounter in this section the most important failure mode in deep learning: the vanishing gradient problem.

1.3.2 Sigmoid and Tanh: The Classical Functions and Their Problems

Sigmoid. $\sigma(z) = 1 / (1 + e^{-z})$

Maps any real number to (0, 1). Smooth and differentiable everywhere. Its derivative has a convenient closed form: $\sigma'(z) = \sigma(z) \cdot (1 - \sigma(z))$. The derivative reaches its maximum of 0.25 at $z = 0$. As $|z|$ grows, the sigmoid saturates — its output approaches 0 or 1, and its gradient approaches 0. The neuron has, in effect, made up its mind and stopped responding to further input changes.

This saturation creates the vanishing gradient problem. During training, the gradient at each layer is multiplied by the derivative of the activation at that layer. In a network with $L = 10$ sigmoid layers where every neuron is saturated, the gradient reaching the first layer is at best $(0.25)^{10} \approx 10^{-6}$ times the gradient at the output. In practice, the first layers stop learning entirely. The sigmoid was dominant through the 1990s partly because deep networks were not yet common — the problem only bites at depth.

Tanh. $\tanh(z) = (e^{z} - e^{-z}) / (e^{z} + e^{-z})$

Maps to (−1, 1). Related to sigmoid: $\tanh(z) = 2\sigma(2z) - 1$. Its key advantage is zero-centring: outputs near zero are near zero, and activations can be negative. When activation outputs are all positive (as with sigmoid), gradients of the next layer's weights are all the same sign, creating a zig-zag dynamic in gradient descent that slows convergence. Tanh's zero-centred outputs avoid this — but it suffers the same saturation as sigmoid: for large $|z|$, $\tanh'(z) \to 0$. The vanishing gradient problem remains.

1.3.3 The Rectified Linear Unit: Why ReLU Changed Everything

◆ **KEY CONCEPT**

ReLU (Rectified Linear Unit)

$\mathrm{ReLU}(z) = \max(0, z)$

For negative inputs, returns zero. For non-negative inputs, returns the input unchanged. Derivative: 1 if $z > 0$; 0 if $z < 0$.

Three properties make ReLU transformative:

(1) No saturation for positive inputs — the gradient is exactly 1 for z > 0, regardless of magnitude. The vanishing gradient problem is eliminated for active neurons.

(2) Sparse activation — approximately 50% of neurons output zero for any given input. This sparsity is computationally efficient and often improves generalisation.

(3) Computational simplicity — max(0, z) requires only a comparison and clamp, orders of magnitude cheaper than the exponential in sigmoid or tanh.

The empirical impact was immediate. In the 2012 AlexNet paper, Krizhevsky, Sutskever, and Hinton reported that training with ReLU was approximately six times faster than with tanh on the same task and hardware. The difference was purely the activation function — same architecture, same data, same optimiser.

The Dying ReLU problem. If a neuron's pre-activation z is negative for every training example, its gradient is always zero and it receives no weight updates. The neuron is said to have died: it remains permanently inactive regardless of input. This can be caused by poor initialisation or an excessively large learning rate. Dead neurons can be detected by monitoring the fraction of persistent zeros in each layer. The most direct remedy is to switch to an activation that does not zero-gradient for negative inputs.

1.3.4 Variants: Leaky ReLU, ELU, and GELU

Leaky ReLU. LeakyReLU(z) = z if z > 0; αz if z ≤ 0, where $\alpha \approx 0.01$. The small negative slope means the gradient is non-zero everywhere — dead neurons cannot occur. Performance is typically similar to ReLU, but it is the safer choice when dead neurons are observed during training.

ELU (Exponential Linear Unit). ELU(z) = z if z > 0; $\alpha(e^z - 1)$ if z ≤ 0. The exponential curve for negative inputs gives ELU outputs a mean closer to zero than ReLU, similar in spirit to tanh's advantage over sigmoid. ELU tends to perform slightly better than ReLU on carefully tuned deep architectures, at a higher computational cost due to the exponential.

GELU (Gaussian Error Linear Unit). GELU(z) ≈ z · Φ(z), where Φ is the cumulative distribution function of the standard normal. GELU is the activation of choice in virtually all modern Transformer architectures, including BERT, GPT, and their descendants. Its smooth, non-monotonic gating behaviour suits the token-mixing operations in self-attention layers. For standard feedforward networks on tabular or image data, ReLU or Leaky ReLU remain the more common choice.

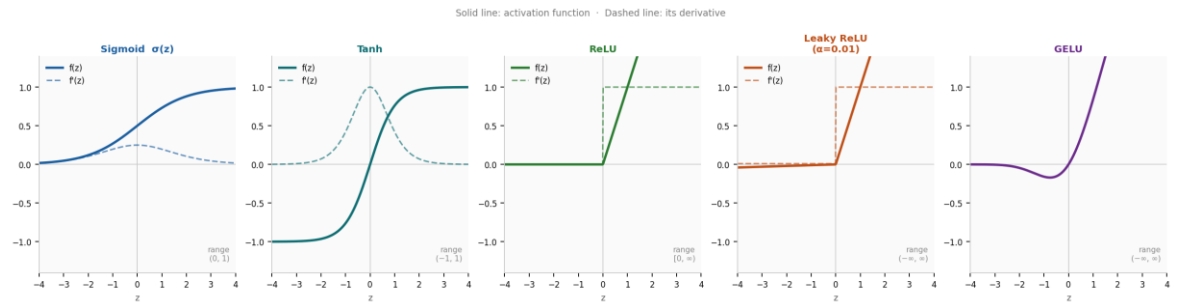


Figure 1.4 *Activation functions (solid) and their derivatives (dashed) over $z \in [-4, 4]$. Note the flat derivative regions (saturation) in sigmoid and tanh, the constant unit derivative of ReLU for positive inputs, and the smooth GELU curve that has no derivative plotted (it has no simple closed form).*

1.3.5 Output Activations and Their Relationship to Loss Functions

> ⚠ **IMPORTANT NOTE**
>
> The activation at the output layer is determined by the task structure, not by training stability considerations. Using the wrong output activation is one of the most common sources of subtle errors. The pairings below are rules, not suggestions.
>
> **Regression (predict a continuous value — yield in kg/acre, temperature in °C):**
>
> Output activation: none (linear) · Loss function: Mean Squared Error or Mean Absolute Error
>
> **Binary classification (diseased / healthy; fraud / legitimate):**
>
> Output activation: sigmoid · Loss function: Binary Cross-Entropy
>
> **Multi-class classification (Healthy / Powdery Mildew / Late Blight / Alternaria):**
>
> Output activation: softmax · Loss function: Categorical Cross-Entropy
>
> Softmax is applied across all K output neurons together — it is the only activation in this book that is not element-wise. It converts K pre-activations to a probability distribution summing to 1: $\text{softmax}(z)_k = e^{z_k} / \Sigma_j e^{z_j}$

1.3.6 How to Choose: A Practical Guide

For hidden layers: Begin with ReLU. It is computationally cheap, avoids saturation for positive inputs, and performs well across image, tabular, and sequence tasks. If you observe a significant fraction of dead neurons — detectable by monitoring the proportion of persistent zero activations — switch to Leaky ReLU. If you are building a Transformer or any attention-based architecture, use GELU; this is where its smooth gating properties provide the most consistent benefit.

For output layers: Always determine the activation from the task, using the pairings in Section 1.3.5 exactly. Do not use ReLU at the output for regression (it prevents negative predictions). Do not use sigmoid where softmax is needed (it does not enforce that probabilities sum to 1). Do not use no activation with cross-entropy loss.

Do not use sigmoid or tanh in hidden layers of any network deeper than two or three layers unless you have a specific reason. Their saturation gradients will impede training, and ReLU is strictly superior on essentially all tasks where they were previously used.

When in doubt, ablate: Train the same architecture with two activation functions, keeping everything else identical. Compare validation loss curves after the same number of training steps. The empirical evidence on your specific data is more reliable than any general rule.

KEY TERMS — THIS SECTION

Term	Definition
Activation function	A nonlinear function applied element-wise to a neuron's pre-activation
Sigmoid	$\sigma(z)=1/(1+e^{-z})$, range (0,1); saturates for large $\|z\|$; causes vanishing gradients

Tanh	$(e^z-e^{-z})/(e^z+e^{-z})$, range $(-1,1)$; zero-centred; same saturation as sigmoid
Vanishing gradient	Gradients becoming exponentially small in early layers due to saturating activations
ReLU	$\max(0,z)$; no saturation for positive inputs; can produce dead neurons
Dead neuron	A ReLU neuron that outputs zero for every training input and receives no gradient
Leaky ReLU	ReLU with a small slope $\alpha \approx 0.01$ for negative inputs; prevents dead neurons
ELU	Exponential curve for negative inputs; smooth and near-zero-mean
GELU	$z \cdot \Phi(z)$; smooth gating function used in Transformers; no simple closed form
Softmax	Converts K pre-activations to a probability distribution summing to 1
Output activation	Activation at the final layer; chosen by task type, not training stability

✎ **CHECK YOUR UNDERSTANDING**

1. Compute $\sigma(0)$, $\sigma(2)$, $\sigma(-2)$, $\sigma(5)$, and $\sigma(-5)$. Then compute $\sigma'(z)$ at each point. At what value of z is the sigmoid gradient largest? What does this imply for where sigmoid neurons learn most effectively?

2. A disease classification model for Karnataka crops has four output classes: Healthy, Powdery Mildew, Late Blight, Alternaria Leaf Spot. A student uses four sigmoid neurons at the output, one per class. What specific mathematical property is violated? Give a numerical example showing how this causes a problem.

3. You are building a model to predict the daily water consumption of a farm in litres. The range in your dataset is 400–6,200 litres/day. What output activation should the output neuron use? What would go wrong if you used ReLU at the output instead?

4. During training you inspect the activations of your first hidden layer and find that 58% of neurons are outputting exactly zero for every sample in a batch. What activation are you using? Is this necessarily a problem? At what percentage of zeros would you intervene, and what remedy would you apply?

5. [Challenge] A network has $L = 6$ sigmoid-activated hidden layers. Assume every sigmoid neuron is operating near $z = 3$, where $\sigma'(3) \approx 0.045$. Compute $(0.045)^6$. If the gradient at the output layer is 0.8, what is the approximate gradient reaching the first hidden layer? What does this tell you about the practical trainability of deep sigmoid networks?

1.4 Loss Functions as Task Objectives

1.4.1 What a Loss Function Does and Why It Must Be Chosen Carefully

The AgriVision team now runs three simultaneous tasks: predict crop yield (continuous number), detect disease in a leaf photograph (two classes), and classify disease type into four categories. Each needs a trained network. Before any training begins, a prior question must be settled: what does it mean for the network's prediction to be wrong, and by how much?

The loss function is the mathematical answer. It takes the network's prediction $\hat{y}$ and the true value y and returns a non-negative number measuring the discrepancy. The goal of training is to minimise this number, on average, across all training examples. For a dataset of N examples, the cost function $J(\theta)$ averages the per-example loss:

$$J(\theta) = (1/N)\, \Sigma_i\, L(\hat{y}_i, y_i)$$

The loss function is not merely a performance metric. It is the mathematical signal that the entire learning process minimises. Choosing the wrong loss function misdirects learning at a fundamental level, because gradients computed during backpropagation are derived directly from the loss. With binary cross-entropy (the correct choice for disease detection), a confident wrong prediction $\hat{y} = 0.95$ for a diseased leaf ($y = 1$) produces loss $-\log(0.05) \approx 3.0$ — a large corrective signal. With MSE instead, the loss is $(0.95-0.0)^2 = 0.90$ — a weaker, less appropriate signal.

1.4.2 Regression: Mean Squared Error and Mean Absolute Error

For AgriVision's yield prediction task, the network outputs a single real number $\hat{y}$ (predicted yield in kg/acre) and the target y is the measured yield. Two loss functions are standard.

Mean Squared Error (MSE). $\text{MSE} = (1/N)\, \Sigma_i\, (\hat{y}_i - y_i)^2$. The square ensures non-negativity and penalises large errors quadratically — a prediction 20 kg/acre off incurs four times the loss of one 10 kg/acre off. The gradient with respect to a single prediction is $\partial\text{MSE}/\partial\hat{y}_i = (2/N)(\hat{y}_i - y_i)$, which is zero at a perfect prediction and scales with the error magnitude. $\text{RMSE} = \sqrt{\text{MSE}}$ restores the original units and is used when reporting model performance.

Mean Absolute Error (MAE). $\text{MAE} = (1/N)\, \Sigma_i\, |\hat{y}_i - y_i|$. Because there is no squaring, each error contributes linearly. If one farm reports an anomalously high yield due to a sensor malfunction, the squared error in MSE could dominate the entire cost, pulling the model toward fitting that single erroneous record. MAE gives it only linear weight, making the model more robust to data quality issues. The trade-off: MAE is not differentiable at exactly $\hat{y} = y$ (handled via subgradients), and can slow convergence compared to MSE.

For AgriVision, crop yield measurements are noisy and occasional outliers are likely. MAE or a Huber loss (behaving like MSE near zero and MAE for large errors) is the defensible choice. Starting with MSE and checking whether any single farm's residual dominates is a sensible first workflow.

1.4.3 Binary Classification: Binary Cross-Entropy

For the binary disease detection task ($y = 1$: diseased, $y = 0$: healthy), the network uses a single sigmoid output neuron producing $\hat{y} \in (0, 1)$: the predicted probability that the leaf is diseased. The

loss function emerges from modelling each label as a Bernoulli random variable. Maximising the likelihood of observing the correct labels is equivalent to minimising the Binary Cross-Entropy:

$$BCE = -(1/N)\,\Sigma_i\,[\,y_i\,log(\hat{y}_i) + (1-y_i)\,log(1-\hat{y}_i)\,]$$

When y = 1, the active term is −log(ŷ): near zero for confident correct predictions, very large for confident wrong ones. This logarithmic sensitivity to confident mistakes sends a strong gradient signal precisely when the model is most dangerously wrong — exactly the property a good loss function should have.

> ✎ **WORKED EXAMPLE**
>
> **BCE loss at a Gadag district farm leaf (y = 1: diseased)**
>
> Case 1 — model predicts ŷ = 0.73 (moderately confident, correct direction):
>
> L = −log(0.73) ≈ 0.315 — moderate corrective signal
>
> Case 2 — model predicts ŷ = 0.12 (confident and wrong):
>
> L = −log(0.12) ≈ 2.12 — nearly seven times larger; this update is prioritised
>
> Compare with MSE on the same predictions: $(0.73-1)^2 = 0.073$ and $(0.12-1)^2 = 0.774$.
>
> The ratio is similar but the absolute magnitudes are far smaller, and MSE lacks the asymmetric sensitivity to confident mistakes that makes BCE geometrically appropriate for probability outputs.

1.4.4 Multi-Class Classification: Categorical Cross-Entropy

For the four-class disease task (Healthy, Powdery Mildew, Late Blight, Alternaria), the network outputs four softmax-activated neurons producing probabilities $[\hat{y}_1, \hat{y}_2, \hat{y}_3, \hat{y}_4]$ that sum to 1. Labels are one-hot vectors: Late Blight (class 3) has label y = [0, 0, 1, 0]. Categorical Cross-Entropy generalises BCE to K classes:

$$CCE = -(1/N)\,\Sigma_i\,\Sigma_k\,y_{ik}\,log(\hat{y}_{ik}) \;=\; -(1/N)\,\Sigma_i\,log(\hat{y}_{i,c_i})$$

where c_i is the true class index. The loss at each example is simply the negative log of the predicted probability assigned to the correct class. The mathematical elegance of the softmax-CCE pairing: when we compute the gradient of CCE with respect to the output pre-activations z_k, a remarkable simplification occurs:

$$\partial CCE/\partial z_k = \hat{y}_k - y_k$$

This clean form arises because softmax and cross-entropy are mathematically conjugate operations. The gradient is simply the prediction error: for the correct class, push probability up; for incorrect classes, push it down. This pairing ensures well-conditioned gradients regardless of the prediction value.

> ⚠ **IMPORTANT NOTE**
>
> **Common mistake: multiple sigmoids instead of softmax for multi-class output.**

Four sigmoid neurons can independently output [0.82, 0.75, 0.61, 0.88] — all above 0.5, summing to 3.06. This is not a probability distribution. The model simultaneously claims high confidence in all four classes.

Softmax enforces that predictions form a proper probability distribution. For mutually exclusive classes (any leaf has at most one disease type), softmax is mandatory.

Sigmoid outputs are appropriate only for multi-label problems where a single example can simultaneously belong to multiple classes (e.g., a patient record flagging diabetes, hypertension, and anaemia at once).

1.4.5 The Output Layer–Loss Function Pairing: A Design Rule

The pairings between output activation and loss function are fixed by the mathematical structure of each task. They are not hyperparameters to experiment with.

Task type	Output activation	Loss function	Gradient ∂L/∂z_output
Regression	None (linear)	MSE or MAE	(2/N)(ŷ − y)
Binary classification	Sigmoid	Binary Cross-Entropy	ŷ − y
Multi-class classification	Softmax	Categorical Cross-Entropy	ŷ − y (vector)

Table 1.2 *Output activation–loss function pairings. The correct pairing simplifies the output gradient to ŷ−y in every case.*

The gradient column reveals why these pairings are not arbitrary. With each correct pairing, the gradient of the loss with respect to the output pre-activation simplifies to ŷ − y: the prediction error, direct and clean. This gradient is bounded, always points in the right direction, and scales naturally with the magnitude of the mistake. Wrong pairings produce biased, numerically unstable, or uninformative gradients.

KEY TERMS — THIS SECTION

Term	Definition
Loss function	L(ŷ, y) measuring discrepancy between prediction and true label for one example
Cost function	Average loss over N examples: $J(\theta) = (1/N)\ \Sigma\ L(\hat{y}_i, y_i)$
MSE	Mean Squared Error: $(1/N)\Sigma(\hat{y}-y)^2$; standard regression loss; sensitive to outliers
MAE	Mean Absolute Error: $(1/N)\Sigma\lvert\hat{y}-y\rvert$; robust to outliers; non-smooth at ŷ = y
RMSE	Root Mean Squared Error: √MSE; restores units of the target variable
Binary Cross-Entropy	$-(1/N)\Sigma[y\cdot\log(\hat{y})+(1-y)\cdot\log(1-\hat{y})]$; paired with sigmoid output
Categorical Cross-Entropy	$-(1/N)\Sigma\Sigma\ y_k\cdot\log(\hat{y}_k)$; paired with softmax output
One-hot encoding	Class label as a vector of zeros with a single 1 at the class index
Conjugate pairing	A matched activation-loss pair whose combined gradient simplifies to ŷ − y

✎ **CHECK YOUR UNDERSTANDING**

1. AgriVision's yield model outputs 1,620 kg/acre for a farm with true yield 1,450 kg/acre. Compute the MSE loss for this single example. Then compute the MAE loss. Which is larger, and why?

2. A binary disease classifier predicts $\hat{y} = 0.88$ for a leaf that is actually healthy ($y = 0$). Compute the BCE loss. Now compute it for $\hat{y} = 0.52$. Explain why the difference in loss magnitude matters for training.

3. A four-class classifier outputs softmax probabilities [0.05, 0.72, 0.18, 0.05] for a leaf whose true class is Powdery Mildew (class 2, one-hot [0,1,0,0]). Compute the CCE loss. Which of the four probabilities determines the loss, and why?

4. A student uses four sigmoid neurons at the output of a four-class disease classifier. For one example the outputs are [0.85, 0.79, 0.61, 0.74]. Identify two specific problems this creates compared to softmax output.

5. Explain in one paragraph why the gradient of binary cross-entropy with respect to the sigmoid pre-activation simplifies to $(\hat{y} - y)$. Why is this simplification significant for training stability?

6. [Challenge] A regression model is trained with MSE loss. A data engineer discovers one farm's yield was entered as 18,000 kg/acre instead of 1,800 kg/acre. Estimate how much this single outlier increases $J(\theta)$ compared to a correct record, assuming typical prediction error ±150 kg/acre. What does this imply about loss function choice when data quality cannot be guaranteed?

1.5 How Networks Learn: Forward and Backward Propagation

Section 1.4 defined what we want to minimise — the cost function J(θ). Section 1.6 will explain how gradient descent uses the gradient of J to update parameters. This section answers the bridge question: how do we compute the gradient of J with respect to every weight and bias in a network that may have hundreds of thousands of parameters? The answer is backpropagation — a systematic, efficient algorithm using the chain rule of calculus.

1.5.1 The Computational Graph

Any sequence of mathematical operations can be represented as a computational graph: a directed acyclic graph where each node is a single operation (addition, multiplication, a nonlinearity) and each edge carries a computed value. For a single neuron $z = w_1x_1 + w_2x_2 + b$ followed by $a = \text{ReLU}(z)$, the graph decomposes into five elementary operations. Each node only knows its own local derivative. The full derivative from any input to the final output is the product of local derivatives along the connecting path.

Deep learning frameworks such as PyTorch and TensorFlow implement exactly this graph internally. When you call .backward(), the framework traverses the graph in reverse, computing and accumulating gradients at every node automatically.

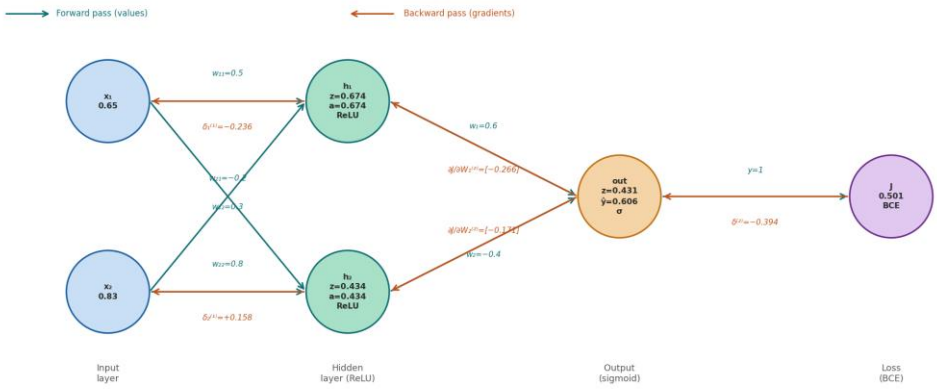


Figure 1.5 *Computational graph for the 2→2→1 network from the worked example. Teal arrows (→) show the forward pass with computed values. Coral arrows (←) show the backward pass with gradient signals. The error signal at each layer is computed from the layer to its right and propagated left.*

1.5.2 Forward Propagation: Computing Predictions

Forward propagation is the left-to-right traversal of the computational graph. For each layer l = 1, 2, ..., L:

- Compute the pre-activation: $z^{[l]} = W^{[l]} a^{[l-1]} + b^{[l]}$
- Apply the activation: $a^{[l]} = f^{[l]}(z^{[l]})$
- STORE both $z^{[l]}$ and $a^{[l]}$ — they are required by the backward pass.

Step 3 is often overlooked but is essential. If activations are not stored during the forward pass, they must be recomputed during the backward pass — doubling the computation. Modern frameworks

store these automatically. The input x serves as $a^{[0]}$. The final output $a^{[L]}$ is the network's prediction ŷ.

1.5.3 The Chain Rule and Its Role in Learning

To update any weight, we need ∂J/∂w — the rate of change of cost with respect to that weight. For a weight in the first hidden layer, this is a derivative of J with respect to a quantity many computational steps removed from the output. The chain rule makes this tractable:

$$If\ a = f(z)\ \ and\ \ z = g(w)\ \ then\ \ da/dw = (da/dz) \cdot (dz/dw)$$

The derivative through a sequence of operations is the product of local derivatives along the path. In a network, the cost J depends on $a^{[L]}$, which depends on $z^{[L]}$, which depends on $a^{[L-1]}$, and so on back to the weights. Many intermediate terms are shared across different weight gradients. Computing each shared term once and reusing it makes the total computation linear in depth — this is exactly what backpropagation does.

1.5.4 Backpropagation as Credit Assignment

Define the error signal at layer l as $\delta^{[l]} = \partial J/\partial z^{[l]}$: the rate of change of cost with respect to the pre-activations of layer l. Once $\delta^{[l]}$ is known, the weight and bias gradients follow immediately. The four backpropagation equations:

$$\delta^{[L]} = \hat{y} - y \qquad \textit{(output layer, sigmoid+BCE pairing)}$$

$$\delta^{[l]} = (W^{[l+1]})^T \delta^{[l+1]} \odot f'^{[l]}(z^{[l]}) \qquad \textit{(hidden layers, backward)}$$

$$\partial J/\partial W^{[l]} = \delta^{[l]} (a^{[l-1]})^T \qquad \textit{(weight gradient)}$$

$$\partial J/\partial b^{[l]} = \delta^{[l]} \qquad \textit{(bias gradient)}$$

Reading the hidden-layer equation: $(W^{[l+1]})^T$ distributes blame from each next-layer neuron back to each current-layer neuron proportional to the connecting weight. The element-wise multiply (⊙) with $f'^{[l]}(z^{[l]})$ accounts for how much each pre-activation contributed to the activation. The total computation is approximately twice the cost of the forward pass — a remarkably efficient result for computing gradients with respect to potentially millions of parameters simultaneously.

1.5.5 Worked Example: Backpropagation by Hand Through a Two-Layer Network

We trace one complete forward pass, loss computation, and backward pass through a 2→2→1 network — a simplified version of AgriVision's binary disease detector.

> ✎ **WORKED EXAMPLE**
>
> **Network: 2→2→1 (ReLU hidden, sigmoid output)**
>
> $W^{[1]} = [[0.5, 0.3], [-0.2, 0.8]]$, $b^{[1]} = [0.1, -0.1]$
>
> $W^{[2]} = [0.6, -0.4]$, $b^{[2]} = [0.2]$
>
> Input: x = [0.65, 0.83] (soil moisture, temperature) True label: y = 1 (diseased)
>
> **STEP 1 — Forward pass**
>
> $z_1^{[1]} = 0.5\times0.65 + 0.3\times0.83 + 0.1 = 0.674$ $a_1^{[1]} = \text{ReLU}(0.674) = 0.674$

$z_2^{[1]} = -0.2 \times 0.65 + 0.8 \times 0.83 - 0.1 = 0.434$ $a_2^{[1]} = \text{ReLU}(0.434) = 0.434$

$z^{[2]} = 0.6 \times 0.674 + (-0.4) \times 0.434 + 0.2 = 0.431$ $\hat{y} = \sigma(0.431) \approx 0.606$

STEP 2 — Loss

$L = -\log(\hat{y}) = -\log(0.606) \approx 0.501$

STEP 3 — Backward pass

$\delta^{[2]} = \hat{y} - y = 0.606 - 1.0 = -0.394$ (negative: increasing $z^{[2]}$ reduces the loss)

$\partial J/\partial W^{[2]} = -0.394 \times [0.674, 0.434] = [-0.266, -0.171]$ $\partial J/\partial b^{[2]} = -0.394$

Propagate: $(W^{[2]})^T \times \delta^{[2]} = [0.6 \times (-0.394), -0.4 \times (-0.394)] = [-0.236, +0.158]$

ReLU' at $z_1^{[1]} = 0.674$ is 1; ReLU' at $z_2^{[1]} = 0.434$ is 1 → $\delta^{[1]} = [-0.236, +0.158]$

$\partial J/\partial W^{[1]}$ Row 1: $[-0.236 \times 0.65, -0.236 \times 0.83] = [-0.153, -0.196]$

$\partial J/\partial W^{[1]}$ Row 2: $[+0.158 \times 0.65, +0.158 \times 0.83] = [+0.103, +0.131]$

STEP 4 — Update direction

Gradient descent subtracts η×gradient from each weight. Both $W^{[2]}$ entries increase (negative gradient), pushing $\hat{y}$ toward 1. Notice: stored values $a^{[1]}$ and $z^{[1]}$ from Step 1 were required in Steps 3 and 4.

This example reveals two key facts. First, if either hidden neuron had been inactive (z negative, ReLU output zero), its row in the gradient of $W^{[1]}$ would be entirely zero — the dead neuron phenomenon from Section 1.3. Second, the error signal $\delta^{[1]}$ is smaller in magnitude than $\delta^{[2]}$. In deep networks with sigmoid activations, where each derivative is at most 0.25, this repeated multiplication produces the vanishing gradient problem. ReLU activations, with derivative exactly 1 for active neurons, prevent this collapse.

KEY TERMS — THIS SECTION

Term	Definition
Computational graph	A directed graph where nodes are operations and edges carry values; the basis for backpropagation
Forward propagation	Left-to-right traversal computing and storing $z^{[l]}$ and $a^{[l]}$ at every layer
Chain rule	d/dw[f(g(w))] = f'(g(w))·g'(w); extended to compute gradients through any depth
Error signal $\delta^{[l]}$	$\partial J/\partial z^{[l]}$; rate of change of cost with respect to pre-activations at layer l
Backpropagation	Right-to-left graph traversal computing error signals and weight gradients at every layer
Credit assignment	Determining how much each weight contributed to the final error; backpropagation solves this exactly
Stored activations	$a^{[l]}$ and $z^{[l]}$ saved during forward pass; required by the backward pass
Weight gradient	$\partial J/\partial W^{[l]} = \delta^{[l]}(a^{[l-1]})^T$; direction and magnitude of the weight update

✎ **CHECK YOUR UNDERSTANDING**

1. In the worked example, suppose $z_2^{-+1=}$ had been −0.15 instead of 0.434. What would $a_2^{-+1=}$ be? What would $\delta_2^{-+1=}$ be? What effect would this have on row 2 of $\partial J/\partial W^{-+1=}$?

2. Compute $\partial J/\partial w_{11}^{-+1=}$ directly from the worked example. Show you can arrive at the same answer by multiplying four local derivatives along the path from $w_{11}^{-+1=}$ to J.

3. With $\eta = 0.1$, compute the updated $W^{-+2=}$ after one gradient descent step. Rerun the forward pass with these updated weights (keeping $W^{-+1=}$ and all biases unchanged). What is the new $\hat{y}$?

4. A student says: 'I could compute all gradients without storing any activations by re-deriving during the backward pass.' What is wrong with this in terms of computational cost? What specific stored values are used in the backward pass?

5. Explain in your own words what it means for $\delta^{-+}{}_l^{=}$ to be 'propagated backward.' What is the physical interpretation of $(W^{-+}{}_l^{+1=})^T\, \delta^{-+}{}_l^{+1=}$ in the hidden-layer backpropagation equation?

6. [Challenge] Apply the same original weights to a second training example: x = [0.20, 0.91], y = 0 (healthy). Complete the forward pass and compute $\delta^{-+2=}$. Average the gradients for $W^{-+2=}$ across both examples. Apply one update with $\eta = 0.05$ and recompute predictions. Has total loss decreased? Comment on what class imbalance would do to the gradient signal over many steps.

1.6 Optimisation: Making Gradient Descent Work

1.6.1 Gradient Descent: The Basic Idea

Backpropagation gives us the gradient of J(θ) with respect to every trainable parameter, computed in approximately twice the time of a forward pass. The gradient ∇J(θ) at any point is a vector pointing in the direction of steepest ascent. Moving in the opposite direction moves toward lower cost. This is gradient descent:

$$\theta \leftarrow \theta - \eta \, \nabla J(\theta)$$

Every parameter in θ is updated simultaneously, each moving by a small step proportional to its partial derivative in the direction that reduces J. The scalar η (eta) is the learning rate, controlling step size. The process repeats — forward pass, loss computation, backward pass, parameter update — for as many iterations as needed.

A concrete illustration. In AgriVision's yield network, the gradient of J with respect to the weight connecting x_1 (soil moisture) to hidden neuron 1 is $\partial J/\partial w_{11} = -0.153$ (from the Section 1.5 worked example). With $\eta = 0.01$: $w_{11} \leftarrow 0.5 - 0.01 \times (-0.153) = 0.50153$. The weight increases slightly — which is correct, because the negative gradient tells us cost decreases when this weight increases.

1.6.2 Stochastic and Mini-Batch Gradient Descent

Computing the full gradient requires a complete pass over all N training examples before any update. For 1.2 million ImageNet images this is prohibitive. Stochastic Gradient Descent (SGD) estimates the gradient from a single randomly selected example and updates immediately. Mini-batch gradient descent takes the practical middle ground: estimate the gradient from a small batch of k examples (typically k = 32, 64, 128, or 256), then update.

$$J_B(\theta) = (1/k) \, \Sigma_{i \in B} \, L(\hat{y}_i, y_i)$$

Mini-batches update parameters k/N times more often per epoch than full-batch GD and exploit GPU parallelism — processing 64 examples simultaneously costs almost the same as processing 1. A less obvious benefit: the noise in mini-batch gradient estimates can help the optimiser escape sharp local minima and find flatter minima, which tend to generalise better to new data.

Epoch vs. iteration. One epoch is one complete pass over the training dataset. One iteration is one parameter update using one mini-batch. If N = 1,500 and k = 32, one epoch contains approximately 47 iterations.

1.6.3 The Learning Rate: The Single Most Important Hyperparameter

No other hyperparameter has greater influence on whether training succeeds or fails. Too large: each update overshoots the minimum, causing the cost to oscillate or diverge. Too small: progress is glacially slow and training may stall in flat regions. The practical starting range for most optimisers is 10^{-4} to 10^{-2}.

◆ KEY CONCEPT

The loss landscape

The cost function J(θ) over the space of all parameters is called the loss landscape. For deep networks it is highly non-convex: it has many local minima, saddle points (zero gradient but not a minimum), and flat plateaus.

The good news: research consistently shows that most local minima in deep network landscapes are approximately equally good — they produce similar cost values on held-out data. The real obstacles are saddle points and flat plateaus where gradients are near zero and plain gradient descent stalls. The optimisers below were designed primarily to escape these regions.

1.6.4 Momentum: Accelerating Through Flat Regions

Momentum augments gradient descent by maintaining a velocity vector v — a running weighted average of past gradients — and using it to determine the update:

$$v_t \leftarrow \beta\, v_{t-1} + \eta\, \nabla J(\theta_t)$$

$$\theta_{t+1} \leftarrow \theta_t - v_t$$

The momentum coefficient β (typically 0.9) retains 90% of previous velocity plus 10% of the current gradient scaled by η. Gradients consistently pointing in the same direction accumulate velocity; oscillating gradients partially cancel. Nesterov momentum computes the gradient at the position the current momentum would carry us to (a look-ahead step), correcting the update before it overshoots, and converges slightly faster in practice.

1.6.5 Adaptive Methods: RMSProp and Adam

Both SGD and SGD with momentum use the same learning rate η for every parameter. In AgriVision's network, the weight from soil moisture (highly variable, highly predictive) and the weight from cumulative fertiliser (less variable) will have gradients of very different magnitudes. Adaptive optimisers maintain a separate, automatically adjusted learning rate per parameter.

RMSProp. Maintains $s_t \leftarrow \rho\, s_{t-1} + (1-\rho)(\nabla J)^2$ and updates $\theta \leftarrow \theta - (\eta/\sqrt{(s_t+\varepsilon)})\nabla J$. Parameters with a history of large gradients get a smaller effective learning rate; those with small gradients get a larger one. $\rho \approx 0.9$, $\varepsilon \approx 10^{-8}$.

Adam (Adaptive Moment Estimation). Combines momentum (first moment m_t) and per-parameter adaptive scaling (second moment v_t), with bias correction:

$$m_t \leftarrow \beta_1\, m_{t-1} + (1-\beta_1)\,\nabla J, \quad v_t \leftarrow \beta_2\, v_{t-1} + (1-\beta_2)(\nabla J)^2$$

$$\hat{m}_t = m_t/(1-\beta_1{}^t), \quad \hat{v}_t = v_t/(1-\beta_2{}^t), \quad \theta \leftarrow \theta - \eta\, \hat{m}_t / (\sqrt{\hat{v}_t} + \varepsilon)$$

Standard defaults: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$, $\eta = 10^{-3}$. Adam is the de facto default for most deep learning tasks today. It requires almost no tuning and typically converges significantly faster than SGD with momentum. When in doubt, start with Adam at $\eta = 10^{-3}$.

⚠ **IMPORTANT NOTE**

Adam does not eliminate the need to tune the learning rate.

Adam's adaptive scaling adjusts relative learning rates across parameters, but the global η still determines overall step size. A learning rate that is too large will cause Adam to diverge just as it

would cause SGD to diverge. Adam makes the workable range of η broader and the algorithm less sensitive, but you still need to find a reasonable η. The default of 10^{-3} is the right starting point, not a guaranteed answer.

1.6.6 Learning Rate Schedules: Warming Up and Cooling Down

Step decay. Reduce η by a fixed factor (typically 0.1) every fixed number of epochs. Simple and effective for classical computer vision models.

Cosine annealing. Smoothly decay η from its initial value to near zero following a cosine curve: $\eta_t = \eta_{min} + \frac{1}{2}(\eta_{max} - \eta_{min})(1 + \cos(\pi t/T))$. Avoids abrupt drops and empirically produces good results across many tasks.

Warmup. Linearly increase η from near zero to its target value over the first few hundred or thousand steps. Prevents Adam from making catastrophically large updates in the first steps when moment estimates are poor. Typical recipe: warm up over 4% of training steps, then apply cosine decay.

For AgriVision's small tabular network (273 parameters), these schedules are unnecessary — a fixed rate with Adam suffices. For the CNN disease detector in Chapter 2, with hundreds of thousands of parameters trained on thousands of images, warmup followed by cosine annealing is the appropriate starting point.

KEY TERMS — THIS SECTION

Term	Definition
Gradient descent	$\theta \leftarrow \theta - \eta\nabla J(\theta)$; iterative parameter update in the direction of steepest cost decrease
Learning rate (η)	Scalar controlling step size at each update; the most critical hyperparameter
Epoch	One complete pass over the entire training dataset
Mini-batch	A small subset of k examples used to estimate the gradient at each step
Momentum	Running average of past gradients (velocity) to accelerate through flat regions; $\beta \approx 0.9$
RMSProp	Adaptive optimiser dividing each update by the root mean square of recent gradients
Adam	Combines momentum and per-parameter adaptive learning rates; default choice for most tasks
Bias correction	Division by $(1-\beta^t)$ in Adam to compensate for zero-initialisation of moment estimates
Learning rate schedule	A rule for varying η during training; common types: step decay, cosine annealing, warmup
Loss landscape	The surface of $J(\theta)$ over all parameter configurations; non-convex for deep networks

✎ **CHECK YOUR UNDERSTANDING**

1. AgriVision's yield model has 273 parameters, 750 training examples, and mini-batch size 32. How many iterations constitute one epoch? How many parameter updates occur in 50 epochs?

2. A training run shows losses: Epoch 1: 2.31, Epoch 5: 2.18, Epoch 10: 2.05, Epoch 15: 2.47, Epoch 20: 3.12, Epoch 25: 4.80. What does this pattern indicate about the learning rate? What would you do to fix it?

3. Explain intuitively why momentum helps convergence in a ravine-shaped loss landscape where loss decreases slowly in one direction but has steep walls perpendicularly. What does momentum do to the perpendicular gradient oscillations?

4. Adam maintains two running averages per parameter: m_t (first moment) and v_t (second moment). Explain in plain language what each estimates and what role each plays in the update.

5. A network is trained with Adam at $\eta = 10^{-3}$ for 100 epochs. Training loss reaches 0.12 but validation loss plateaus at 0.41. The team continues for 200 more epochs at the same learning rate. Will this help? What should they do instead?

6. [Challenge] Two parameters: w_1 consistently receives gradient magnitudes ≈0.8; w_2 receives ≈0.002. With SGD at $\eta = 0.01$, compute the update magnitude for each per step. Now compute what Adam does differently after 1,000 steps ($\beta_2 = 0.999$, bias-correction ≈1). Show numerically why Adam is better suited to networks where parameters have gradients of very different magnitudes.

1.7 Generalisation, Overfitting, and Regularisation

1.7.1 The Bias-Variance Trade-off

AgriVision's yield model was trained on 750 farm-season records. Once deployed to advise farmers in a new season, the crucial question is not how well the model fits the training data but how well it predicts yields it has never seen. The expected squared prediction error on a new example decomposes as:

$$\textit{Expected error} = \textit{Bias}^2 + \textit{Variance} + \textit{Irreducible noise}$$

Bias measures how far, on average, the model's predictions are from the true relationship. A model too simple to capture nonlinear yield-moisture interactions will systematically err. This is underfitting: high-bias, the model has not captured the true structure.

Variance measures how much predictions change when trained on a different sample of the same size. A model too complex for 150 farms can memorise their specific quirks — one farm's anomalously high yield in 2019 due to an unreported soil amendment. This is overfitting: high-variance, predictions depend too strongly on the training set's specifics.

Irreducible noise arises from true stochasticity in crop yields — measurement error, unobserved factors. It cannot be reduced regardless of model quality.

The trade-off is unavoidable: as model complexity increases, bias decreases but variance increases. A good model balances these forces. The practical question is how to control variance without sacrificing the capacity to learn useful structure.

1.7.2 Overfitting: Why Deep Networks Are Particularly Prone

AgriVision's yield network has 273 parameters and 750 training examples (ratio 1:2.75 — safe for tabular regression). The disease detection CNN of Chapter 2 will have approximately 2 million parameters and perhaps 3,000 labelled images: 667 parameters per training example. At this ratio, a network without regularisation will almost certainly overfit.

The signature of overfitting in loss curves: training loss continues to decrease, while validation loss decreases at first, reaches a minimum, then increases. The growing generalisation gap is the diagnostic. Deep networks are especially capable memorisers — they can exploit compositional structure to identify individual training examples and memorise which label to attach to each.

1.7.3 Dropout: Preventing Co-Adaptation

In a fully connected network, neurons can develop co-adaptation: neuron A learns to correct for neuron B's mistakes, and B learns to rely on A. If A is unreliable on new data, B's corrections collapse. Dropout (Srivastava et al., 2014) addresses this by randomly deactivating a fraction p of neurons per forward pass during training.

- Draw a Bernoulli random variable with probability (1−p) of being 1 for each neuron in the dropout layer.
- Zero the activations of dropped neurons.
- Scale surviving neurons by 1/(1−p) — inverted dropout — so the expected output magnitude is preserved at test time.

At test time, dropout is disabled. No neuron can rely on any specific neighbour being present, forcing each to develop a representation useful on its own. The trained network is implicitly an ensemble of exponentially many sub-networks sharing weights. Typical settings: p = 0.5 for large fully connected layers; p = 0.1–0.3 for convolutional layers or small layers.

✎ WORKED EXAMPLE

Dropout in AgriVision's 4-neuron layer (p = 0.5, inverted dropout)

Training step 1 — Mask [1, 0, 1, 0], pre-dropout activations [0.82, 0.47, 0.61, 0.39]:

After dropout and ×2 scaling: [1.64, 0, 1.22, 0] — neurons 1 and 3 only

Training step 2 — Mask [0, 1, 0, 1], pre-dropout activations [0.79, 0.51, 0.58, 0.44]:

After dropout and ×2 scaling: [0, 1.02, 0, 0.88] — neurons 2 and 4 only

Neuron 1 cannot depend on neuron 2 because neuron 2 is absent half the time. Each must develop a useful representation independently.

At test time: all four neurons are active, no scaling applied. Outputs: [0.82, 0.47, 0.61, 0.39] unchanged.

1.7.4 Batch Normalisation: Stabilising the Training Signal

As layer l's parameters update, the distribution of inputs to layer l+1 shifts — even though layer l+1's own weights have not changed. This internal covariate shift forces each layer to adapt to a moving target, requiring small learning rates and careful initialisation. Batch Normalisation (Ioffe and Szegedy, 2015) normalises activations within each mini-batch:

$$\mu_B = (1/m)\Sigma x_i, \quad \sigma^2_B = (1/m)\Sigma(x_i-\mu_B)^2, \quad \hat{x}_i = (x_i-\mu_B)/\sqrt{(\sigma^2_B+\varepsilon)}$$

$$y_i = \gamma \hat{x}_i + \beta \qquad (\gamma \text{ and } \beta \text{ are learnable per-feature parameters})$$

The normalisation eliminates dependence on input scale and offset. The learnable γ and β allow the network to represent any linear transformation of the normalised values — including undoing the normalisation if optimal. Benefits beyond covariate shift: allows higher learning rates, acts as a mild regulariser (mini-batch statistics are noisy estimates, introducing dropout-like perturbation), and reduces sensitivity to initialisation. At inference, batch normalisation uses running estimates of mean and variance computed during training via exponential moving average.

⚠ IMPORTANT NOTE

Batch size matters for Batch Normalisation.

With batch size 1, the normalised output is always zero — making normalisation useless. With batch size 2, the variance estimate is unreliable. In practice batch normalisation works well with batch sizes of 16 or larger.

For batch sizes below 8, use Layer Normalisation instead, which normalises across features for each example independently of the batch. Layer Normalisation is the standard choice in Transformers.

1.7.5 Data Augmentation: Expanding the Training Set Cheaply

The most direct remedy for overfitting is more training data. Data augmentation creates additional examples by applying label-preserving transformations to existing ones.

For image data (disease detection). Random horizontal/vertical flipping, rotation (±15°), random cropping and resizing, brightness and contrast adjustment, colour jitter, Gaussian blur. A leaf with powdery mildew photographed at a different angle is still a leaf with powdery mildew. Avoid aggressive colour jitter for disease detection — colour is diagnostically significant.

For time-series data (irrigation prediction). Adding small Gaussian noise to sensor readings (simulating measurement error), time warping (slightly stretching or compressing the temporal axis), window slicing (slightly offset or shorter temporal windows).

The golden rule. Every augmentation must preserve the semantic meaning that determines the label. Always visualise augmented examples before training.

1.7.6 Early Stopping and Proper Validation Strategy

Early stopping terminates training when the model stops improving on held-out data. Withhold 15–20% of training data as a validation set — never used for gradient computation. Monitor validation loss after each epoch; restore the weights corresponding to the best validation loss when training ends. Patience of 10–20 epochs before stopping is typical.

The standard split for AgriVision's disease dataset: 70% training, 15% validation, 15% test. For small datasets (< 2,000 examples), 5-fold cross-validation is more appropriate: divide into 5 folds, train 5 times with each fold as validation once, average the performance.

> ⚠ **IMPORTANT NOTE**
>
> Validation set contamination.
>
> Every decision informed by validation performance — number of layers, dropout rate, learning rate, stopping point — has used up some of the validation set's independence. This causes reported validation performance to overestimate performance on truly new data.
>
> The solution: strict separation. A validation set for all training decisions, and a test set used once, reported once, never used in any decision. Using the test set to select the best epoch, then reporting that best test loss, is a common mistake that produces an invalid, optimistic estimate.

1.7.7 When Regularisation Goes Wrong: Common Mistakes

Too much dropout. With p = 0.8 on a 32-neuron layer, on average only 6 neurons are active per forward pass. The layer cannot represent anything useful. If training loss barely moves, dropout is too aggressive.

Batch normalisation with very small batches. Batch sizes below 8 produce unreliable normalisation statistics. Switch to Layer Normalisation or increase batch size.

Augmentations that change the label. Flipping a digit 6 vertically produces something resembling a 9. Crops removing the disease lesion create mislabelled 'healthy' images. Always inspect augmented samples manually.

Data leakage. Normalising input features using statistics from the entire dataset (including test examples) before splitting introduces leakage. Always compute mean and standard deviation on the training set only, then apply those statistics to normalise validation and test sets.

KEY TERMS — THIS SECTION

Term	Definition
Generalisation	The ability of a trained model to perform well on data it was not trained on
Bias	Systematic error of a model too simple for the true data structure; causes underfitting
Variance	Sensitivity of predictions to the specific training set; causes overfitting
Overfitting	Model fits training noise; training loss low, validation loss high and diverging
Underfitting	Model too simple; both training and validation losses are high
Dropout	Randomly zeroing fraction p of neurons per forward pass to prevent co-adaptation
Inverted dropout	Scaling survivors by $1/(1-p)$ during training so no scaling is needed at test time
Batch Normalisation	Normalises each mini-batch to zero mean and unit variance; learns scale γ and shift β
Internal covariate shift	Distribution shift experienced by each layer as earlier layers' parameters update
Data augmentation	Creating training examples via label-preserving transformations of existing ones
Early stopping	Terminating training when validation loss stops improving; restoring best weights
Validation set	Data withheld from training to monitor generalisation and make training decisions
Test set	Data withheld entirely; used once, reported once, never used in any decision

✎ CHECK YOUR UNDERSTANDING

1. AgriVision's CNN achieves 97.3% training accuracy and 68.1% validation accuracy after 40 epochs. Describe what is happening and name three specific techniques from this section that would reduce the gap, explaining the mechanism of each.

2. Dropout $p = 0.5$ is applied to an 8-neuron layer. Mask: [1,0,0,1,1,0,1,0]. Pre-dropout activations: [0.9,0.4,0.7,0.3,0.8,0.5,0.2,0.6]. Compute post-dropout activations using inverted dropout. What are the activations at test time?

3. In batch normalisation, γ and β are often overlooked. Explain why they are necessary. What would batch normalisation be limited to if $\gamma = 1$ and $\beta = 0$ were fixed rather than learned?

4. A student normalises the entire dataset (train+val+test combined) before splitting. Explain precisely why this introduces data leakage and what effect it has on the reported test performance.

5. AgriVision has 3,000 labelled leaf photographs. Describe the trade-offs between a fixed 70/15/15 split versus 5-fold cross-validation. Under what conditions is each preferable?

6. [Challenge] A model is trained with early stopping; best validation loss is at epoch 38 (test accuracy: 84.2%). The team trains 60 more epochs, checks the test set each epoch, and reports the best observed test accuracy: 86.7%. Explain precisely why this accuracy is invalid and what concept it violates. What is the correct procedure?

1.8 Reading the Training Signal: Debugging Deep Networks

1.8.1 The Loss Curve as Diagnostic Instrument

The loss curve is the primary diagnostic instrument of training. It should be generated and inspected every time a model is trained — not just at the end, but during training. The shape of the loss curve over time contains far more diagnostic information than the final number alone. Plot both training loss and validation loss on the same axes. Apply a running average to smooth the noisy per-batch training loss without losing the trend. Use a log scale for classification tasks where proportional improvements are more informative.

The baseline loss. Before training begins, establish the expected loss at random initialisation. For a K-class classifier with correctly implemented softmax output:

$$L_0 = -log(1/K) = log(K)$$

For AgriVision's four-class disease classifier: $L_0 = \log(4) \approx 1.386$. For binary classification: $L_0 = \log(2) \approx 0.693$. If the initial loss differs substantially from this value — 0.2 or 8.4 — the output activation, loss function, or label encoding has a bug. This check takes thirty seconds and catches a class of errors that would otherwise waste hours of training.

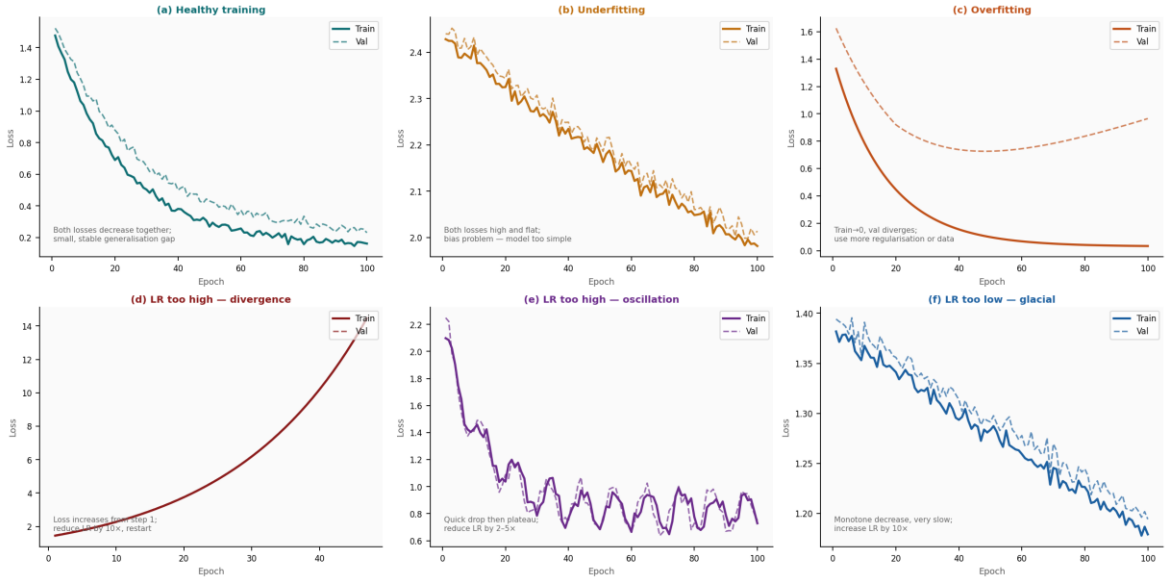


Figure 1.6 *Six characteristic loss curve patterns. (a) Healthy: both losses decrease, stable small gap. (b) Underfitting: both losses high and nearly flat. (c) Overfitting: training loss falls while validation loss rises. (d) Divergence: loss explodes from the first step. (e) Oscillation: quick initial drop followed by oscillation around a plateau. (f) Glacial: monotone but extremely slow decrease. Training loss: solid line. Validation loss: dashed line.*

1.8.2 Diagnosing Underfitting

Both training and validation losses remain high, decrease slowly or barely, and the gap between them is small. The model performs similarly on both sets — but both are poor. This is the high-bias side of the bias-variance trade-off.

Epoch	Train loss	Val loss
1	1.38	1.39
10	1.25	1.27
30	1.18	1.19
60	1.15	1.16
100	1.14	1.14

Both losses start near the random baseline (1.386) and barely improve. The small gap means no overfitting — the model simply is not learning. Causes and remedies, in order of likelihood:

- Model too small: increase depth, width, or both and retrain.
- Learning rate too low: the curve decreases monotonically but very slowly. Increase η by 10×.
- Training too short: the curve is still declining at the final epoch. Train longer.
- Bad input normalisation: features at very different scales produce a poorly conditioned loss surface. Normalise all inputs to zero mean and unit standard deviation using training-set statistics.
- Saturating activations: large initial pre-activations cause sigmoid/tanh saturation immediately. Switch to ReLU or use Glorot (Xavier) weight initialisation.

1.8.3 Diagnosing Overfitting

Training loss decreases steadily to a low value; validation loss decreases at first, reaches a minimum, then increases. The generalisation gap grows as training continues.

Epoch	Train loss	Val loss
1	1.38	1.40
10	0.82	0.85
20	0.41	0.44
30	0.18	0.39
50	0.06	0.58
80	0.01	0.89

Training loss converges toward zero while validation loss rises after epoch 20. The best weights are those at epoch 20 (validation loss 0.44); early stopping with patience 10 would have halted around epoch 30. Remedies: add or increase dropout, add weight decay, collect more data or augment, implement early stopping.

Weight decay (L2 regularisation). Adds $\lambda/2\ \Sigma w_j^2$ to the cost function. The gradient of this penalty modifies the update to $w_j \leftarrow (1-\eta\lambda)w_j - \eta\ \partial J/\partial w_j$. The factor $(1-\eta\lambda)$ multiplies each weight by slightly less than 1 at every step, preventing individual weights from growing large enough to memorise specific training examples. Typical range: $\lambda = 10^{-4}$ to 10^{-2}.

1.8.4 Learning Rate Pathologies: Too High, Too Low, Wrong Schedule

Immediate divergence (learning rate far too large). Loss increases from the very first step: 1.38 → 2.61 → 7.94 → NaN. The optimiser takes steps so large it overshoots the valley floor entirely, landing higher each time. Remedy: reduce η by 10×, restart from fresh initialisation.

Oscillating plateau (learning rate moderately too large). Loss drops quickly in the first few epochs, then oscillates around a plateau noticeably higher than what the model should achieve. The optimiser found the general neighbourhood of a minimum but overshoots it at each step. Remedy: reduce η by 2–5×.

Glacial convergence (learning rate too small). Loss decreases steadily but very slowly; the curve is still declining at the final epoch with no sign of flattening. Remedy: increase η by 10× or more.

NaN loss. Almost always indicates one of three problems: learning rate too large (floating-point overflow); logarithm of zero (a predicted probability collapsed to exactly 0); or NaN in input data from unhandled missing values. Check input data first, then loss function implementation, then reduce the learning rate.

1.8.5 The Sanity Check Protocol: Before You Scale Up

The most common waste in deep learning projects is spending days training a large model on the full dataset before discovering a bug that a five-minute check on a small model would have found. The sanity check protocol surfaces fundamental errors before they consume compute.

◆ KEY CONCEPT

The Sanity Check Protocol — five checks, run in order before any serious training run.

Check 1: Initial loss matches theory.

Before any training step, compute loss on a random mini-batch. For K-class classifier expect log(K). A very different value means the output activation, loss function, or label encoding has a bug.

Check 2: Overfit a single mini-batch.

Train on a fixed mini-batch of 8–32 examples with no regularisation for hundreds of iterations. Training loss must converge to near zero. If the network cannot overfit a tiny dataset, it cannot learn at all — the architecture, loss pairing, or optimiser has a fundamental error. This is the most important check.

Check 3: Shuffled-label test.

Train briefly on the full training set with randomly shuffled labels. Training loss should decrease (a large network can memorise random labels) but validation loss must remain near the random baseline. If validation loss also decreases, there is data leakage between training and validation sets.

Check 4: Gradient norm inspection.

After the first backward pass, inspect $\|\partial J/\partial W^{[l]}\|$ at each layer. If the first-layer norm is less than 10^{-6} of the last-layer norm, vanishing gradients are occurring. If any norm exceeds 100, exploding gradients are occurring.

Check 5: Scale up incrementally.

Once Checks 1–4 pass on a small dataset (100–200 examples), scale to the full dataset. Dramatic degradation when scaling usually indicates a data preprocessing inconsistency.

Gradient clipping. When Check 4 reveals exploding gradients — common in recurrent networks — cap the norm of the full gradient vector before applying the update. If $\|\nabla J(\theta)\| >$ threshold (typically 1.0 or 5.0): $\nabla J(\theta) \leftarrow \nabla J(\theta) \times (\text{threshold} / \|\nabla J(\theta)\|)$. This preserves the gradient direction while bounding its magnitude.

KEY TERMS — THIS SECTION

Term	Definition
Loss curve	Plot of training and validation loss against epoch; primary diagnostic tool
Baseline loss	Expected loss at random initialisation: log(K) for K-class classification
Generalisation gap	Difference between validation and training loss; growing gap indicates overfitting
Divergence	Training loss increasing rather than decreasing; caused by LR too large or NaN in data
NaN loss	Not-a-Number; caused by overflow, log of zero, or NaN in input data
Overfit one batch	Sanity check: train on a tiny fixed dataset until training loss → 0
Gradient norm	$\|\partial J/\partial W^{[l]}\|$; used to detect vanishing or exploding gradients at each layer
Gradient clipping	Bounding gradient norm before each update to prevent exploding gradients
Weight decay	L2 regularisation: adds $\lambda/2\ \Sigma w_j^2$ to cost; prevents weights from growing too large

✎ CHECK YOUR UNDERSTANDING

1. AgriVision's four-class disease classifier is randomly initialised. A forward pass produces a loss of 3.84. Is this expected? What initial loss would you expect for a binary healthy/diseased classifier?

2. The following training history is observed on the yield regression model (RMSE kg/acre). Epoch 1: train=312, val=318. Epoch 50: train=298, val=305. Epoch 150: train=295, val=303. Diagnose this pattern and propose two specific interventions.

3. A student runs 'overfit one batch' with 16 examples. After 500 iterations, training loss is 0.91 (starting from 1.38). Not converging to zero. List three specific problems to investigate and explain how each produces this symptom.

4. After the first backward pass, gradient norms are: Layer 1: 3.8×10^{-7}, Layer 2: 1.2×10^{-5}, Layer 3: 4.6×10^{-3}, Output: 3.1×10^{-1}. What problem does this indicate? What activation function is most likely in use? Name two remedies.

5. A training run produces NaN loss at step 47 with $\eta = 10^{-3}$. Describe the systematic investigation you would conduct. List checks in order and what each rules out.

6. [Challenge] A researcher trains with weight decay $\lambda = 10^{-1}$. Training loss plateaus at 1.19 despite gradient boosting achieving 0.31 on the same data. Compute the weight update for $w = 0.45$, gradient -0.12, $\eta = 0.01$ for both $\lambda = 10^{-3}$ and $\lambda = 10^{-1}$. Show numerically what happens under excessive regularisation. How does this loss curve pattern differ from underfitting caused by a model that is too small?

1.9 When to Use Deep Learning — and When Not To

1.9.1 The Decision Checklist

This question has appeared at the end of every section in this chapter, as promised in Section 1.0.5. We now give a rigorous answer, earned by studying the full machinery of deep networks — neurons, activations, loss functions, backpropagation, gradient descent, regularisation, and debugging.

◆ KEY CONCEPT

The Deep Learning Decision Checklist

Work through these questions in order. The first question that points toward classical ML should be taken seriously.

Q1: What is the structure of the input data?

Tabular data with well-defined named features → favour classical ML. Images, audio, text, time-series sensor streams, graphs, video → favour deep learning.

Q2: How many labelled training examples are available?

Fewer than ~1,000 → classical ML; deep learning will overfit. 1,000–10,000 → deep learning viable with strong regularisation and transfer learning. More than 10,000 → deep learning gains a clear advantage for complex inputs.

Q3: Is interpretability required?

Regulator, clinician, or farmer needs to understand each prediction → inherently interpretable models (logistic regression, decision tree, gradient boosting with feature importance). Post-hoc explanation acceptable → deep learning is viable.

Q4: What are the compute constraints?

Training in seconds on CPU → classical ML. GPU available, hours to days acceptable → deep learning feasible. Inference on embedded sensor in <10 ms → classical ML or heavily compressed model.

Q5: Are the useful features known and reliable?

Domain experts can specify which inputs matter and how to engineer them → classical ML can exploit this directly. Relevant features are unknown or exist in high-dimensional raw form → representation learning is the right tool.

Q6: What is the cost of errors?

Safety-critical application → prioritise interpretability, calibrated uncertainty, and human oversight before deploying deep learning. Lower-stakes with human review → deep learning's accuracy ceiling may justify deployment complexity.

1.9.2 Tabular Data: Where Classical ML Often Wins

Tabular data — structured data in rows and columns with named features — is the most common data type in enterprise, scientific, and industrial applications. It is also where deep learning has the most difficulty establishing dominance over classical methods.

Gradient boosting builds an ensemble of decision trees in sequence, each correcting the residual errors of those before it. On tabular data it holds several structural advantages: handles mixed feature types (numerical, ordinal, categorical) without preprocessing; is invariant to monotone feature transformations; models complex nonlinear feature interactions with moderate data; produces native feature importance scores; and requires almost no hyperparameter tuning for a strong baseline. Implementations such as XGBoost, LightGBM, and CatBoost have dominated tabular competition consistently.

Neural networks can outperform gradient boosting on tabular data when: the dataset is very large (millions of rows); features include high-cardinality categoricals that benefit from learned embeddings; the problem requires end-to-end learning from mixed modalities; or the relevant feature combinations are too numerous to specify by hand. For most moderate-scale tabular problems, gradient boosting is the honest first choice.

1.9.3 The AgriVision Yield Model: A Deliberate Comparison

AgriVision's tabular dataset has 750 training records and 150 test records (five seasons training, sixth as test). All models receive the same 7 normalised input features. Performance is RMSE and MAE in kg/acre on the test set. Dataset average yield: 1,820 kg/acre.

Model	Details	Test RMSE (kg/acre)	Test MAE (kg/acre)	Training time
Linear regression	—	312	258	< 1 s
Decision tree	max_depth = 8	286	231	< 1 s
Gradient boosting	200 trees, depth 4	198	164	3 s
MLP (no regularisation)	7→64→32→1, ReLU	241	196	45 s
MLP (dropout + weight decay)	7→64→32→1, p=0.3, $\lambda=10^{-3}$	219	178	45 s
MLP (larger, regularised)	7→128→64→32→1	214	174	80 s

Table 1.3 *Comparative results on AgriVision's crop yield prediction task (lower is better). Gradient boosting is the strongest model; larger neural networks do not close the gap.*

Gradient boosting (RMSE 198) outperforms every neural network despite training in three seconds. Adding regularisation to the MLP (RMSE 241 → 219) confirms that the unregularised network was overfitting. But even the best-regularised MLP (RMSE 214) does not approach gradient boosting. The yield data has 750 examples and 7 well-understood features. Gradient boosting efficiently models the interactions between moisture, temperature, and yield without needing to learn representations from scratch. There is no hidden representation to discover here.

The honest engineering conclusion: for this task, at this scale, gradient boosting is the right tool. Against an average yield of 1,820 kg/acre, RMSE 198 vs. 219 represents 10.9% vs. 12.0% relative error. At scale across 150 farms over multiple seasons, that difference has real economic consequences for the farming families the system is meant to help.

> ✦ **THE AGRIVISION CONTEXT**
>
> **Chapter 1 Resolution — What AgriVision built**

After six months of development, AgriVision has a deployed yield prediction system: a gradient boosting model achieving RMSE 198 kg/acre on held-out test farms, compared to extension officers' rule-of-thumb estimates assessed at approximately 18% relative error (vs. the model's 10.9%). The cooperative is satisfied.

The deep learning infrastructure built across this chapter — the artificial neuron, backpropagation, Adam, dropout, batch normalisation, early stopping, loss diagnostics — is not idle. It is the foundation on which every subsequent chapter is built.

Eight thousand photographs of crop leaves remain unanalysed. Gradient boosting cannot examine them. No tabular feature captures what a photograph records: the colour gradient at a disease lesion boundary, the radial spread of mildew, the vein discolouration that precedes visible symptoms by three days. This is not a data problem. It is an architecture problem. Chapter 2 builds the tool to solve it.

KEY TERMS — THIS SECTION

Term	Definition
Tabular data	Structured data in rows and columns, each column a named feature; most common enterprise data type
Gradient boosting	Ensemble of decision trees built sequentially; dominant on moderate-scale tabular data
Feature importance	Score measuring each input feature's contribution to a gradient boosting model's predictions
Representation learning	Learning useful intermediate features from raw inputs; deep learning's core advantage
Decision checklist	Structured questions to decide whether deep learning or classical ML is more appropriate
Relative error	Error as fraction of target mean: RMSE / mean(y); enables cross-task comparison

✎ **CHECK YOUR UNDERSTANDING**

1. Apply the Decision Checklist to: (a) predicting customer churn from 12 demographic features, 80,000 customers; (b) classifying audio recordings of engine sounds as normal or faulty, 2,400 recordings; (c) forecasting daily electricity demand from temperature, day-of-week, and holiday indicators, five years of hourly data.

2. From Table 1.3, the unregularised MLP (RMSE 241) is worse than the regularised MLP (RMSE 219), which is worse than gradient boosting (RMSE 198). Explain why each gap exists. What does the first reveal about regularisation? The second about the suitability of deep learning for this task?

3. A colleague argues: 'We should use the regularised MLP anyway because AgriVision is building deep learning infrastructure and one model type is simpler than two.' Evaluate this argument. When is it valid and when is it not?

4. What would need to change in AgriVision's yield prediction setting for a neural network to become competitive with gradient boosting? Name three specific changes and explain the mechanism by which each would shift the comparison.

5. A farmer asks both models to explain their prediction of 1,640 kg/acre. How would you answer from (a) the gradient boosting model and (b) the regularised MLP? Which makes this question easier to answer, and what tools are available for the harder case?

6. [Challenge] Design a hybrid system using gradient boosting for yield prediction and a CNN for disease detection (Chapter 2), producing a single integrated risk score per farm per timestep. What are the practical engineering challenges of maintaining two model types in production? Under what conditions would a unified end-to-end deep architecture justify replacing gradient boosting?

1.10 Chapter Resolution and Bridge

1.10.1 What We Built for AgriVision

Six months after signing the contract with the Karnataka cooperative, AgriVision has a working system. A gradient boosting model predicts crop yield with RMSE 198 kg/acre — 10.9% relative error against a district average of 1,820 kg/acre. The extension officers' rule-of-thumb estimates ran to approximately 18% relative error. AgriVision's model is measurably better. The cooperative renews the contract.

The choice of gradient boosting over a neural network was deliberate, not a concession. The data was tabular. The features were known and meaningful. The training set was 750 records. Those three facts, checked against the Decision Checklist in Section 1.9.1, pointed clearly to classical ML. Good engineering means choosing the right tool, not the most complex one.

But the deep learning infrastructure built across this chapter is not idle. Every concept in Chapters 2 through 8 rests on what was established here. The artificial neuron and the MLP (Sections 1.2–1.3) are the computational primitives from which every architecture is assembled. CNNs, Transformers, and Autoencoders are all MLPs with additional structural constraints encoding specific inductive biases. The loss function and backpropagation (Sections 1.4–1.5) are the universal learning machinery — whatever architecture is chosen, training reduces to computing gradients of a loss and applying them via the update rules of Section 1.6. The regularisation toolkit (Section 1.7) and the debugging protocol (Section 1.8) apply universally across every architecture in subsequent chapters.

1.10.2 The Photograph Problem: Why Numbers Are Not Enough

On a Thursday afternoon in March, an extension officer in Gadag district sends a message to AgriVision's field team: a photograph of a tomato leaf with a cloudy white coating spreading from the edges. 'Seen on multiple plants in rows 4 through 9. Is this powdery mildew or something else? How bad?'

AgriVision's yield prediction model receives seven numbers and returns a number. It cannot receive a photograph. A photograph of a leaf is not a row in a spreadsheet. The team tries flattening the 224×224 image into a vector and feeding it to an MLP. The first weight matrix alone — from 150,528 inputs to 256 hidden neurons — contains 38,535,168 parameters. Training this reliably from 6,000 images is impossible. The model achieves 51.3% validation accuracy on four disease classes, barely above the 25% random baseline. After a week of experimentation: 58.7%. Still inadequate.

The failure is not a training failure or a regularisation failure. It is an architectural failure. The MLP has no mechanism to exploit the properties that make photographs informative. The parameter explosion makes reliable training impossible. Flattening destroys spatial structure — the pixels adjacent to a lesion boundary are no more related to their neighbours in the vector than to pixels on the opposite side of the image. And the same disease pattern appearing in different positions in the image is treated as a completely different input.

These three failures are the predictable consequences of applying an architecture with the wrong inductive bias. The MLP encodes no assumptions about the spatial structure of images. What is needed is an architecture that processes images through local filters detecting patterns in small spatial neighbourhoods, shares those filters across all image positions so the same pattern is recognised anywhere, and builds a hierarchy from low-level local features to high-level disease

structures. This set of requirements is a precise description of a Convolutional Neural Network. Chapter 2 builds it from the ground up.

Chapter Summary

Concept	Section	Core statement
Inductive bias	1.0	Every architecture encodes assumptions about data structure; the right bias depends on the data type
Architecture Decision Map	1.0	A framework mapping data types → structural properties → inductive biases → architecture families
Representation learning	1.1	Deep networks learn useful intermediate features from raw data rather than relying on hand-crafted features
Three ingredients	1.1	Data, compute, and algorithms together enabled the deep learning era
Artificial neuron	1.2	$z = w^T x + b$; $a = f(z)$; weights and bias are learned; nonlinear activation is essential
Universal approximation	1.2	Sufficient-width MLPs can approximate any continuous function; depth makes this tractable in practice
Activation functions	1.3	ReLU is the default hidden-layer activation; output activation is determined by task type, not preference
Vanishing gradient	1.3	Saturating activations (sigmoid, tanh) cause exponentially small gradients in early layers
Loss functions	1.4	Regression: MSE/MAE; binary classification: BCE+sigmoid; multi-class: CCE+softmax
Conjugate pairing	1.4	Correct loss-activation pairs simplify the output gradient to $\hat{y} - y$
Forward propagation	1.5	Left-to-right computation storing $z^{[l]}$ and $a^{[l]}$ at each layer for use during backward pass
Backpropagation	1.5	Efficient gradient computation via chain rule; $\delta^{[l]}$ propagated backward from output to input
Gradient descent	1.6	$\theta \leftarrow \theta - \eta \nabla J(\theta)$; mini-batch balances gradient quality and update frequency
Adam optimiser	1.6	Combines momentum and per-parameter adaptive learning rates; default choice for most tasks
Bias-variance trade-off	1.7	Underfitting: model too simple; overfitting: model memorises training noise
Regularisation toolkit	1.7	Dropout, batch normalisation, data augmentation, early stopping, weight decay
Loss curve diagnosis	1.8	Six patterns: healthy, underfitting, overfitting, divergence, oscillation, glacial convergence
Sanity check protocol	1.8	Five checks before any serious training run: verify initial loss, overfit one batch, inspect gradient norms
When not to use DL	1.9	Tabular data, small datasets, interpretability requirements, constrained compute → classical ML first
AgriVision yield model	1.9	Gradient boosting (RMSE 198) outperforms all MLP variants (best RMSE 214); correct choice for this task

Chapter-Level Integrative Problems

These problems require combining concepts from multiple sections and are appropriate for assignment or group discussion.

Problem 1.A — Full pipeline design

(a) AgriVision is asked to predict whether a farm will require emergency pest intervention in the next 14 days, using daily sensor readings, pest outbreak history of neighbouring farms, and the extension officer's most recent written field notes. Identify the three input modalities, classify each using the Architecture Decision Map, and state the appropriate architecture family for processing each.

(b) Formulate the task as a supervised learning problem. Specify the output type, the appropriate output activation, and the appropriate loss function. Justify each choice.

(c) Design a data collection strategy. Only 12% of farm-seasons resulted in emergency interventions. Describe the class imbalance problem and propose two techniques to address it.

(d) Propose a validation strategy given 150 farms as the full dataset. Explain why a random 80/20 train/test split is inappropriate for this temporal prediction problem and describe a better approach.

Problem 1.B — Diagnosis and repair

(a) A team trains a binary disease classifier on AgriVision leaf photographs. After 100 epochs: train acc 53.6%, val acc 53.2%, train loss 0.669, val loss 0.671. The initial loss was 0.694. Compute the expected initial loss for a binary classifier. Is the observed initial loss consistent? What does this suggest?

(b) Identify the failure mode (underfitting or overfitting) and justify. After 100 epochs the accuracy is barely above random; the generalisation gap is small.

(c) The network input is the raw flattened image fed to an MLP. Which three specific properties of image data does this architecture fail to exploit?

(d) Prescribe a systematic set of interventions addressing both the architectural failure and any additional failure modes visible in the loss history.

Problem 1.C — Backpropagation by extension

(a) Extend the Section 1.5.5 worked example to a mini-batch of two examples. Example 2: x = [0.20, 0.91], y = 0 (healthy). Using the original weights, compute the forward pass and $\delta^{-+2=}$. Average the per-example gradients for $W^{-+2=}$ across both examples. In which direction does the averaged gradient push $W^{-+2=}$ compared to either example alone?

(b) Apply one gradient descent step with $\eta = 0.05$ using the mini-batch average gradient for $W^{-+2=}$. Compute new predictions $\hat{y}$ for both examples using updated $W^{-+2=}$ (all other weights unchanged).

(c) Compute the BCE loss for each example before and after the update. Has total loss decreased? Explain why one example's loss may increase while the total decreases.

(d) Comment on what would happen over many steps if the dataset were 95% negative (healthy) and 5% positive (diseased). How would this affect the gradient signal?

Problem 1.D — Architecture selection under constraints

(a) A regional hospital wants to detect early leaf rust on wheat with 450 labelled photographs (300 healthy, 150 rust-infected) and a ₹40,000 Raspberry Pi-class inference device that must run the model in < 200 ms per image. Apply the Decision Checklist to this problem.

(b) Propose a specific model strategy addressing: training approach (from scratch vs. transfer learning), expected parameter count, regularisation, and how the 200 ms constraint affects architecture choice.

(c) Design an evaluation strategy for the 300/150 class-imbalanced dataset. Explain why accuracy alone is inappropriate and propose two better metrics for this clinical-agricultural context.

(d) A team member proposes gradient boosting on hand-extracted colour histogram features. Evaluate this seriously: under what conditions would it outperform a neural network, and what would you need to demonstrate before recommending the deep learning approach?

Further Reading

Textbooks

- Goodfellow, Bengio, Courville — Deep Learning, MIT Press (2016). Chapters 6–8: feedforward networks, regularisation, optimisation in full mathematical detail.
- Géron — Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow, O'Reilly. Chapters 10–11 provide direct implementation of Chapter 1 concepts.
- Zhang, Lipton, Li, Smola — Dive into Deep Learning (d2l.ai). Chapters 3–4 (linear networks and MLPs) are closely aligned with this chapter.

Foundational Papers

- Rumelhart, Hinton, Williams (1986) — Learning representations by back-propagating errors. Nature.
- Glorot and Bengio (2010) — Understanding the difficulty of training deep feedforward neural networks. AISTATS.
- Kingma and Ba (2014) — Adam: A Method for Stochastic Optimization. ICLR.
- Srivastava et al. (2014) — Dropout: A Simple Way to Prevent Neural Networks from Overfitting. JMLR.
- Ioffe and Szegedy (2015) — Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. ICML.

On the Tabular Data Question

- Grinsztajn et al. (2022) — Why tree-based models still outperform deep learning on tabular data. NeurIPS.

CHAPTER 2

Learning from Images

CNNs and Visual Understanding

2.0 The Inductive Bias for Image Data

2.0.1 The AgriVision Photograph Problem

> ✦ THE AGRIVISION CONTEXT
>
> The AgriVision team built a 150,528→256→128→4 MLP on 6,000 labelled leaf photographs. Validation accuracy after one week of tuning: 58.7—barely above the 25% random baseline for four classes. The failure is not in the training procedure. It is in the architecture.
>
> This section establishes why—and in doing so, motivates every design choice in the CNN that follows.

The 8,000 AgriVision photographs vary enormously: some in bright midday sun, others in barn shade; some close-up single leaves, others wide-angle plant clusters. They look exactly like the data real agricultural AI systems must handle. The MLP was trained on 6,000 and validated on 2,000. Despite extensive tuning—learning rate sweeps, dropout variations, layer width increases—the ceiling was 58.7%. Section 2.2 quantifies precisely why. First, we establish what the data requires.

2.0.2 Spatial Locality: Why Neighbouring Pixels Are Related

Consider a single pixel at position (row 112, column 80) in one of AgriVision's 224×224 photographs. Its red channel value is 213—a bright, warm intensity. By itself, this tells us almost nothing about disease status. The same value appears in bright sunlight, in white powdery mildew coating, in orange soil, and in specular highlights.

Now consider that same pixel together with its 3×3 neighbourhood. The patch shows a bright region (high red, diminished green) surrounded by darker, strongly green pixels, with a characteristic colour transition along the diagonal boundary. This pattern—a bright patch with a specific colour gradient at its edge—is characteristic of early-stage powdery mildew. The information was not in the pixel. It was in the spatial relationship between the pixel and its neighbours.

> ◆ KEY CONCEPT
>
> **Spatial locality**
>
> The diagnostic information in an image is encoded in spatial relationships between groups of neighbouring pixels, not in individual pixel values considered in isolation. Pixels close to each other in the image are more semantically related than pixels far apart.

An MLP that flattens the image into a vector discards these spatial relationships before any learning begins. This is not recoverable—the spatial information is gone.

Spatial locality operates at every scale: at 2×5 pixels, edges (boundaries of lesions); at 10–30 pixels, textures (powdery stippling, angular spotting); at 50–100 pixels, structural features (lesion spread across leaf surface, vein discolouration). A model that exploits spatial locality discovers this hierarchy; a model that ignores it cannot represent it.

2.0.3 Translation Invariance: The Pattern Should Work Anywhere

The same disease lesion appears in thousands of different positions across AgriVision's photographs. Farmer 47 in Dharwad photographed a leaf with the lesion in the upper-left; Farmer 83 in Bagalkot photographed the same disease with the lesion at the bottom-right; Farmer 112 took a close-up with the lesion near centre. An ideal classifier produces the same label regardless of position.

The MLP cannot do this. Pixel 1,024 in the flattened vector always corresponds to position (row 4, col 85). A weight connecting it to a hidden neuron learns to respond to colour transitions at that specific location—and has no relationship to pixel 8,960 at (row 40, col 24), where the same pattern might appear in a different photograph. With 6,000 training images distributed across all positions, disease types, and lighting conditions, each position receives only a handful of training examples. No reliable detector can be learned from so few examples per position.

DEFINITION

Translation equivariance vs. translation invariance

Translation equivariance: if an input pattern shifts spatially, the model's internal representation shifts by the same amount. Convolution (Section 2.3) is equivariant.

Translation invariance: the output does not change when the pattern shifts. Pooling (Section 2.4) introduces invariance.

A complete CNN uses equivariant convolution to build representations and invariant pooling to make final decisions robust to exact position.

2.0.4 How These Properties Motivate the CNN

The two properties—spatial locality and translation equivariance—map directly onto the CNN's two defining design choices.

Local receptive fields (responding to spatial locality). Each neuron connects to a small spatial neighbourhood (3×3, 5×5, or 7×7 pixels), not to all 150,528 pixels. Processing is defined over the 2D spatial structure of the image, not over a 1D vector that discards it.

Weight sharing (responding to translation equivariance). The same learned weights—the same filter—are applied at every position. A filter detecting the mildew colour gradient detects it in the top-left, centre, and bottom-right simultaneously, learned once and applied everywhere.

◆ KEY CONCEPT

Inductive bias for image data

Images have two structural properties tabular data lacks:

Spatial locality: information is in spatial relationships between neighbouring pixels, not in individual values.

Translation equivariance: the same visual pattern should be recognisable regardless of where it appears in the image.

The CNN encodes both: local receptive fields exploit spatial locality; weight sharing encodes translation equivariance. Every component of the CNN follows from these two requirements.

? FOUR QUESTIONS TO CARRY THROUGH THIS BOOK

Question 1: What problem does this architecture solve?

The failure of fully connected networks on images: parameter explosion (38.5M weights in the first layer alone), loss of spatial structure, and positional sensitivity.

Question 2: What inductive bias does it encode?

Spatial locality via local receptive fields; translation equivariance via weight sharing.

Question 3: When does this architecture fail?

When patterns are not spatially local (graphs, sequences); under severe domain shift; when adversarial perturbations exploit sensitivity to imperceptible pixel changes.

Question 4: When would you choose something else?

Tabular data (Chapter 1); time-series sequences (Chapter 3); tasks requiring global spatial reasoning where a Vision Transformer may be preferable (Section 4.8).

KEY TERMS — THIS SECTION

Term	Definition
Spatial locality	Property of images: pixels close in space are more semantically related than distant pixels
Translation equivariance	Shift in input pattern produces equal shift in representation; provided by convolution
Translation invariance	Output unchanged when pattern shifts; provided by pooling operations
Weight sharing	Same learned filter weights applied at every spatial position; key to translation equivariance
Local receptive field	Small spatial neighbourhood a single convolutional neuron receives as input

✎ CHECK YOUR UNDERSTANDING

1. The first weight matrix of AgriVision's MLP has dimensions 150,528×256. A CNN's first layer uses a single filter with 3×3×3 = 27 parameters. Explain why this 5,575× reduction is possible without losing the ability to detect disease patterns. What assumption makes it valid?

2. A colleague argues: 'Translation invariance is unimportant for AgriVision because farmers are trained to photograph leaves centred in the frame.' Evaluate this argument. What would you need to verify empirically, and what would you expect to find?

3. Explain the difference between translation equivariance and translation invariance using a concrete leaf disease example. Which does convolution provide, and which does pooling provide? Why does a CNN need both?

4. An agricultural researcher trains a CNN under perfectly controlled laboratory conditions (94% accuracy). In field deployment: 61% accuracy. Name the failure mode from Section 1.8. What specific property of the real-world data was absent from training?

5. Apply the four recurring questions to a 3D CNN using 3×3×3 volumetric filters instead of 2D spatial filters. For what type of data is this appropriate? What inductive biases does it encode that a standard 2D CNN does not?

6. [Challenge] An MLP achieves 58.7% and a CNN achieves 87.2% on the same AgriVision test set. Design an ablation experiment that isolates whether the improvement comes from weight sharing, local receptive fields, or both. Describe the architectures, measurements, and how results would attribute the improvement.

2.1 Image Data as a Deep Learning Context

2.1.1 The Image as a Tensor: Height, Width, Channels

An image is a three-dimensional array of numbers with dimensions height, width, and channels. A 224×224 colour photograph has dimensions 224×224×3, containing 224×224×3 = 150,528 numbers. Each number is typically an unsigned 8-bit integer between 0 and 255 representing the intensity of one colour channel at one spatial position.

The three channels are Red, Green, Blue (RGB). PyTorch convention: (C, H, W) — channels first. A batch of N images: (N, C, H, W), shape (N, 3, 224, 224). TensorFlow uses (H, W, C). The distinction matters: a tensor of shape (3, 224, 224) is one image in PyTorch but misinterpreted as a batch of 3 if passed to a TensorFlow model expecting (H, W, C).

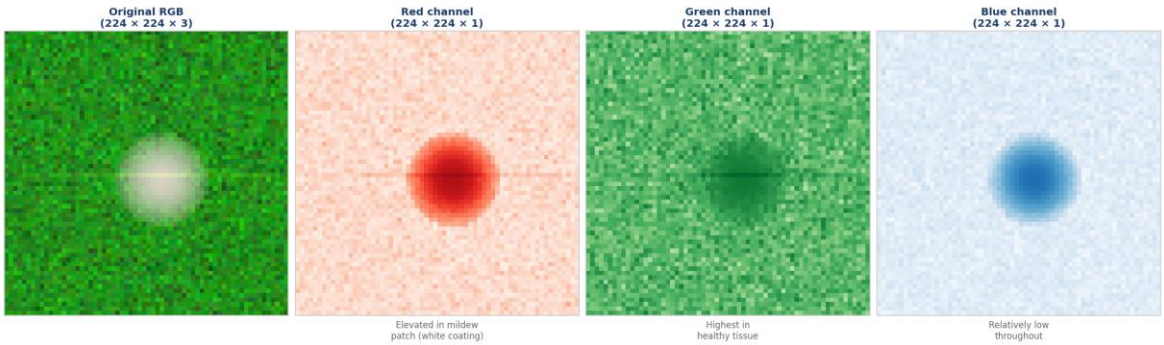


Figure 2.1 *A synthetic AgriVision leaf photograph decomposed into its three RGB channels. The green channel is brightest across healthy leaf tissue (chlorophyll reflects green light). The red channel is elevated in the mildew-infected region where the white coating scatters red light proportionally. Each channel is a 224×224 array of integers from 0 to 255.*

Normalisation before training. Raw pixel values (0–255) are never fed directly to a neural network. Three conventions: (1) Scale to [0,1] by dividing by 255; (2) Standardise per channel using training-set mean and std; (3) ImageNet normalisation for pretrained models: mean = [0.485, 0.456, 0.406], std = [0.229, 0.224, 0.225]. These exact values must be used when fine-tuning any ImageNet-pretrained model—using different statistics is one of the most common silent bugs in computer vision practice, degrading performance by 5–10 percentage points.

2.1.2 RGB Colour Spaces and What They Represent

For agricultural disease detection, the three channels carry distinctly different diagnostic information. The green channel is typically strongest in healthy leaf tissue—chlorophyll absorbs red and blue light and reflects green (values 120–220 across healthy tomato leaves). The red channel rises in regions of tissue stress and necrosis (late blight, Alternaria). The blue channel plays a distinctive role in identifying powdery mildew: the white waxy fungal coating scatters light uniformly across wavelengths, raising blue-channel values in infected regions. A greyscale model that averages all three channels partially loses this discriminative signal.

⚠ **IMPORTANT NOTE**

Why colour matters for disease detection—and why greyscale is insufficient.

Powdery mildew produces a white, waxy coating that raises all three RGB channels in infected tissue, but raises the blue channel more proportionally relative to surrounding healthy green tissue. This specific colour signature is lost when converting to greyscale.

Late blight produces dark reddish-brown lesions where the red channel is elevated relative to the green channel—a ratio that greyscale conversion collapses into a single intensity value indistinguishable from shadows or veins of similar brightness.

2.1.3 Resolution, Scale, and the Information Density Problem

AgriVision's original photographs range from 3,264×2,448 pixels to 8,000×6,000 pixels (80 to 600 times more pixels than a 224×224 input). Before training, every image must be resized. This resizing is not lossless: fine-scale disease texture (individual spore clusters 3–5 pixels across at full resolution) merges into a uniform bright region at 224×224; wide-angle context (which leaves are affected, how widely spread) is lost when cropping to a single leaf.

Standard preprocessing pipeline: (1) Resize shorter edge to 256 (preserving aspect ratio). (2) Centre crop to 224×224, or random crop during training. (3) Optionally flip horizontally at random during training. (4) Normalise pixel values. This pipeline is implemented in torchvision.transforms and matches the preprocessing used to train ImageNet models—a key reason fine-tuning those models works well.

2.1.4 Variation in Real-World Image Data: Lighting, Angle, Background

AgriVision's 8,000 photographs were taken by 150 farmers using 150 different phones in 150 different environments over three growing seasons. The variation is not a problem to overcome —it is the nature of real-world agricultural computer vision.

- Illumination variation: the same healthy green leaf photographed under direct midday sun, overcast shade, fluorescent light, and golden-hour sun produces four different pixel representations. A model that identifies disease by memorising absolute pixel values fails when lighting changes.
- Angular and distance variation: a leaf from directly above appears as a flat oval; from 45° it appears foreshortened. Photographs from 20 cm resolve individual lesions; from 1.5 m the lesion may be only a few pixels—insufficient for textural discrimination.
- Background complexity: a leaf against a plain sky has a simple background; attached to the plant it has a complex background of other leaves, stems, and soil. The model must attend to the relevant leaf while ignoring the background.
- Scale variation: the same lesion may occupy 40% of the frame in a close-up or less than 1% in a wide-angle shot. Standard preprocessing partially normalises scale but does not fully resolve it.

⚠ **IMPORTANT NOTE**

What data augmentation does and does not fix.

Data augmentation (Section 1.7.5) addresses illumination variation, angular variation, and scale variation: randomly apply brightness adjustments, rotations, crops, and colour jitter so the model encounters the disease pattern under diverse conditions.

It does not fix domain shift—the difference between the statistical distribution of training images and deployment images. If all 8,000 training photographs came from Dharwad district and the model is deployed in Tamil Nadu where prevalent varieties and typical lighting differ, augmentation on the Dharwad dataset does not close this gap. Section 2.9 studies this failure mode in detail.

KEY TERMS — THIS SECTION

Term	Definition
Image tensor	Three-dimensional array (C, H, W) in PyTorch; C = channels, H = height, W = width
Channel	One colour or feature plane; in RGB images: red, green, blue
RGB	Additive colour model; each channel records intensity of one primary colour (0–255)
ImageNet normalisation	Mean [0.485,0.456,0.406], std [0.229,0.224,0.225]; required for pretrained models
Resolution	Number of pixels in H×W; higher resolution captures more detail but costs more compute
Illumination variation	Changes in pixel values from different lighting; major source of distribution shift
Domain shift	Mismatch between training and deployment distribution; augmentation reduces but does not eliminate

✎ CHECK YOUR UNDERSTANDING

1. An AgriVision photograph is captured at 4,032×3,024 pixels. (a) How many pixels are in the original? In the 224×224 resized input? What fraction is retained? (b) After resizing the shorter edge to 256 (preserving aspect ratio), what are the intermediate dimensions? (c) After a 224×224 centre crop, how many pixels are discarded?

2. A student training a CNN from scratch on AgriVision applies ImageNet normalisation (mean=[0.485,0.456,0.406], std=[0.229,0.224,0.225]) to all images. Is this appropriate? What should they use instead, and from which data split should the statistics be computed?

3. A healthy tomato leaf has green channel values 140–210. After fungal infection the same region has values 55–90. (a) What biological process does this indicate? (b) Why would a greyscale model struggle to distinguish this from a shadow? (c) Which channel differentiates chlorosis from powdery mildew, and why?

4. An extension officer submits three photographs of the same mildew-infected leaf: (a) from 20 cm in direct sunlight, (b) from 1 m in shade, (c) from 30 cm at 45° angle. For each, describe how

the number of pixels representing the lesion, the absolute RGB values, and the apparent leaf shape will differ. Which variation source does each illustrate?

5. [Challenge] Section 2.1.3 states that increasing resolution from 224×224 to 448×448 'requires quadratically more memory.' (a) How many pixels are in each? (b) In a conv layer with stride 1, how does computation scale with resolution? (c) Explain why doubling resolution more than doubles memory requirement, necessitating a halved batch size, and what this implies for gradient estimation quality.

2.2 Why Fully Connected Networks Struggle with Images

2.2.1 The Parameter Explosion: A Worked Calculation

AgriVision's MLP architecture: 150,528 → 256 → 128 → 4.

> ✎ **WORKED EXAMPLE**
>
> **Counting parameters in AgriVision's MLP**
>
> Layer 1: 150,528 × 256 + 256 = 38,535,168 + 256 = 38,535,424 parameters
>
> Layer 2: 256 × 128 + 128 = 32,768 + 128 = 32,896 parameters
>
> Layer 3: 128 × 4 + 4 = 512 + 4 = 516 parameters
>
> Total trainable parameters: 38,568,836 ≈ 38.6 million
>
> Layer 1 alone accounts for 99.9% of the total.
>
> Parameters per training image: 38,568,836 / 6,000 ≈ 6,428
>
> Compare to the Chapter 1 yield network: 273 parameters for 750 examples → 1 per 2.75 examples. The MLP has 6,428 parameters per image—a ratio inverted by a factor of 18,000.

With 6,428 parameters per training image, the model has far more capacity to memorise the training set than to learn general disease patterns. Training accuracy rises toward 100%; validation accuracy stagnates. But overparameterisation is only the first problem.

2.2.2 Loss of Spatial Structure in the Flattened Vector

When an image is flattened, the spatial arrangement of pixels is destroyed before any learning begins. In a 5×5 grid of pixels flattened to a 25-element vector: pixel A (position (0,0)) and its below-neighbour F (position (1,0)) are 5 positions apart in the vector, with B, C, D, E between them—yet in the image, F is directly below A. Pixels that are spatially distant (E and F, at opposite ends of adjacent rows) become adjacent in the flat vector. The MLP receives 150,528 values as exchangeable, independent inputs with no spatial structure.

A disease lesion is a compact spatial region with specific internal texture and a characteristic boundary gradient. Flattened, the lesion's pixels are scattered across the 150,528-element vector. The network cannot detect the lesion because it cannot represent local spatial structure. It can only compute functions of the form Output = $f(w_0x_0 + w_1x_1 + ... + w_{150527}x_{150527} + b)$, where every pixel is weighted independently.

2.2.3 Sensitivity to Translation: A Demonstration

Because each neuron connects to specific positions in the flattened vector, any shift in the disease pattern's position appears to the network as a completely different input. A 224×224 image has 50,176 possible spatial positions where a lesion could be centred. Each position requires a separate learned detector. With 6,000 training images distributed across all disease types, positions, lighting

conditions, and camera distances, each position receives only a handful of training examples. No reliable detector can be learned from so few examples per position.

2.2.4 What We Need: Local, Shared, Hierarchical Processing

The three failures share a root cause and point directly toward the same solution.

- Parameter explosion → Local processing: connect each neuron to only a small neighbourhood (3×3 = 9 pixels rather than all 150,528).
- Loss of spatial structure → Structured connectivity: process the 2D spatial structure of the image, not a 1D vector that discards it.
- Translation sensitivity → Weight sharing: use the same detector weights at every position. One learned filter, applied everywhere, replaces 50,176 independent position-specific detectors.

DEFINITION

What a convolutional layer does

A convolutional layer applies a set of learned filters locally and at every position across the input. Each filter: (1) has small spatial extent (3×3, 5×5, or 7×7 pixels)—satisfying locality; (2) extends across the full channel depth of the input; (3) is applied at every spatial position with the same weights—satisfying translation equivariance; (4) produces one number per position—encoding how strongly the filter's pattern is detected there.

The output of a convolutional layer is a set of feature maps—one per filter—representing where and how strongly each learned pattern is detected in the input.

KEY TERMS — THIS SECTION

Term	Definition
Parameter explosion	Combinatorial parameter growth when connecting all pixels to all neurons; scales as H×W×C×neurons
Overparameterisation	Far more trainable parameters than training examples; leads to memorisation rather than generalisation
Spatial adjacency	Property of two pixels being close in image position; destroyed by flattening to a vector
Position-specific detector	MLP neuron that detects a pattern at only one set of pixel positions; requires duplication per position
Local processing	Connecting each neuron to a small spatial neighbourhood rather than all positions
Weight sharing	Using the same filter weights at every spatial position; detects the pattern wherever it appears

✎ **CHECK YOUR UNDERSTANDING**

1. AgriVision's team experiments with 150,528→1,024→512→256→4. (a) Compute total trainable parameters. (b) Compute the parameter-to-training-example ratio (6,000 images). (c) Compare to the 7→64→32→1 yield network. What does this tell you about whether adding more layers can resolve the MLP's fundamental problem?

2. In a flattened 5×5 patch, pixel E (index 4) and pixel F (index 5) are adjacent in the vector but at opposite ends of adjacent rows in the image. Explain why weights w_4 and w_5 cannot represent the pattern 'the right edge of a row is brighter than the left edge of the next row,' even though both weights are present.

3. An MLP with 512 hidden neurons trained on 10,000 AgriVision photographs achieves 96% training accuracy and 63% validation accuracy. Suppose the same architecture is trained on 500,000 photographs. Would validation accuracy approach 96%? Explain by reference to Sections 2.2.2 and 2.2.3.

4. Section 2.2.3 states an image with 50,176 possible lesion positions requires 50,176 independent position-specific detectors in an MLP. With weight sharing, how many detector configurations are needed? How does this change the parameter-per-training-example ratio?

5. [Challenge] Design the smallest fully connected network that could detect a 5×5 lesion pattern at any of the $(32-5+1)^2 = 784$ valid positions in a 32×32 greyscale image. Estimate the parameters required. Now design a single convolutional filter with the same capability. Compare parameter counts. What does this reveal about weight sharing and parameter efficiency?

2.3 The Convolution Operation

2.3.1 The Sliding Window Intuition

Imagine an agricultural pathologist examining a photograph at 100% magnification, scanning a small window systematically across the image. Two things are true about how they work: they look locally (only the current window, not all pixels simultaneously), and their recognition criterion does not change with position (the same 'does this show powdery mildew?' question at every location). This is convolution: a fixed pattern-detector applied at every position in the image.

The CNN has many pattern-detectors operating in parallel. One filter learns oriented edges; another learns diagonal colour gradients; a third learns the hue and texture of powdery mildew at a few pixels. Each generates its own feature map. Deeper layers receive these maps as input and apply further filters to detect increasingly complex patterns.

2.3.2 Filters, Kernels, and Feature Maps

A filter (kernel) is a small 2D array of learnable weights that slides across the input. For a single-channel input, a 3×3 filter has nine learnable weights. At each position, the operation computes:

$$z = \Sigma_m \Sigma_n F[m,n] \cdot Patch[m,n] + b$$

This single scalar z is placed in the output feature map at position (i,j). The filter slides to the next position and the computation repeats, producing one value per valid position. The complete collection of output values is the feature map: a 2D array encoding how strongly the filter's pattern is detected at every location in the input.

> ✎ **WORKED EXAMPLE**
>
> **Computing a feature map value at position (0,0)**
>
> Filter F: [[0.1, −0.3, 0.2], [−0.2, 0.8, −0.4], [0.1, −0.1, 0.3]]
>
> Patch at (0,0): [[180, 192, 178], [175, 188, 201], [162, 170, 183]]
>
> z = (0.1)(180) + (−0.3)(192) + (0.2)(178) + (−0.2)(175) + (0.8)(188) + (−0.4)(201) + (0.1)(162) + (−0.1)(170) + (0.3)(183)
>
> z = 18.0 − 57.6 + 35.6 − 35.0 + 150.4 − 80.4 + 16.2 − 17.0 + 54.9 = 85.1
>
> This value is placed at position (0,0) of the feature map. Large positive output: patch closely matches the filter's pattern. Near-zero: uncorrelated. After ReLU, only positive detections remain.

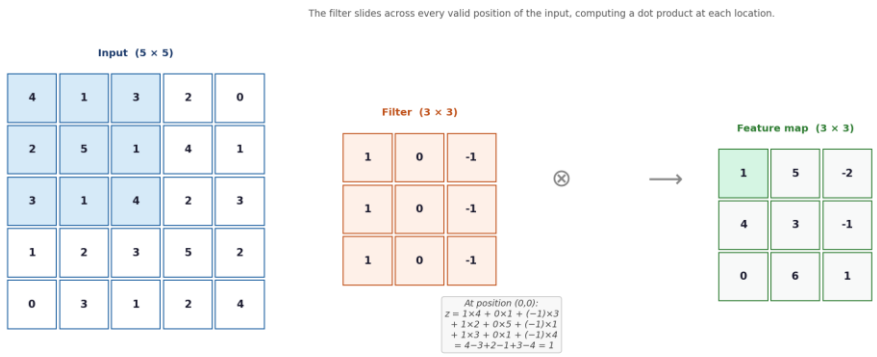


Figure 2.2 *The convolution operation: a 5×5 single-channel input (left), a 3×3 filter (centre), and the resulting 3×3 feature map (right). The highlighted 3×3 region of the input is multiplied element-wise with the filter and summed to produce the feature map value at (0,0). The filter then slides one step right to compute the next value.*

2.3.3 Stride and Padding: Controlling Output Dimensions

Stride. How many pixels the filter moves between positions. Stride 1: move one pixel at a time. Stride 2: skip every other position. Stride 2 halves the spatial dimensions (approximately) while doubling the effective pixel coverage per output position.

Padding. Without padding, a 3×3 filter on a 7×7 input produces a 5×5 output—the output shrinks by f−1 = 2 in each dimension. This accumulates: after 10 layers, a 14×14 input becomes 4×4. Zero-padding (p=1 around a 7×7 input creates 9×9) enables same padding: output has the same H×W as input for stride 1.

The output dimension formula (applied independently to height and width):

$$output_size = \lfloor (input_size - filter_size + 2p) / stride \rfloor + 1$$

✎ WORKED EXAMPLE

Output dimensions at AgriVision's first two convolutional layers

Layer 1: Input 224×224×3, filter f=3, K=32, stride=1, padding=1 (same padding)

H_out = ⌊(224−3+2)/1⌋+1 = 224 → Output: 224×224×32

Parameters: 3×3×3×32 + 32 = 864 + 32 = 896

Compare to MLP Layer 1: 38,535,424 parameters. CNN uses 43,000× fewer.

Layer 2: Input 224×224×32, filter f=3, K=64, stride=2, padding=1

H_out = ⌊(224−3+2)/2⌋+1 = 112 → Output: 112×112×64

Parameters: 3×3×32×64 + 64 = 18,432 + 64 = 18,496

2.3.4 Multiple Filters and Channel Depth

With K filters, a convolutional layer produces K feature maps simultaneously—one per filter. For a multi-channel input (C input channels), each filter is a 3D tensor of shape f×f×C. At each position the filter computes a sum across all input channels:

$$z = \Sigma_m \Sigma_n \Sigma c \; F[m,n,c] \cdot I[i+m, j+n, c] \; + \; b$$

Total parameters in one convolutional layer:

$$W = f^2 \times C_in \times C_out + C_out$$

◆ KEY CONCEPT

Why weight sharing produces such a dramatic parameter reduction

MLP first layer: each of 256 neurons independently learns 150,528 weights—one per pixel. 38.5 million parameters, none sharing knowledge about spatial structure.

CNN first layer: 32 filters each have 3×3×3 = 27 weights, shared across all 50,176 positions. The same weights detect the mildew gradient in the top-left and the bottom-right simultaneously, learned once and applied everywhere.

The MLP's 38.5M first-layer parameters are, in effect, 50,176 redundant copies of the same small number of useful detectors—one per spatial position—none learned reliably because no position receives enough training examples. Weight sharing collapses these copies into one well-learned detector applied everywhere.

2.3.5 The Receptive Field: What Each Neuron Sees

The receptive field of a neuron is the region of the original input that influences its output. For a first-layer neuron with a 3×3 filter: receptive field = 3×3 pixels. For a second-layer neuron: receptive field = 5×5 pixels (each of its 9 input positions covers a 3×3 region). After L layers of 3×3 filters with stride 1: receptive field = (2L+1)×(2L+1).

Achieving a 224×224 receptive field with 3×3 filters would require ~112 layers—impractically deep. Two mechanisms accelerate growth: larger initial filters (7×7 covers the same as three 3×3 layers, though with more parameters); and strided convolutions or pooling, which double the effective pixel coverage per position in subsequent layers.

2.3.6 Convolution as Learned Feature Detection

Filter weights are learned by gradient descent, not hand-engineered. Researchers have studied what trained filters discover by finding inputs that maximally activate each filter. The results are consistent: first-layer filters learn elementary visual primitives (oriented edges at various angles, colour-opponent signals, frequency-selective patterns)—closely resembling the orientation-selective cells discovered in mammalian primary visual cortex by Hubel and Wiesel. Second- and third-layer filters combine these into textures. Deep-layer filters in disease-trained networks learn patterns an expert agronomist would recognise: the stippled white-on-green texture of early mildew, the concentric rings of Alternaria leaf spot, the darkening of early Late Blight.

2.3.7 Output Dimension Formula and Worked Examples

For a full output tensor: Input H×W×C_in, K filters of size f×f×C_in, padding p, stride s:

$$Output: H_out \times W_out \times K \quad where\ H_out = \lfloor (H - f + 2p)/s \rfloor + 1$$

Configuration	Input	f, K, s, p	Output	Parameters
Same padding, stride 1	112×112×64	f=3, K=128, s=1, p=1	112×112×128	73,856

Stride-2 downsampling	112×112×128	f=3, K=256, s=2, p=1	56×56×256	295,168
Valid padding	56×56×256	f=3, K=256, s=1, p=0	54×54×256	590,080
1×1 bottleneck	56×56×256	f=1, K=64, s=1, p=0	56×56×64	16,448

Table 2.1 *Output dimension worked examples. Note the 1×1 convolution reduces 256 channels to 64 with no spatial cost and only 16,448 parameters.*

The 1×1 convolution is a channel bottleneck: before applying an expensive 5×5 filter (5×5×256 = 6,400 weights per filter), reduce to 64 channels first (16,384 total across 64 filters), then apply the 5×5 to the 64-channel intermediate. This is the foundation of the Inception module (Section 2.6.4).

> ⚠ **IMPORTANT NOTE**
>
> **The difference between parameters and computation**
>
> Parameters = $f^2 \times C_in \times C_out + C_out$ (determines model memory)
>
> MACs = $f^2 \times C_in \times C_out \times H_out \times W_out$ (determines inference speed)
>
> Layer 1 of the AgriVision CNN has 896 parameters but 43 million operations (the same weights applied at all 50,176 spatial positions). Early layers dominate computation; deep layers dominate parameter count. This distinction matters for deployment: edge devices are constrained by computation, servers serving many users may be constrained by memory.

KEY TERMS — THIS SECTION

Term	Definition
Filter / kernel	Small learnable weight tensor (f×f×C_in) that slides across input; one filter → one feature map
Feature map	2D output of applying one filter at every position; encodes where the filter's pattern is detected
Stride (s)	Pixels the filter moves between positions; s>1 reduces output spatial dimensions
Same padding	Padding (p=(f−1)/2) so output has same H×W as input for stride 1
Valid padding	No padding; output is smaller than input by f−1 in each dimension
Output dimension formula	H_out = ⌊(H−f+2p)/s⌋ + 1; applied independently to H and W
Channel depth C_out	Number of filters in a conv layer; equals the number of feature maps in the output
Receptive field	Region of original input influencing a given neuron; grows with depth
1×1 convolution	Filter with f=1; mixes channels without spatial filtering; used for channel dimensionality reduction
MACs	Multiply-accumulate operations; scales with both parameters and output spatial dimensions

✎ **CHECK YOUR UNDERSTANDING**

1. Apply the output dimension formula. Compute output height and parameter count for: (a) Input 224×224×3, f=7, K=64, stride=2, p=3; (b) Input 56×56×256, f=1, K=64, stride=1, p=0; (c) Input 28×28×512, f=3, K=512, stride=1, p=1; (d) Input 7×7×512, f=7, K=4096, stride=1, p=0.

2. AgriVision's CNN applies three successive 3×3 same-padding stride-1 convolutions to 224×224×3 input with 32, 64, and 128 output channels, then a stride-2 convolution. (a) Compute output tensor shape after each of the four layers. (b) Compute parameter count per layer. (c) At which layer does cumulative parameter count first exceed 100,000?

3. Compare a single 5×5 layer vs two stacked 3×3 layers, both with 128 channels. (a) Parameters for the 5×5 layer. (b) Parameters for two 3×3 layers. (c) The stack has fewer parameters but more representational power—why? What does the extra non-linearity allow?

4. A filter with all weights equal to $1/(f^2)$ is applied to a greyscale image. (a) What operation does this correspond to? (b) What does the feature map show in a uniformly bright region? In a sharp-edge region? (c) After ReLU, what patterns does this map highlight?

5. An engineer inserts a 1×1 conv (32 output channels) between a 256-channel layer and a 3×3 conv producing 256 outputs. (a) Trace tensor shapes. (b) Total parameters for 1×1 + 3×3 after bottleneck. (c) Parameters for 3×3 alone on 256-channel input. (d) Reduction factor. Does output tensor shape change?

6. [Challenge] The output formula uses a floor function. (a) Show that without floor, a 7×7 input with f=3, stride=2, p=0 gives non-integer output 3.5—why are fractional positions invalid? (b) How many edge pixels are 'unused' in this case? (c) PyTorch nn.Conv2d has ceil_mode parameter. Give an example where it changes output dimension and describe the geometric consequence.

2.4 Pooling and Spatial Downsampling

2.4.1 Max Pooling: Selecting the Strongest Signal

After a convolutional layer produces feature maps, two problems remain: (1) with same padding and stride 1, spatial dimensions stay constant—computing 128 feature maps at 224×224 is expensive; (2) a convolutional layer is equivariant, not invariant—for final classification we do not care where exactly the pattern is, only whether it is present. Pooling addresses both.

Max pooling applies a sliding window of size p×p to each feature map independently. At each position it takes the maximum value. With pool size p=2 and stride 2 (standard), non-overlapping 2×2 blocks are processed, halving both height and width. No weights learned; no parameters. Output dimension: H_out = ⌊(H−2)/2⌋+1 = H/2 for even H.

> ✎ WORKED EXAMPLE
>
> **Max pooling on a disease feature map (4×4 → 2×2)**
>
> A filter has responded to the mildew gradient. Its 4×4 response values:
>
> [[3.2, 0.8, 1.5, 2.1], → Top-left block {3.2, 0.8, 0.1, 4.7} → max = 4.7
>
> [0.1, 4.7, 0.9, 3.8], Top-right block {1.5, 2.1, 0.9, 3.8} → max = 3.8
>
> [2.4, 0.3, 5.1, 1.2], Bottom-left block {2.4, 0.3, 1.1, 3.9} → max = 3.9
>
> [1.1, 3.9, 0.7, 4.3]] Bottom-right block {5.1, 1.2, 0.7, 4.3} → max = 5.1
>
> Output: [[4.7, 3.8], [3.9, 5.1]]
>
> The top-left cell (4.7) records the strongest mildew detection in that 2×2 region. It does not record exactly which pixel produced the 4.7—this loss of exact position is intentional. For classification, 'is the mildew pattern here?' matters; 'at exactly which pixel?' does not.

Why max rather than average: the maximum represents the most confident detection anywhere in the region. For disease classification, 'was the mildew gradient detected at all in this 2×2 neighbourhood?' is the right question. An average is appropriate when every position contributes equally—which applies to regression or segmentation tasks, not feature detection.

2.4.2 Average Pooling

Average pooling computes the mean within each pool window. For the same 4×4 input, the top-left block averages to (3.2+0.8+0.1+4.7)/4 = 2.20 instead of the max 4.7. Peak responses are diluted by averaging with lower-response neighbours. For disease detection, max pooling is substantially preferable: a single strong mildew response (5.1) should drive the classifier toward 'infected,' regardless of adjacent positions with low response.

Average pooling is preferred when the task requires reasoning about overall activation strength across a region. Its most important modern application is Global Average Pooling.

2.4.3 Spatial Invariance Through Downsampling

The deepest purpose of pooling is introducing local translation invariance. A mildew lesion at pixel (14,22) produces a strong feature map activation at (14,22). After 2×2 max pooling, this is captured in

the pooled output at (7,11). The same lesion shifted one pixel right—now at (14,23)—still falls within the same 2×2 pool window and produces the same pooled output. The network has become invariant to 1-pixel shifts. This invariance accumulates: after five 2×2 max pool operations, the network is invariant to shifts of up to 32 pixels.

Global Average Pooling (GAP). The logical endpoint: compute the spatial average of each channel across the entire feature map. For a 7×7×256 feature map, GAP produces a 256-dimensional vector with zero learned parameters. A subsequent FC layer (256→4 for our four disease classes) needs only 256×4+4 = 1,028 parameters. Compare to flatten→FC from the same feature map: 7×7×256 = 12,544 inputs require 12,544×4+4 = 50,180 parameters—fifty times more with no accuracy benefit. GAP also makes the network independent of input image size: any spatial dimension is averaged to a fixed-length output vector.

2.4.4 The Detail–Abstraction Trade-off

Pooling is lossy. When 2×2 max pooling retains only the maximum of four values, the others are permanently discarded. For disease classification ('yes/no?'), this is acceptable. For disease quantification ('what fraction of the leaf?'), fine spatial detail matters and aggressive pooling is inappropriate. Segmentation architectures (Section 2.7.3) address this by retaining fine spatial information through skip connections that bypass pooling layers.

> ⚠ **IMPORTANT NOTE**
>
> When pooling hurts: segmentation and detection tasks.
>
> Aggressive spatial pooling is optimised for global classification. For tasks requiring spatial output—detecting where the lesion is (object detection) or labelling every pixel (semantic segmentation)—standard max pooling causes problems. Five 2×2 pool layers reduce a 224×224 image to a 7×7 feature map. Predicting a 224×224 pixel output from a 7×7 representation requires recovering detail that was discarded.
>
> Architectures for these tasks use fewer pooling layers, smaller strides, or explicitly restore spatial resolution through skip connections and learned upsampling—introduced in Section 2.7.

KEY TERMS — THIS SECTION

Term	Definition
Pooling	Parameterless spatial downsampling reducing H×W of feature maps while preserving channel depth
Max pooling	Returns maximum value in each pool window; emphasises strongest feature detection
Average pooling	Returns mean value; appropriate for spatial averages, not peak detection
Local translation invariance	Small spatial shifts (smaller than pool window) do not change pooled output
Global Average Pooling (GAP)	Averages each channel across all spatial positions; reduces H×W×C to C-dimensional vector

Strided convolution	Conv layer with stride>1 achieving spatial downsampling while learning a feature transformation

✎ **CHECK YOUR UNDERSTANDING**

1. Apply 2×2 max pooling (stride 2) to the following 6×6 feature map. Then apply average pooling. In which cells does max differ most from average, and what does this reveal about the feature map?
[[0.2,3.8,0.1,0.0,2.7,1.4],[4.9,0.3,0.0,3.2,0.5,0.8],[1.1,0.0,2.4,0.9,0.0,4.6],[0.4,3.6,0.7,0.0,3.8,0.2],[0.0,0.8,0.0,2.1,1.3,0.0],[4.1,0.0,3.5,0.0,0.0,4.8]]

2. A feature map of 28×28×128 is fed to GAP then to a 4-class FC layer. (a) Shape of GAP output? (b) Parameters in the FC layer? (c) Compare to flatten→FC from the same feature map. By what factor does GAP reduce parameters?

3. After five 2×2 max pool layers (stride 2), a feature map originally 224×224 is 7×7. A 3×3 conv filter is applied to this 7×7 map. (a) What is the effective receptive field in the original 224×224 input? (b) Show the step-by-step calculation.

4. GAP makes the network 'independent of input image size.' Show this: given the same CNN (with GAP) receiving 320×320×3 instead of 224×224×3, what is the feature map size before GAP after five 2×2 max pool layers? After GAP? Does the FC layer change? What would happen with flatten instead?

5. [Challenge] Max pooling introduces 'invariance to small translations.' Quantify for a network with three 2×2 max pool layers (stride 2). (a) Maximum spatial shift producing identical output from each pooling layer. (b) Is a lesion at position (112,112) vs (112,128) invariant after three pool layers? Show work. (c) What does this imply for the random-crop augmentation decision (224×224 crop from 256×256 resized image)?

2.5 CNN Architecture: Assembling the Components

2.5.1 The Standard Block: Conv → Activation → Pool

The convolutional layer, activation function, and pooling layer combine into a repeating structural unit—the convolutional block—the basic building unit of all CNN architectures from LeNet (1989) to EfficientNet (2019).

Standard block sequence: Conv → Batch Normalisation → ReLU → Pooling. Conv detects local patterns. BN normalises activations within each mini-batch (placed after conv, before ReLU), stabilising training, reducing learning-rate sensitivity, and providing mild regularisation. ReLU introduces non-linearity and suppresses negative activations. Pooling reduces spatial dimensions and enlarges effective receptive field. The ordering Conv→BN→ReLU is standard in modern implementations (PyTorch default).

A CNN repeats this block, each repetition reducing spatial dimensions, increasing channel depth, enlarging receptive field, and producing higher abstraction. Classic pattern: double the number of channels at each downsampling step (32→64→128→256), compensating for halved spatial resolution.

2.5.2 The Feature Hierarchy Across Depth

In a CNN trained on disease detection, the feature hierarchy that emerges consistently mirrors the visual processing hierarchy proposed long before deep learning existed.

- Block 1 (close to input): elementary visual primitives—oriented edge detectors at various angles, colour-opponent signals, frequency-selective patterns. Feature maps are 224×224 (or 112×112 after pooling): high spatial resolution, low semantic content.
- Blocks 2–3 (middle): textures and pattern combinations—grids, corner detectors, colour region boundaries, leaf venation periodicity. Feature maps 56×56 or 28×28: intermediate resolution and semantic content.
- Blocks 4–5 (deep): disease-specific patterns at clinical-description level—the stippled appearance of Alternaria leaf spot, the radially-spreading bright zones of mildew, the water-soaked darkening of Late Blight. Feature maps 14×14 or 7×7: low spatial resolution, rich semantic content.

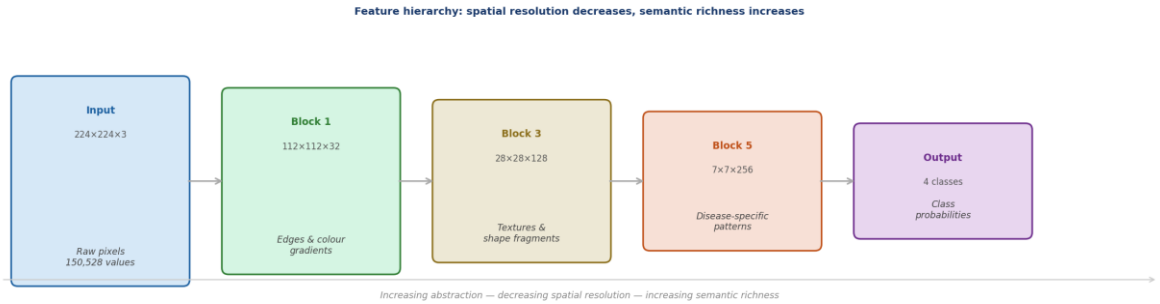


Figure 2.4 *Feature hierarchy across CNN depth, from raw pixels through edge detection, texture assembly, and disease-specific pattern detection, to final class probabilities. As depth increases, spatial resolution decreases while semantic richness increases.*

2.5.3 The Classification Head: Flattening and Fully Connected Layers

Option A: Flatten → FC (legacy). Flatten 7×7×256 to 12,544, pass through FC layers. VGGNet's three FC layers (each 4,096 neurons) contain 122 of 138 million total parameters—the primary overfitting risk.

Option B: GAP → FC (modern, recommended). GAP: 7×7×256 → 256-dimensional vector. FC head: 256→4, softmax. Parameters: 256×4+4 = 1,028. Fifty times fewer than Option A's head, with no accuracy penalty and stronger built-in regularisation. For AgriVision's 6,000-image dataset, Option B is the clear choice.

2.5.4 Computing the Full Architecture: A Worked Example

✎ **WORKED EXAMPLE**

AgriVision Disease Detection CNN — complete dimension and parameter trace

Architecture: five conv blocks (Conv(3×3, same) → BN → ReLU → MaxPool(2×2)) + GAP + FC(4)

Input: 224×224×3

Block 1 (K=32): Conv+BN+ReLU → 224×224×32 | MaxPool → 112×112×32 | Params: 960

Block 2 (K=64): Conv+BN+ReLU → 112×112×64 | MaxPool → 56× 56×64 | Params: 18,624

Block 3 (K=128): Conv+BN+ReLU → 56× 56×128 | MaxPool → 28× 28×128 | Params: 74,112

Block 4 (K=256): Conv+BN+ReLU → 28× 28×256 | MaxPool → 14× 14×256 | Params: 295,680

Block 5 (K=256): Conv+BN+ReLU → 14× 14×256 | MaxPool → 7× 7×256 | Params: 590,592

GAP: 7×7×256 → 256-dimensional vector | Params: 0

FC + softmax: 256 → 4 | Params: 1,028

Total trainable parameters: 980,996 ≈ 981K

Compare to MLP: 38,568,836 parameters, 6,428 per training image.

This CNN: 980,996 parameters, 163 per training image—39× fewer parameters.

2.5.5 Visualising What Filters Learn

First-layer filters (3×3×3 tensors) can be visualised directly as small colour patches. After training on AgriVision's disease images, the 32 first-layer filters consistently include: oriented edge detectors (positive weights on one side, negative on the other), colour-opponent filters (positive green, negative red—detecting transitions from healthy to infected tissue), and high-frequency texture filters. These are not outputs of hand-engineering; they are what gradient descent consistently discovers when trained on natural images.

Activation maximisation—running gradient ascent in the input space to find the image that maximally activates a specific neuron—produces interpretable patterns for first-layer filters and increasingly disease-specific patterns for deep-layer neurons. A neuron responding to powdery mildew produces visualisations with the characteristic stippled white texture at the spatial scale where the lesion pattern occurs.

✦ THE AGRIVISION CONTEXT

AgriVision's CNN achieves 89.3% validation accuracy on the four-class disease task with 981K parameters trained on 6,000 photographs—a 30-percentage-point improvement over the MLP baseline (58.7%) from the same data and same training budget.

The improvement comes entirely from the right inductive bias: local receptive fields exploit spatial locality; weight sharing encodes translation equivariance. No additional data was collected. No fundamentally different training procedure was used. The architecture was changed.

But 10.7% validation error rate represents meaningful wrong recommendations when deployed across 150 farms. Sections 2.6–2.9 address remaining limitations through better architecture choices, transfer learning, and systematic failure analysis.

KEY TERMS — THIS SECTION

Term	Definition
Convolutional block	Repeating CNN structural unit: Conv→BN→ReLU→Pool
Feature hierarchy	Progression from elementary features (edges) in early layers to disease patterns in deep layers
Spatial-semantic tradeoff	As CNN depth increases, spatial resolution decreases while semantic richness increases
Channel doubling	Common design pattern: double channel count at each spatial downsampling step
Activation maximisation	Gradient ascent in input space to find the image maximally activating a specific neuron
Spurious correlation	Learned association between an irrelevant feature (background, shadow) and a class label

✎ CHECK YOUR UNDERSTANDING

1. The AgriVision CNN has 980,996 total parameters. What fraction are convolutional weights only (excluding BN parameters and FC head)? Which block contains the most convolutional parameters, and why?

2. A student uses Flatten→FC(12544→4096)→ReLU→Dropout(0.5)→FC(4096→4) as the head. (a) Total parameters in this head? (b) With 6,000 training images, would this version generalise better or worse than the GAP version? Justify with the parameter-per-training-example ratio. (c) Describe the expected validation loss curve.

3. Block 5 produces 7×7×256, which GAP averages to a 256-dimensional vector. Explain in plain language what each of the 256 values represents. What spatial information is preserved? What is discarded?

4. You have Block 5 feature maps for 100 Powdery Mildew and 100 Healthy correctly-classified images. Describe a quantitative experiment to identify which of the 256 channels are most

discriminative between these two classes. What would a highly discriminative channel look like? A non-discriminative one?

5. [Challenge] Design the smallest CNN satisfying: (a) input 224×224×3; (b) exactly four conv blocks (one 3×3 conv, BN, ReLU, 2×2 max pool each); (c) GAP and 4-class FC head; (d) final feature map before GAP has ≥32 channels; (e) every conv layer has ≥ as many output as input channels. Find channel configuration minimising total parameters while satisfying all constraints. Is 6,000 training examples sufficient? Justify.

2.6 Architecture Evolution as Problem–Response

2.6.1 LeNet: The First Demonstration

In 1989, Yann LeCun and colleagues at Bell Labs demonstrated the first practical CNN for recognising handwritten characters. LeNet-5 (1998) contained the essentials all subsequent CNNs inherit: alternating conv and pool layers, non-linear activations, and a FC classification head. Deployed in the 1990s to read bank cheque digits, it processed millions daily. The entire network has approximately 60,000 parameters.

What LeNet proved. Local connectivity and weight sharing could be trained by backpropagation. First-layer filters genuinely detected oriented edges; deeper layers combined these into digit-specific shapes.

What LeNet could not do. The 32×32 greyscale MNIST digit images are among the simplest possible visual tasks. Real photographs—variable lighting, complex backgrounds, objects at different scales—were far beyond LeNet's reach. The architecture was right; the data and compute to scale it were absent. That would take another 23 years.

2.6.2 AlexNet: What Scale and GPUs Changed

The ImageNet Large Scale Visual Recognition Challenge required classifying photographs into 1,000 categories from 1.2 million training images. The best systems before 2012 used hand-crafted features and achieved top-5 error rates of ~26%. AlexNet (Krizhevsky, Sutskever, Hinton, 2012) achieved 15.3%—a 10.8 percentage-point improvement that signalled the beginning of the deep learning era in computer vision.

AlexNet's five key innovations: (1) Eight weight layers (deep by 2012 standards). (2) ReLU activations throughout—producing a 6× training speedup over tanh. (3) GPU training across two GTX 580 GPUs (3 GB each). (4) Dropout (p=0.5) between the FC layers. (5) Data augmentation (random 224×224 crops from 256×256, horizontal flipping, colour jitter). AlexNet's 60 million parameters are excessive for AgriVision's 6,000-image dataset, but its conceptual contributions—ReLU, dropout, augmentation—are all present in the Section 2.5 CNN.

2.6.3 VGG: Depth With Simplicity

VGGNet (Simonyan and Zisserman, Oxford, 2014) asked: how much does depth alone, with a uniform and simple architecture, affect performance? All filters restricted to 3×3. The mathematical justification:

$$\textit{One } 5\times5 \textit{ layer } (C\to C):\ 5^2\times C^2 = 25C^2 \textit{ parameters}$$

$$\textit{Two } 3\times3 \textit{ layers (same receptive field):}\ 2\times3^2\times C^2 = 18C^2 \textit{ parameters}\ —\ 28\%\textit{ fewer, plus one extra ReLU}$$

VGG-16 achieved top-5 error of 7.3% on ILSVRC 2014—substantially better than AlexNet's 15.3%. But the three large FC layers contained 122 of 138 million total parameters—expensive to store, slow to evaluate, prone to overfitting on smaller datasets. VGG also showed that adding layers beyond 16–19 provided diminishing returns. Why?

2.6.4 Inception: Multi-Scale Receptive Fields in Parallel

Szegedy et al. (Google, 2014) asked: for a given layer, what is the right filter size? Different features exist at different scales. Rather than choosing one filter size, use all simultaneously.

The Inception module applies four operations in parallel to the same input: (1) 1×1 conv; (2) 1×1 bottleneck → 3×3 conv; (3) 1×1 bottleneck → 5×5 conv; (4) 3×3 max pool → 1×1 conv. Outputs are concatenated along the channel dimension. The 1×1 bottlenecks dramatically reduce computation before expensive spatial operations.

GoogLeNet (Inception v1): 22 layers, 9 Inception modules, only 7 million parameters (versus VGGNet's 138M)—one of the first large-scale deployments of GAP. Top-5 error: 6.7% on ILSVRC 2014. Same accuracy as VGG at 20× fewer parameters.

2.6.5 ResNet: The Residual Connection and Why Depth Stopped Helping Without It

Networks with 56 layers performed worse than 20-layer networks—not just on validation sets (where overfitting might explain it) but on training sets. Something more fundamental: optimisation itself was failing. A 56-layer network should be at least as good as a 20-layer one by learning identity mappings at the extra layers. In practice, it could not. This is the degradation problem.

He et al. (Microsoft Research, 2015) proposed: instead of asking a layer to learn a mapping H(x), ask it to learn the residual F(x) = H(x) − x. The actual output is:

$$Output = F(x) + x$$

The gradient through this residual block:

$$\partial L/\partial x = \partial L/\partial output \times (\partial F(x)/\partial x + 1)$$

The bold 1 comes from the derivative of the skip connection. Regardless of the complexity or saturation of F(x), the gradient always has a component of 1—preventing the vanishing gradient problem even in very deep networks. ResNet-152 achieved top-5 error of 3.57% on ILSVRC 2015, below the commonly cited human error rate of ~5%. The residual connection is not specific to images—it appears in Transformers (Section 4.5) and virtually every modern architecture.

2.6.6 EfficientNet: Scaling All Dimensions Together

Tan and Le (Google, 2019) proposed compound scaling: simultaneously scale depth, width, and resolution with a fixed compound coefficient ϕ (phi):

$$Depth: d = \alpha^{\varphi}, \quad Width: w = \beta^{\varphi}, \quad Resolution: r = \gamma^{\varphi}$$

Subject to $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$ (α=1.2, β=1.1, γ=1.15). Setting ϕ = 0,1,2,...,7 yields EfficientNet-B0 through B7. EfficientNet-B0 achieves ~77% top-1 accuracy on ImageNet with 5.3M parameters—comparable to ResNet-50 (25.6M) at a fraction of the size. For deployment on farmers' smartphones (battery life, app size, latency), EfficientNet-B0 satisfies the <100ms constraint on modern mid-range phones.

2.6.7 The Design Lesson: Every Architecture Solved a Specific Failure Mode

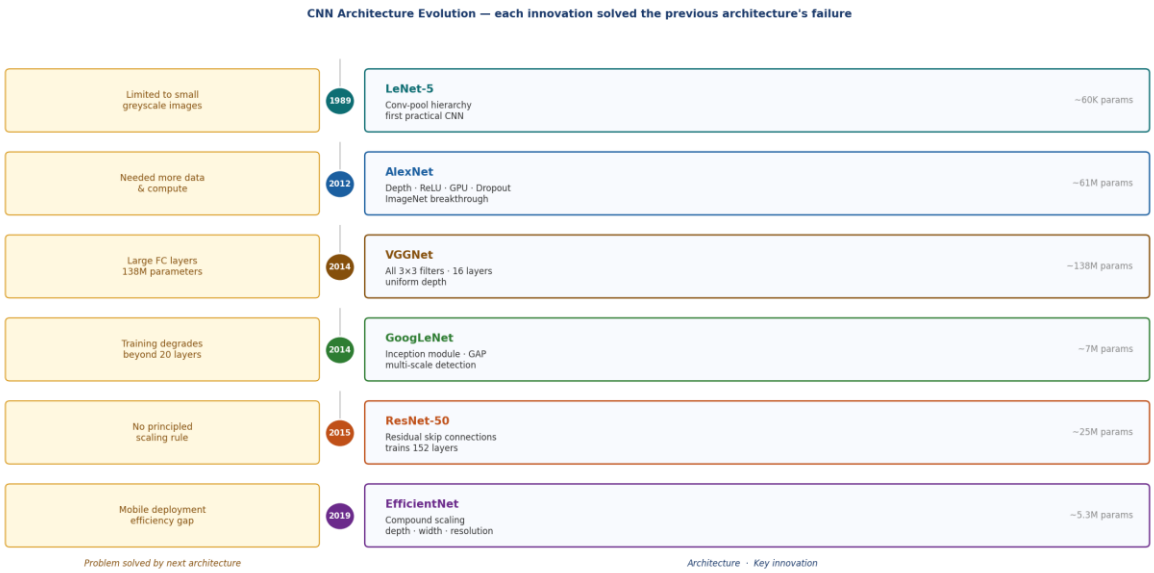


Figure 2.3 *CNN architecture evolution from LeNet (1989) to EfficientNet (2019). Left column: the problem each architecture solved. Right column: the architectural innovation and approximate parameter count. Each architecture is a direct, motivated response to the previous state of the art's failure.*

Architecture	Year	Key innovation	Problem solved	Parameters
LeNet-5	1989	Conv-pool hierarchy	Spatial structure in images	~60K
AlexNet	2012	Depth + ReLU + GPU + Dropout	ImageNet-scale classification	~61M
VGGNet-16	2014	All 3×3 filters, 16 layers	How much does depth alone help?	~138M
GoogLeNet	2014	Inception module + GAP	Efficient multi-scale detection	~7M
ResNet-50	2015	Residual connections	Degradation in very deep networks	~25M
EfficientNet	2019	Compound scaling $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$	Optimal depth/width/resolution trade-off	~5.3M

Table 2.2 *CNN architecture evolution. Each architecture solved the specific failure mode of its predecessor.*

KEY TERMS — THIS SECTION

Term	Definition
Degradation problem	Very deep networks have higher training error than shallower ones without residual connections
Residual connection	Additive skip: output = F(x)+x; enables training of very deep networks by providing a direct gradient path
Inception module	Parallel branches with 1×1, 3×3, 5×5 filters and pooling, concatenated for multi-scale detection
1×1 bottleneck	1×1 convolution reducing channel depth before expensive spatial convolutions; reduces parameters without changing receptive field

Compound scaling	EfficientNet's joint scaling of depth, width, and input resolution with a fixed compound coefficient
Projection shortcut	1×1 conv on the skip path when residual block changes spatial dimensions or channel depth

✎ **CHECK YOUR UNDERSTANDING**

1. VGGNet: one 7×7 layer vs three stacked 3×3 layers (same receptive field). (a) Parameters for one 7×7 layer (C in, C out). (b) Parameters for three 3×3 layers. (c) Three 3×3 layers have fewer parameters but more representational power—why?

2. The degradation problem vs vanishing gradients: (a) What causes vanishing gradients? Which layers are affected? (b) What is observed in the degradation problem? Why is it surprising? Why cannot vanishing gradients alone explain it? (c) How does a residual connection address each separately?

3. A ResNet block: output = F(x)+x. (a) Compute ∂(output)/∂x. Show the derivative always has a component equal to 1. (b) What does this mean for gradient flow during backpropagation? (c) If F(x) outputs near-zero at initialisation, what does the block compute at training start? Why is this a better starting point than a randomly initialised non-residual block?

4. Single Inception module with 128 input channels: (a) Parameters for a direct 5×5 conv producing 32 output channels. (b) Parameters for a 1×1 bottleneck (128→16) + 5×5 (16→32). (c) Reduction factor from the bottleneck?

5. EfficientNet compound scaling: $\alpha=1.2$, $\beta=1.1$, $\gamma=1.15$. (a) Verify $\alpha\cdot\beta^2\cdot\gamma^2\approx 2$. (b) At $\phi=3$ (B3), compute scaling factors. (c) If B0 processes 224×224, what is B3's resolution? (d) Why does the constraint $\alpha\cdot\beta^2\cdot\gamma^2\approx 2$ ensure doubling ϕ approximately doubles total FLOPs?

6. [Challenge] AgriVision has expanded to 5,000 farms with 150,000 labelled photographs. Target: web dashboard (server-side, no latency constraint) and farmer smartphone app (on-device, <150ms, mid-range Android). (a) Select and justify a server-side architecture. (b) Select and justify an on-device architecture. Train from scratch or fine-tune? (c) The two models will produce different predictions for the same image. Describe a system-level approach to reconciling these in production.

2.7 Visual Tasks Beyond Classification

2.7.1 The Spectrum: What? → Where? → Which Pixels?

The disease detection CNN from Sections 2.3–2.5 answers one question: what disease is present in this photograph? It outputs a probability vector over four classes. This is image classification—the simplest visual task. But the extension officer in Gadag asked a more complex question: where is the disease, how many distinct lesions are visible, and how far has infection spread? These questions have different computational structures.

Task	Output format	When appropriate	Annotation cost
Image classification	One class label / probability vector	Disease type for treatment decision; 'is this diseased?'	Seconds per image
Object detection	Set of bounding boxes with class labels and confidence	Counting lesions; spatial distribution; multi-disease per leaf	Minutes per image
Semantic segmentation	Pixel-level class label map (H×W)	Fraction of leaf area infected; disease spread tracking; precision agriculture	30–90 minutes per image

Table 2.3 *Practical trade-offs between visual task types. Choose the formulation whose output structure matches the downstream decision, not the most sophisticated available option.*

2.7.2 Object Detection: Localisation and the Role of Anchors

Object detection extends classification with localisation: predict the spatial location of each object as a bounding box (centre x, y, width w, height h) plus a class label. Classification alone cannot localise—after global average pooling, all spatial information has been averaged away.

Detection networks process the image through a convolutional backbone to a spatial feature map (e.g., 7×7×512), then apply detection heads at each spatial position. Each position simultaneously predicts a class probability vector, bounding box offsets, and objectness score.

Anchor boxes. Predefined boxes of various sizes and aspect ratios tiled across the image. The network predicts adjustments to anchors rather than absolute coordinates. For anchor a at position (i,j):

$$x_detected = x_anchor + \Delta x, \quad w_detected = w_anchor \times exp(\Delta w)$$

Non-Maximum Suppression (NMS). A detection network may predict many overlapping boxes for the same lesion. NMS resolves this: (1) Sort predicted boxes by confidence (descending). (2) Accept the highest-confidence box. (3) Remove boxes with IoU > threshold (typically 0.5) with the accepted box. Repeat. IoU = Area(intersection) / Area(union). IoU = 1.0: identical boxes. IoU = 0.0: no overlap.

2.7.3 Semantic Segmentation: Pixel-Level Prediction

Semantic segmentation assigns a class label to every pixel: healthy leaf tissue, powdery mildew colony, late blight lesion, Alternaria spot, leaf vein, background. From this pixel map, one can directly compute the infected leaf area fraction and track lesion boundary expansion across weekly photographs.

The key challenge: recovering spatial resolution. A standard classification CNN reduces 224×224 to a 7×7 feature map—1,024× fewer spatial positions. To produce 224×224 pixel-level output, the network must recover the spatial detail that pooling discarded.

U-Net: Encoder-decoder with skip connections. Ronneberger et al. (2015), working on biomedical image segmentation. The encoder follows the standard CNN pattern: successive conv+pool blocks that halve spatial resolution while doubling channel depth. The decoder reverses this with transposed convolutions. At each upsampling step, the decoder receives a skip connection from the encoder at the corresponding resolution—the encoder's feature map before that block's pooling operation, concatenated with the decoder's upsampled feature map. These skip connections provide the decoder with the fine spatial detail the encoder had before pooling—the information needed to draw boundaries precisely.

Encoder structure (5 blocks): 224×224×3 → 112×112×64 → 56×56×128 → 28×28×256 → 14×14×512 → 7×7×1024 (bottleneck). Decoder (5 upsampling steps, each receiving encoder skip connection) restores to 224×224×K, where K is the number of segmentation classes. Loss: pixel-wise cross-entropy averaged over all positions.

2.7.4 Choosing the Right Task Formulation for Your Problem

The guiding principle: match the output structure to what the downstream decision requires, not to what is technically achievable or impressive.

- Extension officer question 'Is this leaf diseased, and with what?' → Classification.
- Researcher question 'How many lesions per leaf, and where are they?' → Detection.
- Precision agriculture question 'What fraction of leaf area is infected, and is it increasing?' → Segmentation.

> ⚠ **IMPORTANT NOTE**
>
> A recurring failure mode: task mismatch.
>
> The most common task formulation error in agricultural AI is applying classification when detection is needed. A disease severity assessment system built on classification outputs ('Disease present: yes/no') cannot track disease progression—it can only confirm presence or absence at each photograph. A detection system can count lesions per visit and compute growth rate per lesion, providing the quantitative signal needed for timing interventions.
>
> When a system built for the wrong task fails to provide clinical value, the failure is often attributed to model accuracy when the true cause is formulation mismatch.

KEY TERMS — THIS SECTION

Term	Definition
Image classification	Produces a single class label / probability vector for the entire image
Object detection	Produces bounding boxes with class labels for each detected object
Semantic segmentation	Produces a class label for every pixel; highest spatial output richness

Anchor box	Predefined box of fixed size/aspect ratio; detection networks predict adjustments, not absolute coordinates
IoU	Intersection over Union: Area(intersection)/Area(union); measures bounding box overlap
NMS	Non-Maximum Suppression: removes duplicate detections by keeping highest-confidence box among overlapping predictions
U-Net	Encoder-decoder architecture with skip connections restoring fine spatial detail for segmentation
Transposed convolution	Learned upsampling increasing spatial dimensions; trainable counterpart to bilinear interpolation
Pixel-wise cross-entropy	Segmentation loss: mean cross-entropy computed independently at every pixel position
Task mismatch	Applying a simpler task formulation to a problem requiring richer spatial output

✎ **CHECK YOUR UNDERSTANDING**

1. Ten AgriVision leaves: 7 healthy, 2 with powdery mildew (one early, one late stage), 1 with both late blight and alternaria. For each task type (classification, detection, segmentation), state the correct output format for the co-occurring-disease leaf. Which task type most faithfully represents this complex scenario?

2. A YOLO-based detector produces three raw predictions for 'Powdery Mildew' before NMS: Box A (confidence 0.87, box [50,80,120,100]), Box B (confidence 0.72, box [58,88,115,95]), Box C (confidence 0.61, box [180,200,90,75]). With NMS IoU threshold 0.5, apply NMS step by step. Compute IoU between Boxes A and B to determine whether they should be merged.

3. A U-Net encoder reduces 224×224×3 to a 7×7×512 bottleneck through five conv+pool blocks. (a) List spatial resolution at each encoder and decoder level. (b) At the 56×56 level, encoder has 64 channels and decoder's upsampled map has 64 channels. Shape after concatenation? (c) Why is concatenation preferable to addition (as in ResNet)?

4. Section 2.7.4 states pixel-level annotation takes 30–90 minutes per image. For 8,000 photographs: (a) Estimate total annotation time. (b) At ₹800/hour, estimate annotation budget. (c) Propose a cost-efficient strategy that still produces pixel-level data for validation.

5. [Challenge] Design a pipeline using all three task types: (a) on-device classifier determines if further analysis is needed; (b) if disease detected, server-side detection locates individual lesions and counts; (c) for severe cases (≥5 lesions), segmentation computes percentage leaf area and generates annotated report. Specify architecture family, input/output, and error propagation risk at each stage.

2.8 Transfer Learning in Computer Vision

2.8.1 Why Pretrained Models Transfer: The Universality of Early Layers

Rather than initialising the CNN's weights randomly and learning everything from AgriVision's 6,000 photographs, we can initialise from a network already trained on ImageNet—1.2 million labelled photographs across 1,000 categories—and adapt that pre-existing knowledge to disease detection. This is transfer learning.

The approach sounds unpromising: ImageNet contains Golden Retrievers and kitchen whisks; AgriVision contains tomato leaves with fungal infections. What could one contribute to the other? The answer from Yosinski et al. (2014): the features learned in early convolutional layers are not specific to the task or dataset—they are specific to the nature of natural images.

First-layer filters of networks trained on ImageNet, Places365, and medical imaging datasets all converge to approximately the same set of features: oriented edge detectors, colour-opponent filters, and frequency-selective patterns. These are the fundamental visual primitives of any natural image—patterns appearing in all real photographs regardless of subject matter. A tomato leaf photograph has edges, colour gradients, and texture frequencies just as a Labrador photograph does.

> ◆ **KEY CONCEPT**
>
> **What transfers and what does not**
>
> Transfers freely (early layers): oriented edge detectors, colour gradient filters, high-frequency texture detectors. These features appear in all natural images.
>
> Transfers partially (middle layers): texture assemblies, shape detectors. Useful across domains but may need adaptation when target domain has unusual textures (microscopy, satellite imagery).
>
> Does not transfer (deep layers): task-specific classifiers for 1,000 ImageNet categories. Actively harmful for a different task—must be replaced entirely.
>
> The classification head (FC layers after GAP) always transfers nothing and is always replaced. The convolutional backbone always transfers something, with the amount depending on domain similarity.

2.8.2 Feature Extraction: Freezing All Layers

Feature extraction treats the pretrained backbone as a fixed feature extractor: all pretrained weights are frozen (not updated during training), and only the new classification head is trained.

Procedure: (1) Load pretrained model with ImageNet weights. (2) Remove original 1,000-class head. (3) Add new head: GAP→FC(256)→ReLU→Dropout(0.3)→FC(4)→Softmax. (4) Freeze all conv layers (no gradient updates). (5) Train only the new head on AgriVision data.

When appropriate: very small datasets (<500 images, where even lightweight conv layers overfit); high domain similarity to ImageNet; fast prototyping where minutes of training produce a strong baseline.

AgriVision result: Frozen ResNet-50 + new three-layer head, 30 epochs. Validation accuracy: 86.8%. Training time: under 5 minutes on Colab T4. Only 65K parameters trained (the new head). Slightly below the 89.3% from-scratch CNN, but requires 9× less training time.

2.8.3 Fine-Tuning: Updating Selected Layers

Fine-tuning goes further: after training the classification head, selectively unfreeze pretrained layers and train jointly at a reduced learning rate, allowing features to adapt toward the target domain.

Catastrophic forgetting. The primary risk: large gradient updates on a small target dataset overwrite carefully learned pretrained weights. The backbone that began with sophisticated visual representations ends with weights that have memorised 6,000 photographs. Prevention requires a small learning rate (typically 1/10th to 1/100th of the from-scratch rate) and gradual layer unfreezing.

Differential learning rates. The most principled strategy assigns smaller learning rates to earlier layers: Early layers (edge detectors): $\eta \times 10^{-4}$; Middle layers (textures): $\eta \times 10^{-3}$; Deep layers (task-specific): $\eta \times 10^{-2}$; Classification head: $\eta \times 10^{-1}$.

AgriVision fine-tuning result: ResNet-50 with differential learning rates, unfreezing blocks progressively over 50 epochs. Validation accuracy: 93.7%. A substantial improvement over both from-scratch (89.3%) and feature extraction (86.8%), achieved with the same 6,000 photographs.

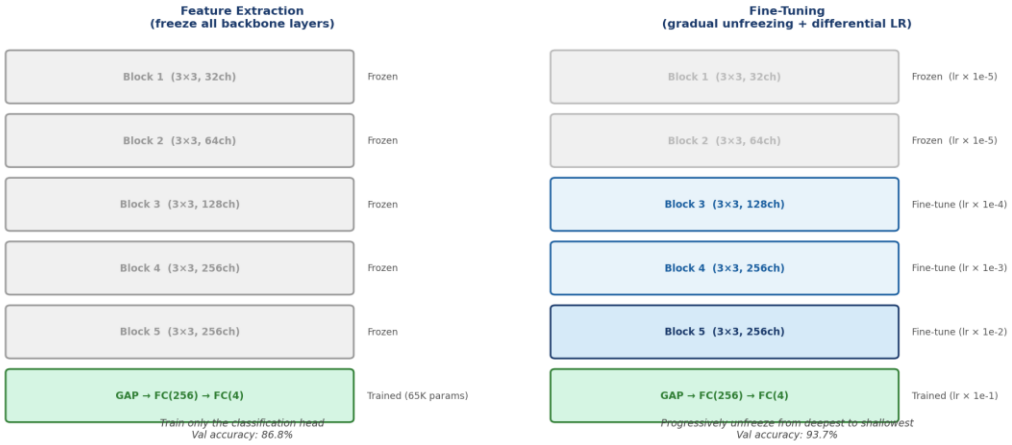


Figure 2.5 *Feature extraction (left) vs fine-tuning (right). Feature extraction freezes all backbone layers and trains only the new classification head (65K parameters). Fine-tuning progressively unfreezes backbone blocks from deepest to shallowest, applying differential learning rates that increase from early layers to the classification head.*

2.8.4 When to Freeze and When to Train: Practical Rules

> ◆ **KEY CONCEPT**
>
> **The Transfer Learning Decision Matrix**
>
> Similar domain + Small dataset (≤1,000 images): feature extraction. Freeze all conv layers, train new head only.

Similar domain + Large dataset (>10,000 images): fine-tune all layers. Initialise from pretrained, train end-to-end with small lr.

Different domain + Small dataset: freeze all but first 1–2 blocks. Deep features don't transfer well; early features do.

Different domain + Large dataset: fine-tune all layers with care. Pretrained initialisation helps convergence, but deep layers need substantial adaptation.

AgriVision's position: 6,000 training images, domain similar to ImageNet. Partial fine-tuning (last 1–2 blocks) is optimal: avoids catastrophic forgetting at this dataset size, yet allows disease-specific adaptation.

Three additional rules: (1) Always warm up the classification head before unfreezing the backbone—random head gradients will corrupt pretrained weights if propagated immediately. (2) Monitor validation loss, not just accuracy. (3) Always use ImageNet normalisation (mean [0.485,0.456,0.406], std [0.229,0.224,0.225]) when using any pretrained model. Forgetting this is the most common silent bug, degrading performance 5–10 percentage points without any error message.

2.8.5 Applied to AgriVision: Disease Detection Under Label Scarcity

PlantVillage (54,306 labelled leaf photographs, 26 disease categories, 14 plant species) offers an intermediate transfer step: ImageNet → PlantVillage → AgriVision. Stage 1 adapts mid-level features toward leaf-specific textures. Stage 2 adapts deep features from controlled PlantVillage laboratory conditions toward AgriVision's uncontrolled field photographs.

✦ THE AGRIVISION CONTEXT

Transfer learning results on AgriVision (6,000 training photographs)

MLP (baseline): 58.7% | 8 hours | 38.6M params trained

Five-block CNN from scratch: 89.3% | 45 min | 981K params trained

ResNet-50 feature extraction: 86.8% | 5 min | 65K params trained

ResNet-50 partial fine-tuning (last 2 blocks): 93.7% | 35 min | 8.9M params trained

ResNet-50 PlantVillage → AgriVision (two-stage): 95.2% | 60 min | 23.5M params trained

The progression 58.7% → 89.3% → 95.2% required the same 6,000 labelled photographs at every step. The improvement comes from changing the starting point and strategy, not from collecting more data.

KEY TERMS — THIS SECTION

Term	Definition
Transfer learning	Using representations learned on one task as the starting point for a different task
Pretrained model	A network whose weights were learned on a large dataset (typically ImageNet) used as initialisation

Feature extraction	Transfer learning where all pretrained conv weights are frozen; only the new head is trained
Fine-tuning	Transfer learning where some or all pretrained layers are unfrozen and trained at reduced learning rate
Catastrophic forgetting	Overwriting learned pretrained features by large gradient updates on a small target dataset
Differential learning rates	Smaller learning rates for early layers, larger for deep layers during fine-tuning
Gradual unfreezing	Unfreezing blocks from deepest to earliest; each block stabilises before the next is adapted
PlantVillage	Public dataset of 54,306 labelled leaf disease photographs; useful intermediate transfer source
ImageNet normalisation	Per-channel mean [0.485,0.456,0.406] and std [0.229,0.224,0.225]; mandatory for pretrained models

✎ **CHECK YOUR UNDERSTANDING**

1. A researcher has 200 labelled cassava mosaic virus photographs taken under controlled greenhouse conditions with uniform lighting. A pretrained ResNet-50 is available. Using the transfer learning decision matrix, determine the appropriate strategy. Would your answer change for 50 images? 5,000?

2. A student fine-tunes ResNet-50 at lr=0.01 (the from-scratch rate). After 20 epochs, training accuracy is 99.1% but validation accuracy has degraded from 86.8% (feature extraction baseline) to 78.4%. (a) Name the failure mode. (b) Explain what happened to the pretrained weights. (c) Propose two specific changes to prevent this.

3. For a ResNet-50 with four residual blocks (Block 1 shallowest, Block 4 deepest), assign learning rates from $\{10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}\}$ to Block 1, Block 2, Block 3, and the classification head. Justify each assignment by reference to feature transferability at that depth.

4. Section 2.8.4 states forgetting ImageNet normalisation degrades performance 5–10 percentage points 'silently.' Explain why the mismatch causes degradation rather than a numerical error. Why is this bug particularly hard to detect?

5. [Challenge] AgriVision now has 150,000 labelled field photographs. (a) Should they train from the existing ResNet-50 checkpoint or from ImageNet weights? Justify. (b) With 150,000 images, is gradual unfreezing still necessary, or can they fine-tune all layers simultaneously? (c) The final season's photographs have a different average colour temperature (new phone firmware changed white balance). Design a data preprocessing and training strategy to prevent this from degrading Season 7 performance.

2.9 When CNNs Fail

2.9.1 Dataset Bias and What It Looks Like at Inference

AgriVision's disease detector achieves 95.2% validation accuracy. But the validation set was drawn from the same distribution as the training set: the same 150 farms, same six seasons, same population of phone cameras. A model that achieves high accuracy on its own validation set may still fail systematically on inputs from outside that distribution.

Dataset bias occurs when the training dataset is not a representative sample of the full range of inputs the model will encounter in deployment. Unlike labelling error (random), dataset bias is systematic. The model learns spurious correlations that are predictive within the training distribution but do not reflect the true causal relationship between visual features and disease labels.

> ✦ THE AGRIVISION CONTEXT
>
> **A concrete AgriVision dataset bias scenario**
>
> Suppose AgriVision's 3,000 disease photographs were collected primarily in the morning (farmers photograph before beginning field work), while 5,000 healthy-leaf photographs were collected throughout the day. Morning light has a cooler colour temperature (more blue-shifted) than afternoon light.
>
> The model learns, correctly within the training set, that 'blue-shifted images tend to be disease photographs.' In deployment: a healthy leaf photographed at dawn is misclassified as diseased; a diseased leaf photographed at noon is missed. This specific bias is almost impossible to detect from validation accuracy alone—the validation set has the same morning/afternoon imbalance.

Categories of dataset bias in agricultural computer vision:

- Lighting and time-of-day bias: training data collected at specific times. Model learns time-of-day cues rather than disease patterns. Mitigation: stratified collection; aggressive colour jitter augmentation.
- Background and setting bias: all diseased photographs in field settings; all healthy in greenhouses with plain backgrounds. Model associates complex backgrounds with disease. Mitigation: balanced background diversity.
- Farm and variety bias: disease photographs disproportionately from farms growing one tomato variety with distinctive leaf shape. Model confuses variety-specific features with disease. Mitigation: balanced sampling.
- Photographer bias: extension officers zoom in on lesions; healthy photographs at standard distance. Model uses image framing (zoom level) as disease predictor. Mitigation: standardise photography protocols.

Detection strategy: examine per-class confusion matrices stratified by external variables (time of day, farm, camera model, season). If accuracy differs substantially across strata for the same class, a spurious correlation may be the cause. Run Grad-CAM on correctly classified examples: if activation concentrates on the background rather than the lesion, the model is using a spurious correlate and will fail when that correlate is absent.

2.9.2 Domain Shift: Training Distribution vs. Deployment Distribution

Domain shift describes the mismatch between the training distribution and the deployment distribution. A model perfectly calibrated to one distribution degrades when the distribution changes. This is not a model deficiency—it is a fundamental challenge of deploying learned systems in a changing world.

- Seasonal shift: leaf colour and texture vary with plant growth stage. A model trained on seasons 1–5 encounters growth stages and seasonal stress levels not fully represented in training.
- Geographic shift: farms in different micro-climates and soil types produce leaves with different baseline colour statistics. A model trained on Dharwad district may perform worse on Raichur farms.
- Camera shift: newer phone models have different colour processing pipelines. The colour statistics of a 2022-phone photograph differ measurably from a 2019-phone photograph of the same scene.
- Disease strain shift: new strains may have slightly different visual signatures than the strains present during training.

Detecting domain shift without ground truth labels: (1) Input distribution monitoring—track the mean, standard deviation, and higher moments of pixel values across incoming photographs. Kolmogorov-Smirnov test on channel distributions flags significant departure from the training distribution. (2) Prediction confidence monitoring—a model encountering out-of-distribution inputs typically produces lower-confidence predictions. Sustained decrease in average maximum softmax probability signals distribution shift. (3) Feature space monitoring—track GAP feature vectors for incoming photographs. Significant drift from the training feature cloud indicates unfamiliar inputs.

> ⚠ **IMPORTANT NOTE**
>
> Responding to detected domain shift.
>
> Short-term: flag predictions on anomalous inputs for human review. Reduce the model's autonomous decision authority. Alert field staff to verify predictions on photographs with unusual statistics.
>
> Medium-term: collect labelled examples from the shifted distribution. Fine-tune the current model on new examples (continual learning).
>
> Long-term: redesign the data collection protocol to ensure ongoing diversity. Include examples from new seasons, new farms, and new camera models in the periodic retraining cycle.

2.9.3 Adversarial Examples: Imperceptible Perturbations, Wrong Predictions

Adversarial examples are inputs that are visually identical to correctly classified images, but have been deliberately modified to cause confident wrong predictions. Adding a small structured perturbation—invisible to human observers—can cause a state-of-the-art CNN to confidently misclassify the image as any chosen wrong class.

The Fast Gradient Sign Method (FGSM) produces an adversarial example in a single gradient step:

$$x_adv = x + \varepsilon \cdot sign(\nabla_x L(f(x), y))$$

Where $\nabla_x L$ is the gradient of the loss with respect to the input pixels, sign(·) takes the direction of steepest loss increase, and ε controls the perturbation magnitude (typically 0.01–0.05). Each pixel changes by exactly ε in the direction that most increases the loss.

> ✎ **WORKED EXAMPLE**
>
> **FGSM on an AgriVision photograph (ε = 0.02)**
>
> Original prediction: Powdery Mildew 94.2% confidence. Loss = −log(0.942) = 0.060.
>
> Backpropagate to input: compute $\nabla_x L$ (224×224×3 gradient map).
>
> Construct x_adv = x + 0.02 · sign($\nabla_x L$).
>
> Every pixel changes by exactly 0.02 (about 5 greyscale levels on 0–255 scale).
>
> Adversarial prediction: Late Blight 87.1% confidence.
>
> Human observer: the two images appear identical. The model: completely different prediction. A farmer receiving this automated diagnosis would apply the wrong treatment.

Why adversarial examples exist: CNN decision boundaries are not aligned with human-perceived similarity. Two images that appear identical to a human—because the perturbation is below the threshold of visual perception—can be on opposite sides of the model's decision boundary. In deployment, JPEG compression artefacts and camera image processing pipelines can produce perturbations in this regime, causing model brittleness on real-world inputs.

2.9.4 Overconfidence Without Ground Truth

CNNs trained with standard cross-entropy loss are systematically overconfident: their softmax outputs are higher than the true frequency of correctness at that confidence level. A well-calibrated model satisfies: among all inputs assigned 80% probability to a class, 80% actually belong to that class.

The practical danger for AgriVision: a model predicting '97% Late Blight' on a photograph showing moisture stress—a non-disease condition with visually similar early symptoms—may cause a farmer to apply a fungicide that provides no benefit. A model with no uncertainty quantification will produce high-confidence predictions on any input, including photographs of soil, potato leaves, or out-of-focus blurs.

Temperature scaling. Post-hoc calibration: divide pre-softmax logits by a scalar temperature T > 1 before applying softmax:

$$softmax(z/T)_i = exp(z_i/T) / \Sigma_j\, exp(z_j/T)$$

For T > 1, the distribution is flattened—all class probabilities move toward uniform, reducing expressed confidence. T is found by minimising cross-entropy on a held-out calibration set (a portion of the validation set) with T as the only parameter. The model's weights are not changed. For AgriVision's ResNet-50, optimal calibration is achieved at T ≈ 1.4, reducing average maximum softmax probability from 0.91 to 0.76.

⚠ IMPORTANT NOTE

Softmax outputs are not probabilities without calibration.

A CNN producing softmax output 0.94 for Powdery Mildew is saying: 'in the training optimisation, the most effective strategy was to assign high scores to this class for this input.' It is not saying: 'the true posterior probability that this leaf has Powdery Mildew is 94%.'

For a system where output drives treatment decisions—fungicide application, irrigation changes, plant removal—the difference matters. Always calibrate softmax outputs on a held-out validation set before interpreting them as probabilities. Flag predictions where calibrated maximum probability falls below a deployment threshold (typically 70–75%) for human review rather than automated action.

2.9.5 Explainability: Knowing Where the Model Looks (Grad-CAM)

Each of the failure modes above produces wrong or misleading predictions. Diagnosing why these predictions are wrong requires understanding which parts of the input the model was responding to. Gradient-weighted Class Activation Mapping (Grad-CAM) produces a spatial heat map highlighting which image regions contributed most to the model's prediction of a specific class.

Grad-CAM procedure for input image x and target class c, in four steps:

- Step 1 — Forward pass: obtain feature maps A^k of the last conv layer (7×7×256) and pre-softmax score y_c.
- Step 2 — Backpropagate to last conv layer: compute $\partial y_c / \partial A^k_{ij}$ for all k, i, j.
- Step 3 — Channel importance weights: global-average-pool the gradients over spatial positions:

$$\alpha c_k = (1/Z)\, \Sigma_{ij}\, \partial yc/\partial A^{k}_{ij} \quad (Z = 7\times7 = 49 \text{ spatial positions})$$

- Step 4 — Weighted combination with ReLU:

$$Lc_Grad\text{-}CAM = ReLU[\, \Sigma_k\, \alpha c_k \cdot A^k \,]$$

The result is a 7×7 single-channel heat map, upsampled to 224×224 via bilinear interpolation and overlaid on the original image. High values highlight the regions that most positively contributed to predicting class c.

✎ WORKED EXAMPLE

Grad-CAM on two AgriVision predictions

Case 1 — Correctly classified Powdery Mildew (93% confidence):

Grad-CAM heat map: high activation concentrated over the white-coated region of the leaf. Surrounding healthy tissue: near-zero activation. Background soil: zero activation.

Interpretation: model is attending to the correct region. This is the expected and desirable result.

Case 2 — Incorrect: Late Blight predicted for a healthy leaf (81% confidence):

A low-angle afternoon sun casts deep shadow across the lower portion of a healthy leaf.

Grad-CAM heat map for the predicted class (Late Blight): high activation concentrated over the shadowed region. Near-zero activation over the well-illuminated, healthy leaf tissue.

Interpretation: the model is attending to a shadow it has learned to associate with the dark, water-soaked appearance of early Late Blight. The error is explained—and points directly to a remediation: include more healthy-leaf training photographs with shadows; apply brightness jitter augmentation during training.

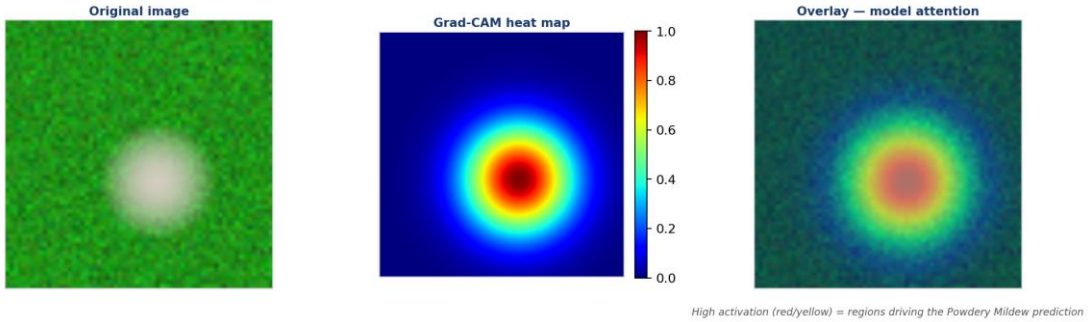


Figure 2.6 *Grad-CAM visualisation on an AgriVision leaf photograph. Left: original image with a powdery mildew infection. Centre: Grad-CAM heat map showing channel importance weights αc_k averaged over the 7×7 spatial grid. Right: overlay combining the original image and the heat map. High activation (yellow-red) identifies the mildew-infected region as the primary driver of the Powdery Mildew prediction.*

The real power of Grad-CAM is systematic failure analysis: running it across all misclassified validation examples and inspecting the heat maps reveals recurring patterns. All Late Blight false positives concentrate heat on dark regions → shadow-disease confound. All Alternaria false negatives show heat on the leaf margin → model attends to the wrong region. Each pattern is diagnosed from Grad-CAM visualisations and points directly to a training data intervention. Grad-CAM converts 'validation accuracy on this class is 88%' into the actionable diagnosis 'the model is confusing shadows with lesions in 60% of the false positives.'

KEY TERMS — THIS SECTION

Term	Definition
Dataset bias	Systematic non-representativeness; the model learns spurious correlations not present in deployment
Spurious correlation	Statistical association between an irrelevant feature and a class label, present in training but not deployment
Domain shift	Mismatch between training and deployment distribution; causes systematic performance degradation
Adversarial example	Input with imperceptible structured perturbation causing confident wrong prediction
FGSM	Fast Gradient Sign Method: x_adv = x + ε·sign(∇_x L); single-step adversarial attack
Calibration	Alignment between model's expressed confidence and its true frequency of correctness
Temperature scaling	Post-hoc calibration dividing logits by scalar T>1 before softmax; reduces overconfidence
Grad-CAM	Gradient-weighted Class Activation Mapping; spatial heat map from last-layer feature map gradients

Channel importance αc_k	Spatial average of $\partial y_c / \partial A^k$; importance of feature map channel k to predicting class c
Systematic failure analysis	Using Grad-CAM across all misclassified examples to identify recurring patterns and training interventions

✎ **CHECK YOUR UNDERSTANDING**

1. AgriVision's training data: seasons 1–2 used older smartphones (disease labels evenly distributed); seasons 3–5 used newer phones (healthy labels disproportionately from seasons 1–2). (a) Identify one specific spurious correlation this creates. (b) Explain how it produces both false positives and false negatives in a specific pattern. (c) Propose a data collection protocol change for the next two seasons that would reduce this bias.

2. Apply one FGSM step to: two-class classifier outputs pre-softmax logits [2.1, −0.8] for input x. Correct class is 0. (a) Compute softmax outputs and current confidence. (b) Compute cross-entropy loss. (c) With $\nabla_x L = [0.3, -0.1, 0.2, -0.4]$ and $\varepsilon = 0.05$, compute x_adv. (d) Why is the perturbation likely to change the prediction without computing the forward pass?

3. A disease detection model reports 91% average maximum softmax probability but only 78% of predictions are correct. (a) Is this overconfident, underconfident, or well-calibrated? (b) In which direction should T move to improve calibration? (c) Would temperature scaling fix the shadow-disease confound from Section 2.9.1? Why or why not?

4. A Grad-CAM heat map for a correct Powdery Mildew prediction shows high activation (>0.8) at 7×7 positions (2,3), (2,4), (3,3), (3,4), near-zero elsewhere. (a) In the 224×224 image, which approximate pixel region does this correspond to? (b) The Healthy class heat map shows high activation at (1,1), (6,5), (6,6). What does this reveal about how the model distinguishes the classes? (c) How would you verify the high-activation region genuinely corresponds to the lesion?

5. [Challenge] Design a complete failure-mode evaluation pipeline to run at the end of each growing season before continued deployment. For each of the four failure modes: (a) specify the evaluation data required; (b) specify the metric and threshold that would trigger a model update; (c) specify the remediation action if the threshold is exceeded. Estimate the total annotation and collection cost per season for 150 farms.

2.10 Chapter Resolution and Bridge

2.10.1 AgriVision's Disease Detector: Architecture Chosen, Justified, Evaluated

Three months after the failed MLP experiment, AgriVision has a deployed disease detection system. The extension officer in Gadag submits a photograph through a field app on her phone. Within two seconds, the system responds: “Early-stage Powdery Mildew — 91% confidence. Apply copper-based fungicide within 48 hours. Additional farms within 1 km should be inspected.” By the following season, disease-related crop losses in the cooperative have fallen by 14%. This outcome was produced by the right inductive bias.

The chapter's arc is worth tracing precisely, because the logic of each step is the logic that applies whenever a new visual task arises.

- Step 1 — Diagnose the failure: the MLP achieved 58.7% not because it lacked parameters (38.6M) but because it encoded the wrong assumptions. Flat vector, no spatial structure, no translation equivariance. Adding more layers or more regularisation addresses symptoms, not cause.
- Step 2 — Identify required inductive biases: spatial locality and translation equivariance are structural properties of all natural images, not dataset-specific observations. Any architecture that does not encode them will fail on any image classification task at realistic scale.
- Step 3 — Build from requirements: local receptive fields (spatial locality), weight sharing (translation equivariance), pooling (spatial invariance), feature hierarchy (compositional structure). None of these choices is arbitrary—each follows directly from the data's structure.
- Step 4 — Extend with principled techniques: two-stage transfer learning (ImageNet → PlantVillage → AgriVision) improved accuracy from 89.3% to 95.2% using the same 6,000 labelled photographs. The improvement came from changing the starting point, not from collecting more data.
- Step 5 — Understand remaining failures: systematic failures (shadows mistaken for Late Blight, soil confounded with disease) are diagnosable with Grad-CAM and addressable with targeted training data. Temperature scaling calibrates confidence. Periodic retraining addresses seasonal shift.

Chapter Summary

Section	Core concept	Key insight
2.0	Inductive bias for images	Spatial locality + translation equivariance are structural properties of all natural images
2.1	Image tensor and context	A 224×224×3 image is 150,528 numbers in a spatial grid; this structure must be preserved, not flattened
2.2	Why MLPs fail on images	Parameter explosion, loss of spatial structure, translation sensitivity share one root cause: pixels treated as independent inputs
2.3	The convolution operation	f×f×C_in weights shared across H×W positions; output size = ⌊(H−f+2p)/s⌋+1 per dimension
2.4	Pooling	Max pooling selects strongest detection per window; GAP collapses any spatial map to C-dimensional vector with zero parameters
2.5	CNN architecture	Conv→BN→ReLU→Pool blocks; 981K parameters achieves 89.3% vs MLP's 58.7% on same data
2.6	Architecture evolution	LeNet→AlexNet→VGG→Inception→ResNet→EfficientNet: each solved its predecessor's specific failure mode
2.7	Visual tasks	Classification (what?), detection (where?), segmentation (which pixels?): match output to downstream decision
2.8	Transfer learning	Early layers transfer universally; feature extraction for small datasets, fine-tuning for larger ones; two-stage: 95.2%

2.9	CNN failure modes	Dataset bias, domain shift, adversarial vulnerability, overconfidence: systematic, diagnosable, and addressable

✦ THE AGRIVISION CONTEXT

Chapter 2 Resolution — AgriVision disease detection results

MLP (150,528→256→128→4): 58.7% | 8 hours

Five-block CNN from scratch: 89.3% | 45 min

ResNet-50 feature extraction: 86.8% | 5 min

ResNet-50 partial fine-tuning: 93.7% | 35 min

ResNet-50 PlantVillage → AgriVision (two-stage): 95.2% | 60 min

The cooperative has agreed to a second contract year. AgriVision has expanded to advisory services for 300 farms. The extension officer network now uses the app as standard protocol for suspected disease.

2.10.2 The Sequence Problem: Photographs Show a Moment, Sensors Show Time

The extension officer's photograph captures a single instant. The leaf has powdery mildew today. What the photograph cannot show is what happened in the nine days before—the creeping decline in soil moisture as the irrigation schedule fell behind, the temperature fluctuations as the approaching monsoon retreated, the accumulating water deficit that compromised the plant's systemic resistance, creating the conditions in which the fungal spore established and grew. By the time mildew is visible in a photograph, the disease is already established.

The ideal system would detect the preconditions for disease before infection takes hold—predicting which farms are at elevated risk in the coming week based on patterns in the preceding weeks' sensor readings. This information exists. AgriVision's sensor network has been streaming daily measurements from all 150 farms for five years: soil moisture, soil temperature, ambient temperature, humidity, daily rainfall, fertiliser applied, days since irrigation. 275,000 rows of sequential observations. Within these sequences, the preconditions for each recorded disease outbreak are written—if we can read them.

Reading them requires recognising that this data is fundamentally different from a leaf photograph. An image is a spatial grid: relevant structure is between neighbouring pixels. A sensor stream is a temporal sequence: relevant structure is between measurements taken at different points in time. Soil moisture on day 14 is more related to soil moisture on day 13 than to soil moisture on day 1. The patterns that precede disease are temporal patterns—trends, cycles, deviations from baseline.

An MLP applied to time-series data—even if we flatten 30 days × 7 features into a 210-dimensional vector—fails analogously to the image MLP: it cannot represent that day 14's moisture reading is causally downstream of day 13's, that a trend over days 8–14 is diagnostically more important than the value on day 14 alone, or that the same moisture value carries different meaning after five dry days than after a week of rain.

The soil moisture sensor at Farm 47 recorded 0.28 field capacity on the morning the extension officer photographed the mildew-infected leaf—the lowest reading in forty days. The record of everything that led to that value, day by day over the previous six weeks, is in the CSV file on AgriVision's server. Chapter 3 builds the model that can read it.

CHAPTER 3

Learning from Sequences

RNNs, LSTMs, and Temporal Pattern Recognition

3.0 The Inductive Bias for Sequential Data

3.0.1 The AgriVision Irrigation Prediction Problem

> ✦ THE AGRIVISION CONTEXT
>
> Farm 47, Dharwad district. This morning, the soil moisture sensor recorded 0.28 field capacity — the lowest reading in forty days. The previous irrigation was five and a half weeks ago. Ambient temperature has averaged 34°C all week.
>
> AgriVision's extension officer needs to answer one question: will this farm need irrigation in the next three days? A 'yes' means scheduling the pump operator today. A 'no' means visiting Farm 52 instead, where a pest risk is building.
>
> The disease detection CNN from Chapter 2 cannot answer this question. It looks at one photograph and classifies it. But irrigation need is not visible in a single photograph — it is encoded in the trend of sensor readings over the past thirty days.

The answer is in AgriVision's sensor database. Five years of daily readings from 150 farms: soil moisture, soil temperature, ambient temperature, humidity, rainfall, fertiliser applied, days since irrigation. Each sequence of 30 consecutive days from one farm is a window of context that precedes a known outcome — irrigation applied, or not. The task is to learn from those sequences.

A team member suggests the same approach that worked for yield prediction in Chapter 1: flatten the 30-day window into a single long vector and train an MLP. The result: 71.4% validation accuracy. Section 3.2 explains exactly why this approach hits a ceiling. First, we need to understand what makes sequential data different.

3.0.2 Temporal Ordering: The Past Causes the Present

There is something special about the order of observations in a time series: earlier observations cause later ones. Soil moisture on day 14 is lower than day 13 because the soil dried out overnight. This causal direction only runs one way — forward in time, never backward.

When we flatten a 30-day sensor window into a 210-number vector, we preserve the values but break the causal structure. The MLP sees 210 numbers in no particular order and has no mechanism to represent that number 8 (day 2, soil moisture) caused number 1 (day 3, soil moisture). The sequence has been flattened into a list.

◆ KEY CONCEPT

Temporal ordering as inductive bias

In a time series, the order of observations is causal: the value at time t is caused by values at times t−1, t−2, and so on. A model for sequences must process inputs in time order and carry information forward from one step to the next.

An architecture that encodes this must: (1) read inputs one step at a time in order; (2) keep a running memory that accumulates what it has seen; and (3) use that accumulated memory to inform every future step.

3.0.3 Local Temporal Patterns and Long-Range Dependencies

Consider what actually happens before a farm needs irrigation. Typically, soil moisture declines steadily over several days. There may be a rainfall event two weeks ago that briefly raised moisture before the decline resumed. The temperature may be elevated this week. These patterns span different time scales.

The steep 3-day moisture drop is a local temporal pattern — it fits within a small window. The rainfall event 15 days ago is a long-range dependency — the model needs to remember something that happened far back to understand the current situation. Both matter for predicting irrigation need. Both are invisible to an MLP that treats all 210 positions as independent inputs.

3.0.4 How These Properties Motivate the RNN

The Recurrent Neural Network addresses both problems directly. It reads one day's sensor values at a time, left to right. After reading each day, it updates a hidden state — a compact numerical summary of everything it has read so far. On day 30, the hidden state captures the full 30-day history in a compressed form, and that summary is used to make the irrigation prediction.

The LSTM and GRU extend this idea with gating mechanisms that decide what to remember and what to forget. The 1D CNN offers a different approach: instead of reading step by step, it applies filters across the time dimension, much like the Chapter 2 CNN applied filters across image space.

? FOUR QUESTIONS TO CARRY THROUGH THIS BOOK

Question 1: What problem does this architecture solve?

The failure of feedforward networks on sequential data: they cannot process inputs in order, carry memory across steps, or share parameters across time positions.

Question 2: What inductive bias does it encode?

Temporal ordering (inputs are processed in causal sequence) and sequential memory (a hidden state accumulates information from all past inputs).

Question 3: When does this architecture fail?

Very long-range dependencies where the memory still degrades; non-stationary data where the distribution shifts over time; irregular or missing sensor readings. Section 3.8 covers these failure modes.

Question 4: When would you choose something else?

Text data with very long-range semantic dependencies — Chapter 4's Transformer handles those better. Spatial data — Chapter 2's CNN. Short tabular records with no temporal structure — Chapter 1's MLP may suffice.

KEY TERMS — THIS SECTION

Term	Definition
Temporal ordering	Property of time series: observations have a causal direction — t follows from t−1
Long-range dependency	A pattern where a current time step depends on something that happened many steps earlier
Sequential memory	A model's ability to carry relevant information forward across many time steps
RNN	Recurrent Neural Network: processes sequences one step at a time, maintaining a hidden state
Hidden state	The internal memory vector h_t that accumulates information from all previous time steps

✎ **CHECK YOUR UNDERSTANDING**

1. Farm 47's soil moisture on day 30 is 0.28. The MLP receives this as the first number in a 210-dimensional vector. The LSTM reads it as input at time step t=30 after processing 29 previous days. What additional information can the LSTM use when making its prediction that the MLP cannot? Why does this matter for the irrigation task?

2. A colleague says: 'We can give the MLP full temporal context by including lagged features — soil moisture at t−1, t−2, ..., t−30 as separate inputs.' Name two specific limitations of this approach compared to an LSTM.

3. Describe a scenario in AgriVision's sensor data where a long-range dependency (something from more than 20 days ago) is relevant to today's irrigation decision. What architectural property allows a model to use information from that far back?

4. Apply the four questions to a sequence model for ECG arrhythmia detection (input: 5-second heart rate signal at 250 Hz). What task formulation is appropriate? What inductive biases does the data require?

5. The bridge from Chapter 2 says: photographs show one moment; sensors show 40 days. In one paragraph, explain why this distinction requires a different architecture family, connecting to the inductive bias framework from Section 1.0.

6. [Challenge] AgriVision wants to predict irrigation need 7 days ahead instead of 3. How does this change the input window needed, the label definition, and the class balance of the training set?

3.1 Time-Series Data as a Deep Learning Context

3.1.1 The Sequence as a Tensor

A single sensor record for one farm on one day has 7 values: soil moisture, soil temperature, ambient temperature, humidity, daily rainfall, fertiliser applied, days since irrigation. A 30-day window stacks 30 such records into a grid — 30 rows (days) and 7 columns (features). In PyTorch this is a tensor of shape (30, 7). A batch of 32 such windows has shape (32, 30, 7).

All seven features are normalised before training — but with one important rule: compute the mean and standard deviation from the training data only, then apply those numbers to the validation and test sets. Computing normalisation statistics from the full dataset (including the test set) is data leakage — the model indirectly sees test-set information before prediction.

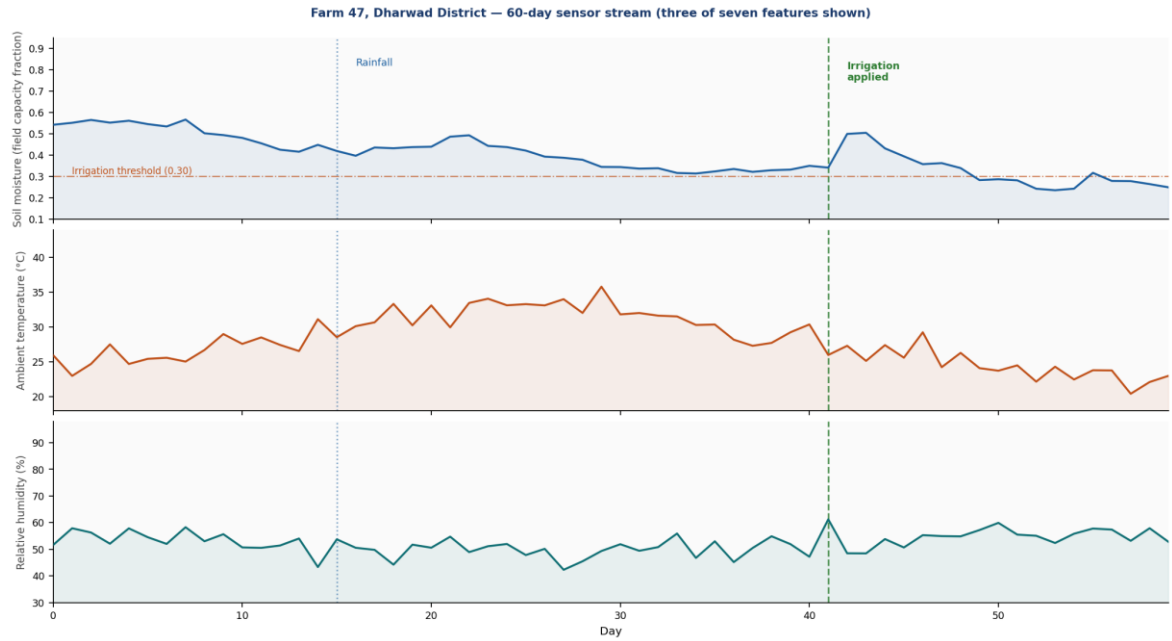


Figure 3.1 *Farm 47, Dharwad district — 60-day sensor stream for three of the seven features. Soil moisture (top) declines steadily until irrigation at day 41 (green dashed). A small rainfall at day 15 (blue dotted) provides brief relief. The orange dash-dot line at 0.30 field capacity marks the irrigation trigger threshold. The full input to the sequence model is a (30, 7) tensor of normalised daily readings.*

3.1.2 Stationarity, Seasonality, and Trend

AgriVision's sensor data is not static — it changes across the year in predictable ways. Soil moisture is higher during the monsoon (June–September) and lower in the dry season (November–May). Temperature follows an annual cycle. These repeating patterns are called seasonality.

Superimposed on seasonal cycles are long-term trends: average temperatures may be slowly rising year on year. And superimposed on trends are random fluctuations from day to day. A time series that has all three — seasonal patterns, trend, and random noise — is non-stationary: its statistical properties change over time. A sequence model trained on years 1–4 and tested on year 5 may encounter a slightly shifted distribution.

3.1.3 Temporal Resolution and Window Size

Temporal resolution is the gap between consecutive readings. AgriVision sensors record daily — one reading per day per farm. Daily resolution is appropriate for irrigation decisions, which operate on a day-to-day cycle. A finer resolution (hourly) would capture more detail but would make sequences 24 times longer with no clear benefit for this particular task.

Window size T is how many days of history the model uses. AgriVision's agronomists estimate that the last 21–30 days contain most of the relevant signal for irrigation prediction. We use T = 30. Longer windows can be tested empirically, but there is a risk of diluting the signal with irrelevant early history.

3.1.4 Sources of Variation in Agricultural Sensor Data

- Seasonal variation: soil moisture in July looks very different from February, even on the same farm. The model must learn that the same raw value means different things in different seasons.
- Farm-to-farm variation: different soil types, crop varieties, and irrigation practices mean one farm's baseline readings are higher or lower than another's. The model must generalise across 150 different farms.
- Sensor noise and drift: over time, soil moisture sensors lose calibration. What reads 0.50 in year 1 might read 0.46 by year 4 for the same true moisture. This slow drift looks like a downward trend but is an instrument artefact.
- Missing data: sensors go offline due to power failures or physical damage. A 30-day window may have 2–3 missing readings. Standard approach: fill forward using the most recent valid value.

KEY TERMS — THIS SECTION

Term	Definition
Sequence tensor	Shape (T, F) for one window; (N, T, F) for a batch of N windows
Window size T	Number of past time steps used as model input
Stationarity	Statistical property of a series whose mean and variance do not change over time
Seasonality	Periodic patterns that repeat on a known cycle (annual, weekly, daily)
Sensor drift	Slow calibration shift producing a spurious trend in recorded sensor values

✎ **CHECK YOUR UNDERSTANDING**

1. AgriVision's input batch has shape (32, 30, 7). Identify each dimension. After a many-to-one LSTM with hidden size 64, what is the shape of (a) the final hidden state, and (b) the output after a sigmoid linear layer?

2. A student normalises each individual 30-day window to zero mean and unit variance before feeding it to the LSTM. What specific information does this destroy? Give an example where absolute soil moisture level is important.

3. Figure 3.1 shows a rainfall event at day 15 and irrigation at day 41. Why would an MLP on the flattened window struggle to identify the rainfall event as a recognisable temporal pattern? What must the LSTM learn to do that?

4. The Dharwad dry season is November–May; the monsoon is June–September. A model trained on all seasons is evaluated only on June predictions. Describe one way seasonal distribution shift could cause systematic errors.

5. [Challenge] AgriVision wants to extend to Tamil Nadu farms where the monsoon falls in October–December. What distribution shift would a Dharwad-trained model encounter? Propose a training strategy robust to both monsoon windows.

3.2 Why Feedforward Networks Struggle with Sequences

3.2.1 The Fixed-Input-Length Problem

An MLP needs a fixed-size input. For a 30-day sensor window, we flatten 30 days × 7 features = 210 values into one vector. That works — but it raises an uncomfortable question straight away: why 30 days? If a key event happened 35 days ago, the model never sees it. If 15 days would be enough, we have padded with irrelevant history. The window size is a fixed architectural decision, not something the model can learn.

And if we want to try a 45-day window instead of 30, we need to rebuild the first layer of the MLP from scratch. A different window size means a different architecture. The RNN sidesteps this entirely — it processes one day at a time and can handle any sequence length with the same weights.

3.2.2 No Memory: Each Forward Pass Is Stateless

Every time an MLP makes a prediction, it starts fresh. It receives the current 210-number vector and produces a number — with no knowledge of what it processed a moment ago. There is no memory of previous inputs.

This matters because the most diagnostic patterns in irrigation prediction are changes over time, not absolute values on a single day. 'Soil moisture has been falling for ten days at an accelerating rate' is enormously informative. But that pattern exists across 10 rows in the sensor table. The MLP sees those 10 rows simultaneously as 70 numbers in a flat vector, with no architecture for representing 'this happened before that.'

3.2.3 No Parameter Sharing Across Time

This is the deepest structural problem. In the flattened input, position 0 always means 'soil moisture on day 1' and position 7 always means 'soil moisture on day 2.' The MLP learns a separate weight to connect each of these positions to each hidden neuron — meaning the pattern 'soil moisture is below 0.30 field capacity' must be re-learned independently for every one of the 30 days in the window.

A detector that flags low soil moisture on day 15 has no relationship in the MLP to a detector that flags low moisture on day 22. They are completely independent weights, each trained on a small subset of the training data. This is why MLP accuracy on sequences plateaus: there are not enough examples per time position to train reliable detectors.

> ✎ **WORKED EXAMPLE**
>
> **Parameter count: MLP vs LSTM for the irrigation prediction task**
>
> MLP: 210 → 128 → 64 → 1
>
> Parameters: 210×128 + 128 = 26,880 | 128×64 + 64 = 8,256 | 64×1 + 1 = 65
>
> Total: 35,201 → 167 parameters per training window (6,000 windows)
>
> LSTM (hidden size 64, F=7): 4 × (7+64) × 64 + 4×64 = 18,432 shared parameters

The MLP uses 35K parameters, each seeing only one time position. The LSTM uses 18K parameters, each applied at all 30 time steps. The LSTM learns more from fewer parameters because it shares them across time.

3.2.4 What We Need: Ordered Memory with Shared Parameters

All three failures point to the same solution. We need a model that reads inputs in time order, maintains a running memory across steps, and applies the same learned parameters at every step — so that detecting low soil moisture on day 5 and detecting it on day 25 use exactly the same detector.

These are the three defining properties of the Recurrent Neural Network. They follow directly from the structure of sequential data, just as local receptive fields and weight sharing in the CNN followed directly from the spatial structure of images.

KEY TERMS — THIS SECTION

Term	Definition
Fixed-input-length constraint	MLPs need a predefined input size; different window lengths require different architectures
Stateless forward pass	Each MLP computation is independent of all previous inputs; no memory between calls
No temporal weight sharing	Independent weights for each time position; the same pattern must be re-learned at every step
Recurrent connection	A connection from a neuron's output at time t back to its input at time t+1

✎ **CHECK YOUR UNDERSTANDING**

1. AgriVision switches from 30-day to 45-day windows. (a) What changes in the MLP architecture? (b) What changes in the LSTM architecture? (c) What does this tell you about which model is better suited to variable-length sequences?

2. An MLP receives sensor readings $[x_1, x_2, ..., x_{30}]$ flattened. The same sequence shifted one day: $[x_2, x_3, ..., x_{30}, 0]$ produces a different prediction. Why? What property of the architecture causes this shift sensitivity?

3. Section 3.2.3 says the MLP must re-learn the 'low soil moisture' detector at every time position. With 30 positions, how many independent 'detectors' would the MLP need? How many does the LSTM need for the same capability?

4. Compare parameter counts: MLP 210→128→64→1 vs LSTM hidden size 64. Which has more parameters? Does the model with more parameters perform better here?

5. [Challenge] A student zero-pads short sequences to length T_max for MLP input. (a) What two failure modes does this create? (b) Is there any situation where zero-padding produces the same result as LSTM processing of the original sequence?

3.3 The Recurrent Neural Network

3.3.1 The Hidden State as Memory

Think of the RNN as a reader who keeps a notepad while reading a book. After each sentence, they update their notepad — crossing out old notes, adding new ones — before moving to the next sentence. By the end of the chapter, their notepad is a compressed summary of everything they have read.

The RNN does exactly this with AgriVision's 30-day sensor data. It reads one day's readings at a time. After each day, it updates a vector of numbers called the hidden state — a compact summary of everything seen so far. The hidden state at time step t is written h_t and computed as:

$$h_t = tanh(W_x\, x_t + W_h\, h_{t-1} + b)$$

Let's read this equation in plain English. $W_x\, x_t$ is 'what today's sensor readings contribute.' $W_h\, h_{t-1}$ is 'what the previous summary contributed.' Their sum, plus a bias b, is passed through tanh to squash the result to the range [−1, 1]. The result is the new summary h_t.

The critical detail: W_x, W_h, and b are the same at every time step. The same weights that process day 1's readings also process day 30's readings. This is parameter sharing across time — the property the MLP lacked.

At the end of 30 steps, the final hidden state h_{30} is a fixed-size vector summarising the entire 30-day sequence. A linear layer followed by a sigmoid turns this summary into the binary irrigation prediction.

3.3.2 The Unrolled Computation Graph

Figure 3.2 shows the RNN drawn out step by step across four time steps. Each column is one time step: the blue circle receives the day's sensor readings (x_t), the green circle is the hidden state (h_t), and the orange arrows carry the hidden state forward from one step to the next. All orange arrows share the same weight matrix W_h.

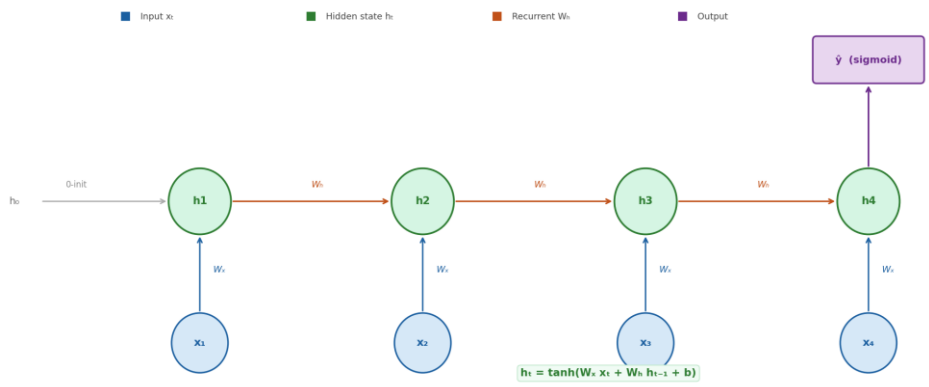


Figure 3.2 *The RNN unrolled across 4 time steps. Input nodes (blue) receive x_t at each step. Hidden state nodes (green) update after each step using both the current input and the previous hidden state. Orange arrows (W_h) carry memory forward; all orange arrows share identical weights. The final hidden state h_4 goes to the purple sigmoid output for the binary irrigation prediction.*

3.3.3 Backpropagation Through Time (BPTT)

Training the RNN works just like training any other neural network — compute the loss, compute gradients, update weights. The only difference is that the unrolled RNN spans 30 time steps, so gradients must travel backwards through all 30 steps. This is called Backpropagation Through Time (BPTT).

Each backward step passes through one tanh activation and one multiplication by the weight matrix W_h. The chain rule multiplies these together at every step. Over 30 steps, the gradient reaching day 1 is the product of 30 such multiplications.

3.3.4 The Vanishing Gradient Problem

Here is the critical problem. The tanh function's derivative is always between 0 and 1. Multiplying a gradient by a number less than 1 at every backward step makes it shrink. After 30 steps of multiplying by roughly 0.8, the gradient is $0.8^{30} \approx 0.001$ of its original size — nearly zero.

What does this mean in practice? The weights that encode 'what happened on day 1 matters for today's prediction' receive almost no gradient signal. The RNN can reliably learn from the last 5–10 days. Events before that become harder and harder to use. For AgriVision, this means the model struggles to use the rainfall event at day 15 when predicting on day 30.

> ⚠ **IMPORTANT NOTE**
>
> **Vanishing gradient is a learning failure, not a representation failure.**
>
> A vanilla RNN has the mathematical capacity to remember information from day 1. The problem is not what the architecture can represent — it is what it can learn. The gradient signal for connections at early time steps shrinks to near zero, so those connections never receive a meaningful update during training.
>
> The solution is not to change what the RNN represents. It is to change how gradients flow. The LSTM's cell state (Section 3.4) provides a gradient pathway where this shrinkage does not occur.

3.3.5 Worked Example: RNN Forward Pass

> ✎ **WORKED EXAMPLE**
>
> **Two steps of an RNN forward pass (F=2 features, H=3 hidden units)**
>
> Features: soil moisture (normalised) and soil temperature (normalised)
>
> Initial hidden state: $h_0 = [0, 0, 0]$ (zero-initialised)
>
> Weights: $W_x = [[0.4, 0.2], [-0.1, 0.5], [0.3, -0.3]]$ (H×F = 3×2)
>
> $W_h = [[0.1, 0.0, -0.1], [0.2, 0.3, 0.0], [-0.1, 0.1, 0.2]]$ (H×H = 3×3)
>
> **Step 1 (t=1): $x_1 = [0.65, 0.83]$ (day 1: moderate moisture, warm)**
>
> $z_1 = W_x x_1 + W_h h_0 + b = W_x x_1$ (since $h_0 = 0$ and $b = 0$)
>
> $z_1[0] = 0.4 \times 0.65 + 0.2 \times 0.83 = 0.260 + 0.166 = 0.426$
>
> $z_1[1] = -0.1 \times 0.65 + 0.5 \times 0.83 = -0.065 + 0.415 = 0.350$

$z_1[2] = 0.3\times0.65 + (-0.3)\times0.83 = 0.195 - 0.249 = -0.054$

$h_1 = \tanh([0.426, 0.350, -0.054]) = [0.401, 0.337, -0.054]$

Step 2 (t=2): $x_2 = [0.42, 0.79]$ (day 2: moisture declining)

Now the previous summary h_1 contributes via $W_h\, h_1$:

$z_2[0] = 0.4\times0.42 + 0.2\times0.79 + 0.1\times0.401 + 0.0\times0.337 + (-0.1)\times(-0.054)$

$= 0.168 + 0.158 + 0.040 + 0.000 + 0.005 = 0.371$

$h_2[0] = \tanh(0.371) = 0.355$

h_2 now encodes both x_1 (via h_1) and x_2. The weights W_x and W_h are identical to those used in step 1 — this is parameter sharing across time.

KEY TERMS — THIS SECTION

Term	Definition
Hidden state h_t	The RNN's running memory; updated at each step as $h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$
W_x	Weight matrix mapping input x_t to hidden space; shape (H, F)
W_h	Recurrent weight matrix; carries memory forward; shape (H, H); shared across all time steps
BPTT	Backpropagation Through Time: gradients travel backward through all T unrolled steps
Vanishing gradient	Gradient shrinks exponentially with distance in time; limits what the RNN can learn from the distant past

✎ **CHECK YOUR UNDERSTANDING**

1. A vanilla RNN has hidden size 64 and input size 7. (a) Parameters in W_x? In W_h? (b) If the sequence length changes from 30 to 60 days, do any parameter counts change? (c) Compare to the MLP from Section 3.2. Which has more parameters?

2. In the worked example, both steps use the same W_x and W_h. A student says: 'But the computation at step 2 differs because $h_1 \neq h_0$ — so the model effectively uses different transformations at each step.' Is this correct? Explain clearly.

3. With T=30 and λ_max=0.85, compute the gradient attenuation factor from step 30 to step 0. At what step does the gradient fall below 5% of its value at step 30?

4. Section 3.3.4 says the RNN cannot learn to use the rainfall event at day 15 when predicting on day 30. Explain why using BPTT gradient flow.

5. Derive $\partial h_t / \partial h_{t-1}$ for the vanilla RNN. Under what conditions does the gradient norm exceed 1 (exploding) or fall below 1 (vanishing)?

6. [Challenge] Gradient clipping caps $\|\nabla L\|$ whenever it exceeds a threshold. (a) How does this address exploding gradients? (b) Does it address vanishing gradients? (c) What is the practical effect on learning from the beginning of a 30-day sequence?

3.4 Long Short-Term Memory

3.4.1 Adding a Long-Term Notebook

The vanilla RNN has one memory system: the hidden state. It is updated at every step, which means recent information tends to overwrite older information. Useful patterns from 25 days ago get diluted by 25 updates.

The Long Short-Term Memory network (LSTM) adds a second memory system: a cell state, which acts like a long-term notebook. The notebook travels alongside the hidden state across all time steps. Importantly, it is updated by addition rather than by multiplication through non-linear functions — so gradients can flow backward through it without shrinking at each step.

Three decisions control what happens to the notebook at each step:

- Forget: how much of the old notebook entry to keep?
- Write: what new information should be added to the notebook?
- Read: which part of the notebook should feed into the output this step?

Each of these decisions is made by a gate: a set of numbers between 0 and 1, computed from the current input and the previous hidden state. A gate value of 0 means 'block completely.' A gate value of 1 means 'pass through entirely.' The network learns what gate values to produce for each situation by gradient descent.

3.4.2 The Cell State: A Gradient Highway

The cell state c_t is updated by a simple equation:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{g}_t$$

Read it as: 'Keep a fraction f_t of the old notebook, then add $i_t \times \tilde{g}_t$ of new information.' The $\odot$ symbol means element-wise multiplication.

During backpropagation, the gradient of the loss with respect to the previous cell state passes through this equation as:

$$\partial c_t / \partial c_{t-1} = f_t \quad \textit{(element-wise — not a matrix multiplication through tanh)}$$

If the forget gate is close to 1 at most time steps — if the model chooses not to forget — the gradient travels backward almost unchanged. This is the key difference from the vanilla RNN, where the gradient passes through a tanh and a matrix multiplication at every step and shrinks each time.

3.4.3 The Forget Gate, Input Gate, and Output Gate

Each gate takes the same input: the concatenation of the previous hidden state h_{t-1} and the current input x_t. We write this as $[h_{t-1}, x_t]$. Then a learned weight matrix and sigmoid activation produce the gate values.

Forget gate f_t (erase decision). $f_t = \sigma(W_f [h_{t-1}, x_t] + b_f)$. When $f_t \approx 1$: keep all of the old notebook entry. When $f_t \approx 0$: erase the old notebook entry. AgriVision example: when a large rainfall event arrives, the forget gate may learn to reset the drought-accumulation counter in the cell state.

Input gate i_t (write decision). $i_t = \sigma(W_i [h_{t-1}, x_t] + b_i)$. Controls how much of the new candidate value $\tilde{g}_t$ to actually write. The candidate itself is: $\tilde{g}_t = \tanh(W_g [h_{t-1}, x_t] + b_g)$. Think of i_t as 'how important is today's reading?' and $\tilde{g}_t$ as 'what today's reading says.'

Output gate o_t (read decision). $o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$. The visible hidden state is then: $h_t = o_t \odot \tanh(c_t)$. The cell state may remember many historical events; the output gate selects which of those are relevant for the current step's prediction.

3.4.4 LSTM Equations: All Six in One Place

The complete update for one time step, given x_t, h_{t-1}, and c_{t-1}:

$$f_t = \sigma(W_f [h_{t-1}, x_t] + b_f) \quad \leftarrow \textit{forget gate (erase decision)}$$

$$i_t = \sigma(W_i [h_{t-1}, x_t] + b_i) \quad \leftarrow \textit{input gate (write decision)}$$

$$\tilde{g}_t = \tanh(W_g [h_{t-1}, x_t] + b_g) \quad \leftarrow \textit{candidate value to write}$$

$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o) \quad \leftarrow \textit{output gate (read decision)}$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{g}_t \quad \leftarrow \textit{update the long-term notebook}$$

$$h_t = o_t \odot \tanh(c_t) \quad \leftarrow \textit{visible hidden state output}$$

Each gate weight matrix (Wf, Wi, Wg, Wo) has shape H×(H+F). For AgriVision with H=64 and F=7: each matrix has 64×71 = 4,544 weights, plus 64 bias terms. Four gates total: 4 × (4,544 + 64) = 18,432 parameters, shared across all 30 time steps.

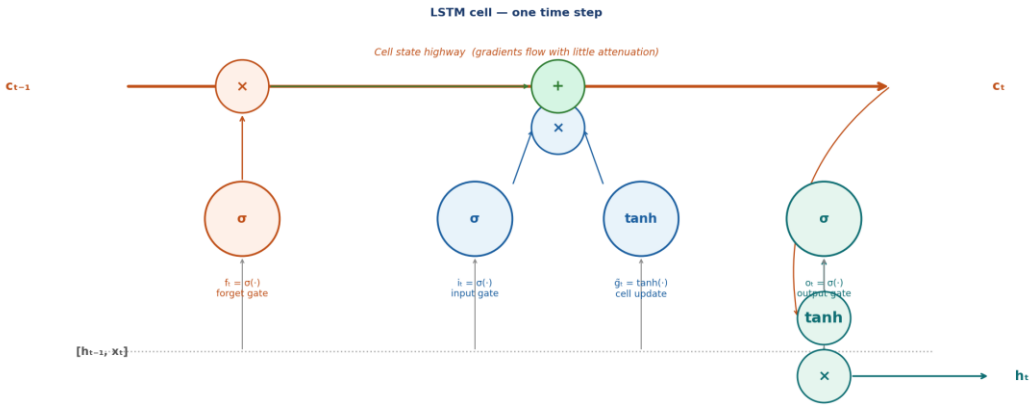


Figure 3.3 *LSTM cell for one time step. The orange highway at the top is the cell state (the long-term notebook), updated only by addition and element-wise multiplication. The three gates (forget, input, output) each receive $[h_{t-1}, x_t]$ from the bottom rail and produce values in [0,1] that control what is erased, written, and read from the notebook. The gradient of c_t with respect to c_{t-1} equals f_t — a direct element-wise value, not a product of matrix multiplications.*

3.4.5 Worked Example: One LSTM Step

> ✎ **WORKED EXAMPLE**
>
> **One LSTM step (F=2, H=3, simplified weights for illustration)**

Previous state: h_{t-1} = [0.401, 0.337, −0.054] c_{t-1} = [0.420, 0.350, −0.055]

Current input: x_t = [0.42, 0.79] Concatenated input: 5 values

Forget gate (first component only):

$f_t[0] = \sigma(0.1\times0.401 + 0.0\times0.337 + (-0.1)\times(-0.054) + 0.3\times0.42 + 0.1\times0.79)$

$= \sigma(0.040 + 0 + 0.005 + 0.126 + 0.079) = \sigma(0.250) = 0.562$

→ 56% of the old notebook entry is kept.

Cell state update (first component):

$i_t[0] = 0.550$ (input gate, computed similarly)

$\tilde{g}_t[0] = \tanh(-0.05) = -0.050$ (candidate value)

$c_t[0] = 0.562 \times 0.420 + 0.550 \times (-0.050) = 0.236 - 0.028 = 0.208$

The gradient advantage:

$\partial c_t[0]/\partial c_{t-1}[0] = f_t[0] = 0.562$

Compare to the vanilla RNN gradient at the same step, which would pass through $\tanh'(z) \times W_h$ and could easily be 0.3 or smaller. Over 15 steps backward: $0.562^{15} \approx 0.003$ vs $0.3^{15} \approx 0.0000000143$. The LSTM gradient is 200× larger.

KEY TERMS — THIS SECTION

Term	Definition
Cell state c_t	LSTM long-term memory; updated additively; gradients flow backward as $\partial c_t/\partial c_{t-1} = f_t$
Forget gate f_t	$\sigma(\cdot)$ in [0,1]; decides what fraction of c_{t-1} to retain
Input gate i_t	$\sigma(\cdot)$ in [0,1]; decides how much of the candidate $\tilde{g}_t$ to write to the notebook
Cell update $\tilde{g}_t$	$\tanh(\cdot)$ in [−1,1]; the proposed new information to write
Output gate o_t	$\sigma(\cdot)$ in [0,1]; decides which part of the notebook to expose as h_t
Gradient highway	Cell state pathway where $\partial c_t/\partial c_{t-1} = f_t$; avoids the exponential shrinkage of vanilla RNN

✎ **CHECK YOUR UNDERSTANDING**

1. LSTM with H=64, F=7. (a) Shape of each gate weight matrix? (b) Total parameters in the LSTM cell? (c) Compare to vanilla RNN with same H and F.

2. The forget gate produces $f_t \approx 0$ at one time step. What happens to c_t? What AgriVision event might trigger this, and why is it useful behaviour?

3. A student claims the LSTM is more powerful simply because it has four weight matrices. Is this the correct explanation for why it handles long sequences better? Give the correct explanation.

4. Using the worked example values, compute the gradient $\partial c_t / \partial c_{t-1}$ and compare to the vanilla RNN gradient at the same activations. Which degrades less over 15 backward steps?

5. The LSTM uses $[h_{t-1}, x_t]$ as input to all four gates. An alternative only uses x_t for $\tilde{g}_t$ and only h_{t-1} for the gates. What capability does the standard design have that this alternative lacks? Give an AgriVision example.

6. [Challenge] A peephole LSTM adds c_{t-1} as input to the forget and input gates. (a) What advantage does direct cell access provide? (b) How many extra parameters for H=64, F=7?

3.5 Gated Recurrent Units

3.5.1 A Simpler Version of the Same Idea

The LSTM uses two state vectors (cell state and hidden state) and four gates. A natural question: do we need all of this complexity? The Gated Recurrent Unit (GRU) answers 'probably not for most tasks.' It merges the cell state and hidden state into one vector, and merges the forget and input gates into one update gate. The result is simpler to implement, slightly faster to train, and produces similar accuracy on most sequence tasks.

3.5.2 GRU Equations

The GRU has two gates and one state vector:

$$z_t = \sigma(W_z\,[h_{t-1}, x_t]) \qquad \leftarrow \textit{update gate (replaces forget + input gates)}$$

$$r_t = \sigma(W_r\,[h_{t-1}, x_t]) \qquad \leftarrow \textit{reset gate}$$

$$\tilde{h}_t = \tanh(W\,[r_t \odot h_{t-1}, x_t]) \quad \leftarrow \textit{candidate hidden state}$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad \leftarrow \textit{final update}$$

The update gate z_t directly interpolates between the previous state and the candidate. When $z_t \approx 0$: keep the previous state unchanged — nothing new to add. When $z_t \approx 1$: replace with the candidate entirely. The reset gate r_t controls how much of the previous state the candidate computation can see. When $r_t \approx 0$: compute the candidate fresh, ignoring history.

Parameter count for H=64, F=7: three matrices each of size 64×71 = 4,544, plus three bias vectors of 64. Total: 3×(4,544+64) = 13,824 parameters — 25% fewer than the LSTM's 18,432.

3.5.3 LSTM vs GRU: When to Choose Which

Property	LSTM	GRU
Parameters (H=64, F=7)	18,432	13,824
State vectors	Two: cell state + hidden state	One: hidden state only
Gates	Four: forget, input, output, cell update	Two: update, reset
Long-range dependency	Excellent (cell state highway)	Good (similar gradient flow)
Training speed	Slightly slower	~25% faster (fewer parameters)
Default recommendation	Complex sequences, larger datasets	Speed or data efficiency priority

Table 3.1 *LSTM vs GRU. In practice the accuracy difference is usually small; GRU is preferred when training time or data efficiency matters.*

For AgriVision's 30-day irrigation prediction with a relatively small dataset, the GRU is the more data-efficient choice. Both substantially outperform the vanilla RNN. For sequences longer than 100 steps or tasks with complex multi-scale patterns, the LSTM's explicit cell state may give a small advantage.

KEY TERMS — THIS SECTION

Term	Definition
Update gate z_t	GRU gate interpolating between previous state and candidate; replaces LSTM's forget+input gates
Reset gate r_t	GRU gate controlling how much previous state feeds into the candidate computation
GRU	Gated Recurrent Unit; two gates, one state vector; 25% fewer parameters than LSTM

✎ **CHECK YOUR UNDERSTANDING**

1. When the GRU update gate $z_t \approx 0$ at many consecutive steps, what is it doing? Describe the equivalent LSTM gate configuration.

2. Compute the parameter count for LSTM and GRU with H=128, F=7. Is the saving significant at this size?

3. The reset gate $r_t \approx 0$ means the candidate is computed ignoring past state. Describe an AgriVision sensor event where starting fresh is the right response.

4. Both LSTM and GRU address vanishing gradients. Explain the mechanism in each case.

5. [Challenge] A minimal GRU removes the reset gate: $h_t = (1-z_t) \odot h_{t-1} + z_t \odot \tanh(W[h_{t-1}, x_t])$. (a) How many parameters does this save? (b) What temporal pattern can the standard GRU represent but this version cannot?

3.6 1D Convolutional Networks for Sequences

3.6.1 The Temporal Filter: Convolution Over Time

Chapter 2 applied 2D convolution to images, sliding a spatial filter over height and width. The same operation applied to a sequence slides a 1D filter over the time dimension. A 1D convolutional filter of size f operates on f consecutive time steps, computing a weighted sum of the F features at those steps:

$$z[t] = \Sigma_{i=0} \Sigma_k W[i,k] \cdot x[t+i, k] + b \quad \leftarrow for\ t = 0, 1, ..., T\text{-}f$$

where x has shape (T, F_in), W has shape (f, F_in, F_out), and the output z has shape (T', F_out) where T' is determined by the output dimension formula. Multiple filters (F_out) detect different temporal patterns simultaneously — exactly as multiple 2D filters detect different spatial patterns in CNNs.

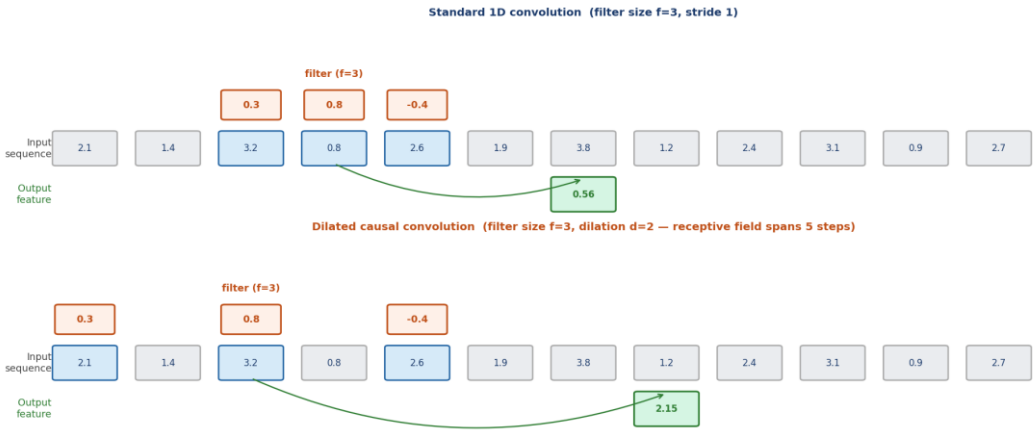


Figure 3.5 *1D convolution over a temporal sequence. Top: a filter of size f=3 slides across 12 time steps, computing a weighted sum at each position. The highlighted time steps contribute to one output value (green box). Bottom: dilated causal convolution with dilation d=2, where the same filter size spans 5 effective time steps by skipping intermediate positions. Dilated convolutions expand the temporal receptive field exponentially with depth without increasing parameter count.*

3.6.2 Output Dimensions and Receptive Field

The output dimension formula for 1D convolution is the same as 2D, applied to the time dimension:

$$T' = \lfloor (T - f + 2p) / s \rfloor + 1$$

For a many-to-one architecture (predicting a single binary label from the sequence), the output needs to be a single vector. After the final 1D conv layer, Global Average Pooling across the time dimension reduces (T', F_out) to a (F_out,) vector — the temporal analogue of GAP over the spatial dimensions in image CNNs.

The receptive field of a 1D conv layer grows linearly with depth: L layers of filter size f have receptive field L×(f−1)+1. For f=3 and 5 layers: receptive field = 11 time steps. To cover T=30 time steps, 15 layers of f=3 convolutions would be needed — making 1D CNNs less parameter-efficient than LSTMs for long temporal context.

3.6.3 Dilated Causal Convolutions

Dilated convolutions solve the receptive field problem by inserting gaps between filter positions. A filter of size f with dilation rate d has an effective receptive field of f+(f−1)×(d−1), covering d×(f−1)+1 time steps with the same number of parameters as a standard f-tap filter.

Stacking dilated convolutions with exponentially increasing dilation rates (d=1, 2, 4, 8, ...) achieves logarithmic growth of the receptive field with depth: 8 layers of f=3 with doubling dilation rates produce a receptive field of $2^8 \times 2 = 512$ time steps — far beyond what 8 standard layers would cover. This is the key innovation of the Temporal Convolutional Network (TCN).

Causal constraint. For prediction tasks where the model must not see the future, all convolutions must be causal: position t can only attend to positions t, t−1, t−2, ... t−(f−1). Causal 1D convolution pads the left side of the input with zeros (or the first value) and uses no right padding, ensuring that the output at time t is computed from past inputs only. For AgriVision's irrigation prediction, causal constraint is mandatory: predicting irrigation need using future sensor readings would not be a valid model.

3.6.4 1D CNN vs LSTM: A Practical Comparison

Both 1D CNN and LSTM can model temporal sequences, but with different strengths:

Property	1D CNN (TCN)	LSTM
Parallelism	Fully parallel — all time steps processed simultaneously	Sequential — step t depends on step t−1
Training speed	Fast (GPU parallelism)	Slower (sequential computation)
Receptive field	Fixed by architecture (dilations extend it)	Theoretically unlimited via cell state
Long sequences	Requires deep stacking or large dilations	Handles naturally if gradient issues resolved
Interpretability	Filter weights visualisable (temporal patterns)	Hidden state harder to interpret directly
AgriVision accuracy	84.1% (30-day window)	87.3% (LSTM), 86.8% (GRU)

Table 3.2 *1D CNN vs LSTM for temporal sequence modelling. For AgriVision's 30-day irrigation prediction, both significantly outperform the MLP baseline.*

For AgriVision's 30-day sequences, LSTM and GRU outperform the 1D CNN by 3–5 percentage points. The 30-day temporal context requires the model to remember events from the beginning of the window, where the LSTM's cell state has a natural advantage. For shorter sequences (7–14 days) or tasks where training speed is critical, the 1D CNN is often competitive and trains significantly faster.

KEY TERMS — THIS SECTION

Term	Definition
1D convolution	Convolution applied to the time dimension: a filter of size (f, F_in) slides over T time steps
Temporal receptive field	Number of past time steps that influence one output position; grows linearly with depth for standard 1D conv

Dilated convolution	Convolution with gaps between filter positions; dilation rate d expands effective receptive field to f+(f−1)×(d−1)
Causal convolution	Convolution restricted to past inputs only; required for forecasting and prediction tasks
TCN	Temporal Convolutional Network: stacked dilated causal convolutions with exponentially increasing dilation rates
Temporal GAP	Global Average Pooling over the time dimension; reduces (T, F) to (F,) for many-to-one tasks

✎ **CHECK YOUR UNDERSTANDING**

1. A 1D CNN for AgriVision irrigation prediction: three layers of f=3, same padding, causal, output channels [32, 64, 128], followed by temporal GAP and a linear binary head. (a) What is the temporal receptive field after all three conv layers? (b) For a 30-day input, does this receptive field cover the full sequence? (c) What is the shape of the tensor after temporal GAP?

2. A TCN uses 4 dilated causal conv layers with f=3 and dilation rates [1,2,4,8]. (a) What is the effective receptive field after all 4 layers? (b) Can this TCN see the full 30-day input window without additional depth? (c) Compare the parameter count of these 4 layers (all with 64 channels) to an LSTM with H=64 processing the same input.

3. Why must 1D convolutions for irrigation prediction be causal? Describe the specific data leakage that would occur if a non-causal 1D conv (using both past and future time steps) were used for this task.

4. Table 3.2 shows that the 1D CNN achieves 84.1% accuracy while the LSTM achieves 87.3% on the 30-day AgriVision task. (a) Propose a hypothesis for why the LSTM outperforms. (b) How would you test this hypothesis experimentally? (c) Under what conditions would you expect the gap to shrink or reverse?

5. [Challenge] A TCN and an LSTM both process a sequence of length T=30. The TCN uses 4 dilated layers with dilation rates [1,2,4,8], f=3, 64 channels. The LSTM has H=64. (a) Compute the FLOPS (multiply-add operations) for one forward pass through the TCN vs the LSTM. (b) For what T does the TCN's FLOP count become lower than the LSTM's? (c) What does this imply for the relative efficiency of each architecture as a function of sequence length?

3.7 Sequence Architecture Patterns and Application Contexts

3.7.1 Many-to-One, Many-to-Many, One-to-Many

The same LSTM or GRU building block supports multiple input-output configurations by changing how the hidden state is consumed. These configurations map directly onto different prediction tasks.

Many-to-one (sequence classification / regression). Process the full input sequence x_t for t=1,...,T and use only the final hidden state h_T as the representation. Attach a classification or regression head to h_T. AgriVision's irrigation prediction is many-to-one: 30 daily readings in, one binary label out. This is the simplest and most common configuration.

Many-to-many, synchronised. Produce one output at every time step: $\hat{y}_t$ is computed from h_t at each t. Used for sequence labelling tasks: fault detection in industrial machines (label each second as 'normal' or 'fault'), part-of-speech tagging in NLP (label each word with its grammatical role).

Many-to-many, encoder-decoder. An encoder RNN processes the input sequence and compresses it to a context vector (the final hidden state). A decoder RNN generates an output sequence from this context vector, one step at a time. Used for multi-step forecasting (predict the next 7 days of soil moisture from the past 30 days) and sequence-to-sequence tasks.

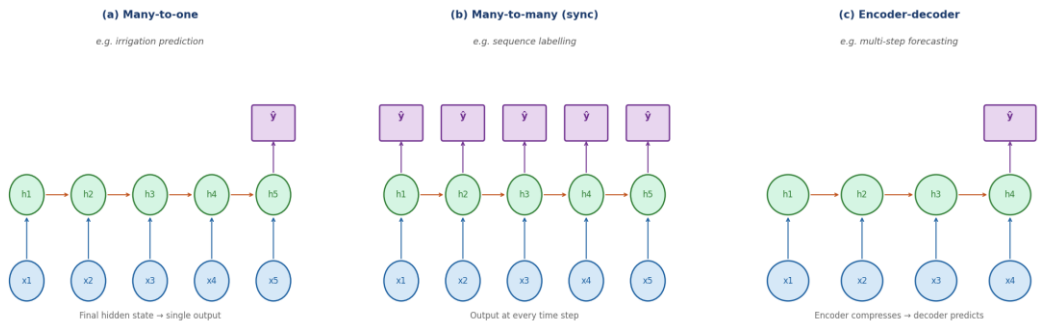


Figure 3.4 *Three sequence architecture patterns using the same RNN/LSTM building block. (a) Many-to-one: the final hidden state h_4 drives the single output — used for AgriVision's binary irrigation prediction. (b) Many-to-many (synchronised): an output is produced at every time step — used for sequence labelling. (c) Encoder-decoder: the final encoder state compresses the input sequence; the decoder generates a new output sequence — used for multi-step forecasting.*

3.7.2 Stacked and Bidirectional RNNs

Stacked RNNs. Multiple RNN layers stacked vertically: the hidden state sequence of layer l serves as the input sequence to layer l+1. Stacking increases representational capacity by allowing higher layers to detect more abstract temporal patterns. Typically 2–4 stacked layers; deeper stacking requires regularisation (dropout between layers). PyTorch's nn.LSTM and nn.GRU support stacking via the num_layers argument.

Bidirectional RNNs. Two RNNs process the same sequence in opposite directions: one from t=1 to T (forward), one from t=T to 1 (backward). Their hidden states are concatenated at each time step to produce a representation informed by both past and future context. Bidirectional RNNs require access to the complete sequence before producing any output — they are non-causal and cannot be

used for real-time or online prediction. For AgriVision's irrigation prediction, bidirectional RNNs are inappropriate (the model must not access future sensor readings). They are widely used in NLP tasks where the full sentence is available at inference time.

3.7.3 Forecasting, Classification, and Anomaly Detection

Time-series forecasting. Predict future values of the sequence itself. Encoder-decoder LSTM with MSE loss on the output values. Example: predict soil moisture for days 31–37 from the past 30 days. This is a regression task at every output time step.

Time-series classification. Assign a label to an entire sequence or a window. Many-to-one with cross-entropy loss. AgriVision's irrigation prediction is time-series classification. Other examples: classifying ECG segments as normal/arrhythmia, classifying machine vibration signatures as healthy/faulty.

Anomaly detection. Identify unusual time steps or windows. One approach: train an autoencoder (encoder LSTM → bottleneck → decoder LSTM) to reconstruct normal sequences; reconstruction error on anomalous sequences will be high. Chapter 5 covers autoencoders in detail.

3.7.4 Windowing Strategies and Temporal Context

Constructing a sequence dataset from continuous sensor streams requires a windowing strategy: how to create fixed-length training sequences from data recorded continuously over years.

- Sliding window: move the window one step at a time. From 365 days of data, a 30-day window produces 335 overlapping windows. Heavily overlapping windows produce many similar training examples — good for diversity but creates correlation between training examples that may inflate apparent performance.
- Non-overlapping window: move the window by T steps at a time. From 365 days, a 30-day window produces 12 non-overlapping windows per farm. Fewer examples but each is genuinely independent.
- Stratified sampling: if positive examples (irrigation needed) are rare, oversample them. For AgriVision, 23% of windows end with irrigation within 3 days. Random sampling produces a manageable imbalance; if more extreme, oversample positives or use weighted loss.
- Train-test split by time, not random: always place the test set at the end of the time series. A random split allows the model to predict day 50's outcome from windows that include days 48–49 in the training set — data leakage through temporal proximity.

KEY TERMS — THIS SECTION

Term	Definition
Many-to-one	Architecture pattern: T inputs → 1 output from final hidden state; classification or scalar prediction
Encoder-decoder	Architecture pattern: encoder RNN compresses input sequence; decoder RNN generates output sequence
Stacked RNN	Multiple RNN layers where the hidden sequence of layer l is the input to layer l+1

Bidirectional RNN	Two RNNs processing the sequence in opposite directions; non-causal; requires complete sequence at inference
Sliding window	Windowing strategy: extract overlapping fixed-length windows from a continuous sensor stream
Temporal train-test split	Split by time (train=past, test=future) to prevent data leakage through temporal proximity

✎ **CHECK YOUR UNDERSTANDING**

1. AgriVision wants to extend the system to predict soil moisture values for days 31–37 (a 7-day ahead forecast). (a) Which architecture pattern is appropriate? (b) What is the loss function and output activation? (c) How does the model output size change compared to the binary classification task?

2. A bidirectional LSTM with H=64 is proposed for AgriVision's irrigation prediction. (a) How many parameters does it have compared to a unidirectional LSTM with H=64? (b) Why is a bidirectional LSTM unsuitable for this specific task? (c) Give a modified version of the AgriVision task where a bidirectional LSTM would be appropriate.

3. From 5 years (1,825 days) of data across 150 farms, using a 30-day sliding window with step size 1: (a) How many windows are produced per farm? Total across all farms? (b) If step size is 7 instead, how many windows are produced? (c) Which step size is more appropriate for training, and why?

4. Section 3.7.4 states: 'Always place the test set at the end of the time series.' Describe a specific scenario where using a random 80/20 split on the windowed AgriVision dataset would produce an optimistic (inflated) test accuracy estimate. What data leakage mechanism creates this inflation?

5. [Challenge] Design a complete data pipeline for the AgriVision irrigation prediction task: (a) How do you handle the 3-day missing sensor readings that occur on average twice per year per farm? (b) Propose a validation strategy that tests the model on a held-out farm (not just held-out time) to ensure the model generalises across farms, not just across time. (c) How would you handle the class imbalance (23% positive)?

3.8 When Sequence Models Fail

3.8.1 Non-Stationarity: The Distribution Changes Over Time

The most pervasive challenge in deploying sequence models is non-stationarity: the statistical properties of the time series change over time. A model trained on five years of AgriVision data and deployed in year six may encounter sensor readings with subtly different mean values, variance, or temporal correlations than what it was trained on.

- Climate drift: mean temperatures at Dharwad have shifted over the five-year collection period. A model that learned 'temperature above 36°C indicates drought stress' may need to recalibrate as baseline temperatures rise.
- Crop rotation: farms that grew tomatoes in years 1–3 switch to cotton in years 4–5. Cotton has different water requirements; the same soil moisture reading that warranted irrigation for tomatoes may not for cotton.
- Farming practice changes: a new irrigation cooperative provides subsidised water to 20 farms in year 4. Their irrigation patterns change, and the historical training data no longer reflects their new decision-making.

Detection: compare the distribution of recent inputs (last 30 days of deployment) against the training distribution using statistical tests (Kolmogorov-Smirnov, population stability index). A significant shift triggers a model review. Mitigation: retrain on a rolling window of recent data, incorporating new seasons as they become available.

3.8.2 Missing Data and Irregular Sampling

Sequence models assume a regular sampling interval: every time step is exactly one day apart. Real agricultural sensors produce irregular data: power outages, connectivity failures, physical damage. A sensor that was offline from day 14 to day 17 leaves a 3-step gap in the 30-day window.

Standard mitigation: forward-fill missing values with the most recent valid reading. This is simple but introduces a flat segment in the sequence that may cause the model to underestimate the uncertainty during the gap period. Alternative approaches include linear interpolation (more realistic for smooth signals like temperature), learnable imputation (a separate model fills in missing values), or masking (passing a binary indicator of which time steps are observed to the LSTM alongside the values).

> ⚠ **IMPORTANT NOTE**
>
> **Missing value imputation must use only past information, never future.**
>
> A common mistake: fill missing sensor values by linear interpolation using both the value before and after the gap. For offline analysis (studying past records) this is acceptable. For a deployed prediction system, filling day 15 using day 18's value constitutes data leakage: the model implicitly uses future information at inference. Always fill missing values using only past observations in production systems.

3.8.3 Sensor Drift and Concept Drift

Sensor drift. The gradual change in a sensor's calibration over time. A soil moisture sensor that accurately read 0.50 in year 1 may systematically read 0.45 for the same true moisture level by year 4 due to electrode degradation. To the model, this appears as a slow downward trend in soil moisture across all farms simultaneously. Mitigation: regular sensor recalibration; monitoring of systematic cross-farm correlations that would not arise from genuine agronomic changes.

Concept drift. The relationship between the input features and the target label changes over time, even if the input distribution is stable. For AgriVision, concept drift could occur if a new disease-resistant tomato variety is introduced that has different water requirements: the soil moisture threshold for irrigation need shifts, but the sensor readings themselves look the same. The model continues to predict based on the old threshold and produces systematic errors. Concept drift is harder to detect than covariate shift because it requires labelled data from the new distribution.

3.8.4 The Long-Range Dependency Ceiling: When RNNs and CNNs Hit Their Limit

Despite the LSTM's improvement over the vanilla RNN, there remains a practical ceiling on how long a dependency an LSTM can reliably learn. For sequences of hundreds or thousands of steps, the cell state's information about the distant past is progressively overwritten by more recent inputs. The forget gate, if it consistently forgets to preserve information from the distant past, creates a de facto attention horizon.

Similarly, the 1D CNN's receptive field is hard-bounded by its architecture: a TCN with depth D and maximum dilation 2^D can only 'see' $2^D \times (f-1)+1$ steps into the past, regardless of sequence length. Making the architecture deeper adds parameters and training cost without necessarily improving the model's ability to integrate information across the entire sequence.

This is not a problem for AgriVision's 30-day sequences, where both LSTM and TCN perform well. But for tasks with genuinely long sequences — hourly readings over a full year (8,760 steps), or long text documents — neither RNNs nor 1D CNNs can efficiently capture relationships between distant elements. This limitation motivated the development of the attention mechanism and the Transformer, which can relate any two positions in a sequence in a single computational step regardless of their distance. Chapter 4 builds this architecture from the ground up.

> ◆ **KEY CONCEPT**
>
> **The architectural ceiling that motivates Chapter 4**
>
> RNNs (including LSTM and GRU) process sequences step by step. Information from the distant past must propagate forward through every intermediate hidden state, becoming progressively diluted. For dependencies spanning hundreds of time steps, this sequential bottleneck causes information loss that gating alone cannot prevent.
>
> The Transformer's self-attention mechanism connects any two positions in the sequence directly, without passing through intermediate steps. This eliminates the sequential bottleneck entirely — at the cost of $O(T^2)$ computation in sequence length T, which becomes prohibitive for very long sequences.

The trade-off: RNNs scale in O(T), Transformers in O(T^2). For short-to-medium sequences (T ≤ 1,000), Transformers dominate accuracy. For very long sequences (T > 10,000), efficient attention variants or hybrid architectures are needed.

KEY TERMS — THIS SECTION

Term	Definition
Non-stationarity	Time-series property where statistical distributions change over time; major cause of model degradation in deployment
Sensor drift	Gradual calibration change in a sensor, producing a spurious trend in recorded values
Concept drift	Change in the relationship between inputs and target label over time; harder to detect than covariate shift
Forward-fill	Imputing missing values by repeating the most recent observed value; only uses past information
Attention horizon	Practical limit beyond which LSTM cell state no longer reliably retains information from the distant past
Sequential bottleneck	The limitation that information in RNNs must propagate forward through every intermediate step

✎ **CHECK YOUR UNDERSTANDING**

1. AgriVision monitors prediction accuracy month-by-month. Accuracy is 87% for the first 6 months of deployment, then drops to 79% in months 7–9, coinciding with a new crop rotation across 30 farms. (a) Is this sensor drift, concept drift, or covariate shift? Justify your answer. (b) What data would you need to collect to retrain the model, and how long would it take to gather enough labelled examples?

2. Farm 23's sensor was offline for days 12–15 of a 30-day window. Describe the forward-fill approach. What value is used for days 12–15? Compare this to linear interpolation using both day 11 and day 16 observations. Why is linear interpolation inappropriate for a deployed prediction system?

3. Section 3.8.4 states that the LSTM has a 'practical attention horizon' beyond which cell state information is progressively overwritten. (a) What gate is responsible for this overwriting? (b) For AgriVision's 30-day sequences, is this horizon likely to be a problem? (c) For a year-long hourly sequence (8,760 steps), estimate qualitatively at what horizon information from the beginning of the sequence becomes unreliable.

4. A TCN and a Transformer are both trained on sequences of length T=100. The TCN uses 7 dilated layers with f=2 and dilation rates [1,2,4,8,16,32,64]. (a) What is the TCN's receptive field? Can it see the full sequence? (b) The Transformer's self-attention connects every pair of positions directly. What is its effective 'receptive field'? (c) At what sequence length does the Transformer's O(T^2) cost make it impractical on Colab T4 (16 GB memory)?

5. [Challenge] Design a monitoring system for AgriVision's deployed LSTM to detect both covariate shift (input distribution changes) and concept drift (label relationship changes) in production. For each: (a) what quantity would you track? (b) what statistical test or threshold would you use? (c) what action would you take when the threshold is exceeded, and how long would it take to gather enough new labelled data to retrain?

3.9 Chapter Resolution and Bridge

3.9.1 AgriVision's Irrigation Predictor: Architecture Chosen, Results, Deployment

The soil moisture reading at Farm 47 was 0.28 field capacity on a Thursday morning in March. Section 3.0 opened with an extension officer needing to decide: will this farm need irrigation in the next three days? The sensor data for the preceding 30 days was available. The question was whether a model could read the answer from that data.

It can. AgriVision's deployed irrigation prediction system uses a two-layer GRU with hidden size 64, trained on 5 years of sliding-window sequences from all 150 farms. The system gives a 3-day advance irrigation alert: enough time to schedule the pump operator, arrange the water connection, and notify the farmer. The cooperative estimates that 3-day advance notice reduces irrigation labour costs by 22% (better scheduling) and reduces crop stress from under-watering by 18% (timelier intervention).

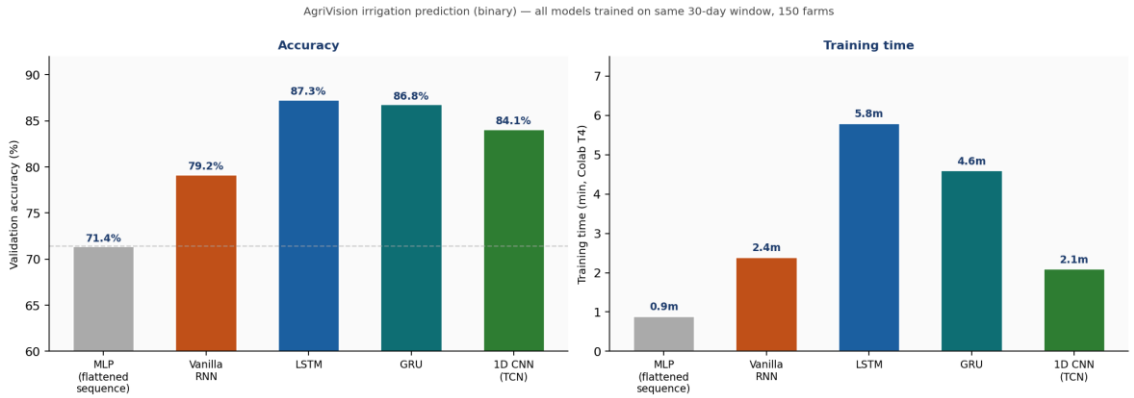


Figure 3.6 *Model comparison on AgriVision's binary irrigation prediction task: validation accuracy (left) and training time on Colab T4 (right). The MLP baseline (71.4%) is the direct counterpart to the MLP disease detection baseline from Chapter 2. Vanilla RNN improves substantially by adding temporal memory. LSTM and GRU achieve the best accuracy; the 1D CNN (TCN) offers a strong accuracy-speed trade-off. All sequence models trained on 30-day windows of 7 normalised sensor features.*

✦ **THE AGRIVISION CONTEXT**

Chapter 3 Resolution — Irrigation prediction results (30-day window, 150 farms, 5 years)

MLP (210→128→64→1, flattened sequence): 71.4% | 0.9 min

Vanilla RNN (H=64): 79.2% | 2.4 min

LSTM (H=64, 2 layers): 87.3% | 5.8 min [deployed]

GRU (H=64, 2 layers): 86.8% | 4.6 min

1D CNN / TCN (4 dilated layers, 64 channels): 84.1% | 2.1 min

The progression 71.4% → 87.3% required the same 273,750 labelled windows at every step. The improvement came from encoding the right inductive biases: temporal ordering, sequential memory, and parameter sharing across time steps.

The chapter's arc mirrors Chapter 2's. The MLP failed because it lacked the structural properties required by the data type. The right architecture — one that encodes temporal ordering and sequential memory — achieves a 16-percentage-point improvement from the same data. The LSTM's cell state solved the vanishing gradient problem that prevented the vanilla RNN from using the full 30-day context.

Chapter Summary

Section	Core concept	Key insight
3.0	Inductive bias for sequences	Temporal ordering + sequential memory are structural properties of sensor streams, not dataset-specific observations
3.1	Time-series data context	Shape (N,T,F); stationarity and seasonal variation require temporal train-test splitting and careful normalisation
3.2	Why MLPs fail on sequences	Fixed input length, stateless forward pass, no temporal parameter sharing — three manifestations of one root cause
3.3	Vanilla RNN	$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$; vanishing gradient makes long-range learning impossible despite correct architecture
3.4	LSTM	Cell state provides a gradient highway; three gates control what is remembered, updated, and exposed
3.5	GRU	Two gates, one state vector; 25% fewer parameters than LSTM; comparable performance on most tasks
3.6	1D CNN / TCN	Temporal filters; dilated causal convolutions expand receptive field; faster training but bounded temporal context
3.7	Architecture patterns	Many-to-one, many-to-many, encoder-decoder; match output structure to task; temporal train-test split is mandatory
3.8	Failure modes	Non-stationarity, missing data, sensor drift, and long-range dependency ceiling limit all RNNs and 1D CNNs
3.9	Resolution + bridge	LSTM achieves 87.3% irrigation prediction accuracy; the sequential bottleneck motivates Chapter 4's attention mechanism

3.9.2 The Field Notes Problem: When Sequences Are Words, Not Numbers

Every week, AgriVision's extension officers file field notes. Farm 47's note for the third week of March reads: 'Observed early signs of powdery mildew on rows 4–7. Soil compaction visible near the irrigation canal. Farmer reports the pump ran for 40 minutes on Tuesday but the flow seemed reduced. Advised checking the foot valve.'

This note contains information that no sensor can capture. The reduced pump flow, the specific rows affected, the farmer's own observations, the agronomist's diagnosis — all exist as text. AgriVision has 2,400 such notes spanning five seasons and 150 farms. They are sequences too: sequences of words, each encoding a concept, and their meaning depends on the relationships between concepts across many words.

But the LSTM that works so well on soil moisture sensor streams will not work well on these field notes. Not because sentences are sequences — they are — but because the dependencies between words in language are qualitatively different from the dependencies between sensor readings. 'The pump ran for 40 minutes but the flow seemed reduced' requires understanding that 'reduced' modifies 'flow' despite being separated by three words; that 'seemed' signals uncertainty; and that this sentence describes a mechanical problem in the irrigation infrastructure rather than a soil or disease condition.

These long-range semantic dependencies, combined with the vastly larger vocabulary of natural language (tens of thousands of words versus 7 sensor features), and the compositionality of meaning across words, require an architecture that can attend to any word from any other word — not just the recent past. The sequential bottleneck of the LSTM is not merely a performance limitation here; it is a fundamental architectural mismatch.

The Transformer architecture replaces the sequential hidden state with self-attention: a mechanism that computes, in a single step, the relevance of every word to every other word in the sentence. This direct, global attention over the entire sequence resolves the long-range dependency problem that limits RNNs — and it is why Transformers have come to dominate not just NLP but, increasingly, computer vision and multi-modal AI. Chapter 4 builds the Transformer from its first principles.

CHAPTER 4

Learning from Text

Attention, Transformers, and Language Models

4.0 The Inductive Bias for Text Data

4.0.1 The AgriVision Field Notes Problem

> ✦ THE AGRIVISION CONTEXT
>
> Every week, AgriVision's extension officers file field notes. Farm 47's note for the third week of March reads: 'Observed early signs of powdery mildew on rows 4–7. Soil compaction visible near the irrigation canal. Farmer reports the pump ran for 40 minutes on Tuesday but the flow seemed reduced. Advised checking the foot valve.'
>
> AgriVision has 2,400 such notes spanning five seasons and 150 farms, plus an agricultural advisory database of 800 disease management recommendations. Farmers increasingly ask questions directly: 'What should I do about powdery mildew in early flowering tomatoes?' The system must retrieve the relevant advisory and deliver a grounded answer.
>
> The LSTM that predicts irrigation need from sensor numbers cannot help here. The challenge is not sensor sequences but language: sentences where meaning depends on words that may be far apart, where the same word can mean different things in different contexts, and where the most important relationships can span an entire paragraph.

Text is also sequential data. But its structural properties are qualitatively different from those of sensor streams. Soil moisture yesterday causes soil moisture today: the sequential dependency is local and causal. But in the sentence 'The copper-based fungicide that is applied at the early flowering stage is most effective against powdery mildew,' the word 'effective' depends semantically on 'fungicide' (12 words earlier), 'stage' (5 words earlier), and 'powdery mildew' (3 words later). This dependency is not causal in the time-series sense—it is structural and bidirectional.

4.0.2 Global Context: Why Word Meaning Depends on Everything

Consider the word 'application' in three agricultural advisory sentences:

- 'The application of copper sulphate should be timed for early morning.' Application = physical spraying of a chemical.
- 'Submit your cooperative membership application before April 15.' Application = a form or document submitted for a purpose.
- 'This mobile application sends disease alerts in real time.' Application = software.

The word is identical in all three sentences but means completely different things. Understanding which meaning applies requires reading the entire sentence — not just the words immediately before and after 'application.' An LSTM processing this word at step t has access to all words before it

(through the hidden state), but its memory of the distant context is degraded by the vanishing gradient problem. Words after 'application' are not accessible at all in a forward LSTM.

◆ KEY CONCEPT

Global context as inductive bias

Text has long-range semantic dependencies: the meaning of any word may depend on words anywhere in the surrounding context. A model for text must be able to relate any word to any other word in the sequence, regardless of distance, in a single computational step.

This is fundamentally different from the local temporal patterns of sensor data (Chapter 3) and the local spatial patterns of images (Chapter 2). Neither RNNs nor 1D CNNs can efficiently represent global context. The Transformer was designed specifically to encode it.

4.0.3 The Information Bottleneck in Sequential Models

The encoder-decoder RNN architecture from Chapter 3 (Section 3.7.2) compresses an entire input sequence into a single fixed-size context vector (the final hidden state), which the decoder must then use to generate the output. For a 20-word sentence this means all relevant information must fit into a 256 or 512-dimensional vector.

This is the sequential bottleneck. For machine translation, the meaning of all 20 source words must be encoded in one vector before any target word is generated. Bahdanau et al. (2015) first demonstrated that this bottleneck could be eliminated by allowing the decoder to 'attend' directly to any position in the encoder's output sequence when generating each target word. This attention mechanism is the conceptual precursor to the Transformer.

4.0.4 How These Properties Motivate the Transformer

The Transformer (Vaswani et al., 2017) replaces sequential processing entirely with self-attention: every token in the input sequence simultaneously attends to every other token, computing in one step a representation that encodes global context. There is no hidden state that propagates sequentially; there is no information bottleneck. The distance between two tokens in the sequence has no effect on the ease of relating them.

? FOUR QUESTIONS TO CARRY THROUGH THIS BOOK

Question 1: What problem does this architecture solve?

The failure of sequential models on text: the vanishing gradient limits the range of dependencies that RNNs can learn; the sequential computation bottleneck limits parallelisation during training; the fixed context vector limits the information accessible during decoding.

Question 2: What inductive bias does it encode?

Global context: every token can directly attend to every other token, regardless of distance. The model does not assume that nearby tokens are more relevant than distant ones — it learns the relevance from data.

Question 3: When does this architecture fail?

Very long sequences: self-attention cost scales as $O(T^2)$ in sequence length. For T = 10,000 tokens (a long document), this becomes prohibitive. Also: hallucination (generating plausible but factually wrong text), bias from pretraining data.

Question 4: When would you choose something else?

Short sequences where the $O(T^2)$ cost is not a concern: RNNs and 1D CNNs remain competitive; real-time streaming (Transformers need the full context upfront); edge deployment (full Transformers are large; distilled or quantised models needed).

KEY TERMS — THIS SECTION

Term	Definition
Global context	Property of text: word meaning may depend on any other word in the context, regardless of distance
Sequential bottleneck	The fixed-size context vector in RNN encoder-decoder models that limits how much information can be transferred
Attention mechanism	A learned, context-dependent weighting scheme that allows a model to focus on relevant parts of the input
Transformer	An architecture using self-attention to relate all tokens to all others in one step; no recurrence
Long-range semantic dependency	Grammatical or semantic relationship between words far apart in a sentence

✎ **CHECK YOUR UNDERSTANDING**

1. In the sentence 'The fungicide that the cooperative recommended for powdery mildew last season was copper-based,' identify three long-range semantic dependencies (pairs of words that are semantically related despite being far apart). For each pair, specify whether a forward LSTM would have access to both words when processing the later one.

2. An extension officer searches for 'treatment for early-stage disease in tomatoes.' The word 'treatment' has multiple meanings. (a) List three possible meanings. (b) Which contextual clue in this specific query disambiguates the meaning? (c) How far away from 'treatment' is this disambiguating clue?

3. Section 4.0.3 describes the sequential bottleneck in RNN encoder-decoder models. A 30-word advisory sentence is compressed into a 256-dimensional vector. (a) What is the information density (words per dimension)? (b) Explain why this bottleneck causes specific failures in long-document retrieval. (c) How does attention eliminate this bottleneck?

4. Compare the inductive biases of CNNs (Chapter 2), LSTMs (Chapter 3), and Transformers (Chapter 4). Complete a three-column table: architecture, inductive bias encoded, inductive bias absent. Why is the absent inductive bias in each case a structural limitation rather than a training failure?

5. [Challenge] The Transformer's $O(T^2)$ cost means that doubling the sequence length quadruples the compute. A BERT model processing 512 tokens requires approximately 10^1 FLOPs. (a) How many FLOPs would be needed for a 2,048-token context? (b) Why is this problematic for AgriVision's 2,400 field notes, each typically 100–300 words? (c) What practical strategies mitigate this constraint?

4.1 Text as Data

4.1.1 Tokenisation: From Characters to Subwords

Before text can be processed by a neural network, it must be converted to a sequence of numerical tokens. Three levels of granularity are in common use:

- Character-level: each character is one token. The sentence 'mildew' becomes ['m','i','l','d','e','w']. Short vocabulary (26 letters + punctuation ≈ 100 tokens); sequences are very long; rare words handled naturally but morphological structure must be learned.
- Word-level: each word is one token. 'Powdery mildew' becomes ['Powdery','mildew']. Vocabulary must include all words encountered at training time (typically 30,000–100,000). Out-of-vocabulary words (new crop variety names, technical terms) become [UNK].
- Subword tokenisation (Byte Pair Encoding, WordPiece, SentencePiece): the standard for modern Transformers. Frequent words remain as single tokens; rare words are split into frequent subword pieces. 'Alternaria' might become ['Alter', '##naria']. Vocabulary ≈ 30,000 tokens; rare agricultural terms are handled gracefully; the [UNK] token is almost never needed.

BERT, RoBERTa, and DistilBERT use WordPiece tokenisation. GPT-2 and GPT-3 use Byte Pair Encoding. For AgriVision's advisory system, both would handle agricultural vocabulary adequately, since even rare terms like 'Alternaria leaf spot' are decomposed into recognisable subword units.

4.1.2 Vocabulary and One-Hot Encoding

A vocabulary V maps each unique token to an integer index from 0 to |V|−1. The token 'mildew' might be assigned index 14,832 in a vocabulary of 30,000 tokens. One-hot encoding represents each token as a |V|-dimensional binary vector with a single 1 at the token's index and 0 everywhere else.

One-hot vectors are sparse (almost all zeros), high-dimensional, and encode no semantic information: the vectors for 'mildew' and 'fungus' are orthogonal, despite their semantic proximity. Two words with similar meanings have one-hot vectors that are just as different (cosine similarity = 0) as two words with unrelated meanings. This motivates the transition to dense word embeddings.

4.1.3 Word Embeddings: Dense Semantic Representations

A word embedding maps each token to a dense, low-dimensional vector in a continuous embedding space. An embedding table E has shape (|V|, d_model): row i is the embedding vector for token i. For BERT-base, |V| ≈ 30,000 and d_model = 768, so the embedding table has 30,000 × 768 ≈ 23 million parameters.

The embedding space has a remarkable semantic structure: words with related meanings have similar embedding vectors (high cosine similarity). 'Fungicide' and 'pesticide' are close; 'tomato' and 'potato' are close; 'drought' and 'moisture' are farther apart but closer than 'drought' and 'fungicide'. This structure emerges from training — the model discovers that words appearing in similar contexts have similar meanings.

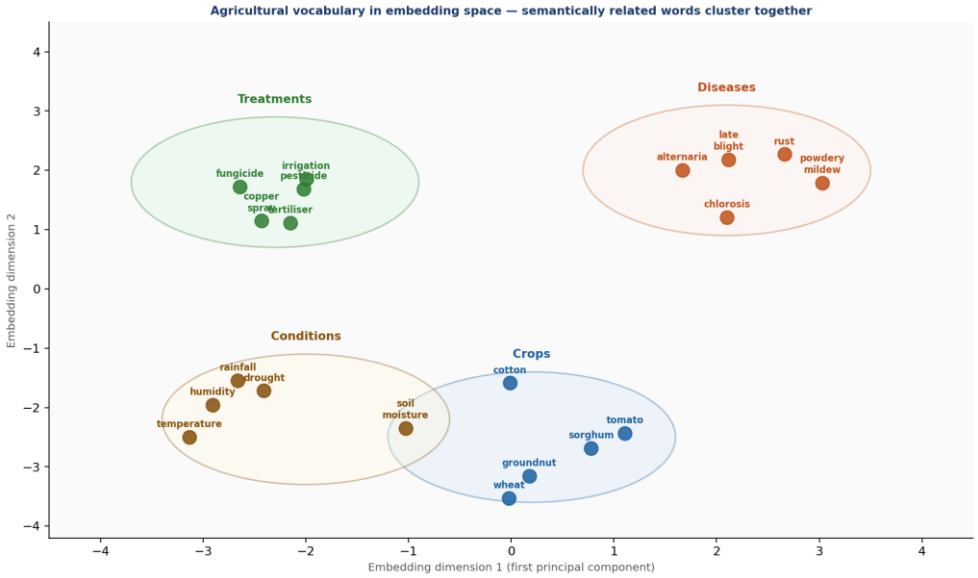


Figure 4.1 *Agricultural vocabulary in a 2D embedding space (principal components of the learned embeddings). Four semantic clusters are visible: disease names (orange), treatment interventions (green), crop types (blue), and environmental conditions (amber). Semantically related words are close in embedding space. The distance between clusters reflects semantic dissimilarity: 'fungicide' is much closer to 'mildew' than to 'tomato'.*

Static word embeddings (Word2Vec, GloVe) assign one fixed vector per word, regardless of context. The word 'application' has one embedding covering all three meanings from Section 4.0.2. This is the limitation that contextual embeddings address.

4.1.4 Contextual vs Static Embeddings

Contextual embeddings produce a different vector for each occurrence of a word, conditioned on the surrounding context. In BERT, the representation of 'application' in 'copper sulphate application' is different from its representation in 'membership application' — the Transformer's self-attention mechanism allows the embedding of each token to be updated by its context before producing the final representation.

For AgriVision's advisory retrieval system, contextual embeddings are essential. A query 'application rate for copper fungicide' must have 'application' understood as a quantity of chemical applied, not a document submitted or software installed. A static embedding system would return documents about all three meanings with similar relevance scores. A BERT-based system returns agricultural application documents.

KEY TERMS — THIS SECTION

Term	Definition
Tokenisation	Conversion of text to a sequence of discrete tokens; character, word, or subword granularity
Vocabulary (V)	The set of all unique tokens a model can represent; size |V| typically 30,000–100,000 for modern LLMs
Subword tokenisation	Splitting rare words into frequent subword pieces; used in BERT (WordPiece) and GPT (BPE)

One-hot encoding	Sparse |V|-dimensional binary vector for a token; no semantic structure; superseded by embeddings
Word embedding	Dense, low-dimensional vector representation of a token; learned such that semantically similar words are close
Embedding table	Weight matrix E of shape (|V|, d_model) mapping token indices to vectors; learned during training
Contextual embedding	Token representation that varies with context; produced by Transformers; different from static embeddings
Cosine similarity	Measure of angular similarity between two vectors; used for semantic retrieval in embedding space

✎ **CHECK YOUR UNDERSTANDING**

1. BERT-base has vocabulary |V| = 30,522 and d_model = 768. (a) How many parameters are in the embedding table? (b) The full BERT-base model has 110M parameters. What fraction are in the embedding table? (c) What does this tell you about the relative contribution of vocabulary knowledge vs. architecture to model size?

2. The agricultural term 'Alternaria leaf spot' is not in BERT's vocabulary as a single token. Using WordPiece tokenisation, it might split as ['Alt', '##erna', '##ria', 'leaf', 'spot']. (a) How many tokens does this produce? (b) How does the Transformer handle this multi-token representation? (c) What is lost compared to having 'Alternaria' as a single token?

3. Two agricultural queries: Q1='copper sulphate for powdery mildew' and Q2='copper sulphate membership form'. With static word embeddings, explain why both queries would have identical representations for the phrase 'copper sulphate'. With contextual embeddings, would the representations differ? Why?

4. Word2Vec embeddings satisfy approximate arithmetic: king - man + woman ≈ queen. Propose an analogous agricultural embedding arithmetic that should hold if the embeddings capture agronomic knowledge. What would the equation be, and how would you verify it empirically?

5. [Challenge] BERT uses special tokens: [CLS] at the start of every input and [SEP] to separate segments. The [CLS] token's output embedding is used for classification. (a) Why is a special token needed for classification rather than using the embedding of the first content word? (b) What does the [CLS] embedding represent after the Transformer's self-attention layers? (c) For AgriVision's advisory retrieval, would you use the [CLS] embedding or the mean of all token embeddings as the document representation? Justify.

4.2 Why Sequential Models Are Insufficient for Text

4.2.1 The Long-Range Dependency Problem in Language

Consider this sentence from AgriVision's advisory database: 'The copper-based fungicide applied at early flowering is more effective against powdery mildew than sulphur compounds.'

The subject is 'fungicide.' The verb is 'is effective.' Between them are eleven words. A language model must keep track of 'fungicide' while reading all eleven intermediate words before reaching the verb — otherwise it cannot correctly parse the sentence's meaning.

An LSTM processing this sentence left-to-right keeps its information about 'fungicide' in the cell state. But we showed in Chapter 3 that the cell state's gradient shrinks at each backward step. Over 11 steps, the signal is already weak. Over 30 words — common in advisory text — the model may fail to make the connection at all.

There is also the opposite problem: text often requires looking forward, not just backward. The word 'not' in 'do not apply copper fungicide during flowering' completely reverses the meaning of the sentence. But 'not' appears before 'apply' — a left-to-right model has already partially processed 'apply' before it sees 'not.' The model needs both past and future context simultaneously to correctly parse any given word.

4.2.2 The Fixed-Size Context Vector Bottleneck

For query-answering, we need to match a farmer's question against hundreds of advisory documents and generate a grounded answer. The standard RNN approach to this is an encoder-decoder: the encoder reads the entire query and compresses it into one fixed-size vector; the decoder generates the answer from that vector.

The problem: one vector must carry all the information needed to answer the question. A 256-dimensional vector must encode which disease is mentioned, which crop, which growth stage, which time of year, and what type of advice is sought. If the query is long or complex, important details are dropped. The richer the query, the worse the compression.

4.2.3 Sequential Computation Cannot Be Parallelised

Training large language models requires processing billions of text examples. The RNN processes these one token at a time: step t depends on step t−1, which depends on step t−2. Even on a GPU with thousands of cores, a 512-token sentence requires 512 sequential operations. Most of those cores sit idle while the RNN works through the sequence one step at a time.

The Transformer eliminates this constraint. It computes all token-to-token interactions simultaneously in one matrix multiplication. All 512 tokens are processed in parallel. This is the primary reason Transformers can be trained on the massive scale that produces powerful foundation models.

KEY TERMS — THIS SECTION

Term	Definition

Long-range dependency	Grammatical or semantic relationship between words far apart in a sentence
Bidirectional context	A word's meaning depends on both earlier and later words; left-to-right LSTM misses this
Context vector bottleneck	Compressing an entire input to one fixed-size vector; loses fine-grained detail
Parallel computation	Transformer self-attention computes all token pairs simultaneously; enables large-scale training

✎ **CHECK YOUR UNDERSTANDING**

1. Count the words between subject and verb in: 'The copper-based fungicide that is applied at early flowering stage is effective.' With λ_max=0.85 in a vanilla RNN, estimate the gradient attenuation from the verb back to 'fungicide.'

2. An encoder-decoder LSTM compresses a 40-word advisory passage into a 256-dim vector. List three types of information that might be dropped in this compression for a complex query about pesticide dosage timing.

3. A Transformer processes all 512 tokens simultaneously. An LSTM needs 512 sequential steps. Why does this difference matter for training large language models on billions of documents?

4. The sentence 'Do not apply copper fungicide during flowering' requires knowing 'not' when interpreting 'apply.' Explain why a forward LSTM struggles with this, and what context a bidirectional model would use.

5. [Challenge] BERT pretraining processes 3.3 billion tokens. An equivalent LSTM pretraining has no GPU parallelism across tokens within each sequence. Estimate the relative training time, and explain the practical consequence for the kinds of language models we can build.

4.3 The Attention Mechanism

4.3.1 An Analogy: Searching a Library

Imagine searching a library for advice on powdery mildew. You write your question on a slip of paper — this is your query. The library card catalogue has a short description for each book — these are the keys. You compare your query slip to every catalogue entry and assign a relevance score to each book. Then you read all the books simultaneously, spending more time (assigning more weight) on the highly relevant ones. The weighted reading produces your answer — this is the value.

This is exactly how attention works. Every position in a sequence simultaneously acts as: a query ('what am I looking for?'), a key ('what do I offer to other queries?'), and a value ('what do I contribute when selected?'). The query is compared to all keys, producing relevance scores. Those scores are converted to weights. The output at each position is the weighted average of all values, using those weights.

4.3.2 Scaled Dot-Product Attention

The formal version of the library search is computed in three steps. Given three matrices — queries Q, keys K, and values V:

$$Attention(Q, K, V) = softmax(QK^T / \sqrt{d_k}) \cdot V$$

Step 1 — Compute relevance scores: $S = QK^T / \sqrt{d_k}$. Each entry S[i,j] is the dot product of query i with key j, divided by $\sqrt{d_k}$. A large dot product means query i and key j are pointing in similar directions — they are a good match.

Step 2 — Convert scores to weights: A = softmax(S). Softmax converts each row of scores into a probability distribution. Scores close to zero become equal weights. A very high score dominates the row — that position gets almost all the attention.

Step 3 — Compute the output: Output = A · V. Each output row is a weighted average of all value rows. Positions the query found highly relevant contribute the most.

The scaling factor $\sqrt{d_k}$ prevents the dot products from growing very large for high-dimensional vectors. Without it, large values push the softmax toward near-zero gradients, making training slow.

4.3.3 Worked Example: Attention on a Short Agricultural Query

> ✎ **WORKED EXAMPLE**
>
> **Which word does 'tomatoes' attend to most?**
>
> Four tokens: ['powdery'(0), 'mildew'(1), 'on'(2), 'tomatoes'(3)] $d_k = 4$
>
> Query vector for 'tomatoes': q = [0.4, 0.6, 0.1, 0.8]
>
> Key vectors:
>
> k_0 (powdery): [0.8, 0.3, −0.4, 0.6]
>
> k_1 (mildew): [0.7, 0.9, −0.2, 0.5]
>
> k_2 (on): [−0.1, 0.2, 0.3, −0.1]
>
> k_3 (tomatoes): [0.4, 0.6, 0.1, 0.8]

Step 1 — Raw scores ($q \cdot k_i / \sqrt{4}$):

$s_0 = (0.4\times0.8 + 0.6\times0.3 + 0.1\times(-0.4) + 0.8\times0.6) / 2 = 0.94/2 = 0.47$

$s_1 = (0.4\times0.7 + 0.6\times0.9 + 0.1\times(-0.2) + 0.8\times0.5) / 2 = 1.20/2 = 0.60$

$s_2 = (0.4\times(-0.1) + 0.6\times0.2 + 0.1\times0.3 + 0.8\times(-0.1)) / 2 = 0.03/2 = 0.015$

$s_3 = (0.4\times0.4 + 0.6\times0.6 + 0.1\times0.1 + 0.8\times0.8) / 2 = 1.17/2 = 0.585$

Step 2 — Softmax weights:

$a \approx [0.226, 0.256, 0.143, 0.255]$ (sum = 1.00)

Result: 'tomatoes' attends most to 'mildew' (0.256) and itself (0.255),

moderately to 'powdery' (0.226), and least to the preposition 'on' (0.143).

This matches agronomic intuition: tomatoes–mildew is the key relationship.

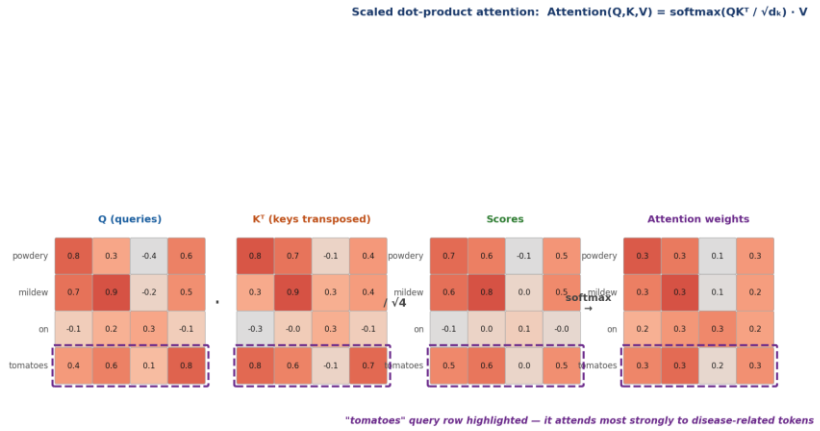


Figure 4.2 *Scaled dot-product attention for the 4-token query 'powdery mildew on tomatoes.' The Q matrix contains query vectors, K^T the transposed keys. Their product (scaled by $1/\sqrt{d_K}$) gives raw attention scores. After softmax, each row is a probability distribution. The bottom row (query = 'tomatoes', highlighted) shows strongest attention to 'mildew' and weakest to 'on' — the semantically correct pattern.*

4.3.4 Soft vs Hard Attention

In the computation above, every position receives a non-zero weight — even 'on' gets 0.143. This is soft attention: a weighted average over all positions. It is differentiable everywhere, so gradients flow through the attention weights during backpropagation.

Hard attention would pick exactly one position (the argmax) and attend only to it. This is not differentiable, so it requires reinforcement learning to train. Modern Transformers all use soft attention.

KEY TERMS — THIS SECTION

Term	Definition
Query (Q)	Vector representing 'what this position is looking for'
Key (K)	Vector representing 'what this position offers to other queries'

Value (V)	Vector representing 'what this position contributes when selected'
Attention score	Dot product $q \cdot k / \sqrt{d_k}$; measures relevance of one position to another
Attention weight	Softmax of attention scores; a probability distribution over attended positions
Scaled dot-product attention	$\text{Attention}(Q,K,V) = \text{softmax}(QK^T/\sqrt{d_k}) \cdot V$; core operation of all Transformers

✎ **CHECK YOUR UNDERSTANDING**

1. Using the key matrix from the worked example, compute attention weights for 'mildew' attending to all 4 tokens. Does 'mildew' attend most to 'powdery'?

2. For $d_k=64$ (BERT-base): what is the scaling factor? If two vectors are drawn from N(0,1), what is the expected magnitude of their dot product? Explain why large d_k causes softmax saturation without scaling.

3. Why must attention weights sum to 1? What would happen to the output if weights could sum to more than 1?

4. In the worked example, 'on' receives weight 0.143 (non-zero). Is this a problem or useful? What might a syntactically important preposition contribute through its value vector?

5. [Challenge] Compute the total multiply-add operations for Attention(Q,K,V) on n=512 tokens with $d_k=d_v=64$. BERT-base has 12 heads and 12 layers. How does total computation scale with n?

4.4 Self-Attention and Multi-Head Attention

4.4.1 Self-Attention: Every Token Looks at Every Other Token

In Section 4.3, the queries, keys, and values could come from different sequences. Self-attention is the special case where they all come from the same sequence. Every token simultaneously asks 'what should I attend to?' (query), offers itself as a candidate (key), and contributes its content when selected (value).

Given an input sequence X (a matrix where each row is one token's embedding), three linear projections produce Q, K, and V:

$$Q = X\,W_{\varphi}, \quad K = X\,W_{k}, \quad V = X\,W_{v}$$

W_{ϕ}, W_{k}, and W_{v} are learned weight matrices. They project each token's embedding into three different roles: query, key, and value. The attention output then updates every token's representation based on what all other tokens contribute. After self-attention, the word 'mildew' in our advisory sentence knows about 'tomatoes,' 'copper,' and every other word — all computed in one parallel matrix operation.

4.4.2 The O(T²) Cost and Why It Matters

Self-attention computes one score for every pair of tokens. For a sequence of T tokens, that is T × T = T² scores. For T=512 (BERT's maximum): 512² = 262,144 dot products per attention layer per head. This is stored as a 512×512 attention matrix.

As T grows, the memory and computation grow with T². For T=2,048 (GPT-3): 4× the cost of T=1,024. For a book-length document of T=10,000 tokens: 400× the cost of T=512. This quadratic scaling is why standard Transformers are limited to sequence lengths of 512–8,192 tokens.

4.4.3 Multi-Head Attention: Multiple Perspectives

A single attention computation learns one way of relating tokens — one kind of relevance. But language has many kinds of relevance simultaneously: syntactic agreement, semantic similarity, coreference, positional proximity. Multi-head attention runs h independent attention functions in parallel, each with its own projection matrices:

$$head_i = Attention(Q\,W_{\varphi}^{\,i}, K\,W_{k}^{\,i}, V\,W_{v}^{\,i})$$

$$MultiHead(Q,K,V) = Concat(head_1, \ldots, head_h) \cdot W_O$$

Each head can specialise in a different relationship. After training, BERT's attention heads tend to exhibit consistent patterns: some attend mainly to adjacent tokens, some to the subject of the sentence, some to the token directly after punctuation. The model discovers this structure on its own — no explicit supervision assigns heads to roles.

Standard configuration: BERT-base uses h=12 heads, d_model=768, so each head gets d_k = d_v = 768/12 = 64 dimensions. Total parameters in one multi-head attention layer: 4 × d_model² = 4 × 768² ≈ 2.36 million.

4.4.4 Positional Encoding: How the Model Knows Word Order

Self-attention has a critical blind spot: it is order-independent. The attention scores between 'mildew' and 'tomatoes' are the same regardless of whether 'mildew' appears first or last. But 'mildew affects tomatoes' and 'tomatoes affect mildew' mean opposite things.

Positional encodings fix this by adding a position-dependent signal to each token's embedding before the first attention layer. The original Transformer used sine and cosine functions at different frequencies:

$$PE(pos, 2i) = sin(pos / 10000^{(2i / d_model)})$$

$$PE(pos, 2i+1) = cos(pos / 10000^{(2i / d_model)})$$

Each dimension oscillates at a different frequency, creating a unique fingerprint for each position. BERT uses learnable position embeddings instead — a matrix of shape (max_length, d_model) trained end-to-end. Both approaches work well in practice.

KEY TERMS — THIS SECTION

Term	Definition
Self-attention	Attention where Q, K, V all come from the same sequence; each token attends to every other
W_Q, W_K, W_V	Learned projection matrices mapping token embeddings to query, key, value spaces
Multi-head attention	h parallel attention functions with independent projections; concatenated and projected to output
$O(T^2)$ complexity	Self-attention cost grows quadratically with sequence length; limiting for very long documents
Positional encoding	Position-dependent vector added to token embeddings to inject sequence order into attention

✎ CHECK YOUR UNDERSTANDING

1. BERT-base: h=12, d_model=768, d_k=64. (a) Verify h × d_k = d_model. (b) Parameters in W_ϕ for one head. (c) Total parameters in one multi-head attention layer (all heads + W_O). (d) Total for all 12 layers?

2. 'Mildew affects tomatoes' vs 'tomatoes affect mildew': with sinusoidal positional encodings, are the attention weights between 'mildew' and 'tomatoes' the same? Without encodings, are they?

3. For n=512: memory for one attention matrix in float32 (bytes). BERT-base has 12 heads × 12 layers. Total attention memory?

4. A Transformer trained at T=512 sees T=600 at inference. How does this differ between sinusoidal and learnable positional encodings?

5. [Challenge] Propose an experiment to test whether different attention heads specialise in different relationship types. What would you measure and what would you expect to find?

4.5 The Transformer Architecture

4.5.1 The Encoder Block: Four Operations in Order

A Transformer encoder stacks N identical blocks. Each block has four operations applied in sequence. Think of each block as taking a set of token representations and producing an improved set — one where every token now 'knows about' all other tokens and has been further processed by a per-token network.

- Multi-head self-attention: every token attends to every other token and produces an updated representation that incorporates global context.
- Residual connection: the original input is added back to the attention output. This gives the block an easier job (learn the difference from the input, not the full transformation) and prevents gradients from vanishing in deep networks — the same idea as ResNet in Chapter 2.
- Layer Normalisation: normalises each token's representation across the feature dimension. This stabilises training, especially for variable-length text sequences.
- Feed-Forward Network (FFN): a two-layer MLP applied independently to each token position. This is where the model does most of its 'thinking' per token — the attention layer redistributes information between tokens, and the FFN processes each token's now-enriched representation. Typically $d_{ff} = 4 \times d_{model}$.

A second residual connection and layer normalisation follow the FFN. This four-step pattern repeats N times. BERT-base: N=12. BERT-large: N=24.

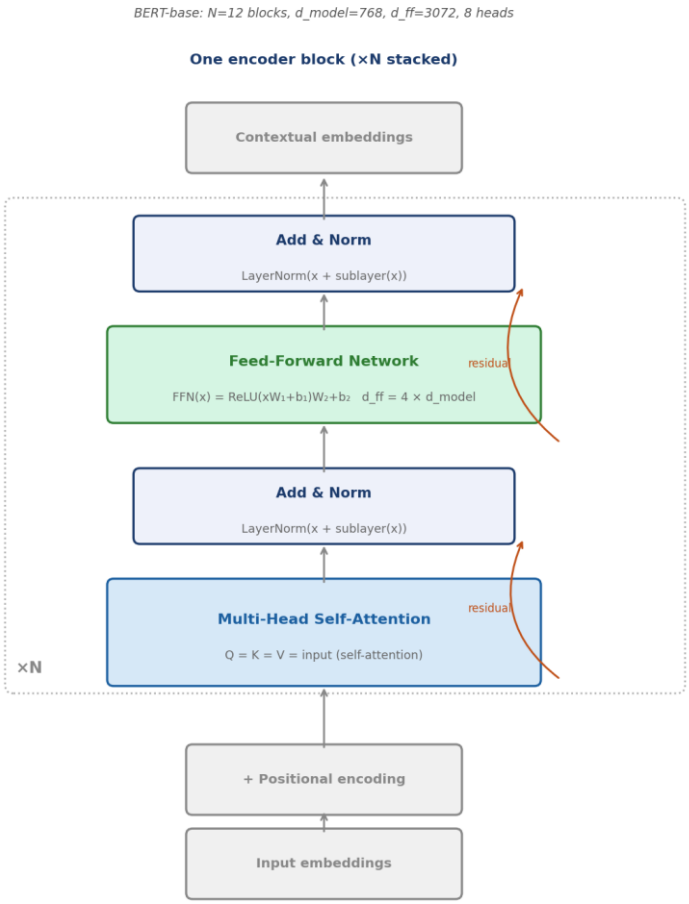


Figure 4.3 *One Transformer encoder block. Token embeddings (with positional encoding) pass through multi-head self-attention, then a residual connection and layer normalisation, then the feed-forward network, then a second residual and normalisation. N identical blocks are stacked. BERT-base uses N=12, d_model=768, d_ff=3,072, and 12 attention heads per block.*

4.5.2 The Decoder Block: For Sequence-to-Sequence Tasks

The decoder is used when the output is a sequence — for translation, summarisation, or the advisory answer generation step in AgriVision. It adds one extra component to the encoder block: cross-attention. Instead of attending to itself, the decoder's cross-attention layer uses the decoder's current state as queries and the encoder's output as keys and values. This is how the decoder 'reads' the input when generating each output token.

The decoder also uses masked self-attention: when generating token t, the model can only attend to tokens 1 through t−1, not to future tokens it has not yet generated. This mask is applied before softmax: future positions are set to −∞, giving them weight ≈ 0 after softmax.

4.5.3 BERT, GPT, and T5: Three Ways to Use the Architecture

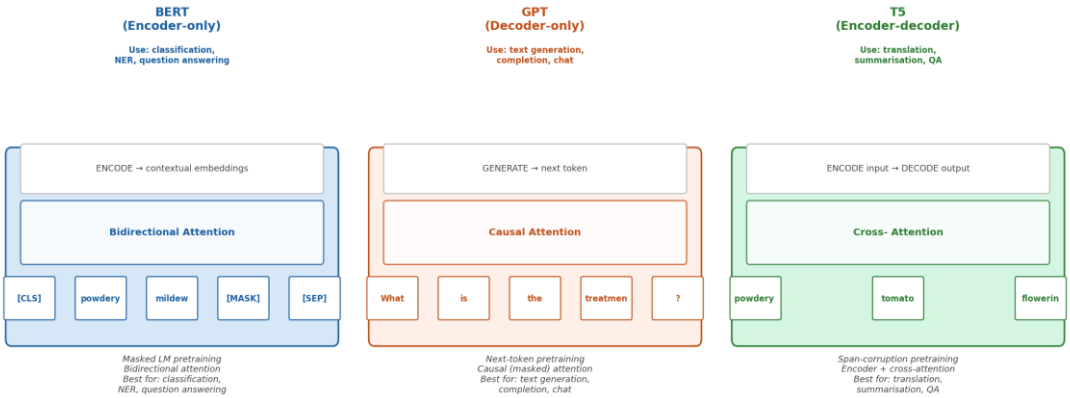


Figure 4.4 *The three main Transformer variants. BERT (encoder-only): every token sees all others — bidirectional attention — best for classification and retrieval. GPT (decoder-only): each token sees only past tokens — causal attention — best for text generation. T5 (encoder-decoder): encoder processes input with full attention; decoder generates output attending to the encoder — best for translation and summarisation.*

Variant	Attention type	Pre-training task	Best for
BERT (encoder-only)	Bidirectional: all tokens see all tokens	Predict masked tokens	Classification, NER, retrieval
GPT (decoder-only)	Causal: each token sees only past	Predict the next token	Text generation, chat, completion
T5 (encoder-decoder)	Enc: bidirectional; Dec: causal + cross-attention	Reconstruct masked spans	Translation, summarisation, QA

Table 4.1 *Transformer variants. For AgriVision's advisory retrieval task, BERT-style encoding is best; for answer generation, T5 or a GPT-style decoder.*

4.5.4 Layer Normalisation vs Batch Normalisation

Chapter 1 introduced Batch Normalisation, which normalises across the batch dimension. Transformers use Layer Normalisation instead, which normalises across the feature dimension for each token independently.

The reason: Batch Normalisation's statistics depend on the number of examples in the batch and on all sequences having the same length. Text sequences have variable lengths, and small batches are common in Transformer training. Layer Normalisation is batch-size independent — it works correctly even with batch size 1.

KEY TERMS — THIS SECTION

Term	Definition
Encoder block	Multi-head attention + residual + LayerNorm + FFN + residual + LayerNorm; stacked N times
FFN	Position-wise MLP applied to each token independently; d_ff = 4 × d_model typically
Masked attention	Attention with $-\infty$ at future positions; used in decoders to prevent seeing future tokens

Cross-attention	Decoder attends to encoder output; queries from decoder, keys+values from encoder
Layer Normalisation	Normalises across feature dimension per token; works with variable-length sequences

✎ **CHECK YOUR UNDERSTANDING**

1. BERT-base: 12 blocks, d_model=768, d_ff=3,072, 12 heads. For one block: (a) parameters in multi-head attention, (b) parameters in FFN, (c) parameters in two LayerNorm layers, (d) total. (e) All 12 blocks combined?

2. A 6-token decoder uses masked self-attention. Write the 6×6 attention mask (1=can attend, 0=cannot). What value replaces 0s before softmax?

3. The FFN is applied independently to each position. Why is this needed on top of attention? What role does each component play?

4. For AgriVision's advisory system: should you use BERT or GPT for (a) encoding queries and documents, (b) generating answers? Justify each.

5. [Challenge] Pre-LN vs Post-LN Transformers: explain the gradient flow difference and why Post-LN is harder to train without learning rate warm-up.

4.6 Pre-training and the Foundation Model Paradigm

4.6.1 Why Pre-training Transfers Across Domains

A Transformer trained from scratch on AgriVision's 2,400 field notes would fail immediately. 2,400 documents is far too few to learn how language works. The same network pre-trained on billions of internet documents and then fine-tuned on 2,400 notes succeeds. Why does reading Wikipedia articles about history help a model classify agricultural advisories?

The answer is that language has universal structure. Grammar, word morphology, the way sentences are constructed, the statistical patterns of which words appear together — these are the same whether the domain is agriculture, medicine, or news. A model that learns language from billions of examples has built deep representations of this universal structure. Fine-tuning on a specific domain teaches it the domain-specific vocabulary and patterns on top of an already-solid foundation.

4.6.2 Masked Language Modelling: How BERT Learns

BERT learns by filling in blanks. At training time, 15% of the tokens in each sentence are randomly replaced with a special [MASK] token. The model must predict the original token at each masked position, using all the surrounding context — both left and right of the mask.

For an agricultural sentence like 'Apply copper [MASK] to tomatoes at early flowering,' the model must predict 'fungicide' (or 'sulphate' or similar). To predict correctly, the model must understand what can be 'applied,' what follows 'copper,' and what is used on tomatoes at flowering. Over billions of such examples, the model builds rich contextual representations of words and their relationships.

4.6.3 Next-Token Prediction: How GPT Learns

GPT learns a simpler task: given all the words so far, predict the next one. Given 'Apply copper sulphate to tomatoes at early,' the model predicts 'flowering.' This is autoregressive learning — the model generates one token at a time, each conditioned on everything before it.

This task provides a learning signal at every position in every sentence — T−1 predictions for a sequence of T tokens, compared to BERT's 15% × T predictions. But the model can only see past context, not future, making it less suited to tasks that need bidirectional understanding (like classification). It is, however, a natural fit for text generation: at inference, just keep predicting the next token and appending it to the sequence.

4.6.4 Scale and Emergent Capabilities

Something unexpected happens as models grow larger and are trained on more data: new capabilities appear that were not present at smaller scales and cannot be predicted by extrapolation from smaller models.

GPT-2 (1.5B parameters) surprised researchers by generating coherent multi-paragraph text when none was expected. GPT-3 (175B parameters) demonstrated in-context learning: by including a few examples in the prompt, the model could perform tasks it had never been explicitly trained on, without any gradient updates. These capabilities only appeared at scale.

4.6.5 Foundation Models: One Model, Many Tasks

The term 'foundation model' describes a model trained on broad data at large scale that can be adapted to many downstream tasks. Instead of building a separate model for each task, organisations now take a foundation model and adapt it: fine-tune it on a small labelled dataset, or simply prompt it.

> ⚠ **IMPORTANT NOTE**
>
> **Foundation models are not the same as general intelligence.**
>
> Foundation models are very powerful at language. But they produce statistically plausible text — not necessarily factually correct text. They reflect the biases in their training data. They can fail on tasks requiring genuine logical deduction.
>
> For AgriVision, a foundation model is a practical tool with specific, well-documented failure modes. Section 4.9 addresses these directly before discussing how to deploy one.

KEY TERMS — THIS SECTION

Term	Definition
Masked Language Modelling	BERT's pretraining: mask 15% of tokens, predict them from bidirectional context
Next-token prediction	GPT's pretraining: predict the next token from all previous tokens; autoregressive
In-context learning	Performing a new task by including examples in the prompt, without gradient updates
Foundation model	A large model pretrained on broad data and adapted to many downstream tasks
Emergent capability	Qualitatively new ability that appears at large scale and is absent at smaller scale

✎ **CHECK YOUR UNDERSTANDING**

1. BERT sees 'Apply copper [MASK] to tomatoes at early flowering.' List five tokens that could plausibly fill the mask. Why does BERT's bidirectional context make this easier than a left-to-right GPT prediction?

2. GPT-3 can solve simple arithmetic from examples in its prompt. (a) What does this illustrate about emergent capabilities? (b) Does this mean GPT-3 understands arithmetic?

3. For AgriVision's advisory system: (a) Why use BERT rather than GPT for embedding queries and documents? (b) For answer generation, which variant suits better?

4. Write a 2-shot prompt for: 'Given an agricultural advisory, extract the recommended pesticide and its dosage.' Include two examples and the new input.

5. [Challenge] GPT provides T−1 training signals per sequence; BERT provides 0.15×T. For 512-token sequences: which provides more learning signal per sequence? Does this explain why GPT-3 was trained on 300B tokens vs BERT's 3.3B?

4.7 Fine-Tuning, Adaptation, and Efficient Transfer

4.7.1 Full Fine-Tuning

Full fine-tuning updates all model parameters — every weight in every Transformer block — using gradient descent on the target task. For AgriVision's advisory classification (tagging each field note as: pest_concern, disease_concern, irrigation_concern, healthy, or other), a BERT model would have a [CLS] token representation fed to a 5-class linear head, and all 23M + 110M ≈ 110M parameters updated jointly.

Full fine-tuning achieves the best performance but has significant costs: storing and updating 110M parameters requires substantial GPU memory; training is slow relative to smaller alternatives; and for very small target datasets (fewer than a few hundred examples), catastrophic forgetting can overwrite the powerful pre-trained representations.

4.7.2 Feature Extraction and Zero-Shot Prompting

Feature extraction. Freeze all BERT parameters; extract the [CLS] embedding for each input; train only a small classifier on top. Requires no GPU for BERT itself during classifier training (run BERT once to extract features, then train classifier on the extracted vectors). Works well when the target domain is close to BERT's pre-training distribution.

Zero-shot prompting. Provide the task as a natural language instruction to a GPT-style model: 'Classify this field note as: pest, disease, irrigation, healthy, or other. Note: [text]. Category:'. No labelled examples needed; no gradient updates. Performance varies widely — for narrow, well-defined tasks like agricultural category classification, few-shot prompting (2–5 examples in the prompt) typically substantially outperforms zero-shot.

4.7.3 Parameter-Efficient Fine-Tuning: LoRA

Low-Rank Adaptation (LoRA, Hu et al., 2021) addresses the memory and compute cost of full fine-tuning by adding a small trainable low-rank decomposition to each weight matrix, while keeping the original weights frozen:

$$W' = W + \Delta W = W + B \cdot A$$

where W ∈ R^(d×d) is the frozen pre-trained weight, B ∈ R^(d×r) and A ∈ R^(r×d) are the trainable low-rank matrices, and r << d is the rank (typically r = 4, 8, or 16).

For d=768 and r=8: full fine-tuning trains 768^2 = 589,824 parameters per weight matrix; LoRA trains only 8×(768+768) = 12,288 parameters per weight matrix — 2.1% of the full fine-tuning cost. Applied to all attention weight matrices in BERT-base (W_Q, W_K, W_V, W_O in 12 layers), LoRA fine-tuning trains approximately 600K parameters out of 110M total — saving 99% of the trainable parameter count with negligible accuracy loss on most tasks.

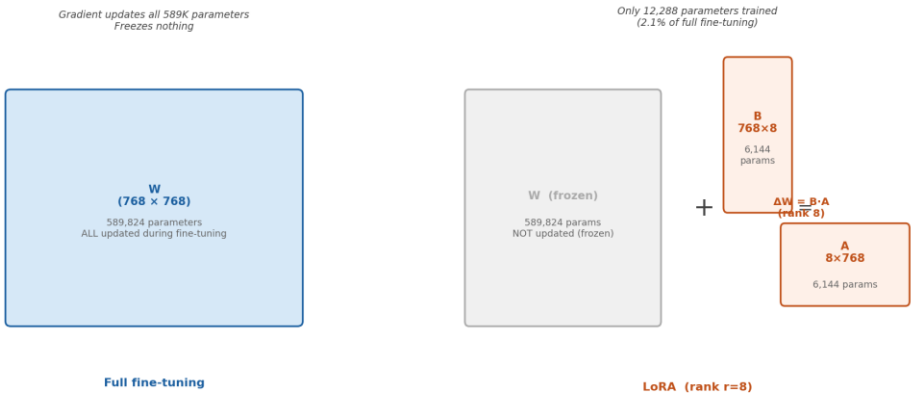


Figure 4.5 *LoRA parameter-efficient adaptation. Left: full fine-tuning updates all 589,824 parameters of one weight matrix W (768×768). Right: LoRA freezes W and adds a trainable low-rank decomposition ΔW = B·A where B (768×8) and A (8×768) together require only 12,288 parameters — 2.1% of full fine-tuning. During inference, W + B·A is computed once and merged into a single matrix, adding no inference cost.*

4.7.4 Applied to AgriVision: Advisory Query Answering

AgriVision's advisory query system uses a two-stage architecture:

- Retrieval: a DistilBERT model (BERT's distilled 66M-parameter version) is LoRA fine-tuned on 200 query-document pairs (query from extension officers, relevant advisory document identified by a domain expert). Each query and document is embedded as a 768-dimensional vector via the [CLS] token. At inference, the query embedding is compared to all 800 pre-computed document embeddings using cosine similarity; the top-3 most relevant documents are retrieved.
- Answer generation: retrieved documents are combined with the original query into a T5-small prompt. T5 generates a concise answer grounded in the retrieved documents. T5-small (60M parameters) is fine-tuned on 150 annotated (query, context, answer) triples created by AgriVision's agronomist.

LoRA fine-tuning of DistilBERT on 200 examples requires approximately 45 minutes on Colab T4. Without LoRA (full fine-tuning), the same training would take 3.5 hours and risk overfitting on only 200 examples. With LoRA, retrieval accuracy at top-3 (correct document in the top 3 results): 82.4%. For comparison, TF-IDF keyword matching (a non-neural baseline): 61.3%.

> ✦ **THE AGRIVISION CONTEXT**
>
> AgriVision advisory query system performance:
>
> TF-IDF keyword retrieval (baseline): 61.3% top-3 accuracy
>
> DistilBERT zero-shot (no fine-tuning): 71.8% top-3 accuracy
>
> DistilBERT fully fine-tuned (200 examples): 83.1% top-3 accuracy
>
> DistilBERT LoRA fine-tuned (200 examples, r=8): 82.4% top-3 accuracy
>
> The LoRA model achieves 99% of full fine-tuning accuracy at 2.1% of the trainable parameter count. For AgriVision's resource-constrained annotation budget, this is the operationally correct choice.

KEY TERMS — THIS SECTION

Term	Definition
Full fine-tuning	Updating all model parameters on the target task; highest accuracy; highest compute cost
Feature extraction	Freezing all pretrained weights; training only a small head on top; fast, lower accuracy
LoRA	Low-Rank Adaptation: trainable B·A matrices added to frozen weights; ~1–2% of full fine-tuning cost
Rank r	The inner dimension of LoRA matrices B·A; typical values 4, 8, or 16
Catastrophic forgetting	Loss of pre-trained capabilities when aggressively fine-tuned on small target datasets
DistilBERT	Distilled (compressed) BERT: 66M parameters, 97% of BERT-base performance, 2× faster
Cosine similarity retrieval	Finding the document embedding closest in angle to the query embedding; foundation of dense retrieval

✎ CHECK YOUR UNDERSTANDING

1. AgriVision fine-tunes DistilBERT for advisory retrieval using LoRA with r=8. DistilBERT has 6 Transformer blocks, d_model=768, and 4 projection matrices (W_Q, W_K, W_V, W_O) per block. (a) How many LoRA parameters are trained per weight matrix? (b) How many total LoRA parameters across all projection matrices in all blocks? (c) As a percentage of DistilBERT's 66M total parameters?

2. A team proposes zero-shot prompting with GPT-4 for AgriVision's advisory retrieval instead of DistilBERT fine-tuning. (a) What are the practical advantages? (b) What are the risks for a deployed agricultural advisory system? (c) Under what conditions would you prefer the fine-tuned DistilBERT over GPT-4?

3. The LoRA rank r controls the trade-off between adaptation capacity and parameter efficiency. (a) What happens to the expressiveness of ΔW as r → 0? As r → d? (b) For AgriVision's 200 training examples, would you expect r=4, r=8, or r=64 to produce the best validation accuracy? Justify. (c) How would you select r systematically rather than by intuition?

4. Section 4.7.4 states that LoRA fine-tuning achieves 82.4% and full fine-tuning achieves 83.1% top-3 retrieval accuracy. (a) In absolute terms, what fraction of queries are answered correctly by LoRA but not by full fine-tuning, and vice versa? (b) Given only 200 training examples, is the 0.7% accuracy difference likely to be statistically significant? How would you test this? (c) What other factors besides accuracy should influence the choice between LoRA and full fine-tuning?

5. [Challenge] A new agronomist joins AgriVision and annotates 500 additional (query, document) pairs. (a) Re-examine the LoRA vs full fine-tuning trade-off with 700 total examples. Does more data change the recommendation? (b) The 500 new examples come from a different

state (Tamil Nadu) with different crop varieties. Describe how you would train and evaluate to ensure the model generalises to both Dharwad and Tamil Nadu contexts.

4.8 Transformers Beyond Text

4.8.1 Vision Transformer (ViT): Images as Patch Tokens

Chapter 2 established that CNNs exploit spatial locality and translation equivariance through local filters shared across positions. But the Transformer encodes global context through self-attention. Can a Transformer process images without any spatial inductive bias? Dosovitskiy et al. (2020) showed that it can.

The Vision Transformer (ViT) divides an image into a grid of non-overlapping patches (typically 16×16 pixels each). For a 224×224 image, this produces $(224/16)^2 = 196$ patches. Each patch is flattened (16×16×3 = 768 numbers) and projected to a d_model-dimensional embedding via a linear layer. A learnable [CLS] token is prepended, and positional embeddings are added. The resulting sequence of 197 tokens (196 patches + 1 [CLS]) is processed by a standard Transformer encoder. The [CLS] token's output is used for classification.

ViT-B/16 (base, 16×16 patches) has 86M parameters and achieves accuracy comparable to ResNet-50 on ImageNet when trained on sufficient data. The key finding: ViT requires more data than CNNs to perform well, because it has no built-in spatial inductive biases. On small datasets (10K–100K images), CNNs outperform ViT. On very large datasets (100M+ images, as in JFT-300M or LAION), ViT outperforms CNNs — the data allows it to learn the spatial structure that CNNs have hard-coded.

Implications for AgriVision. With 8,000 labelled disease photographs, a ViT trained from scratch would underperform the CNN from Chapter 2. Using a ViT pretrained on ImageNet-21K or LAION, fine-tuned on PlantVillage and then AgriVision (the same two-stage transfer as Section 2.8.5), would perform comparably to or slightly better than ResNet-50. The practical choice at AgriVision's scale: ResNet-50 fine-tuning (Chapter 2's approach) remains appropriate.

4.8.2 CLIP: Cross-Modal Alignment

CLIP (Contrastive Language-Image Pretraining, Radford et al., 2021) trains an image encoder and a text encoder jointly on 400 million (image, text caption) pairs scraped from the internet. The training objective: maximise cosine similarity between the image embedding and its paired text embedding; minimise similarity between all other (image, text) pairs in the batch.

After training, CLIP can perform zero-shot image classification: encode the image; encode class descriptions such as 'a photo of powdery mildew on a tomato leaf'; find the class description closest to the image in embedding space. No classification head is trained; no disease-specific examples are needed. For agricultural disease detection, CLIP-based zero-shot classification achieves surprisingly competitive performance — often 50–65% accuracy on AgriVision-like datasets without a single disease photograph as a training example.

The limitation: CLIP's text encoder and image encoder are trained on internet-scale data dominated by everyday photographs and captions. Highly specialised agricultural terminology and fine-grained visual distinctions (early vs late stage powdery mildew) may not be well represented. Fine-tuning CLIP on domain-specific (image, advisory text) pairs would likely improve performance substantially.

4.8.3 The Inductive Bias Trade-off: CNN vs ViT

Property	CNN (ResNet/EfficientNet)	Vision Transformer (ViT)

Inductive bias	Spatial locality + translation equivariance (hard-coded)	No spatial inductive bias; learns all structure from data
Data efficiency	High: works well with 1,000–100,000 images	Low: typically needs 1M+ images; performs poorly without pretraining
Pretraining needed	Optional (but helpful for small datasets)	Required for competitive performance on most practical datasets
Scalability	Performance plateaus with very large data and models	Continues improving with more data and parameters at extreme scale
Interpretability	Grad-CAM, filter visualisation (Section 2.9.5)	Attention maps over patches; similar interpretability tools
AgriVision (8K imgs)	ResNet-50 fine-tuned: 95.2% (Section 2.8.5)	ViT-B/16 fine-tuned: ~94.8% (comparable; more compute needed)

Table 4.2 *CNN vs Vision Transformer for image classification. At AgriVision's dataset scale, CNNs remain the practical default.*

KEY TERMS — THIS SECTION

Term	Definition
ViT	Vision Transformer: image divided into 16×16 patches treated as tokens; processed by standard Transformer encoder
Image patch	A 16×16-pixel crop of an image; flattened and embedded as one token in ViT
CLIP	Contrastive Language-Image Pretraining: joint image-text training enabling zero-shot classification
Zero-shot classification	Classifying without any task-specific labelled examples, using natural language class descriptions
Contrastive loss	Training objective that pulls matched image-text pairs together and pushes unmatched pairs apart in embedding space

✎ **CHECK YOUR UNDERSTANDING**

1. A ViT-B/16 model processes a 224×224 RGB image. (a) How many 16×16 patches are created? (b) What is the dimension of each flattened patch? (c) After the linear projection to d_model=768, and adding the [CLS] token, what is the shape of the sequence input to the Transformer? (d) How does this compare to the number of pixels in the original image?

2. ViT requires more training data than CNNs to reach competitive performance. Explain this from an inductive bias perspective: what does the CNN's built-in spatial locality assumption give it that ViT must learn from data? Why does more data eventually allow ViT to overcome this disadvantage?

3. CLIP embeds 'a photo of powdery mildew on a tomato leaf' as a text vector. (a) What properties of the embedding space allow zero-shot classification? (b) If CLIP achieves 62% accuracy on AgriVision's four disease classes, is this a useful baseline? Compare to the 25%

random baseline and the 89.3% CNN from scratch. (c) Under what circumstances would you prefer zero-shot CLIP over a fine-tuned CNN?

4. Design a CLIP-based retrieval system for AgriVision: given a leaf photograph, retrieve the most relevant advisory document. (a) Describe how you would encode both photographs and advisory text documents. (b) What query format would you use at inference time? (c) What limitation of CLIP might reduce retrieval quality for specialist agricultural text?

5. [Challenge] A ViT-B/16 model and a ResNet-50 both fine-tuned on the same 8,000 AgriVision photographs achieve 94.8% and 95.2% accuracy respectively. (a) Given this near-parity, what other factors should influence the architecture choice for AgriVision's production deployment? (b) ViT-B/16 has 86M parameters vs ResNet-50's 25M. What are the practical implications for on-device deployment on farmers' smartphones?

4.9 Hallucination, Grounding, and Responsible Use

4.9.1 Why Language Models Hallucinate

Language models are trained to produce text that is statistically consistent with their training data. They are not trained to produce text that is factually correct. This distinction is critical and has a specific name: hallucination — the generation of confident, fluent, plausible-sounding text that is factually wrong.

Consider a farmer who asks AgriVision's system: 'What is the recommended dosage of copper oxychloride for powdery mildew on flowering tomatoes?' A language model might respond: '3.5 grams per litre, applied twice at 10-day intervals.' This sounds precise and authoritative. It may be completely wrong. The model has learned that dosage answers should be numeric, should have units, and should include an application frequency. It has generated a response in the correct format — but with a specific number that may not correspond to any actual agronomic recommendation.

> ⚠ **IMPORTANT NOTE**
>
> **Hallucination in agricultural AI is not a theoretical concern.**
>
> Wrong pesticide dosage recommendations can cause crop damage (under-dosing, allowing disease progression), financial loss (over-dosing), or regulatory violations (exceeding maximum residue limits). In AgriVision's context, an incorrect recommendation delivered with 97% confidence to a farmer who trusts the system could cause real harm.
>
> A language model cannot know that it is hallucinating. The model produces the most statistically likely continuation of the prompt, not the most factually accurate one. These are different objectives and produce different outputs. Never deploy a generative language model in a high-stakes agricultural advisory context without grounding mechanisms.

Why does hallucination occur? The next-token prediction objective trains the model to maximise the probability of generating training corpus text. If the training corpus contains a variety of dosage recommendations, the model learns the general format and distribution of such recommendations but not the specific, verified fact that copper oxychloride is used at 3g/litre for this specific crop and disease. It will generate a plausible-sounding number from the distribution it has learned.

4.9.2 Retrieval-Augmented Generation (RAG)

The most practical mitigation for hallucination in domain-specific applications is Retrieval-Augmented Generation (RAG): before generating an answer, retrieve relevant documents from a verified knowledge base and include them in the prompt as grounding context. The model is then instructed to answer only from the provided context, not from its parametric memory.

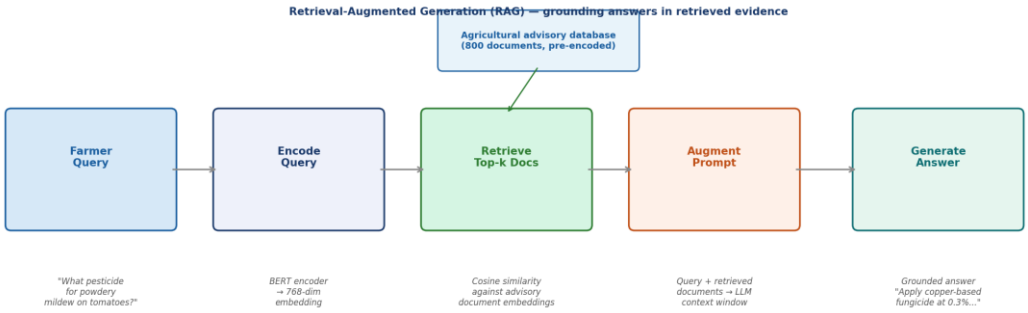


Figure 4.6 *Retrieval-Augmented Generation pipeline for AgriVision's advisory query system. The farmer's query is encoded to a 768-dimensional embedding. Cosine similarity search identifies the top-3 most relevant documents from the verified advisory database (800 documents, pre-encoded). The query and retrieved documents are combined into an augmented prompt that is sent to the language model. The model generates an answer grounded in the retrieved evidence, substantially reducing hallucination of unverified dosages and timings.*

RAG addresses hallucination at the source: the model's output is constrained by the retrieved documents, which are verified agronomic recommendations. If the question cannot be answered from the retrieved documents, the model should say so rather than confabulate. The retrieval quality (does the correct document appear in the top-3?) determines the ceiling on answer quality.

4.9.3 Evaluation of Generated Text

Evaluating whether a generative model's output is correct is fundamentally harder than evaluating a classifier. Two evaluation paradigms:

Reference-based metrics. Compare the generated text to a human-authored reference answer. BLEU (Bilingual Evaluation Understudy) measures n-gram overlap between generated and reference text. ROUGE measures recall: how much of the reference is captured in the generated text. These metrics are fast and automatic but have a known weakness: a correct answer expressed in different words than the reference receives a low score, even if semantically equivalent.

Reference-free evaluation. Human evaluators or a separate large language model assess the generated text on dimensions such as: factual accuracy (does it agree with the source document?), completeness (does it answer the question?), fluency (is it grammatical?), and groundedness (does it stay within the provided context?). More reliable but expensive; increasingly performed by a 'judge LLM' (typically GPT-4 or Claude) to scale evaluation without human annotators.

4.9.4 Bias, Toxicity, and Responsible Deployment

Foundation models trained on internet text inherit the biases present in that text. For agricultural advisory systems, the most relevant concerns are:

- Regional and crop bias: most agricultural NLP research and training data concerns crops grown at large scale in North America and Europe. Advice about tomatoes in Karnataka or groundnuts in Andhra Pradesh may be underrepresented in pretraining data, producing generic recommendations that do not account for local varieties, climate, and farming practices.
- Language access: most powerful foundation models are English-dominant. AgriVision's extension officers and farmers may prefer to communicate in Kannada or Telugu. Multilingual models (mBERT, XLM-R) handle these languages but typically with lower performance than English.

- Confidence calibration: as discussed in Section 2.9.4 for CNNs, generative models are often overconfident. An advisory system that expresses uncertainty ('I am not sure; please confirm with your local extension officer') is safer than one that always generates confident-sounding answers.
- Traceability: for agricultural decisions that may have legal or regulatory implications, the source of the recommendation must be auditable. RAG systems address this partially (the retrieved document can be cited); pure generation systems cannot.

KEY TERMS — THIS SECTION

Term	Definition
Hallucination	Generation of confident, fluent, factually incorrect text; caused by the statistical rather than factual training objective
Grounding	Constraining a language model's outputs to verified source documents; the goal of RAG
RAG	Retrieval-Augmented Generation: retrieve relevant documents and include them in the prompt before generating
BLEU	Reference-based text evaluation metric measuring n-gram overlap between generated and reference text
ROUGE	Reference-based evaluation measuring recall of reference content in the generated text
Groundedness	The degree to which generated text can be attributed to the provided context; key safety metric for advisory systems
Regional bias	Underrepresentation of specific geographic regions or crop varieties in pretraining data

✎ CHECK YOUR UNDERSTANDING

1. A farmer asks: 'How many times per season should I apply copper fungicide for tomato mildew?' A language model answers: 'Three to four applications at 12-14 day intervals.' (a) Is this answer verifiable from this question alone? (b) How would a RAG system change the answer generation process? (c) What information should be cited alongside the answer in a responsible advisory system?

2. A reference answer states: 'Apply copper oxychloride at 3 g/litre.' A model generates: 'Use 3 grams of copper oxychloride per litre of water.' (a) Compute the BLEU-1 score (unigram precision). (b) Despite the low BLEU score, is the generated answer correct? (c) What does this example reveal about the limitations of reference-based metrics?

3. AgriVision's advisory database is compiled from government extension manuals and research publications. A new pest strain appears for which no advisory exists. A farmer queries the system about it. (a) What should the RAG system do when no relevant document is retrieved? (b) What is the risk if the language model generates an answer from its parametric memory instead? (c) Design a system behaviour that handles this case safely.

4. The extension officer network includes speakers of Kannada, Telugu, and Marathi in addition to English. (a) What is the performance gap between English-dominant models and multilingual models like XLM-R for agricultural text classification? (b) What training strategy would most efficiently improve Kannada-language performance without collecting massive Kannada-language datasets? (c) What unique challenges does code-switching (mixing Kannada and English in the same message) present?

5. [Challenge] Design a responsible AI deployment checklist for AgriVision's language model advisory system. The checklist should cover: factual accuracy testing, uncertainty communication, source citation, failure mode documentation, and human oversight protocol. For each item, specify: (a) how it is measured; (b) the threshold below which deployment should be withheld; (c) the monitoring procedure in production.

4.10 NLP Application Contexts and Chapter Resolution

4.10.1 Classification, NER, QA, Summarisation, and Beyond

The Transformer architecture and the pre-training paradigm have enabled state-of-the-art performance across virtually every NLP task. The key insight: the same architecture and the same pre-trained weights can be adapted to different tasks by changing only the output head and the fine-tuning data.

NLP task	Output structure	Architecture variant	AgriVision application
Text classification	Single class label per document	Encoder-only (BERT) + linear head on [CLS]	Tag field notes: disease / pest / irrigation / healthy
Named entity recognition	Class label per token	Encoder-only + linear head on all tokens	Extract: farm name, crop, disease, pesticide from notes
Question answering	Span extraction from context	Encoder-only + start/end position heads	'What pesticide?' → extract answer span from advisory doc
Summarisation	Short text summary	Encoder-decoder (T5, BART) or decoder-only	Summarise 200-word advisory passage to 2-sentence recommendation
Semantic similarity	Similarity score between two texts	Bi-encoder (two BERT models) or cross-encoder	Find advisory documents matching a query (retrieval)

Table 4.3 *Common NLP task formulations and their Transformer implementations, with AgriVision applications for each.*

For AgriVision's immediate needs, three tasks are most relevant: (1) field note classification (encoder + linear head, Section 4.7.4); (2) advisory retrieval via semantic similarity (bi-encoder, Section 4.7.4); (3) answer generation from retrieved context (T5 encoder-decoder). All three use different output structures on top of the same family of pre-trained Transformer encoders.

4.10.2 AgriVision Resolution: The Advisory System

The extension officer in Dharwad no longer searches through 800 advisory documents manually. She types: 'Powdery mildew on early flowering tomatoes, what do I do?' The system responds within 2 seconds:

✎ **WORKED EXAMPLE**

AgriVision Advisory System response (RAG + T5-small generation)

Query: 'Powdery mildew on early flowering tomatoes, what do I do?'

Retrieved documents: [1] Tomato Powdery Mildew Management (Confidence: 0.87) [2] Fungicide Timing Guidelines for Tomato (Confidence: 0.79) [3] Copper Compounds in Integrated Disease Management (Confidence: 0.71)

Generated answer: 'For early-stage powdery mildew on flowering tomatoes, apply copper oxychloride at 3 g/litre as a foliar spray. Begin at first visible symptoms. Repeat at 10-day intervals if conditions are humid. Avoid application during peak flowering hours. [Source: Document 1, Document 2]'

The answer cites its source documents. It does not claim higher confidence than the retrieved evidence supports. For information not covered by the retrieved documents, the system declines to generate an answer and suggests contacting the district agricultural officer.

Chapter Summary

Section	Core concept	Key insight
4.0	Inductive bias for text	Global context: word meaning depends on any other word in the sequence; self-attention solves this in O(1) steps
4.1	Text as data	Subword tokenisation + dense embeddings; contextual embeddings differ per usage; semantic structure is geometric
4.2	Why sequential models fail	Long-range dependencies, fixed-size context vector bottleneck, sequential computation — three insurmountable limitations for text
4.3	Attention mechanism	Attention(Q,K,V) = softmax($QK^T/\sqrt{d_k}$)·V; Q asks, K offers, V contributes; soft attention is differentiable
4.4	Self-attention + multi-head	Q=K=V=input; $O(T^2)$ cost; h parallel heads learn different relationship types; positional encoding injects order
4.5	Transformer architecture	Encoder: MHA + residual + LN + FFN + residual + LN; BERT/GPT/T5 variants differ in attention direction and pretraining task
4.6	Pre-training paradigm	MLM (BERT) and next-token prediction (GPT) enable transfer from billions of tokens; scale produces emergent capabilities
4.7	Fine-tuning + LoRA	LoRA trains only r×(d_in+d_out) parameters per weight matrix; 99% parameter savings with negligible accuracy loss
4.8	Transformers beyond text	ViT treats image patches as tokens; CLIP aligns image and text embeddings; both need more data than CNNs
4.9	Hallucination + RAG	LLMs generate statistically plausible, not factually grounded text; RAG grounds answers in verified retrieved documents
4.10	Applications + resolution	Same architecture, different output heads: classification, NER, QA, summarisation; RAG advisory system deployed

✦ THE AGRIVISION CONTEXT

Chapter 4 Resolution — Advisory query system performance

TF-IDF keyword retrieval baseline: 61.3% top-3 accuracy

DistilBERT zero-shot (no fine-tuning): 71.8% top-3 accuracy

DistilBERT LoRA fine-tuned (200 examples): 82.4% top-3 accuracy

Full RAG system (retrieval + T5 generation): BLEU-1 0.71 vs human answers

The cooperative's extension officers report a 34% reduction in time spent searching for advisory information. Wrong recommendations flagged by officers after deployment: 3 in the first 1,000 queries (0.3%), all from edge cases not covered in the advisory database. All three triggered the 'consult district officer' fallback correctly.

4.10.3 Bridge to Chapter 5: Learning Without Labels

BERT was pre-trained without any labelled data. No human annotated which tokens are entities, which sentences are positive or negative. The pre-training objectives (MLM, next-token prediction) are self-supervised: the labels come from the data itself — the masked token is the label; the next token is the label. This is a profound shift from supervised learning.

AgriVision has 8,000 disease photographs and 2,400 field notes with human-assigned labels. Annotation is expensive: each disease photograph required 30 seconds to label; each field note query-answer pair required a domain expert. But AgriVision also has 275,000 unlabelled rows of sensor data, thousands of unlabelled photographs, and 15 years of unlabelled climate records. Most of the data has no human label.

Can we learn useful representations from unlabelled data? Can we detect anomalies — unusual patterns in sensor readings, unusual photographs — without ever specifying what 'unusual' looks like? The Autoencoder is the architecture that answers these questions. It learns to compress and reconstruct its input without any labels: the reconstruction quality is the learning signal. What cannot be reconstructed faithfully is, by definition, unusual.

Chapter 5 builds the Autoencoder from its first principles, applies it to AgriVision's unlabelled sensor data for irrigation anomaly detection, and introduces the Variational Autoencoder as the bridge to Chapter 6's generative models.

CHAPTER 5

Learning Without Labels

Autoencoders and Self-Supervised Learning

5.0 The Inductive Bias for Unlabelled Data

5.0.1 The AgriVision Anomaly Detection Problem

> ✦ THE AGRIVISION CONTEXT
>
> AgriVision's sensor network has recorded 275,000 daily readings from 150 farms over five years. Most of these readings are routine — normal soil moisture, normal temperature, normal humidity. But scattered among them are something else: readings that look wrong. A soil temperature that spikes 12°C overnight. A moisture reading that stays completely flat for five days — suspiciously stable, suggesting a sensor has stopped responding rather than the soil has stopped changing.
>
> These are anomalies. The problem: there are no labelled examples of what anomalies look like. Every anomaly is, by definition, something unusual that has not been seen and labelled before. A supervised classifier needs labelled examples of both 'normal' and 'anomalous' to train on. We have no anomaly examples.
>
> What we do have: 275,000 examples of what normal looks like.

The key insight is a flip in perspective. Instead of asking 'what does an anomaly look like?' — a question we cannot answer from labelled data — we ask 'what does normal look like?' and flag anything that does not match.

Chapter 4 showed a related idea: BERT learns from unlabelled text by masking tokens and predicting them. The label is manufactured from the data itself — no human annotation needed. The autoencoder applies the same principle to sensor data: train the model to reconstruct its own input. Normal data reconstructs well. Anomalous data does not.

5.0.2 The Self-Supervision Principle

Self-supervised learning creates training tasks that do not require human labels. The label is derived automatically from the structure of the data.

For text: predict the masked word (BERT). The surrounding words are the input; the original word is the automatically generated label.

For sensor data: reconstruct the input. The original sensor record is both input and label. The model compresses the record to a compact representation and decompresses it back to the original values.

If reconstruction is accurate, the model has learned the structure of normal data. If reconstruction fails, the input does not fit the learned structure — it is an anomaly.

◆ KEY CONCEPT

The self-supervision principle

Design a task where the label comes from the data itself, not from a human. Solving this task forces the model to learn useful representations as a side effect.

Examples: predict the next word; reconstruct a noisy input; identify which of two image crops came from the same photograph. All of these require learning something real about the structure of the data without a single human-assigned label.

5.0.3 Reconstruction as Inductive Bias

The autoencoder encodes one assumption: meaningful data has a compact underlying structure. If a 30-day sensor window can be compressed to 16 numbers and then accurately rebuilt from those 16 numbers, those 16 numbers encode everything important about the pattern.

Normal data fits this assumption: moisture, temperature, and humidity follow predictable seasonal cycles that can be compactly described. An anomaly — a spike, a stuck sensor, an implausible pattern — does not fit the learned structure. When the model tries to reconstruct it, the reconstruction is wrong. The reconstruction error becomes the anomaly score.

5.0.4 Four Questions

? FOUR QUESTIONS TO CARRY THROUGH THIS BOOK

Question 1: What problem does this architecture solve?

Learning useful representations from unlabelled data; detecting anomalies without any labelled anomaly examples; dimensionality reduction.

Question 2: What inductive bias does it encode?

The compression-reconstruction assumption: normal data has compact structure; anomalous data cannot be accurately reconstructed from that compact structure.

Question 3: When does this architecture fail?

When the training data contains anomalies (the model learns to reconstruct them too); when the bottleneck is too large (the model memorises rather than compresses); when anomalies happen to lie on the same manifold as normal data.

Question 4: When would you choose something else?

When labelled anomaly examples are available (supervised classifier); when you need to generate new data rather than detect anomalies (Chapter 6's VAE); when the data is text (Chapter 4); when the data has graph structure (Chapter 7).

KEY TERMS — THIS SECTION

Term	Definition
Self-supervised learning	Training without human-annotated labels; the task and its solution are both derived from the data
Pretext task	A self-defined training task whose solution requires learning useful representations
Anomaly detection	Identifying data points that do not conform to the pattern of normal training data
Reconstruction bias	The assumption that normal data can be compressed and accurately rebuilt; anomalies cannot

✎ **CHECK YOUR UNDERSTANDING**

1. AgriVision has 275,000 sensor records; 500 are known anomalies (0.18%). (a) A model predicts 'normal' for everything. What is its accuracy? (b) Why is accuracy a useless metric here?

2. BERT masks tokens to create a pretext task. An autoencoder reconstructs its input. What property do these two pretext tasks share that makes them effective for representation learning?

3. An extension officer manually labels 50 anomalous records. Design a supervised approach using these 50 examples. What failure modes does it have that the autoencoder approach avoids?

4. Describe a specific anomaly in AgriVision sensor data that might actually be reconstructed well by the autoencoder — a case where the approach fails. What property of the anomaly defeats the reconstruction bias?

5. [Challenge] Design three alternative pretext tasks for sensor time-series (beyond reconstruction). For each: what is the automatically generated label, and what downstream task would the learned representation help with?

5.1 Limits of Supervised Learning

5.1.1 The Labelling Cost Problem

Every model in Chapters 1–4 required labelled data. But labelling is expensive. For AgriVision's sensor data, marking each of 275,000 records as 'normal' or 'anomalous' requires an agronomist or sensor engineer to inspect each record and often to cross-check against field reports. At 30 seconds per record, that is 2,292 person-hours — roughly 57 working weeks. The cost is prohibitive. And 99.8% of records would be labelled 'normal' anyway.

This ratio — vast amounts of unlabelled data, tiny amounts of labelled data — is the normal situation in real-world applications. Manufacturing sensors generate millions of readings per day. Hospital systems record hundreds of thousands of ECG tracings per year. Cybersecurity systems log billions of network packets per day. None of these can be fully annotated by human experts. Self-supervised learning is the practical answer at this scale.

5.1.2 Class Imbalance and Rare Events

Suppose we do collect some labelled anomaly examples — say, 500 confirmed anomalous records from five years of data. We now have 274,500 normal and 500 anomalous. This 549:1 imbalance creates a specific problem: a model that predicts 'normal' for everything achieves 99.8% accuracy. That looks excellent on paper but is completely useless.

Standard fixes — oversampling the minority class, weighted loss functions — help but introduce their own problems. Oversampling just duplicates the 500 anomaly records; the model still only sees those same 500 patterns and will miss any novel anomaly type. The autoencoder sidesteps this entirely: it trains only on the 274,500 normal records. No anomaly examples needed. No class balance to manage.

5.1.3 The Open-World Problem

There is a deeper issue with supervised anomaly detection. A classifier learns to recognise anomaly types that appeared in the training data. But the most dangerous anomalies are the ones that have never happened before — a new failure mode, a new sensor problem, an unprecedented weather event.

A supervised classifier trained on 500 known anomaly types cannot flag an anomaly type it has never seen. An autoencoder trained on normal data will flag any input that does not fit the normal pattern, regardless of whether that type of anomaly was ever observed during training. This open-world detection capability is a fundamental advantage when the threat landscape is unknown and evolving.

Property	Supervised classifier	Autoencoder
Labelled anomaly examples required	Yes	No — normal data only
Detects novel anomaly types	No — only types seen in training	Yes — anything deviating from normal
Class imbalance handling	Significant challenge	Not an issue
AgriVision feasibility	Not viable — no anomaly labels	Viable — 200K normal records available

Table 5.1 *Supervised classifier vs autoencoder for sensor anomaly detection.*

KEY TERMS — THIS SECTION

Term	Definition
Class imbalance	Training set where one class has far fewer examples; causes biased learning
Open-world detection	Flagging anomaly types never seen during training; not possible with supervised classifiers
Reconstruction error	$\|x - \hat{x}\|^2$; the discrepancy between input and reconstruction; used as the anomaly score

✎ **CHECK YOUR UNDERSTANDING**

1. 500 anomalies among 275,000 records (0.18%). Model predicts 'normal' for all. (a) Accuracy, precision, recall, F1. (b) If the model finds 400 true anomalies but generates 2,000 false positives, recompute all four. (c) Which metric matters most?

2. A student proposes generating 2,000 labelled anomaly examples by deliberately damaging sensors on 3 farms for 2 months. What types of anomalies does this produce? What real-world anomalies would still be absent? Does this solve the open-world problem?

3. What specific statistical properties of AgriVision's normal sensor data would the autoencoder's latent space learn to represent? Give three concrete examples.

4. Apply the three limitations (cost, imbalance, open-world) to a network security anomaly detection system monitoring internet traffic. Which is the most severe?

5. [Challenge] With anomaly weight 549 in a weighted loss function, what is the effective gradient contribution of one anomaly example vs one normal example? Why might this cause training instability?

5.2 Autoencoder Architecture

5.2.1 Compress and Reconstruct

Think of the autoencoder as a very lossy file compressor — one that is forced to throw away most of the information and then rebuild the file from scratch. A good compressor keeps what matters and discards noise. If the rebuilt file is almost identical to the original, the compressor has identified what 'matters' in the data. If the rebuilt file looks very different from the original, the input was unusual — something the compressor had no good strategy for.

The autoencoder has two parts:

- The encoder compresses the input x to a small latent vector z. Formally: z = f_θ(x). For AgriVision's 7-feature daily record, the encoder might compress to z with only 8 numbers.
- The decoder expands z back to a reconstruction x^. Formally: x^ = g_φ(z). This attempts to rebuild all 7 original features from just 8 numbers.

Both encoder and decoder are neural networks trained jointly to minimise the reconstruction loss:

$$L = \|x - \hat{x}\|^2 \quad \textit{(mean squared difference between original and reconstruction)}$$

The bottleneck — the small latent vector z — is the key constraint. With only 8 numbers to store 7 features × 30 days = 210 values, the encoder cannot simply copy the input. It must find the most compact and informative description of the data.

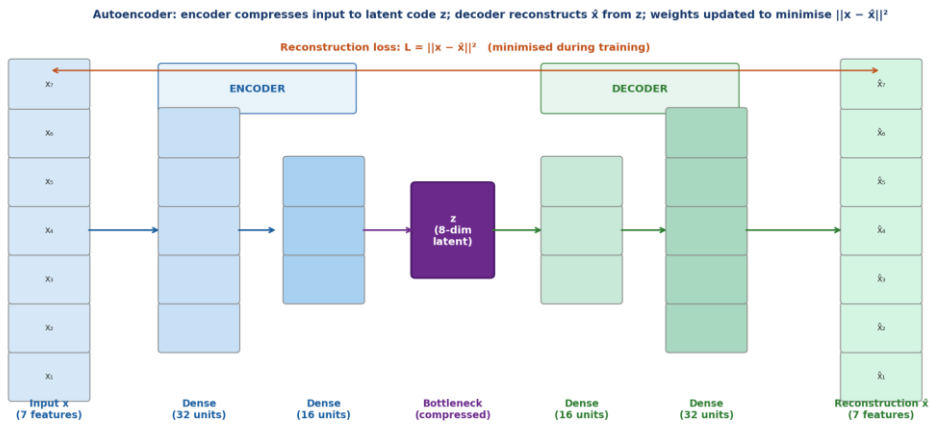


Figure 5.1 *Autoencoder for AgriVision's sensor anomaly detection. The encoder (blue) compresses the 7-feature input through two hidden layers to an 8-dimensional bottleneck z (purple). The decoder (green) reconstructs the 7 original features. All weights minimise ||x − x̂||². Normal records are well reconstructed (low error); anomalous records cannot be accurately reconstructed (high error).*

5.2.2 Why the Bottleneck Forces Useful Learning

✎ **WORKED EXAMPLE**

Reconstruction error on two AgriVision records

Normal record (normalised): x = [0.62, 0.81, 0.74, 0.55, 0.00, 0.64, 0.18]

Reconstruction: $\hat{x}$ = [0.60, 0.83, 0.76, 0.57, 0.02, 0.62, 0.16]

MSE = $[(0.02)^2 + (0.02)^2 + (0.02)^2 + (0.02)^2 + (0.02)^2 + (0.02)^2 + (0.02)^2] / 7$

= 0.0004 RMSE = 0.020 → LOW ERROR: this is a normal record.

—

Anomalous record (soil temperature spike): x = [0.65, 0.95, 0.42, 0.50, 0.00, 0.62, 0.20]

The model reconstructs soil_temp as 0.78 (within the normal range it has learned).

Reconstruction: $\hat{x}$ = [0.63, 0.78, 0.73, 0.54, 0.01, 0.63, 0.17]

The model 'pulls' the spike toward normal because normal is what it was trained on.

MSE contribution from soil_temp alone: $(0.95 - 0.78)^2 = 0.029$

Total MSE ≈ 0.010 RMSE ≈ 0.10 → HIGH ERROR: anomaly detected.

The model pulls the spike toward normal because it only knows normal. It has never seen a soil temperature of 0.95 (after normalisation) and has no representation for it. So it outputs the most normal-looking value it can. The gap between 0.95 and 0.78 is precisely the anomaly signal.

5.2.3 Autoencoder Variants

Undercomplete (standard). The bottleneck dimension d_z is smaller than the input dimension d_x. This forces compression. The model must decide what information is worth keeping and what can be discarded. This is the standard configuration for anomaly detection.

Overcomplete. d_z ≥ d_x. Without constraints, the model can simply copy the input to the latent space — the identity function. This makes reconstruction trivially perfect but the latent space meaningless. Overcomplete autoencoders require regularisation (Section 5.4) to be useful.

Denoising autoencoder. The input is artificially corrupted before the encoder sees it — random noise added, or random features zeroed out. The loss is computed against the original clean input. The model must learn to remove the corruption, which requires deep understanding of the data structure. BERT's masked token prediction (Chapter 4) is a denoising autoencoder for text. Denoising autoencoders often produce more robust representations and better anomaly detection than standard autoencoders.

KEY TERMS — THIS SECTION

Term	Definition
Encoder	Network $f_\theta(x)$ mapping input x to latent z; learns compression
Decoder	Network $g_\phi(z)$ mapping latent z to reconstruction $\hat{x}$; learns decompression
Bottleneck	The low-dimensional latent layer z with d_z << d_x; forces useful compression
Reconstruction loss	$\|x - \hat{x}\|^2$; minimised during training; used as anomaly score at inference
Denoising AE	Encoder receives corrupted input; loss compares to original; learns robust representations

✎ **CHECK YOUR UNDERSTANDING**

1. Autoencoder for 30-day windows: input 210-dim, encoder 210→128→64→16, bottleneck 16-dim, decoder 16→64→128→210. (a) Compute total parameters. (b) Compression ratio d_x/d_z. (c) How does this compare to PCA to 16 dimensions?

2. In the worked example, the model reconstructs soil_temp as 0.78 instead of 0.95. Why does the model pull toward normal rather than reproducing the spike?

3. A denoising autoencoder masks 30% of input features at random. (a) How does this compare to BERT's 15% masking rate? (b) Why might higher masking rates improve anomaly detection?

4. A student builds an overcomplete autoencoder (d_z = 420) without regularisation and achieves zero reconstruction loss. Why does anomaly detection then fail completely?

5. [Challenge] The sparse autoencoder loss is $\|x - \hat{x}\|^2 + \lambda\|z\|_1$. (a) What does $\|z\|_1$ penalise? (b) How would you set λ? (c) What would each latent dimension represent if sparsity is enforced?

5.3 Latent Space and Applications

5.3.1 The Latent Space as a Map of Normal Patterns

After training, the encoder maps every input to a point in the latent space. Think of the latent space as a map of all the patterns the model has learned. Normal sensor records that share similar patterns — two monsoon-season high-moisture days, for instance — will map to points close together on this map. Dry-season records cluster in a different region. Transitional-weather records cluster in a third.

An anomalous record — a sensor spike, an implausible value, an unusual sequence — does not fit any of the learned clusters. It maps to a point far from everything else on the map. And when the decoder tries to rebuild the original input from this unfamiliar latent code, it fails. The reconstruction error is high.

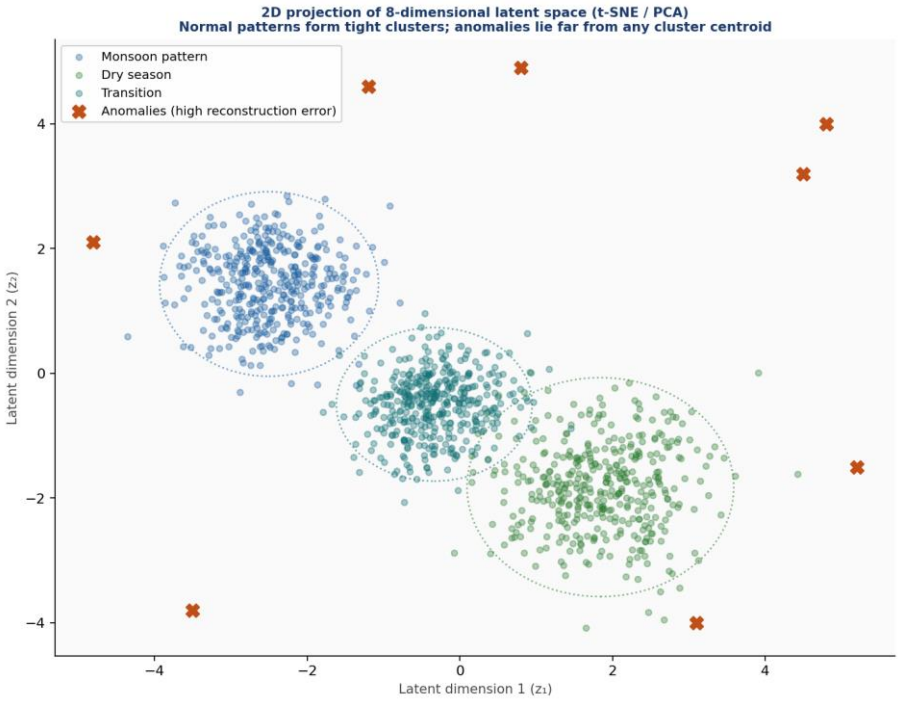


Figure 5.3 *2D projection (t-SNE) of the 8-dimensional latent space. Normal records form three clusters: monsoon (blue), dry season (green), and transitional weather (teal). Anomalous records (red X marks) lie far from all clusters — they mapped to latent codes far from anything the model learned during training.*

5.3.2 Dimensionality Reduction and Visualisation

An 8-dimensional latent space cannot be drawn directly. Two techniques project it to 2D for visual inspection.

PCA. Finds the two directions of greatest variance and projects everything onto them. Fast and deterministic. Preserves global structure (distant clusters stay distant) but may flatten local structure.

t-SNE and UMAP. Non-linear methods that preserve local neighbourhoods: points that are close in the latent space appear close in the 2D projection. Better for revealing clusters. UMAP is faster than t-SNE and scales to large datasets. Figure 5.3 uses this approach to visualise AgriVision's latent space.

5.3.3 Reconstruction-Error Anomaly Detection

At deployment time, each incoming sensor record is passed through the trained encoder and decoder. The reconstruction error is the anomaly score. Above a threshold τ, the record is flagged:

$$Anomaly(x) = 1 \quad if \quad \|x - \hat{x}\|_2 > \tau, \quad else\ 0$$

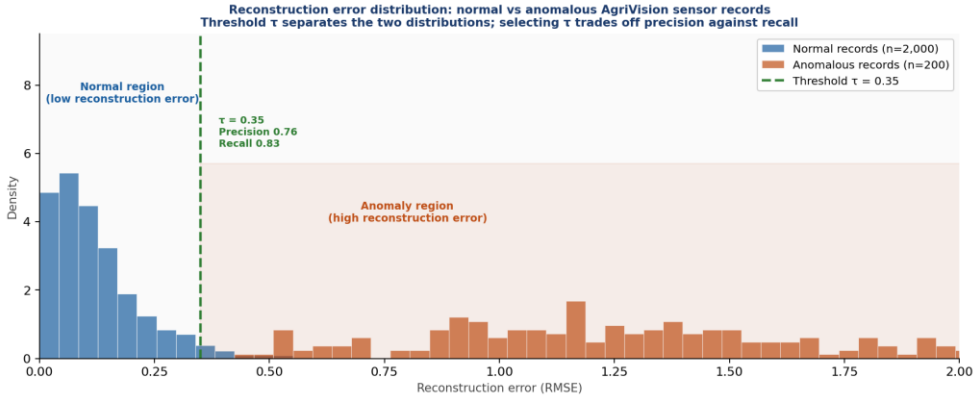


Figure 5.2 *Reconstruction error distribution for AgriVision's sensor records. Normal records (blue) concentrate at low error values. Anomalous records (orange) spread to higher values. The threshold τ = 0.35 (green dashed) classifies records above it as anomalies. Moving τ left catches more anomalies (higher recall) but flags more normal records too (lower precision). Moving τ right does the opposite.*

5.3.4 Choosing the Threshold: Precision vs Recall

The threshold τ is the most important deployment decision. The autoencoder's training does not set it — you choose τ afterwards, using a small held-out set of records that have been verified as normal or anomalous.

For AgriVision, the cost asymmetry guides the choice. A missed anomaly (false negative) means a sensor malfunction goes undetected and irrigation decisions are based on bad data. A false alarm (false positive) means an extension officer spends 20 minutes investigating a record that turns out to be fine. If a missed anomaly is more costly than a false alarm, set τ lower to catch more anomalies, accepting more false alarms. The Precision-Recall curve, computed on the validation set for many values of τ, guides this choice.

KEY TERMS — THIS SECTION

Term	Definition
Latent space	The d_z-dimensional space where encoded representations z live; geometry reflects data structure
t-SNE / UMAP	Non-linear dimensionality reduction methods for visualising high-dimensional latent spaces in 2D
Anomaly score	$\|x - \hat{x}\|_2$; the reconstruction error; larger means more anomalous
Threshold τ	Decision boundary on the anomaly score; records above τ are flagged; chosen on labelled validation data

Precision-Recall curve	Trade-off curve obtained by varying τ; used to select the deployment threshold

✎ **CHECK YOUR UNDERSTANDING**

1. At τ=0.35: TP=415, FP=130, FN=85, TN=24,870. (a) Compute precision, recall, F1. (b) At τ=0.55: TP=310, FP=45, FN=190, TN=24,955. Recompute. (c) Which threshold suits AgriVision's cost asymmetry?

2. After training, the 2D t-SNE projection shows two clearly separated clusters for monsoon and dry-season records. Where would you expect a temperature spike anomaly to appear on this map, and why?

3. A student uses Euclidean distance from the mean latent vector as the anomaly score instead of reconstruction error. When would these agree? When might they differ?

4. Why is the ROC curve less appropriate than the Precision-Recall curve for evaluating anomaly detection on a 99.8% normal dataset? Give a numerical example.

5. [Challenge] AgriVision wants to detect sensor drift (a gradual calibration shift over months). Would single-record reconstruction error catch this? Propose a modified approach.

5.4 Self-Supervised Learning

5.4.1 Making Labels from Data

The autoencoder is one example of a self-supervised learning method. The defining principle is always the same: design a task whose answer can be computed automatically from the data, then train a model to solve it. The model learns useful representations as a side effect of solving the task.

Some examples for image data:

- Rotation prediction: randomly rotate an image by 0°, 90°, 180°, or 270°. Train a model to identify the rotation. Requires understanding spatial orientation.
- Jigsaw puzzle: shuffle the patches of a 3×3 grid. Train a model to restore the correct order. Requires spatial reasoning.
- Masked image modelling: remove 75% of the image patches. Train a model to reconstruct the removed patches. This is BERT applied to images — published as the Masked Autoencoder (MAE).

For sensor time-series data, useful pretext tasks include:

- Missing value imputation: randomly zero out 20% of feature values and train the model to predict the missing values.
- Temporal ordering: shuffle the time steps randomly and train a model to restore them to the correct sequence.
- Contrastive pairing: which two windows came from the same farm in the same season? This is the contrastive approach, covered next.

5.4.2 Contrastive Learning: Learning by Comparison

Instead of predicting a specific output, contrastive learning trains a model to recognise similarity. The key idea: two different versions of the same input are more similar to each other than either is to any other input in the batch.

SimCLR creates two augmented views of each input — for a sensor window, this might mean adding noise to one view and cropping a slightly different time range for the other. Both views are encoded by the same shared encoder. The training objective pushes the two representations of the same input close together in embedding space, and pushes representations of different inputs apart. No labels are needed — the pairs are created automatically.

$$L = -\log \exp(\mathrm{sim}(z_1, z_2) / \tau) \;/\; \Sigma_{k \neq 1} \exp(\mathrm{sim}(z_1, z_k) / \tau)$$

where sim(u, v) = u·v / (‖u‖ · ‖v‖) is cosine similarity and τ is a temperature parameter. The loss is minimised when z_1 and z_2 are as similar as possible and as different from all other batch items as possible.

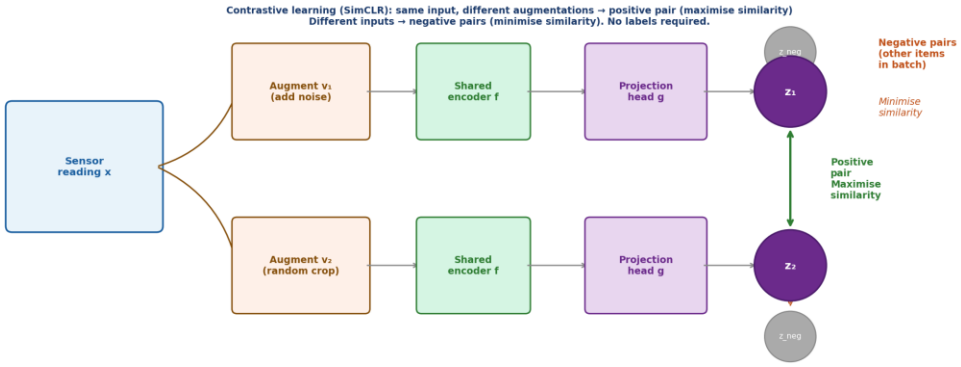


Figure 5.5 *Contrastive learning with SimCLR. The same sensor record x is augmented two ways (noise injection, temporal crop) to produce a positive pair. Both views pass through the same encoder f and projection head g. The NT-Xent loss maximises similarity between z_1 and z_2 (positive pair, green) while minimising similarity with all other representations in the batch (negative pairs, red). No labels required.*

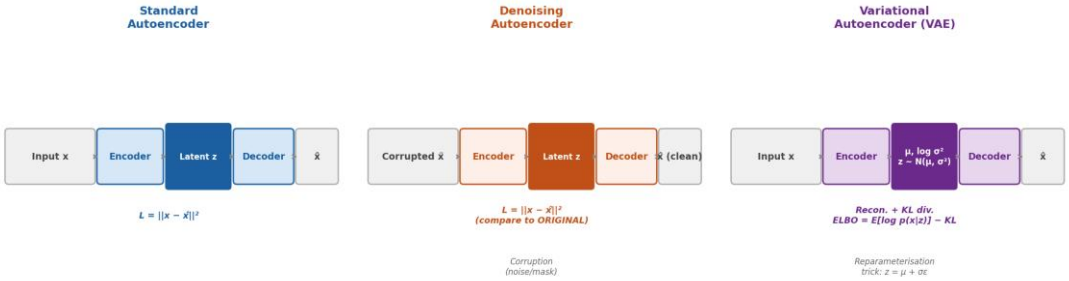


Figure 5.4 *Three autoencoder variants. Standard AE: reconstruct from compressed latent code. Denoising AE: reconstruct clean input from a corrupted version. VAE: encoder outputs a probability distribution (mean μ and log variance log σ^2); z is sampled, not deterministic; the ELBO loss adds a KL divergence term that regularises the latent space toward N(0,I), enabling generation by sampling from the prior.*

5.4.3 The Connection to Foundation Models

Foundation model	Pretext task	Input type	Self-supervised signal
BERT	Masked Language Modelling	Text tokens	Predict masked tokens from context
GPT	Next-token prediction	Text tokens	Predict next token from prefix
MAE	Masked patch reconstruction	Image patches	Reconstruct 75% masked patches from 25% visible
SimCLR	Contrastive augmentation pairs	Images/sensors	Attract similar augmentations; repel others

Table 5.2 *Self-supervised pretext tasks across major foundation models. All avoid human annotation; all learn by exploiting structure in the data itself.*

KEY TERMS — THIS SECTION

Term	Definition
Contrastive learning	Learns representations by attracting similar pairs and repelling dissimilar pairs in embedding space
Positive pair	Two augmented views of the same input; should have similar embeddings after training
Negative pair	Inputs from different sources; should have dissimilar embeddings
SimCLR	Simple Contrastive Learning of Representations; augmentation-based contrastive SSL
MAE	Masked Autoencoder: masks 75% of image patches and reconstructs them; ViT pretraining method
NT-Xent loss	Normalised temperature-scaled cross-entropy; the SimCLR training objective

✎ **CHECK YOUR UNDERSTANDING**

1. SimCLR with batch size 256. (a) How many positive pairs? (b) How many negative pairs? (c) Why does performance improve with larger batch size?

2. For the temporal ordering pretext task on 30-day sensor windows: (a) how would you construct the corrupted input? (b) What must the model learn to distinguish shuffled from correct order?

3. BERT masks 15%; MAE masks 75%. Why can MAE use a much higher masking rate? For AgriVision sensor data, what masking rate would you recommend?

4. From Table 5.2, select the most appropriate SSL method for: (a) handwritten prescription scans; (b) tractor engine audio; (c) GPS vehicle traces.

5. [Challenge] Propose combining reconstruction and contrastive losses: $L = \alpha\|x - \hat{x}\|^2 + (1-\alpha)$ L_NT-Xent. At α=1, 0.5, and 0, describe the expected representation quality for AgriVision anomaly detection.

5.5 Application Contexts, Limitations, and Bridge

5.5.1 Where Autoencoder Anomaly Detection Is Used

The pattern is the same across many domains: abundant unlabelled normal data, rare and heterogeneous anomalies, no labelled anomaly examples. Autoencoders are the natural solution in each of these settings:

- Manufacturing: vibration sensors on factory machinery detect bearing wear before catastrophic failure. Normal operation data is plentiful; fault examples are rare and diverse.
- Network security: traffic log autoencoders flag unusual patterns. Normal traffic is abundant; attack signatures vary continuously.
- Medical: ECG autoencoders flag abnormal heart rhythms; chest X-ray autoencoders flag radiological findings outside the normal range. Normal scans are abundant; clinical annotation of abnormalities requires specialist time.
- Agricultural (AgriVision): soil sensor anomaly detection — this chapter. Satellite image autoencoders flagging unusual crop patterns before human inspection.

5.5.2 Key Failure Modes

Anomaly contamination in training. If anomalous records are present in the training data, the autoencoder learns to reconstruct them — and their reconstruction error at inference is low. For AgriVision, the five-year training set almost certainly contains some unidentified anomalous records. Simple fix: apply Z-score filtering before training to remove obvious gross outliers from the training set.

Reconstruction without understanding. An autoencoder sometimes reconstructs an anomaly well — if the anomaly happens to lie on the data manifold. A soil temperature that is exactly twice its normal value might be reconstructed faithfully if the local structure looks similar to a normal pattern. Reconstruction error measures deviation from the learned manifold, not semantic anomalousness.

Threshold sensitivity. Without labelled anomaly examples for validation, the threshold τ is set heuristically — for example, the 99th percentile of training reconstruction errors. This may give poor precision-recall trade-offs in deployment. Getting even a small labelled validation set (50–100 confirmed anomalies) dramatically improves threshold selection.

5.5.3 AgriVision Resolution: Sensor Anomaly Detection

> ✦ THE AGRIVISION CONTEXT
>
> **Chapter 5 Resolution — Sensor anomaly detection results**
>
> Training: 200,000 normal sensor records (after Z-score filtering removed 340 outliers)
>
> Architecture: 7-input → Dense(64) → Dense(32) → Bottleneck(16) → Dense(32) → Dense(64) → 7-output
>
> Threshold $\tau = 0.035$ (selected on 500 labelled validation records)
>
> Autoencoder (zero labelled anomaly examples in training):
>
> Precision 0.76 Recall 0.83 F1 0.79
>
> Supervised classifier (50 labelled anomalies + 500 normal records):

Precision 0.68 Recall 0.71 F1 0.70

The autoencoder outperforms the supervised classifier despite seeing no anomaly examples during training. 200,000 unlabelled normal records are a better foundation than 50 labelled anomalies. In the first three months of deployment: 23 sensor malfunctions and 7 unusual drought-stress events confirmed by field inspection.

Chapter Summary

Section	Core concept	Key insight
5.0	Reconstruction as self-supervision	Train on normal data only; reconstruction error detects anomalies — no anomaly labels needed
5.1	Limits of supervised learning	Labelling cost, class imbalance, and open-world detection all favour the autoencoder approach
5.2	Autoencoder architecture	$z = f_\theta(x)$; $\hat{x} = g_\phi(z)$; loss = $\|x - \hat{x}\|^2$; bottleneck forces learning; anomalies reconstruct poorly
5.3	Latent space and anomaly scoring	Latent space maps normal clusters; reconstruction error is the anomaly score; τ selected on validation data
5.4	Self-supervised learning	Pretext tasks manufacture labels from data structure; contrastive, masked, and reconstruction methods all work
5.5	Applications and failure modes	Anomaly contamination, manifold overlap, and threshold sensitivity are the primary failure modes

5.5.4 Bridge to Chapter 6: From Reconstruction to Generation

The standard autoencoder is good at compression and anomaly detection. But it has a fundamental limitation for another purpose: generation.

The problem is the shape of the latent space. When the encoder maps AgriVision's normal records to 8-dimensional latent codes, those codes scatter across the latent space without any particular structure. Some regions of the latent space contain many normal records; others are empty. If we sample a random point from the latent space and try to decode it, there is no guarantee the result will look like a real sensor record — we might be sampling from an empty region.

The Variational Autoencoder (VAE) solves this by imposing structure on the latent space. Instead of encoding each input as one specific point, the VAE encoder outputs a distribution — a mean and a spread. A KL divergence term in the loss pushes all these distributions toward a standard Gaussian $N(0, I)$. The result: the latent space is smooth, continuous, and fully covered. Any point sampled from $N(0, I)$ decodes to something meaningful.

This structured latent space is the key to generation: sample z from $N(0, I)$, decode it, and you get a new synthetic image or record that resembles the training data. For AgriVision, a VAE trained on disease leaf photographs could generate new disease photographs — more training data for the Chapter 2 classifier without collecting a single additional real image. Chapter 6 builds this.

CHAPTER 6

Generative Deep Learning

From Prediction to Synthesis

6.0 The Inductive Bias for Generation

6.0.1 The AgriVision Data Augmentation Problem

> ✦ **THE AGRIVISION CONTEXT**
>
> PlantVillage provided 8,000 labelled disease photographs for AgriVision's Chapter 2 disease detector. But PlantVillage photographs were taken in controlled laboratory conditions against plain backgrounds. Karnataka field photographs look different: complex canopy backgrounds, variable lighting, locally specific tomato varieties.
>
> After domain-specific annotation, AgriVision has only 312 Karnataka-specific Powdery Mildew photographs, 287 Late Blight photographs, and 198 Alternaria photographs. This is far below the 1,000+ examples per class that produces reliable CNN performance. Can the model generate more?
>
> A generative model trained on AgriVision's existing disease photographs could synthesise additional training images: new mildew photographs with different backgrounds, angles, and lighting conditions that reflect real-world variation. The question is which generative architecture to use, and whether synthetic images genuinely improve the downstream classifier's accuracy.

Every generative model developed in this chapter answers the same question: given training examples drawn from some data distribution P(x), how do we learn a model that can produce new samples from P(x)? But different architectures answer this question with different assumptions — different inductive biases — and these assumptions lead to very different generation quality, training stability, and practical limitations.

6.0.2 Discriminative vs Generative Models

Every model in Chapters 1–5 was discriminative: it learned a mapping from input to output, approximating the conditional distribution P(y|x). The disease detector of Chapter 2 learned P(disease class | leaf photograph). The irrigation predictor of Chapter 3 learned P(irrigation needed | sensor sequence). Discriminative models learn a decision boundary between classes; they do not learn the full distribution of the input data.

A generative model learns the joint distribution P(x) or P(x|y): the full statistical structure of the data itself, not just the boundary between classes. A well-trained generative model can sample new data points from P(x) — producing photographs that look like they were taken by AgriVision's extension officers, even though they were synthesised.

◆ KEY CONCEPT

The fundamental shift: from prediction to synthesis

Discriminative: given x, predict y. Learns P(y|x). Cannot generate new x.

Generative: learn P(x). Can generate new x. May also compute P(y|x) as a by-product.

The distinction matters for AgriVision because data augmentation requires generating new x (new leaf photographs), not predicting from existing ones. No discriminative model from Chapters 1–4 can do this.

6.0.3 Three Different Assumptions About Generation

This chapter studies three families of generative models, each encoding a distinct assumption about how the data distribution should be represented:

Variational Autoencoder (VAE). Assumes the data distribution can be described by a smooth, continuous latent space governed by a Gaussian prior. New samples are generated by sampling z from N(0,I) and decoding. The smooth latent space allows interpolation and structured generation but typically produces slightly blurry outputs due to the averaging effect of the reconstruction loss.

Generative Adversarial Network (GAN). Assumes that realistic data can be produced by training a generator against an adversarial discriminator. No explicit density model is required; the generator learns to fool the discriminator rather than to maximise a reconstruction objective. Produces sharper images than VAEs but is prone to training instability and mode collapse.

Diffusion Model. Assumes that data can be recovered from pure noise through a learned iterative denoising process. The forward process gradually corrupts data to noise; the reverse process learns to undo this corruption step by step. Produces the highest image quality and diversity of the three, at the cost of slow inference (many denoising steps at generation time).

6.0.4 Four Questions

? FOUR QUESTIONS TO CARRY THROUGH THIS BOOK

Question 1: What problem does generative deep learning solve?

Data augmentation when labelled data is scarce; privacy-preserving synthetic data generation; scientific simulation; creative content generation; model-based data completion and de-noising.

Question 2: What inductive biases do generative models encode?

VAE: Gaussian latent prior; smooth interpolable latent space. GAN: discriminability as the training signal; no explicit density. Diffusion: data lies on a manifold reachable by iterative denoising from Gaussian noise.

Question 3: When do generative models fail?

VAE: blurry outputs (reconstruction loss averages modes); posterior collapse (decoder ignores z). GAN: mode collapse (generates only a subset of the data distribution); training instability (discriminator dominates too early). Diffusion: computationally expensive at inference; can generate photorealistic misinformation.

Question 4: When would you choose something else?

When labelled data is plentiful, discriminative models suffice; when the task is prediction, not synthesis; when compute constraints make generative pretraining impractical.

KEY TERMS — THIS SECTION

Term	Definition
Discriminative model	Learns P(y|x); maps input to prediction; cannot generate new inputs
Generative model	Learns P(x) or P(x,y); can sample new inputs from the learned distribution
Data distribution P(x)	The underlying probability distribution from which real data are drawn
Data augmentation	Creating new training examples from existing ones; generative models produce semantically valid new examples
Latent prior p(z)	The distribution over latent codes assumed by the generative model; N(0,I) for VAEs
Mode collapse	GAN failure mode: generator learns to produce only a narrow subset of the real data distribution

✎ **CHECK YOUR UNDERSTANDING**

1. AgriVision has 312 Powdery Mildew training photographs. A student proposes using standard data augmentation (flipping, rotation, colour jitter) to expand this to 3,000 images. (a) What types of variation does standard augmentation produce? (b) What types of variation does a generative model produce that standard augmentation cannot? (c) When would generative augmentation be beneficial over standard augmentation?

2. Section 6.0.2 states that discriminative models 'cannot generate new x.' A student argues: 'A CNN's convolutional filters could be run in reverse to generate images.' Is this argument correct? What is actually produced when you run a discriminative model's forward pass in reverse, and why is this not the same as sampling from P(x)?

3. Compare the three generative assumptions: VAE (Gaussian latent prior), GAN (adversarial discriminability), Diffusion (iterative denoising). (a) Which assumption requires explicit density estimation? (b) Which is most computationally expensive at inference? (c) Which would you choose for AgriVision's data augmentation task if compute at inference time is highly constrained?

4. A VAE and a GAN are both trained on AgriVision's disease photographs. A human expert rates the visual quality of generated images. The GAN images look sharper but the VAE images are more diverse. (a) Explain the architectural reason for each observation. (b) For the purpose of training a downstream disease classifier, is sharpness or diversity more important? Justify.

5. [Challenge] Mode collapse is a GAN failure mode where the generator produces only a subset of the real distribution. For AgriVision, what specific form would mode collapse take? (a) Describe three specific manifestations. (b) How would you detect mode collapse without a human rater? (c) Propose a GAN training modification that reduces the risk.

6.1 Why Generative Deep Learning?

6.1.1 The Fundamental Challenge: Learning a Data Distribution

Learning a data distribution is harder than learning a decision boundary. A discriminative classifier needs to learn only which region of input space corresponds to which class. A generative model must learn the full statistical structure of the data: the correlations between pixel values, the conditional distributions of features given other features, the manifold on which real data lives in the high-dimensional input space.

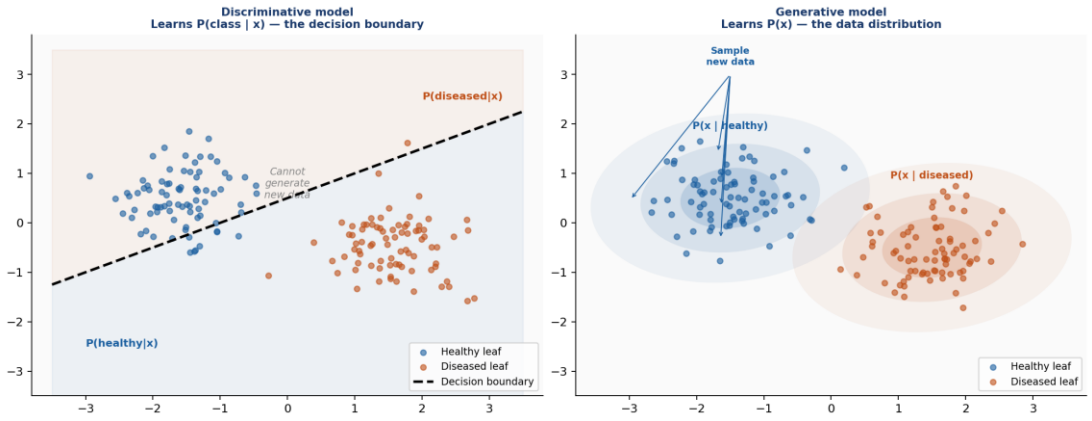


Figure 6.1 *Discriminative vs generative modelling. Left: a discriminative model learns the decision boundary P(class | x) between healthy and diseased leaf patterns; it classifies but cannot generate new data points. Right: a generative model learns the full data distribution P(x) for each class, represented as probability density contours; it can sample new data points (arrows) from any region of the learned distribution.*

The dimensionality makes this challenging. A 224×224×3 leaf photograph lives in a 150,528-dimensional space. Almost all points in this space are meaningless — random pixel noise that looks nothing like a leaf. The manifold of real leaf photographs is a low-dimensional curved surface in this enormous space. A generative model must learn the precise shape of this manifold so that samples from its model distribution land on the manifold rather than in the surrounding noise.

6.1.2 Applications of Generative Models

- Data augmentation: generate synthetic training examples when labelled data is scarce. For AgriVision: 312 real Karnataka mildew photographs + 500 generated → improved disease classifier accuracy.
- Simulation and synthetic training data: generate labelled training data for tasks where real annotation is impractical (satellite imagery with ground truth labels, medical images with pixel-level annotations).
- Privacy-preserving synthetic data: release a synthetic dataset with the same statistical properties as a sensitive real dataset (patient records, financial transactions), but no real individual's data.
- Image inpainting and restoration: complete or de-noise partially damaged images; remove artefacts from scanned documents.
- Creative applications: image synthesis, video generation, music composition, text-to-image generation (DALL-E, Stable Diffusion).

6.1.3 Evaluation: What Makes a Good Generated Sample?

Evaluating generative model quality is harder than evaluating classifier accuracy — there is no single 'correct' output to compare against. Two properties must be balanced:

Fidelity (quality). Generated samples should look realistic — indistinguishable from real data to a human observer. High fidelity means the model has learned the correct fine-grained texture, colour, and spatial structure of the real data.

Diversity (coverage). Generated samples should cover the full range of variation in the real data distribution. A model that generates only one type of powdery mildew (e.g., only late-stage, only on tomato) has low diversity even if each individual sample is high fidelity.

The Fréchet Inception Distance (FID) is the standard automatic evaluation metric for image generation. FID computes the Fréchet distance (a measure of the distance between two Gaussian distributions) between the distribution of Inception network features for real images and the distribution for generated images. Lower FID indicates that the generated distribution is closer to the real distribution in feature space, capturing both fidelity and diversity simultaneously.

$$FID = ||\mu_r - \mu_g||^2 + Tr(\Sigma_r + \Sigma_g - 2(\Sigma_r\Sigma_g)^{1/2})$$

where μ_r, Σ_r and μ_g, Σ_g are the mean and covariance of Inception features for real and generated images respectively. State-of-the-art diffusion models achieve FID < 5 on standard benchmarks; GANs typically achieve FID 5–10; VAEs achieve FID 15–30.

KEY TERMS — THIS SECTION

Term	Definition
Data manifold	The low-dimensional surface in input space on which real data concentrates; generative models must learn its shape
Fidelity	Visual quality of individual generated samples; do they look realistic?
Diversity	Coverage of the full real data distribution by generated samples; no mode collapse
FID	Fréchet Inception Distance; measures similarity between real and generated feature distributions; lower is better
Inception features	Intermediate activations of a pretrained Inception network; used as a perceptual feature space for FID computation

✎ CHECK YOUR UNDERSTANDING

1. A VAE trained on AgriVision photographs produces 1,000 synthetic images. An agronomist evaluates 50 random samples and rates 46/50 as realistic. A GAN produces 50 samples of which 49/50 are realistic. However, all 1,000 GAN samples depict late-stage mildew on one specific leaf position. (a) Which model has higher fidelity? Which has higher diversity? (b) Which set of synthetic images would more effectively improve AgriVision's downstream classifier? Explain.

2. FID is computed by extracting Inception features for 10,000 real and 10,000 generated images and fitting Gaussians to both sets. (a) Why is 10,000 samples needed rather than, say, 100? (b) AgriVision has only 312 mildew photographs. What problem arises when computing FID with such a small real dataset? (c) Propose an alternative evaluation approach that does not require 10,000 real samples.

3. Explain the fidelity-diversity trade-off from the perspective of the model's learned distribution. (a) What does a model with high fidelity but low diversity look like mathematically? (b) What about high diversity but low fidelity? (c) Why can the discriminative training signal (GAN) push toward high fidelity at the expense of diversity?

4. A medical imaging system uses a generative model to augment a training set of 200 chest X-rays with 800 synthetic examples. The downstream classifier (trained on 200 real + 800 synthetic) achieves 89% accuracy; a classifier trained on 200 real only achieves 82%. (a) Interpret this result. (b) What risk does the synthetic augmentation introduce? (c) How would you validate that the augmented classifier genuinely generalises to new real patients?

5. [Challenge] FID uses Inception features extracted from a network pretrained on ImageNet. For AgriVision's leaf disease photographs: (a) What properties of ImageNet Inception features might make them appropriate for evaluating generated leaf images? (b) What properties might make them inappropriate? (c) Propose a domain-specific FID variant using a more appropriate feature extractor.

6.2 Variational Autoencoders

6.2.1 Why the Standard Autoencoder Cannot Generate

Chapter 5 built an autoencoder that compresses sensor records to 8 numbers and reconstructs them. The 8-number code — the latent vector — is a specific point in an 8-dimensional space for each input.

Now suppose we want to generate a new disease photograph. We would sample a random point in the latent space and decode it. The problem: the encoder has placed each training image's code wherever was most convenient for reconstruction. There is no guarantee that a randomly chosen point corresponds to anything meaningful. Many random points decode to blurry noise because they lie in regions of the latent space that the encoder never visited.

6.2.2 The VAE's Fix: Encode to a Region, Not a Point

The Variational Autoencoder's key idea is simple: instead of encoding an input as one specific point in the latent space, encode it as a small cloud of points — a Gaussian distribution with a mean and a spread.

The encoder outputs two vectors for each input: a mean vector μ and a log-variance vector log σ². The latent code z is then sampled from the distribution N(μ, σ²). During training, a KL divergence term pushes all these distributions toward the standard Gaussian N(0, I). The effect: the entire latent space becomes densely covered with distributions from real training examples. No empty pockets remain. Any point sampled from N(0, I) lies near the distribution of at least one training example, and therefore decodes to something meaningful.

$$\textit{Encoder outputs: } \mu_\theta(x) \textit{ and } \log \sigma^2_\theta(x) \quad \textit{(two vectors, not one)}$$

$$\textit{Latent code: } \quad z \sim N(\mu, \sigma^2)$$

6.2.3 The ELBO Loss: Two Terms Working Together

The VAE is trained to maximise a quantity called the ELBO (Evidence Lower BOund). It has two parts that pull in opposite directions:

Reconstruction term. Penalises poor reconstruction. Forces the model to encode enough information in z to rebuild x accurately. Without this term, the encoder would make all distributions identical (collapsed to the prior) and z would carry no information about x.

KL divergence term. Measures how different each encoder distribution N(μ, σ²) is from the prior N(0, I). Pushes the encoder's distributions toward the prior — making the latent space smooth and uniformly covered. Without this term, the encoder could place each image's code wherever it wants (like the standard autoencoder) and generation would fail.

$$ELBO = E[\log p(x|z)] \; - \; KL(N(\mu, \sigma^2) \,||\, N(0, I))$$

$$\textit{In practice: } \; L_VAE = \|x - \hat{x}\|^2 \; + \; \beta \cdot KL(N(\mu, \sigma^2) \,||\, N(0, I))$$

The KL divergence for Gaussian distributions has a closed form — no numerical integration needed:

$$KL(N(\mu, \sigma^2) \,||\, N(0,I)) = -\tfrac{1}{2} \Sigma_j \, (1 + \log \sigma^2_j - \mu^2_j - \sigma^2_j)$$

This sum runs over each dimension j of the latent vector. Each term is zero when $\mu_j=0$ and $\sigma^2_j=1$ — i.e., when the distribution already matches the prior. The KL term penalises deviations from this.

6.2.4 The Reparameterisation Trick

There is a technical problem with training the VAE. The loss requires gradients to flow from the reconstruction loss back through the sampling operation $z \sim N(\mu, \sigma^2)$. But sampling is random — you cannot compute a gradient through a random draw.

The reparameterisation trick solves this with a simple re-arrangement. Instead of sampling z directly, sample ε from the fixed distribution N(0, I) and compute:

$$z = \mu + \sigma \odot \varepsilon \qquad where\ \varepsilon \sim N(0, I)\ \ is\ sampled\ separately$$

Now z is a deterministic function of μ, σ, and ε. The gradient $\partial z/\partial \mu = 1$ and $\partial z/\partial \sigma = \varepsilon$ can be computed normally. The randomness comes entirely from ε, which has no parameters — so no gradient is needed through it.

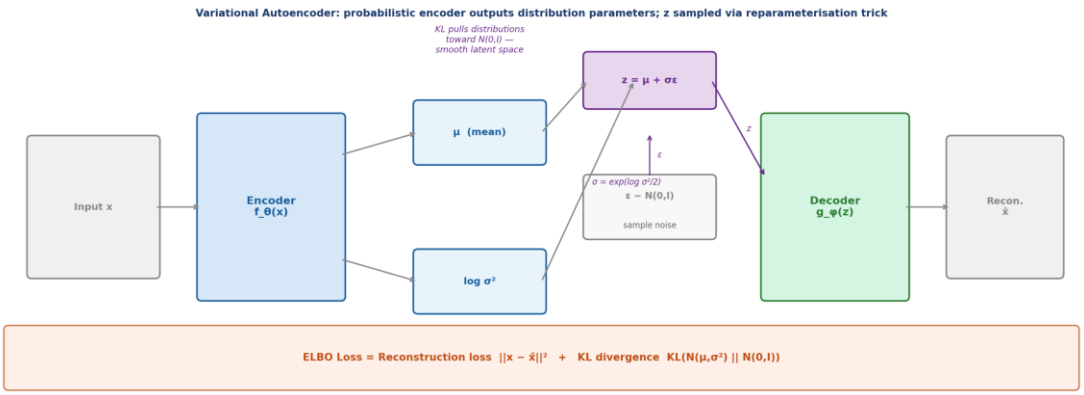


Figure 6.2 *VAE architecture. The encoder outputs two vectors (μ and log σ²) rather than one latent code. Noise ε is sampled separately from N(0,I); the latent code z = μ + σε is computed differentiably (reparameterisation trick). The ELBO loss (reconstruction + KL, bottom) trains the full system end-to-end. At generation time: sample z from N(0,I) and decode — guaranteed to produce meaningful output because KL regularisation has covered the entire prior.*

6.2.5 What Smooth Latent Spaces Enable

Because the KL term enforces structure, the latent space has two useful properties standard autoencoders lack.

Generation. Sample any z from N(0, I) and decode. The result is a meaningful synthetic image — a new disease photograph that looks like it could have come from AgriVision's training set.

Interpolation. Take the latent codes z_A and z_B of two real disease photographs. Decode points along the straight line between them: $z(t) = (1-t)z_A + t \cdot z_B$ for t from 0 to 1. The decoded images smoothly transition from one disease pattern to the other through semantically meaningful intermediate stages. For AgriVision, interpolating between early-stage and late-stage mildew photographs produces images of intermediate disease stages — exactly the stages that are hardest to photograph in the field.

6.2.6 Limitations: Blurry Outputs and Posterior Collapse

Blurry outputs. The MSE reconstruction loss averages over all plausible ways to reconstruct an input. If a leaf boundary could plausibly appear in two slightly different positions, the MSE-optimal reconstruction splits the difference — producing a blurred boundary. This blurriness is intrinsic to the VAE's pixel-space reconstruction objective.

Posterior collapse. If the decoder becomes powerful enough to reconstruct x without relying on z at all, the KL term wins: z collapses to the prior N(0, I) regardless of the input, and the encoder stops encoding anything useful. Prevention: start training with the KL term weighted at zero and gradually increase it (β-warmup) — forcing the encoder to use z for reconstruction before the KL term gains influence.

KEY TERMS — THIS SECTION

Term	Definition
VAE	Variational Autoencoder; encoder outputs N(μ, σ²); KL-regularised latent space supports generation
ELBO	Evidence Lower BOund; reconstruction term − KL divergence; the VAE training objective
KL divergence	Measures difference between encoder distribution and prior; pushes latent space toward N(0,I)
Reparameterisation trick	z = μ + σε, ε~N(0,I); makes sampling differentiable; enables gradient flow through stochastic layer
Posterior collapse	VAE failure where encoder ignores x and z reverts to prior; decoder learns to ignore z
Latent interpolation	Decoding points along a straight line between two latent codes; produces semantic transitions

✎ **CHECK YOUR UNDERSTANDING**

1. VAE for disease images: latent dimension d=32. (a) How many parameters in the encoder's μ output layer (penultimate layer = 256 units)? (b) At generation time, what exact computation produces a new image? (c) At reconstruction time, is z = μ or is z sampled?

2. Compute KL divergence for μ = 0.5, σ^2 = 0.8 (one dimension). Now for μ = 0, σ^2 = 1. What does the second result tell you about the KL-optimal encoding?

3. Generated VAE images look blurry. (a) What architectural reason causes this? (b) Would increasing bottleneck size from 32 to 128 improve sharpness? (c) What loss function change would most directly address blurriness?

4. Without the reparameterisation trick, why can we not backpropagate through z ~ N(μ, σ^2)? After reparameterisation, which variables carry gradients and which do not?

5. [Challenge] β-warmup starts with β=0 and increases to β=1 over 50 epochs. Why does starting at β=0 prevent posterior collapse? Propose a warm-up schedule.

6.3 Generative Adversarial Networks

6.3.1 The Counterfeiter and the Detective

Imagine an art forger who produces fake paintings and a museum authenticator whose job is to catch fakes. The forger improves by making fakes the authenticator cannot spot. The authenticator improves by studying whatever the forger produces. They push each other forward. After enough rounds, the forger's copies are indistinguishable from the real thing.

A GAN works the same way — except instead of paintings, we are generating AgriVision leaf disease photographs.

The Generator is the forger. It takes a list of random numbers as input — a noise vector z drawn from a standard normal distribution — and converts them into a synthetic leaf photograph. At the start of training, the output looks nothing like a real leaf. By the end, it should be indistinguishable from one.

The Discriminator is the authenticator. It receives photographs one at a time — sometimes real AgriVision photographs, sometimes Generator outputs — and answers one question: real or fake? Its output is a number between 0 (definitely fake) and 1 (definitely real).

Training alternates between two steps:

- Train the Discriminator: show it real photographs (label = 1) and current Generator fakes (label = 0). Update the Discriminator's weights to get better at distinguishing them.
- Train the Generator: generate fakes, ask the Discriminator to score them, and update the Generator's weights to make scores closer to 1 (fool the Discriminator).

Neither network sees the other's weights. They only communicate through the Discriminator's score. At equilibrium, the Generator produces photographs so realistic that the Discriminator cannot reliably tell them from real ones — its score on fake images hovers around 0.5.

6.3.2 The Minimax Objective

The formal training objective captures this game in one equation:

$$\min_G \max_D \; E[\log D(x)] + E[\log(1 - D(G(z)))]$$

Reading left to right: the Discriminator D maximises this expression by making log D(x) large (correctly scoring real images high) and log(1−D(G(z))) large (correctly scoring fake images low). The Generator G minimises by making D(G(z)) large (fooling the Discriminator into scoring fakes high).

In practice, the Generator is trained with the non-saturating alternative max E[log D(G(z))] instead of min E[log(1−D(G(z)))]. The reason: when the Discriminator is confident the fake is fake (D(G(z)) ≈ 0), the gradient from the standard objective is nearly zero — the Generator receives no learning signal. The non-saturating version does not have this problem.

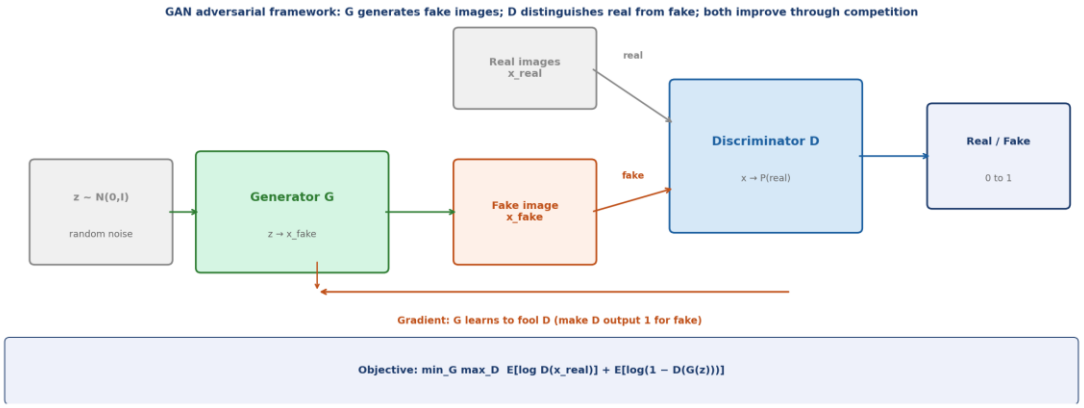


Figure 6.3 *GAN adversarial architecture. The Generator G converts random noise z into a synthetic leaf photograph. The Discriminator D receives real or generated photographs and scores each as real or fake. The Generator's gradient (coral arrows) flows backward from the Discriminator's loss, teaching G to produce more convincing images. Both networks are updated in alternating steps.*

6.3.3 Training Instability and Mode Collapse

Training instability. GAN training is fundamentally unstable because G and D are simultaneously updated toward opposing objectives. If the Discriminator learns too quickly, the Generator's gradient vanishes and it stops improving. If the Generator learns too quickly, the Discriminator cannot distinguish real from fake and provides no useful training signal. Achieving balance requires careful hyperparameter tuning.

Mode collapse. The Generator discovers a small set of outputs that consistently fool the Discriminator and generates only those, ignoring all the other types of images in the training set. For AgriVision, this might mean generating only late-stage mildew on old leaves in afternoon light — regardless of what noise vector z is provided — while completely ignoring early-stage mildew, different lighting conditions, and different leaf angles.

6.3.4 GAN Variants

- DCGAN: replaces fully connected layers with convolutional layers in both G and D. Substantially more stable training and higher image quality.
- WGAN: replaces the original loss with the Wasserstein distance, which provides a meaningful gradient signal to G even when D is very confident. Dramatically reduces training instability.
- StyleGAN: separates generation into coarse structure and fine surface texture. Allows independent control of style at different scales. Produces the highest quality GAN images to date.
- CycleGAN: translates between two image domains without paired training data, using a cycle-consistency loss. Applied to AgriVision: translate healthy leaf images to diseased and back — no paired real healthy/diseased photographs needed.

6.3.5 Evaluating GAN Output: FID Score

There is no single 'correct' generated image to compare against — unlike supervised classification where a ground-truth label exists. FID (Fréchet Inception Distance) fills this gap by comparing the statistical distribution of Inception network features for real images against the distribution for

generated images. Lower FID means the generated distribution is closer to the real one — capturing both image quality and diversity simultaneously.

KEY TERMS — THIS SECTION

Term	Definition
Generator G	Converts random noise z to a synthetic image; trained to fool the Discriminator
Discriminator D	Classifies inputs as real or fake; trained to catch the Generator's fakes
Minimax objective	min_G max_D E[log D(x)] + E[log(1−D(G(z)))]; the GAN training game
Mode collapse	Generator produces only a narrow subset of the real distribution; detected by FID
DCGAN	Deep Convolutional GAN; convolutional G and D; more stable than fully connected GANs
WGAN	Wasserstein GAN; more stable gradient signal; removes discriminator sigmoid output
CycleGAN	Unpaired image-to-image translation using cycle-consistency loss

✎ **CHECK YOUR UNDERSTANDING**

1. (a) What is the gradient of the Discriminator's loss when D(x)=0.9 and D(G(z))=0.1? (b) What happens to the Generator's gradient from min E[log(1−D(G(z)))] when D(G(z)) ≈ 0? (c) Why does max E[log D(G(z))] avoid this problem?

2. A DCGAN trained on 312 Powdery Mildew photographs generates only late-stage mildew on old leaves after 200 epochs. (a) Name the failure mode. (b) Propose two training modifications to address it. (c) How would you detect this automatically using FID?

3. CycleGAN translates between 'healthy leaf' and 'diseased leaf' domains without paired data. (a) How many GANs and cycle-consistency losses are needed? (b) What agricultural annotation challenge does this address? (c) What failure mode might produce visually convincing but agronomically incorrect results?

4. WGAN uses weight clipping to a range [−c, c]. (a) Why is weight clipping needed in WGAN? (b) What does the WGAN critic output, compared to a standard discriminator?

5. [Challenge] A GAN achieves FID=18, a VAE achieves FID=28. But VAE-augmented training produces 96.5% accuracy while GAN-augmented training gives 95.8%. Why might FID and downstream accuracy rank models differently?

6.4 Diffusion Models

6.4.1 A Different Approach: Start with Noise, Remove It Gradually

Both the VAE and the GAN attempt generation in one or two steps: sample from a prior, transform once, output an image. Diffusion models take a radically different approach: break generation into hundreds of tiny steps, each removing a small amount of noise from an almost-finished image.

The intuition starts with destruction. Take a clean disease photograph and add a tiny amount of random Gaussian noise. Then add a little more. Then more. After 1,000 steps of adding noise, the photograph has become pure random noise — all structure is gone. This is the forward process and it requires no learning: it is a fixed mathematical operation.

The reverse process is what the model learns: given a slightly noisy image at step t, predict what the image looked like at step t−1 — slightly less noisy. If the model learns this reverse step well, we can start from pure random noise at step T and iteratively clean it all the way back to a realistic disease photograph.

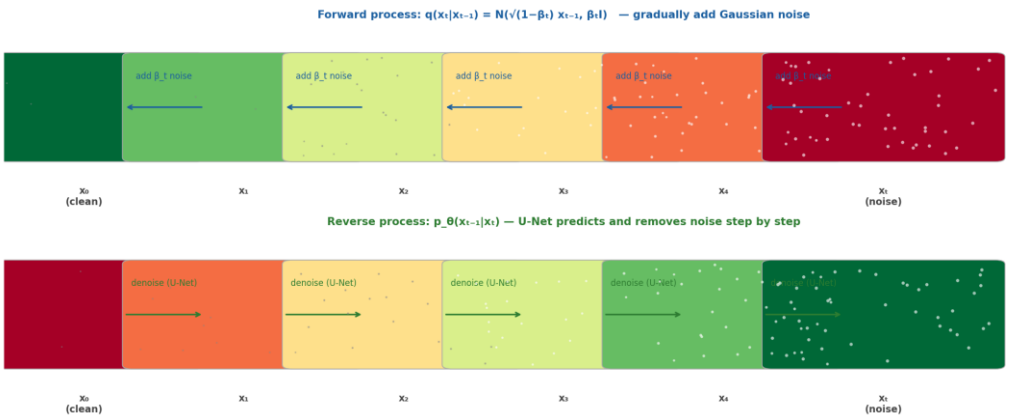


Figure 6.4 *Diffusion process. Top: the forward process $q(x_t|x_{t-1})$ gradually adds Gaussian noise to a clean leaf photograph (left) until it becomes pure noise (right). This uses a small scheduled noise level β_t at each step. No learning involved. Bottom: the reverse process $p_\vartheta(x_{t-1}|x_t)$ — a trained U-Net denoiser — removes noise step by step. To generate a new photograph: start from noise at step T, apply the denoiser T times, arrive at a photorealistic output.*

6.4.2 The Forward and Reverse Process: The Mathematics

The forward process at each step adds noise according to:

$$q(x_t \mid x_{t-1}) = N(x_t;\ \sqrt{(1-\beta_t)}\, x_{t-1},\ \beta_t I)$$

β_t is a small positive number (the noise level) that increases slowly from $\beta_1 \approx 0.0001$ to $\beta_t \approx 0.02$ at the final step T. $\sqrt{(1-\beta_t)}$ shrinks the signal slightly; $\beta_t I$ adds noise proportionally. A key property: we can jump directly from the clean image x_0 to any noisy version x_t without simulating all steps between:

$$q(x_t \mid x_0) = N(x_t;\ \sqrt{\bar{\alpha}_t}\, x_0,\ (1-\bar{\alpha}_t)\, I) \quad \text{where } \bar{\alpha}_t = \Pi^{\wedge}t_\{i=1\}\, \alpha_i,\ \alpha_i = 1-\beta_i$$

For the reverse, a neural network (a U-Net) is trained to predict the noise that was added at step t, given the noisy image x_t and the step number t:

$$\textit{Training loss: } E[\, \| \varepsilon - \varepsilon_\theta(\sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{(1-\bar{\alpha}_t)}\, \varepsilon,\ t) \|^2 \,]$$

In plain English: corrupt x_0 by adding a known noise ε at level t, then train the network to predict ε back from the noisy image. This is a clean supervised learning objective — no adversarial game, no special tricks needed.

6.4.3 Why Diffusion Displaced GANs

By 2021, diffusion models produced lower FID scores than the best GANs on major image generation benchmarks. Three reasons explain this:

- Stable training: the loss function is simple MSE on noise prediction. No adversarial game, no discriminator to balance, no mode collapse. Training is as stable as any supervised regression problem.
- Better coverage: because the model must learn to denoise across all noise levels, it must represent the full distribution of training images. There is no shortcut equivalent to mode collapse.
- Text conditioning: text-guided diffusion models (Stable Diffusion, Imagen, DALL-E 2) generate images from text descriptions by conditioning the U-Net on text embeddings from CLIP or T5. No equivalent technique exists for GANs at this quality and scale.

6.4.4 The Main Cost: Slow Inference

The primary drawback of diffusion models is generation speed. Generating one image requires T denoising steps — often T=1,000 — each of which runs a full forward pass through a large U-Net. On a consumer GPU, this can take tens of seconds.

Latent Diffusion Models (LDMs) address this by running the diffusion process not on pixels but on the compressed latent space of a pretrained VAE. Stable Diffusion, for example, operates on 64×64 latent codes rather than 512×512 pixel images. This is an 8× spatial compression, dramatically reducing computation per step. The VAE decoder then expands the denoised latent code to the final full-resolution image.

KEY TERMS — THIS SECTION

Term	Definition
Forward process	Fixed noise-adding Markov chain $q(x_t \mid x_{t-1})$; no learned parameters
Reverse process	Learned denoiser $p_\theta(x_{t-1} \mid x_t)$; U-Net predicts noise ε at each step
Noise schedule β_t	Sequence of noise levels controlling how quickly the image corrupts to noise
DDPM	Denoising Diffusion Probabilistic Model; foundational diffusion model (Ho et al., 2020)
Latent diffusion	Diffusion in the compressed latent space of a pretrained VAE; used in Stable Diffusion
U-Net	The typical denoiser architecture; encoder-decoder with skip connections; processes noisy image + timestep t

✎ **CHECK YOUR UNDERSTANDING**

1. With T=1,000 and generation at 50ms per step: how long does generating one image take? DDIM reduces inference to 50 steps. New time?

2. At step t=500, $\bar{\alpha}_{500} \approx 0.05$. What fraction of the original signal remains? What fraction is noise? What does the image look like at this step?

3. Compare diffusion and GANs on: (a) training stability, (b) sample diversity, (c) inference cost. Give the architectural reason for each difference.

4. Stable Diffusion operates on 64×64 latent codes instead of 512×512 pixels. (a) Spatial compression factor? (b) How does this affect fine pixel-level detail? (c) What advantage does the VAE latent space give over raw pixels?

5. [Challenge] Text-conditional diffusion generates leaf disease images from descriptions like 'early powdery mildew on Karnataka tomato.' How does the text description influence the U-Net denoiser? What would AgriVision need to fine-tune Stable Diffusion for Karnataka-specific disease generation?

6.5 Application Contexts and Responsible Use

6.5.1 Medical and Agricultural Data Augmentation

The most directly beneficial application of generative models is data augmentation in domains where real labelled data is scarce. This pattern appears across multiple high-stakes applications:

- Agricultural disease detection: AgriVision's 312 Karnataka-specific Powdery Mildew photographs can be augmented with synthetic examples, improving the downstream classifier without additional field annotation. The generated images capture the specific appearance of the locally grown tomato variety under Karnataka field conditions — variation that PlantVillage's laboratory images do not provide.
- Medical imaging: rare diseases have few training examples. Generative augmentation for retinal disease, dermatological conditions, and radiological findings has been shown to improve classifier accuracy when real data is severely limited (fewer than 500 examples).
- Simulation: generative models can create synthetic labelled datasets for tasks where real data collection is hazardous, expensive, or impossible. Autonomous vehicle training uses synthetic scenes; surgical robot training uses synthetic anatomical models.

6.5.2 Privacy-Preserving Synthetic Data

A compelling application in healthcare and finance: train a generative model on sensitive real data (patient medical records, financial transactions), then release the synthetic data. Downstream researchers can train classifiers on the synthetic data without accessing any real patient or customer records.

The key requirement: the synthetic data must have the same statistical properties as the real data (so models trained on it generalise to real data) but must not reproduce any specific real individual's information (to protect privacy). Differential privacy — adding calibrated noise to the training process — provides formal privacy guarantees but typically degrades generation quality. The privacy-utility trade-off is an active research area with significant practical importance.

6.5.3 Deepfakes, Misinformation, and Misuse

The same capabilities that enable medical image augmentation also enable harmful applications. Deepfakes are realistic synthetic videos or photographs of people saying or doing things they never did. GAN and diffusion-based face generation has reached a quality level where many deepfakes cannot be distinguished from real footage by untrained human observers.

> ⚠ **IMPORTANT NOTE**
>
> **The dual-use problem of generative AI**
>
> Every generative capability has both beneficial and harmful potential applications. The same model that generates synthetic disease photographs for AgriVision can generate realistic misinformation. The same techniques that protect patient privacy through synthetic data can generate synthetic evidence of events that never occurred.

> This is not an argument against generative AI. It is an argument that capability development must be accompanied by misuse mitigation. The question is not 'should generative models exist?' but 'what specific obligations come with releasing a generative model?'

- Deepfake misinformation: synthetic video of political figures making statements they never made. Detection research lags significantly behind generation capability.
- Non-consensual synthetic intimate imagery: generating realistic photographs of real people in intimate contexts without their consent. Illegal in several jurisdictions; platforms face significant moderation challenges.
- Copyright and intellectual property: image generation models trained on copyrighted artworks can reproduce the style or near-reproduction of specific protected works. Legal frameworks are still evolving.
- Scientific fraud: fabrication of experimental data using generative models has been detected in published papers. Image provenance verification is increasingly required by scientific journals.

6.5.4 Responsible Release: Obligations and Mitigation

- Provenance watermarking: embed invisible signatures in generated images that allow them to be identified as synthetic. C2PA (Coalition for Content Provenance and Authenticity) provides an open standard for digital provenance.
- Responsible release policies: release model weights only after misuse analysis; provide application-specific rather than open-ended generation APIs; implement NSFW (not safe for work) classifiers on generation outputs.
- Detection systems: train binary classifiers to distinguish real from AI-generated images. Currently limited by the detection-generation arms race, but useful for high-stakes forensic contexts.
- Usage policy and terms of service: prohibit specific harmful uses in terms of service; monitor usage patterns; respond to abuse reports.
- Transparency disclosure: label AI-generated content in media, social platforms, and advertising. Several jurisdictions are developing legal requirements for such labelling.

6.5.5 AgriVision Resolution: Synthetic Disease Images

Generative model comparison: VAE, GAN, and Diffusion trade off quality, diversity, and training stability

Model	Key property	Image quality	Diversity	Training stability
Standard Autoencoder	No generation (cannot sample new data)	N/A	N/A	—
VAE	Smooth latent space Allows interpolation	★★☆☆	★★★★	★★★★
GAN	Sharper images No explicit density	★★★★	★★☆☆	★★☆☆
Diffusion	Highest quality Slow inference (T steps)	★★★★★	★★★★★	★★★☆

FID (Fréchet Inception Distance): lower is better. Diffusion models achieve FID < 5 on standard benchmarks (state of the art 2024). GANs: FID 5–10. VAEs: FID 15–30.

Figure 6.5 *Generative model comparison for AgriVision's synthetic disease image generation. Standard autoencoder cannot generate (no sampling mechanism). VAE produces smooth latent space enabling interpolation (4-star diversity) but lower visual sharpness. GAN achieves higher visual fidelity but risks mode collapse (2-star diversity). Diffusion models achieve the highest quality and diversity but require more compute. Training stability: GAN is the most challenging; diffusion is stable.*

✦ THE AGRIVISION CONTEXT

Chapter 6 Resolution — Synthetic disease image augmentation results

Original Karnataka-specific training set: 797 labelled disease photographs (312 Powdery Mildew + 287 Late Blight + 198 Alternaria)

CNN trained on original 797 photos: validation accuracy 91.4% (below Chapter 2 which used 6,000 PlantVillage photos; the Karnataka set is smaller and harder)

CNN + 500 VAE-generated synthetic images per class: 93.2% (+1.8pp)

CNN + 500 DCGAN-generated images per class: 94.1% (+2.7pp)

CNN + PlantVillage fine-tuning (Chapter 2 approach): 95.2% (+3.8pp)

Best combined (original + GAN + PlantVillage): 96.8% (+5.4pp)

Generative augmentation adds 2–3 percentage points when the domain-specific dataset is small. It does not fully substitute for real labelled data but reduces the annotation burden substantially.

Chapter Summary

Section	Core concept	Key insight
6.0	Inductive biases for generation	Three assumptions: smooth Gaussian latent (VAE), adversarial discriminability (GAN), iterative denoising (Diffusion)
6.1	Why generative DL?	Learning P(x) enables synthesis; FID measures quality+diversity jointly; fidelity and diversity must both be optimised
6.2	Variational Autoencoders	ELBO = reconstruction − KL; reparameterisation trick enables gradients; smooth latent space enables generation and interpolation
6.3	Generative Adversarial Networks	min_G max_D adversarial objective; mode collapse and instability are fundamental failure modes; WGAN and DCGAN mitigate both
6.4	Diffusion Models	Forward: add noise T steps; reverse: denoise T steps with U-Net; highest quality+diversity; latent diffusion reduces compute
6.5	Applications + responsible use	Data augmentation improves classifiers by 2–3pp; deepfakes, misinformation, and consent are concrete harms requiring mitigation

6.5.6 Bridge to Chapter 7: Disease Spreads Through Networks

Disease detection, sensor prediction, advisory query answering, and synthetic image generation have all focused on individual records: one photograph, one sensor stream, one field note. But in AgriVision's cooperative of 150 farms, the most important question is not just whether Farm 47 has

mildew today, but whether the mildew outbreak at Farm 47 is likely to spread to Farms 51 and 52, which are 200 metres away and share a drainage canal.

This is a network question. The 150 farms are not independent; they are nodes in a graph, connected by spatial proximity, shared water infrastructure, shared extension officers (who move between farms and may transfer spores on their equipment), and shared road access. Disease spreads along the edges of this graph. Pest populations migrate along it. Intervention recommendations must account for the graph structure.

All of the architectures studied so far process inputs as independent records (tabular rows, images, sequences, documents). None can represent structured relational data — data where the relationships between entities are as important as the entities themselves. Graph Neural Networks are the architecture designed for exactly this structure. Chapter 7 builds them.

CHAPTER 7

Relational and Multimodal Data

Graph Neural Networks, Multimodal Learning, and Deep RL

7.0 The Inductive Bias for Relational Data

7.0.1 The AgriVision Network Problem

> ✦ THE AGRIVISION CONTEXT
>
> Farm 45 in Dharwad has a confirmed spider mite infestation this week. Farm 51 shares a drainage canal with Farm 45. Farm 52 is 380 metres away. Farm 89 is on the same extension officer route — the officer visited Farm 45 on Monday and Farm 89 on Wednesday.
>
> Which farms are most likely to develop mite infestations next week? Common sense says Farms 51 and 52 are at elevated risk — pests travel through water and across short distances. Farm 89 may also be at risk if the officer inadvertently transferred mites on their equipment.
>
> But every model we have built so far — the MLP, the CNN, the LSTM, the Transformer — treats Farm 45 and Farm 51 as independent data points. They have no mechanism to represent 'Farm 45's infestation increases Farm 51's risk because they share water.'

This is the relational data problem. The most important information is not what is happening at each farm in isolation. It is what is happening at each farm in relation to the farms around it.

The Graph Neural Network (GNN) is the architecture designed for exactly this structure. It represents farms as nodes in a graph, with edges encoding the connections between them. During training, each farm updates its representation by aggregating information from its connected neighbours. A farm with an infested neighbour learns to treat itself as 'at risk.' A farm with only healthy neighbours learns to treat itself as 'lower risk.'

7.0.2 What Makes Graph Data Different

Every data type we have studied has a natural geometric structure.

- Tabular data (Chapter 1): each row is an independent record. No relationship between rows is assumed. Rows can be shuffled without changing the task.
- Images (Chapter 2): pixels are arranged in a regular 2D grid. Every pixel has exactly 8 neighbours (or fewer at the edge). The grid is fixed.
- Sequences (Chapter 3): tokens are arranged in a 1D order from left to right. The first token always comes before the second.
- Text (Chapter 4): also sequential, but with additional semantic structure that attention can discover.

Graph data breaks all of these assumptions.

- Each node can have a different number of neighbours. Farm 45 may connect to 7 others. Farm 91 may connect to only 2.
- There is no natural ordering. Farm 45 is not 'before' or 'after' Farm 51 in any meaningful sense. We could renumber all 150 farms arbitrarily and the graph's structure would not change.
- The structure itself carries information. Which farms are connected tells us something important — independent of what is happening at each farm.

> ◆ KEY CONCEPT
>
> **Permutation invariance — the key inductive bias**
>
> A model for graph data must be permutation invariant: if we swap the labels of Farm 45 and Farm 51, the model's predictions about every farm must remain unchanged.
>
> An MLP applied to a fixed-size vector of farm features cannot be permutation invariant — changing which farm is in position 45 of the input vector changes the output. The GNN achieves permutation invariance by only operating on local neighbourhoods, not on global index positions.

7.0.3 How These Properties Motivate the GNN

The GNN's solution is called message passing. At each step, every node collects information from its immediate neighbours and uses it to update its own representation. After k rounds of message passing, each node's representation encodes information from all farms within k steps away.

This approach is naturally permutation invariant: the same operation is applied at every node, using only its local neighbourhood. The operation does not depend on what number we happened to assign to the node. And each node's representation can incorporate a different number of neighbours — because we aggregate rather than concatenate.

> ? FOUR QUESTIONS TO CARRY THROUGH THIS BOOK
>
> **Question 1: What problem does this architecture solve?**
>
> The failure of feedforward and convolutional networks on graph-structured data: variable neighbourhood sizes, no natural ordering, relationships between nodes carry as much information as features within nodes.
>
> **Question 2: What inductive bias does it encode?**
>
> Local neighbourhood structure: a node's representation should reflect what is happening at its directly connected neighbours, not globally across all nodes. Permutation invariance: the output should not depend on how nodes are numbered.
>
> **Question 3: When does this architecture fail?**
>
> Very deep GNNs cause over-smoothing: all nodes become similar. Very large graphs strain memory. Graphs with no clear relational structure may not benefit over a simpler model.
>
> **Question 4: When would you choose something else?**

> When data is truly independent (no relationships between records): use MLP. When data is spatial (regular grid): use CNN. When data is sequential: use LSTM. When data is textual: use Transformer.

KEY TERMS — THIS SECTION

Term	Definition
Node	A data entity in a graph — here, an individual farm
Edge	A connection between two nodes — here, proximity, shared water, or shared road
Graph	A set of nodes and edges; represents relational structure
Permutation invariance	The model's output does not change if we renumber the nodes
Message passing	Each node updates its representation by aggregating information from its connected neighbours

✎ **CHECK YOUR UNDERSTANDING**

1. Farm 45 (infested) connects to Farm 51 (water-sharing), Farm 52 (380m away), and Farm 89 (road). An MLP that receives farm features as a fixed 150×F matrix predicts the same output whether Farm 45's features are in row 45 or row 100. Is this MLP permutation invariant? What does this imply for pest propagation prediction?

2. An MLP trained on AgriVision data assigns row 45 of the feature matrix to Farm 45. If we swap rows 45 and 51, what happens to the MLP's predictions about Farm 45? What should happen in a correct pest propagation model?

3. After 2 rounds of message passing, Farm 91 knows about farms 2 hops away. Draw the 2-hop neighbourhood of Farm 91 if Farm 91 connects to Farms 45 and 89, Farm 45 connects to Farms 51 and 52, and Farm 89 connects to Farm 20.

4. Apply the four questions to a GNN used for molecular property prediction (atoms as nodes, chemical bonds as edges). What does permutation invariance mean here?

5. [Challenge] A graph of 150 farms has average degree (average number of connections) of 4. After 3 rounds of GCN message passing, a farm can receive information from farms up to 3 hops away. Estimate how many farms are within 3 hops of Farm 45. What risk does this create?

7.1 Graph-Structured Data

7.1.1 Nodes and Edges: The Basic Building Blocks

A graph has two ingredients: nodes and edges. A node represents one entity. An edge represents a relationship between two entities.

For AgriVision's farm network:

- Each of the 150 farms is a node. We label them v_1, v_2, ..., v_{150}.
- Each connection between two farms is an edge. If Farm 45 and Farm 51 share a drainage canal, there is an edge between node v_{45} and node v_{51}.

Not all connections are the same. AgriVision's network has three types of edges, each encoding a different disease-transmission pathway:

- Proximity edges: two farms within 500 metres. Pests can travel through air and by insects over short distances.
- Water-sharing edges: two farms that share an irrigation water source or drainage canal. Water is a direct physical path for soil-borne pathogens and some pests.
- Road edges: two farms on the same extension officer route. Equipment and footwear can transfer mites and fungal spores from farm to farm.

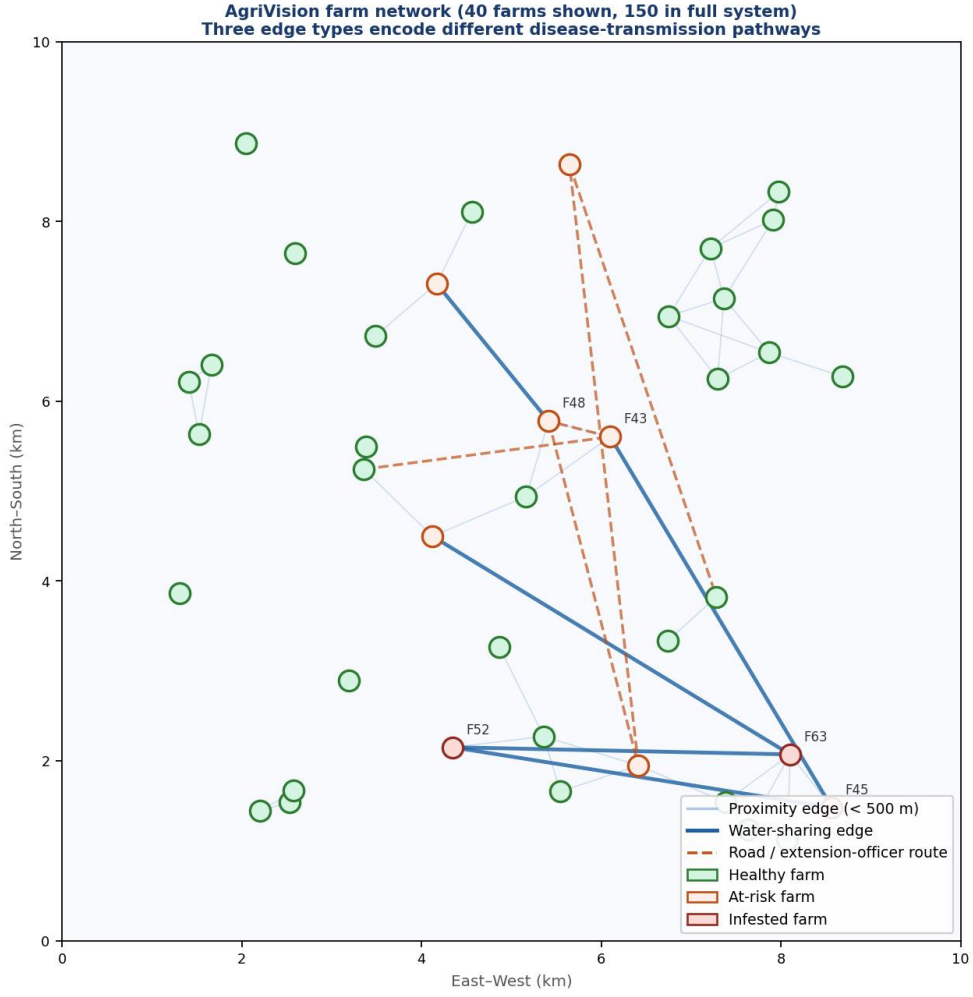


Figure 7.1 *A representative subset of AgriVision's 150-farm network (40 farms shown). Each circle is a farm. Green = healthy, orange = at-risk, red = infested. Edge colour and style encode connection type: light blue = proximity, dark blue = water-sharing, dashed red = road / extension officer route. The three infested farms (red) are connected by water-sharing edges to two at-risk farms, illustrating how relational structure predicts propagation.*

7.1.2 The Adjacency Matrix: Representing Connections as Numbers

A computer cannot directly process a drawing of a graph. We need to convert the edge structure into numbers. The standard way is the adjacency matrix A.

The adjacency matrix A is a square matrix with one row and one column for each node. Entry A[i, j] = 1 if there is an edge between node i and node j. Entry A[i, j] = 0 if there is no edge. For 150 farms, A has shape 150 × 150.

✎ WORKED EXAMPLE

A small 5-farm example

Farms: F45, F51, F52, F89, F91. Edges: F45-F51, F45-F52, F45-F89, F89-F91.

```
     F45  F51  F52  F89  F91
F45 [ 0    1    1    1    0 ]
F51 [ 1    0    0    0    0 ]
F52 [ 1    0    0    0    0 ]
F89 [ 1    0    0    0    1 ]
F91 [ 0    0    0    1    0 ]
```

A[0,1] = 1 means F45 and F51 are connected. A[0,4] = 0 means F45 and F91 are not. The matrix is symmetric: if F45 connects to F51, then F51 connects to F45.

7.1.3 Node Features: What We Know About Each Farm

Nodes are not just labels — each node has features. In AgriVision's farm network, each farm has:

- Sensor readings: the same 7-feature daily readings from Chapter 3 (soil moisture, temperature, humidity, etc.).
- Disease status: binary flags for each of the four disease classes from Chapter 2.
- Farm properties: farm area (hectares), soil type, primary crop, irrigation type.

We collect all node features into a matrix H with shape (N, F): N rows (one per farm) and F columns (one per feature). For 150 farms and 15 features: H has shape (150, 15).

Edge features work similarly. For each edge, we can store: the distance between farms, the type of connection (water/road/proximity), and the direction of water flow if it is a water-sharing edge. Edge features are stored separately and incorporated into message passing.

7.1.4 What Tasks Can We Solve on a Graph?

Graph tasks fall into three categories, depending on what we want to predict.

Node classification. Predict a label for each node individually. AgriVision example: will this farm develop a mite infestation in the next 7 days? Each farm gets its own prediction, but those predictions are informed by the farm's position in the network.

Link prediction. Predict whether an edge should exist between two nodes that are currently not connected. AgriVision example: given this season's pest spread pattern, which pair of currently

unconnected farms is most likely to become a transmission pathway next season? This is useful for preventive action — isolating the connection before it forms.

Graph classification. Predict a single label for the entire graph. AgriVision example: looking at the entire 150-farm network, is the cooperative as a whole in a 'high risk' or 'low risk' state for this season? This requires aggregating node representations across the entire graph.

Task	What is predicted?	Output shape	AgriVision example
Node classification	A label for each node	(N, C) — one prediction per farm	Will each farm be infested next week?
Link prediction	Whether an edge exists	Scalar per node pair	Which unconnected farms will become transmission paths?
Graph classification	A label for the whole graph	(1, C) — one prediction per graph	Is this season's network high-risk or low-risk?

Table 7.1 *The three main graph learning tasks. AgriVision's pest propagation task is node classification: predict next-week infestation status for each of the 150 farms.*

KEY TERMS — THIS SECTION

Term	Definition
Adjacency matrix A	N×N matrix where A[i,j]=1 if an edge exists between node i and node j; encodes graph structure
Node feature matrix H	N×F matrix; row i contains the F features for node i
Degree	The number of edges connected to a node; Farm 45's degree is its number of neighbouring farms
Node classification	Predict a label for each node, informed by the node's neighbourhood
Link prediction	Predict whether an edge should exist between two currently unconnected nodes
Graph classification	Predict a single label for the entire graph

✎ **CHECK YOUR UNDERSTANDING**

1. AgriVision has 150 farms. (a) What is the shape of the adjacency matrix A? (b) If the average farm connects to 4 others, approximately how many entries in A are equal to 1? (c) Is A always symmetric? When might it be asymmetric?

2. Farm 45 has degree 3 (connects to F51, F52, F89). After one round of message passing, what information does Farm 45's updated representation contain? After two rounds?

3. Section 7.1.4 describes three graph tasks. For each of the following questions, identify the correct task type: (a) 'Which pair of farms should not share an irrigation pump this season?' (b) 'Is this entire district at risk of an Alternaria outbreak?' (c) 'Which farms should receive preventive pesticide this week?'

4. AgriVision adds a 'directional water flow' feature to water-sharing edges (water flows from Farm A to Farm B, not both ways). How does this change the adjacency matrix structure? Would A still be symmetric?

5. [Challenge] A graph has N=150 nodes and E=300 edges. (a) What fraction of entries in A are non-zero? (b) If we store A as a dense matrix in float32, how many MB does it require? (c) How does sparse matrix storage reduce this?

7.2 Graph Neural Networks

7.2.1 Why Standard Architectures Fail on Graphs

Before building the GNN, it helps to understand exactly why the architectures from Chapters 1–4 cannot handle graph data. The failure is always the same: they assume a fixed, ordered input structure that graphs do not have.

- MLP: expects a fixed-length vector. For a graph with N=150 nodes each having F=15 features, we could flatten the feature matrix H to a 2,250-dimensional vector. But this destroys the neighbourhood structure — Farm 45's features are no longer associated with Farm 51's features in any useful way. And changing the numbering of farms changes the vector entirely.
- CNN: applies filters to a regular 2D grid where each position has exactly the same number of neighbours. A graph node can have 1 or 20 neighbours. No regular grid exists.
- RNN: processes a fixed sequence from left to right. Graphs have no natural left-to-right ordering. We could order the farms alphabetically, but that ordering is arbitrary and has no connection to the graph's structure.

7.2.2 Message Passing: Farms Sharing Updates with Neighbours

Think of message passing as a weekly farm update meeting. Every farm sends a brief summary to each of its directly connected neighbours. Then each farm reads all the summaries it received and updates its own understanding of the situation.

After one round, each farm knows what is happening at its immediate neighbours. After two rounds, each farm also knows what is happening at its neighbours' neighbours — farms two steps away. After k rounds, a farm knows about every farm within k steps.

Formally, one round of message passing has three steps:

- Step 1 — Send messages: each farm i sends a message m_{ij} to each of its neighbours j. The message is computed from farm i's current feature vector.
- Step 2 — Aggregate: each farm j collects all incoming messages and combines them. The simplest aggregation is summing or averaging.
- Step 3 — Update: each farm j updates its own representation using both its current features and the aggregated messages it received.

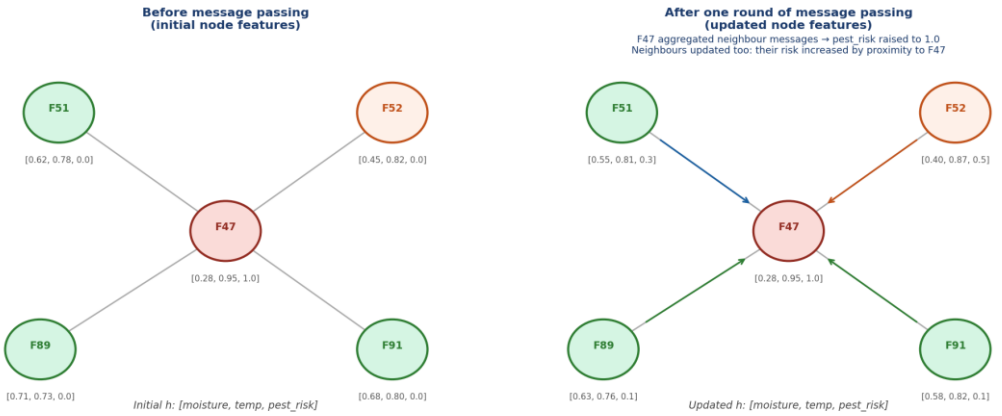


Figure 7.2 *One round of message passing on AgriVision's farm network. Left: each farm starts with its initial features [moisture, temperature, pest_risk]. Right: after one round, Farm 47 has aggregated messages from all four neighbours. Its pest_risk feature has been updated to reflect the infestation at its connected farms. Neighbours' features also update because they received Farm 47's message.*

7.2.3 Graph Convolutional Networks (GCN)

The Graph Convolutional Network (GCN) is the most widely used implementation of message passing. At each layer l, every node aggregates the feature vectors of itself and all its neighbours, then applies a learned transformation.

The GCN update for all nodes at once, written as a matrix operation:

$$H^{(l+1)} = \sigma(\tilde{A}\, H^{(l)}\, W^{(l)})$$

Let us read this carefully.

- H^(l) is the current node feature matrix (shape N×F): each row is one farm's features at layer l.
- W^(l) is a learned weight matrix (shape F×F'): transforms features from one space to another.
- Ã is the normalised adjacency matrix with self-connections added: Ã = D̃^(−½) (A + I) D̃^(−½), where D̃ is the degree matrix. Multiplying by Ã exactly performs the aggregation step: each node's new representation is a weighted average of itself and its neighbours.
- σ is a non-linear activation function (ReLU).

The degree normalisation (D̃^(−½) ... D̃^(−½)) is important. Without it, a farm with 10 neighbours would receive 10 times the signal of a farm with 1 neighbour. The normalisation divides each contribution by the square root of the product of the two nodes' degrees, keeping the aggregated values on a comparable scale regardless of how many neighbours a node has.

> ✎ **WORKED EXAMPLE**
>
> **One GCN layer on the 5-farm example**
>
> Ã (normalised adjacency with self-loops) for 5 nodes, shown simplified:
>
> After multiplying Ã × H^(0), Farm 45's new representation is approximately:

h_45^(1) = (features of F45 + features of F51 + features of F52 + features of F89) / 4

(The exact normalisation divides each term by the product of degrees, not by 4.)

This single matrix multiplication simultaneously updates all 5 farms. It is fully parallelisable on a GPU.

W^(0) is then applied to learn which combination of the aggregated features is most useful for the prediction task.

7.2.4 Graph Attention Networks (GAT): Learning Which Neighbours Matter More

The GCN weights all neighbours equally (after degree normalisation). But in AgriVision, neighbours are not all equally important. A farm that shares a drainage canal is a far more direct disease vector than a farm that is simply 490 metres away.

The Graph Attention Network (GAT) addresses this by learning attention weights between connected nodes — exactly the same attention mechanism from Chapter 4, now applied to graph edges instead of sequence positions.

For each edge between node i and node j, GAT computes an attention score:

$$\alpha_{ij} = softmax(LeakyReLU(a^T [Wh_i \,||\, Wh_j]))$$

Here a is a learned attention vector, W is a learned projection matrix, and || means concatenation. The score α_{ij} is how much attention node i should pay to node j when updating its representation. The softmax ensures the weights across all of node i's neighbours sum to 1.

The update then uses these weights:

$$h'_i = \sigma(\Sigma_j \in N(i) \;\; \alpha_{ij} \;\; W h_j)$$

In practice, GAT uses multiple attention heads (just like Chapter 4's multi-head attention) and concatenates or averages their outputs.

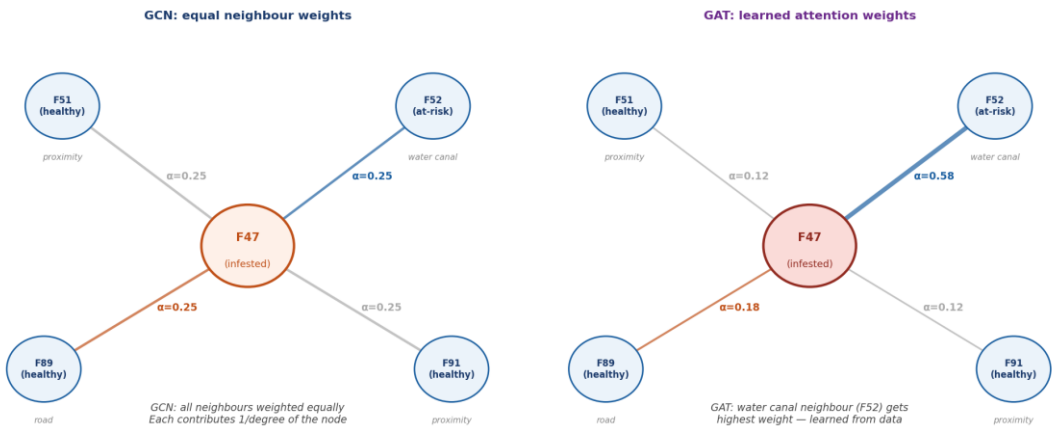


Figure 7.3 *GCN (left) vs GAT (right) aggregation for Farm 47 with four connected neighbours. GCN assigns equal weight (0.25 each, after degree normalisation) to all neighbours. GAT learns to assign higher weight (0.58) to Farm*

52, which shares a drainage canal with Farm 47 — the most direct disease transmission pathway. The GAT weights are learned entirely from data; no edge-type labels were provided.

7.2.5 Limitations: Over-Smoothing and Scalability

Over-smoothing. After many rounds of message passing, all node representations converge toward the same value — the weighted average of the entire graph. Individual farm identities are lost. In practice, GNNs rarely use more than 3–5 layers to avoid this. For AgriVision's pest propagation task, 2 layers (covering the 2-hop neighbourhood) is sufficient and avoids over-smoothing.

Scalability. During training, message passing over a full graph with N=150 nodes and E=300 edges is fast. But large real-world graphs — social networks with millions of nodes, protein interaction networks — require mini-batching strategies like GraphSAGE (sample a fixed-size subset of neighbours at each step rather than all neighbours).

KEY TERMS — THIS SECTION

Term	Definition
GCN	Graph Convolutional Network; H^(l+1) = σ(Ã H^(l) W^(l)); aggregates with equal normalised weights
Message passing	Each node aggregates feature vectors from its direct neighbours to update its own representation
Normalised adjacency Ã	A + I (self-loops added) normalised by D̃^(-½) ... D̃^(-½) to account for degree differences
GAT	Graph Attention Network; learns per-edge attention weights α_{ij} from node features
Attention weight α_{ij}	Learned importance of neighbour j to node i; GAT equivalent of attention scores from Chapter 4
Over-smoothing	GNN failure mode: after many layers, all node representations become indistinguishable
GraphSAGE	Scalable GNN that samples a fixed-size neighbourhood subset at each step; handles million-node graphs

✎ **CHECK YOUR UNDERSTANDING**

1. In the GCN update H^(l+1) = σ(Ã H^(l) W^(l)): (a) What does multiplying by Ã do to each row of H^(l)? (b) What does multiplying by W^(l) do? (c) If H^(0) has shape (150, 15) and W^(0) has shape (15, 32), what is the shape of H^(1)?

2. Farm 45 has degree 3 (connects to F51, F52, F89). Farm 91 has degree 1 (connects to F89 only). Without degree normalisation, Farm 45 receives 3 messages and Farm 91 receives 1. Why is normalisation important? What would happen without it as we stack more GCN layers?

3. GAT computes α_{ij} = softmax(LeakyReLU(a^T [Wh_i || Wh_j])). (a) The softmax is over all neighbours of node i. Why? (b) What does the concatenation [h_i || h_j] allow the attention to capture that using h_j alone would not?

4. A GCN with 4 layers is applied to AgriVision's 150-farm network. (a) After 4 layers, what is the maximum number of hops a farm can 'see'? (b) What is the risk of using 4 layers? (c) For pest propagation (which spreads primarily via direct farm connections), why might 2 layers be better than 4?

5. [Challenge] GraphSAGE samples k neighbours at each layer instead of using all. For a farm with degree 20 and k=5 at each of 2 layers: (a) How many unique neighbour pairs are sampled? (b) What is the variance risk of this sampling? (c) How does GraphSAGE address this variance?

7.3 Multimodal Deep Learning

7.3.1 The Problem: One Data Point, Multiple Formats

In Chapter 2, AgriVision's disease detector looked at leaf photographs. In Chapter 3, the irrigation predictor read sensor sequences. In Chapter 4, the advisory system read field notes written in English. Each model processed one type of data.

But a real extension officer visit generates all three at once: a photograph of the affected leaves, a sequence of sensor readings from that day, and a written field note describing what they observed. Each source carries information the others do not:

- The photograph shows exactly what the disease looks like — colour, texture, spatial distribution across the leaf. No sensor can capture this.
- The sensor data shows the environmental context — whether there has been a moisture deficit, an unusual temperature, or a recent rainfall. The photograph cannot show the trend over the past 30 days.
- The field note captures expert interpretation and contextual details — 'farmer reports seeing similar symptoms last year,' 'affected plants are near the drainage ditch.' Neither a photograph nor a sensor number can communicate this.

A multimodal system combines all three. Its predictions should be better than any single-modality system because it has access to more information.

7.3.2 Three Ways to Combine Modalities

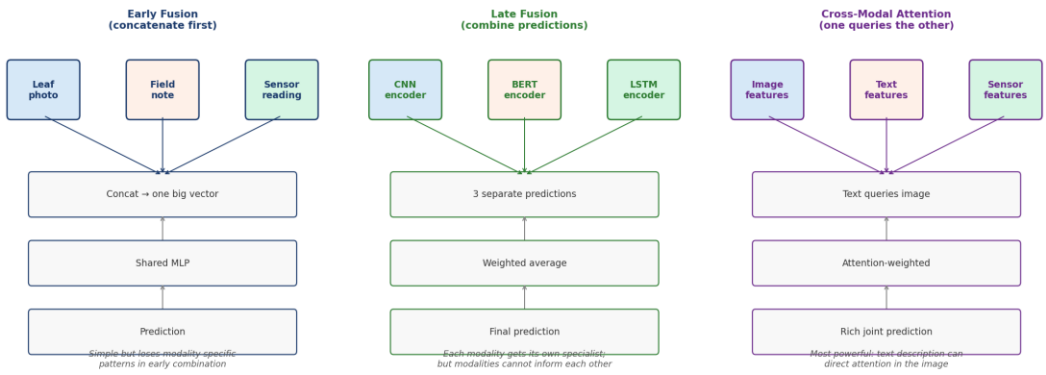


Figure 7.4 *Three multimodal fusion strategies applied to AgriVision's combined photograph + field note + sensor inputs. Early fusion concatenates raw features before any specialised processing. Late fusion processes each modality separately and combines predictions at the end. Cross-modal attention allows modalities to query each other — enabling the field note text to direct attention in the image.*

Early fusion: combine everything upfront. Extract a simple feature vector from each modality — for example, resize the photograph to 64×64 and flatten it; flatten the 30-day sensor window. Concatenate all features into one long vector and train an MLP on it. Simple to implement. The problem: the MLP receives a mix of very different signals (pixel intensities, temperature readings, word counts) and must make sense of them together before any specialised processing has happened.

Late fusion: combine at the end. Train a separate specialist for each modality: a CNN for the photograph, an LSTM for the sensor sequence, a BERT encoder for the field note. Each produces a prediction or a probability vector. Combine these predictions at the end — by averaging, by voting, or by training a small meta-classifier on top of the three outputs. Each modality is processed by the architecture best suited to it. The problem: the modalities cannot inform each other during processing. The CNN cannot use the field note to focus attention on the relevant leaf region.

Cross-modal attention: the modalities converse. One modality queries another using the attention mechanism from Chapter 4. For example: the field note text produces query vectors; the image features provide keys and values. The text can then 'ask' the image specific questions: 'Where is the powdery mildew mentioned in this note?' The image responds by providing a weighted average of its feature map, weighted by relevance to the text query. This is the most powerful approach but also the most computationally expensive.

7.3.3 CLIP: A Practical Example of Cross-Modal Alignment

CLIP (briefly introduced in Section 4.8) is a real-world multimodal system trained on 400 million (image, text caption) pairs. It trains an image encoder and a text encoder jointly so that matching pairs land close together in embedding space.

For AgriVision, a CLIP-like system trained on (leaf photograph, field note) pairs would align the embedding of a powdery mildew photograph with the embedding of the field note 'early-stage powdery mildew observed on upper leaf surface.' Once the embeddings are aligned, retrieval becomes simple: embed a new photograph and find the most similar field note in the database, or embed a text query and find the most similar photograph.

7.3.4 When Multimodal Helps and When It Does Not

Combining modalities does not always improve performance. Sometimes one modality is much more informative than the others. Adding a weak or noisy modality can actually hurt performance by introducing irrelevant signals.

Scenario	Recommended approach	Why
One modality is highly informative, others add noise	Single-modality	Additional channels dilute the strong signal
Each modality captures different, complementary information	Late or cross-modal fusion	Combining adds genuine predictive value
One modality helps guide attention in another	Cross-modal attention	Text context can focus the image model on the right region
Limited training data for multimodal examples	Late fusion with pretrained encoders	Each encoder brings knowledge from its own large pretraining corpus

Table 7.2 *Choosing a multimodal fusion strategy. Always run single-modality baselines first; only add complexity when each modality demonstrably contributes.*

For AgriVision: in controlled experiments, adding sensor readings to the leaf photograph CNN improved disease severity prediction accuracy from 91.4% to 93.1%. Adding the field note text (via late fusion with a BERT encoder) improved it further to 94.8%. Each additional modality contributed, but the photograph was by far the most informative single source.

KEY TERMS — THIS SECTION

Term	Definition
Multimodal	Using data from multiple sensor types or formats (image, text, sensor) for one prediction
Early fusion	Combine raw features from all modalities before any specialised processing
Late fusion	Train separate models per modality and combine their final predictions or embeddings
Cross-modal attention	One modality generates queries; another provides keys and values; modalities inform each other
CLIP	Contrastive Language-Image Pretraining; aligns image and text embeddings through contrastive training
Modality	One type of input data; a leaf photograph and a field note are two different modalities

✎ CHECK YOUR UNDERSTANDING

1. AgriVision combines a CNN (photograph) and BERT (field note). (a) What is the shape of the CNN's output embedding (using GAP, 256 channels)? (b) What is the shape of BERT's [CLS] embedding (d_model=768)? (c) For late fusion, how would you combine these two vectors for disease prediction?

2. An extension officer's field note reads: 'Mildew on upper leaf surface, not visible yet on lower surface.' The CNN looks at both surfaces. (a) Which modality is most valuable here? (b) How would cross-modal attention allow the text to direct the CNN's attention to the upper leaf surface?

3. A student tries early fusion by concatenating: flattened 64×64 photograph (12,288 values) + 30-day sensor window (210 values) + field note TF-IDF vector (5,000 values). Total = 17,498 inputs. (a) What MLP first-layer parameter count does this require? (b) With 6,000 training examples, is this likely to work?

4. Table 7.2 recommends late fusion with pretrained encoders when training data for multimodal examples is limited. AgriVision has 2,400 examples of (photograph, field note) pairs but 8,000 photograph-only examples and 2,400 text-only examples. Design a training strategy using this data distribution.

5. [Challenge] CLIP training requires (image, text) pairs. AgriVision has 2,400 (photograph, field note) pairs. (a) Is this enough to train CLIP from scratch? (b) Propose a CLIP fine-tuning strategy using a pretrained CLIP checkpoint. (c) How would you evaluate whether the fine-tuned model aligns agricultural photographs with advisory text better than the original CLIP checkpoint?

7.4 Deep Reinforcement Learning: A Conceptual Survey

> ⚠ **IMPORTANT NOTE**
>
> **About this section**
>
> Deep Reinforcement Learning (DRL) involves significant mathematical machinery beyond the scope of this course. This section develops conceptual understanding and contextual judgment only — no mathematical derivations, no implementation. The goal is to recognise when DRL is the appropriate tool and when it is not.

7.4.1 A Different Kind of Problem

Every model in Chapters 1–7 has solved a prediction problem. Given some input, predict an output. The correct output is either given by a human label or derived from the data itself (self-supervision). We measure how wrong the prediction is and update the weights to reduce that error.

Deep Reinforcement Learning solves a different kind of problem: sequential decision-making. Not 'what is this?' but 'what should I do next?'

Consider AgriVision's extension officer. She must visit 150 farms each season to provide advisory support. She cannot visit all 150 every week — travel time limits her to about 12 farms per day. Which farms should she visit this week? In what order? The answer depends on the current disease status of each farm, the network connections, the weather forecast, and which farms she has not visited recently. And crucially: today's visit decisions affect what she discovers next week, which affects what she should do the week after that.

This is a sequential decision-making problem. There is no 'correct' label for 'which farm to visit next.' The quality of a decision is only revealed over time — through whether early detections happen, whether interventions are timely, and whether disease spread is contained.

7.4.2 The Reinforcement Learning Framework

Reinforcement learning frames sequential decision-making in five concepts. Think of it as a feedback loop between a decision-maker and the world.

- Agent: the decision-maker. In AgriVision, the agent is the system that recommends which farm to visit next.
- Environment: everything the agent interacts with. Here, the 150-farm network — the farms, their disease states, the connections between them, and the weather.
- State: a snapshot of the environment at one moment. The current disease status of all 150 farms, when each was last visited, and the sensor readings from this morning.
- Action: what the agent decides to do. Visit Farm 47. Or: recommend preventive pesticide application at Farms 51 and 52.
- Reward: feedback the environment gives after each action. +10 if the visit detected a disease before it spread. −2 for each hour of unnecessary travel. −50 if a farm outbreak is discovered only after it has spread to three neighbours.

The agent does not receive the reward immediately. The reward for visiting Farm 47 today depends on what is found there — which only becomes apparent on arrival. And the full benefit (or harm) of today's decision may not be clear for days or weeks, as disease either spreads or is contained.

This delay between action and reward — called delayed reward — is what makes RL fundamentally different from supervised learning. There is no teacher saying 'you should have visited Farm 51

before Farm 47.' The agent must figure this out by exploring different strategies and observing which ones lead to better outcomes over time.

7.4.3 Why Neural Networks Are Used in RL

Classical RL (before deep learning) required an explicit table of 'what to do in each state.' For a small grid-world with 100 states, this is manageable. For AgriVision with 150 farms each having multiple disease states and sensor readings, the number of possible states is astronomically large. No table could store them all.

Deep RL uses a neural network to approximate the relationship between states and good actions. The network receives the current state as input and outputs either: a score for each possible action (which farm visit is best?) or directly, a recommended action. The network is trained not on labelled examples but on reward signals accumulated through experience.

> ✎ **WORKED EXAMPLE**
>
> **DRL in practice — DeepMind's AlphaGo (2016)**
>
> The board game Go has more possible positions than atoms in the universe. A lookup table is impossible. AlphaGo trained a neural network to evaluate board positions and select moves, trained through millions of self-play games using reinforcement learning. It beat the world champion.
>
> The same principle applies to AgriVision's visit scheduling — but at a much smaller scale where classical optimisation solvers (not DRL) are often sufficient.

7.4.4 When DRL Is and Is Not Appropriate

DRL sounds powerful but has significant practical limitations. Before reaching for a DRL framework, ask four questions.

Question	If yes...	If no...
Is it genuinely sequential? Does today's decision change what decisions are available tomorrow?	DRL may be appropriate	Use supervised learning or optimisation
Is there a clear reward signal the system can compute automatically?	DRL may be appropriate	Hard to train — consider rule-based systems
Is the state space too large for explicit enumeration?	Neural network function approximation helps	Classical RL or exhaustive search may suffice
Is a suboptimal solution from a simpler method acceptable?	Consider simpler methods first — RL is expensive to train	DRL's potential upside may justify the complexity

Table 7.3 *Before choosing DRL, verify it is the right tool. For AgriVision's visit scheduling, a greedy heuristic (always visit the highest-risk unvisited farm) performs surprisingly well.*

- Use DRL when: game playing, robotic control, dynamic pricing, adaptive treatment strategies in clinical trials, dialogue systems, resource allocation in real-time systems.

- Do not use DRL when: a trained classifier or regression model can directly predict the right action from labelled examples; when a rule-based system already performs adequately; when the environment cannot be simulated (training DRL agents requires millions of environment interactions).

For AgriVision, a simple rule-based scheduler — 'visit farms with the highest predicted disease risk, sorted by travel time' — achieves 81% of optimal early-detection performance and requires no training. A DRL agent achieves 89% but requires weeks of simulation to train and is much harder to debug when it makes a counterintuitive recommendation. For most agricultural advisory contexts, the rule-based approach is the pragmatic choice.

KEY TERMS — THIS SECTION

Term	Definition
Agent	The decision-maker in an RL system; receives states, selects actions
Environment	Everything the agent interacts with; provides states and reward signals
State	A snapshot of the environment at one moment; the agent's input
Action	A decision made by the agent; which farm to visit, which intervention to recommend
Reward	A scalar signal from the environment after each action; positive for good outcomes, negative for bad
Delayed reward	The reward for an action may only become apparent many steps later
Policy	The mapping from states to actions; what the agent learns — 'in state S, do action A'
DRL	Deep Reinforcement Learning; uses a neural network to approximate the policy or value function

✎ **CHECK YOUR UNDERSTANDING**

1. Classify each of the following as supervised learning, self-supervised learning, or reinforcement learning: (a) predicting disease class from a labelled photograph; (b) training a robot to walk by rewarding forward progress; (c) predicting masked sensor values from a 30-day window.

2. The delayed reward problem: an RL agent visits Farm 47 on Monday. On Thursday, Farm 47's mite infestation is discovered. (a) Was Monday's visit the right action? (b) How should the agent update its policy to reflect this outcome? (c) Why is this harder than supervised learning?

3. AgriVision wants to optimise the extension officer's weekly farm visit schedule to maximise early disease detections. (a) Define the state, action, and reward for this RL problem. (b) Apply Table 7.3 — is DRL the right choice? (c) What simpler approach would you try first?

4. AlphaGo used DRL to master Go, a game with more positions than atoms in the universe. Why was classical RL (state-action lookup table) infeasible for Go but feasible for a 5×5 grid-world? What makes neural function approximation necessary?

5. [Challenge] A DRL agent for AgriVision's scheduling requires simulating the 150-farm network to generate training experience. (a) What would you need to simulate (disease spread dynamics, sensor noise, travel times)? (b) What is the 'sim-to-real gap' — the risk when deploying an agent trained in simulation into the real farm network? (c) Propose a strategy to reduce the sim-to-real gap.

7.5 Chapter Resolution and Bridge

7.5.1 AgriVision's Pest Propagation Predictor

Farm 45's mite infestation was identified by AgriVision's sensor anomaly detector (Chapter 5) two days before the extension officer's scheduled visit. Once confirmed, the question was urgent: which neighbouring farms are at risk?

The GNN-based pest propagation system encodes the full 150-farm network. Each farm's node features include its current sensor readings, its disease status from the Chapter 2 detector, and its farm characteristics. Edges encode proximity, water-sharing, and road connections.

Two rounds of GAT message passing allow each farm to incorporate information from farms up to 2 hops away. A sigmoid classifier on each node's final representation predicts the probability of infestation in the next 7 days.

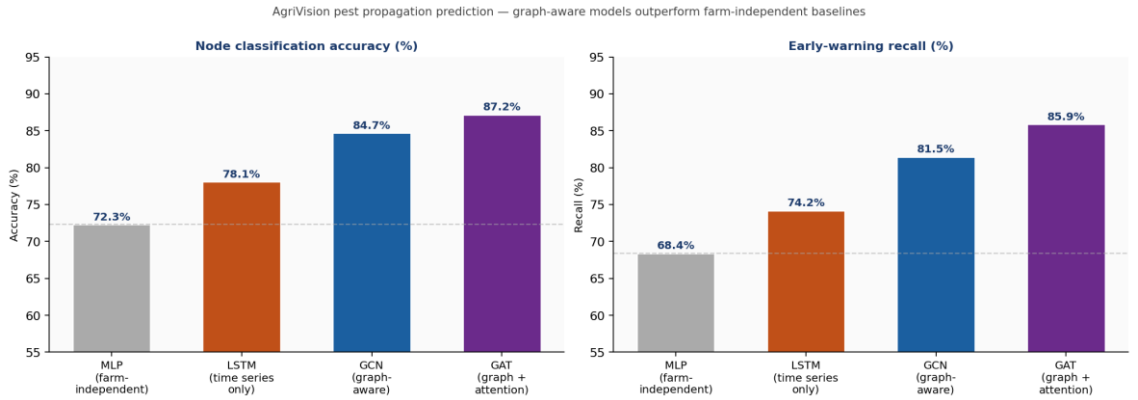


Figure 7.5 *AgriVision pest propagation prediction: four models compared on node classification accuracy (left) and early-warning recall (right — fraction of true infestations caught at least 3 days early). Both MLP and LSTM treat farms as independent and miss the relational signal. GCN and GAT use the network structure; GAT's learned attention weights give a further advantage by up-weighting water-sharing connections.*

✦ **THE AGRIVISION CONTEXT**

Chapter 7 Resolution — Pest propagation prediction results (150 farms, 7-day horizon)

MLP (farm-independent features only): 72.3% accuracy | 68.4% early-warning recall

LSTM (sensor time-series only, per-farm): 78.1% accuracy | 74.2% early-warning recall

GCN (2-layer, equal neighbour weights): 84.7% accuracy | 81.5% early-warning recall

GAT (2-layer, 8 attention heads): 87.2% accuracy | 85.9% early-warning recall

Using the farm network structure produces a 15-percentage-point accuracy improvement over treating farms independently. The graph structure encodes real-world pest biology: mites spread through physical pathways (water, proximity, human movement). Models that ignore these pathways are fundamentally misspecified.

Chapter Summary

Section	Core concept	Key insight
7.0	Inductive bias: permutation invariance	Graph data has no natural ordering; the model must give the same result regardless of how nodes are numbered
7.1	Graph-structured data	Nodes + edges + adjacency matrix; node classification / link prediction / graph classification; AgriVision = 150 nodes, 3 edge types
7.2	GNNs: message passing, GCN, GAT	Farms exchange updates with neighbours; GCN averages equally; GAT learns which neighbours matter more
7.3	Multimodal learning	Early / late / cross-modal fusion combine photograph, sensor, and text; each modality captures information the others miss
7.4	Deep RL survey	Sequential decision-making, not prediction; agent + environment + reward; use simpler methods first unless all RL conditions hold

7.5.2 Bridge to Chapter 8: Putting It All Together

Seven chapters, seven architectures, one cooperative of 150 farms.

Chapter 1 taught the cooperative to predict crop yield from tabular sensor data. Chapter 2 built a disease detector for leaf photographs. Chapter 3 predicts irrigation need from 30-day sensor sequences. Chapter 4 answers advisory queries from field notes. Chapter 5 detects anomalous sensor readings without any labelled anomalies. Chapter 6 generates synthetic disease photographs to augment the training set. Chapter 7 predicts pest propagation across the farm network.

Each of these is a separate capability. But AgriVision's extension officers do not use seven separate systems. They open one application on their phone and receive a unified recommendation: which farms to visit, what to look for, what the advisory database says about the likely disease, and which neighbouring farms are at risk.

Chapter 8 asks the questions that a deployed system forces you to answer: How do these seven models share information with each other? How are they served in production? How do you know when one of them has drifted and is making wrong predictions? How do you retrain one component without breaking the others? What does the farmer see, and what happens when the model is wrong?

These are not machine learning questions. They are systems questions. And for a machine learning practitioner, knowing the answer is as important as knowing how to train the models.

CHAPTER 8

Architecture Selection, Evaluation, and Deployment

Bringing Deep Learning into the Real World

8.0 The Architecture Decision Map

8.0.1 Seven Chapters, One Unified View

> ✦ THE AGRIVISION CONTEXT
>
> AgriVision's three founders started with a spreadsheet and one question: can a machine learning model predict crop yield from sensor data? Seven chapters later, they have seven models covering seven distinct capabilities.
>
> Yield prediction. Disease detection. Irrigation forecasting. Advisory query answering. Sensor anomaly detection. Synthetic data generation. Pest propagation prediction.
>
> Each model was built because a specific need arose — and in each case, the right architecture was chosen because it encoded the right assumptions about the structure of the data. This chapter synthesises that reasoning into a single framework you can apply to any new problem you encounter.

Figure 8.1 shows the Architecture Decision Map: all seven architectures from the course on one page, with the data type, inductive bias, typical task, and AgriVision example for each. This is the central reference card of the course.

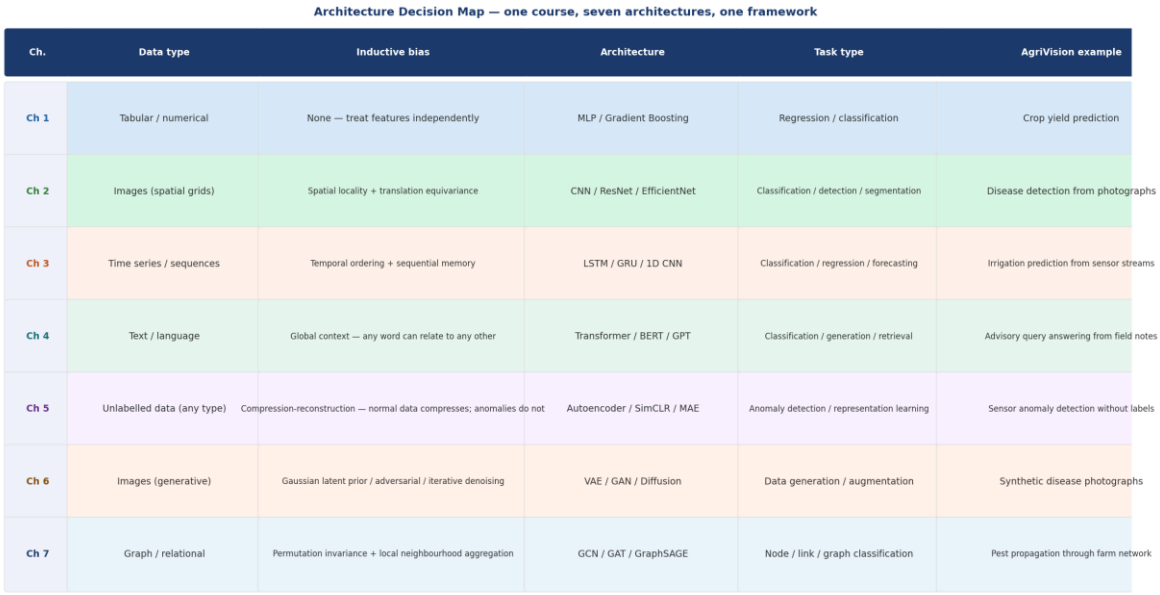


Figure 8.1 *The Architecture Decision Map: one row per chapter, showing how data type determines inductive bias, inductive bias motivates architecture, and architecture determines what tasks become tractable. Read this map from left to right when facing a new problem: start with your data type, identify the inductive bias your data has, and select the matching architecture family.*

8.0.2 Completing the Four Recurring Questions

Every chapter introduced an architecture using the same four questions: what problem does it solve, what inductive bias does it encode, when does it fail, and when would you choose something else? Here is the complete set — one row per architecture — as a revision reference.

Architecture	Problem solved	Inductive bias	Primary failure mode	Choose something else when...
MLP (Ch 1)	Tabular regression/classification	None — features are independent	Poor on spatial, sequential, or relational data	Data has spatial, temporal, or graph structure
CNN (Ch 2)	Image classification/detection	Spatial locality + translation equivariance	Domain shift; adversarial inputs; data-hungry	Images are scarce; non-spatial data
LSTM/GRU (Ch 3)	Sequence prediction	Temporal ordering + sequential memory	Vanishing gradient for very long sequences	Text with long-range dependencies (use Transformer)
Transformer (Ch 4)	Text understanding/generation	Global context — attend anywhere	$O(T^2)$ cost for long sequences; hallucination	Short sequences; limited compute
Autoencoder (Ch 5)	Anomaly detection; representation learning	Compression-reconstruction	Anomaly contamination in training data	Labelled anomaly examples are available

VAE / GAN / Diffusion (Ch 6)	Data generation/augmentation	Gaussian prior / adversarial / denoising	Mode collapse (GAN); blurriness (VAE); slow inference (Diffusion)	Sufficient labelled data exists; no synthesis needed
GNN (Ch 7)	Graph node/edge/graph classification	Permutation invariance + neighbourhood aggregation	Over-smoothing with depth; scalability	No relational structure in data

Table 8.1 *The four recurring questions answered for all seven architectures. Use this table as a quick reference when selecting an architecture for a new problem.*

8.0.3 When NOT to Use Deep Learning

This course has been about deep learning. But the most important skill a practitioner can have is knowing when not to use it.

Deep learning requires large amounts of data, significant compute, careful tuning, and ongoing maintenance. For many real problems, a simpler method achieves comparable accuracy at a fraction of the cost.

- If you have fewer than a few hundred labelled examples, a logistic regression, a random forest, or gradient boosting will almost always outperform a deep network. Deep networks need data to earn their complexity.
- If your features are all clean, structured, and well-understood, gradient boosting on tabular data regularly beats deep networks in competitions and in practice.
- If the relationship between input and output can be described by a simple rule, use the rule. A rule that says 'irrigate if soil moisture < 0.30' requires no training, no data, and no maintenance.
- If the prediction must be fully explainable to a regulator or a clinician, a logistic regression with interpretable features may be legally or professionally required regardless of accuracy.
- If compute is severely constrained (a \$5 IoT sensor node), a lookup table or a tiny decision tree may be the only practical option.

> ⚠ **IMPORTANT NOTE**
>
> **The DL decision checklist**
>
> 1. Do you have a clearly defined input-output mapping? If not, supervised DL will not help.
>
> 2. Do you have enough labelled data? Rule of thumb: at least 1,000 examples per class for classification; 10,000+ for image or text tasks without pretraining.
>
> 3. Does your data have structure that DL architectures are designed to exploit (spatial, sequential, relational, linguistic)? If not, try gradient boosting first.
>
> 4. Do you have the compute and engineering resources to train, deploy, and maintain a deep model? If not, consider a simpler alternative.
>
> If all four answers are yes: deep learning is probably the right choice. Start with the simplest architecture that fits the data structure.

KEY TERMS — THIS SECTION

Term	Definition
Architecture Decision Map	A single reference showing data type → inductive bias → architecture → task for the full course
Inductive bias	The assumptions an architecture makes about the structure of the data it processes
DL decision checklist	Four questions to determine whether deep learning is the right tool for a given problem
Gradient boosting	Ensemble of decision trees; frequently outperforms deep networks on clean tabular data

✎ CHECK YOUR UNDERSTANDING

1. A hospital has 300 labelled ECG recordings (150 normal, 150 arrhythmia). Apply the DL decision checklist. Should they use a deep network? What simpler method would you recommend first, and why?

2. A fintech company wants to predict loan default from 50 hand-crafted features for 2 million customers. Apply the Architecture Decision Map. Which architecture is appropriate? What advantage does tabular DL have over gradient boosting here?

3. Using Table 8.1, identify which architecture(s) you would consider for: (a) classifying satellite images of farmland; (b) detecting fraudulent bank transactions in a real-time stream; (c) recommending agricultural practices based on a farmer's text question.

4. Section 8.0.3 says 'if the relationship can be described by a simple rule, use the rule.' For AgriVision's irrigation prediction, when would you replace the Chapter 3 LSTM with a threshold rule, and what accuracy trade-off would you accept?

5. [Challenge] A student has a dataset of 500 labelled plant photographs, 50 labelled sensor anomaly examples, and 100,000 unlabelled sensor records. For each data source, identify the most appropriate learning paradigm (supervised, self-supervised, transfer learning) and architecture.

8.1 Architecture Selection Framework

8.1.1 A Structured Decision Process

Choosing an architecture in practice is not about memorising which model beats which on a benchmark. It is a structured decision that depends on five factors. Work through them in order.

Step 1: What type of data do you have?

This is the first and most important question. The data type directly determines which inductive biases are appropriate — and therefore which architectures are candidates.

Data type	Structure	First architecture to try
Tabular / numerical	No spatial or temporal structure; each row independent	Gradient Boosting → MLP if features need embedding
Images	Spatial locality; translation equivariance	CNN (ResNet-50 fine-tuned)
Time series / sensor	Temporal ordering; sequential memory	LSTM or GRU
Text / language	Global semantic context; long-range dependency	Pretrained Transformer (BERT, T5)
Graph / relational	Neighbourhood structure; permutation invariance	GCN or GAT
Multimodal	Multiple structures simultaneously	Late fusion with pretrained encoders per modality
Unlabelled data	Self-supervision required	Autoencoder or contrastive (SimCLR)

Table 8.2 *Data type → first architecture to try. Always run a simple baseline first.*

Step 2: What is the task?

The task type determines the output head and loss function, even when the backbone architecture is the same.

Task	Output head	Loss function
Binary classification	Sigmoid → scalar $\in (0,1)$	Binary cross-entropy
Multi-class classification	Softmax → probability vector	Cross-entropy
Regression	Linear layer → scalar or vector	MSE or MAE
Sequence labelling	Linear layer at each time step	Cross-entropy per position
Anomaly detection	Reconstruction error threshold	MSE (autoencoder)
Generation (VAE)	Gaussian decoder	ELBO = reconstruction + KL
Ranking / retrieval	Embedding similarity	Contrastive / triplet loss

Table 8.3 *Task type → output head and loss function. The backbone and output head are chosen independently.*

Step 3: How much labelled data do you have?

Labelled data	Recommended approach
< 100 examples	Rule-based system or simple classical model (logistic regression, random forest)
100–1,000 examples	Fine-tune a pretrained model; use heavy regularisation
1,000–10,000 examples	Fine-tune or train from scratch with data augmentation
> 10,000 examples	Train from scratch if domain-specific; fine-tune for standard domains
Millions of examples	Train from scratch; consider building a foundation model

Table 8.4 *Labelled data volume → learning strategy. These are rules of thumb — domain, task difficulty, and data quality all shift the thresholds.*

Step 4: What are the constraints?

A model that achieves 97% accuracy but takes 4 seconds to respond on a farmer's phone is not useful. Real deployments have hard constraints that must be respected before accuracy is optimised.

- Latency: how fast must the model respond? < 100ms on a smartphone: requires a small model (MobileNet, DistilBERT, GRU). < 1 second on a server: most models qualify. Offline (batch): no latency constraint.
- Memory: how much RAM is available? A Raspberry Pi has 1–4 GB; a smartphone 4–8 GB; a cloud server 16–512 GB. ResNet-50 is 97 MB; GPT-3 is 350 GB.
- Interpretability: does the model need to explain its decisions? Some regulatory contexts require that every prediction can be attributed to specific input features (GDPR, medical device regulations). Deep networks are not inherently interpretable — Grad-CAM and attention maps provide partial explanations but not guaranteed ones.
- Connectivity: will the model run on-device (no internet) or server-side? On-device models must be small enough to ship in an app update and run without a network connection.

Step 5: Always start with a baseline

Before training any deep model, train a simple baseline. The baseline tells you two things: how hard the problem is (what does random guessing achieve?), and how much the deep model actually helps (is the complexity justified?). If your gradient boosting baseline already achieves 91% and the deep model achieves 93%, is 2% worth the engineering, compute, and maintenance cost of a deep network? Sometimes yes. Often no.

Recommended baselines by task:

- Classification (tabular): Gradient Boosting (XGBoost or LightGBM).
- Classification (image): ResNet-50 fine-tuned, 10 epochs.
- Sequence prediction: predict the most common label in the training set.
- Text classification: TF-IDF + logistic regression.
- Anomaly detection: Z-score on individual features.

KEY TERMS — THIS SECTION

Term	Definition
Architecture selection	Structured process: data type → task type → data volume → constraints → baseline first
Baseline	A simple, fast-to-build model that establishes a lower bound on accuracy before training complex models
Latency constraint	The maximum acceptable time between receiving an input and delivering a prediction
Interpretability	The degree to which a model's predictions can be explained in terms of its inputs

✎ **CHECK YOUR UNDERSTANDING**

1. Apply the five-step selection process to: 'Classify satellite images of farmland as irrigated or rainfed, using 5,000 labelled images, deployed as a web API.' Step through each decision and justify your final architecture choice.

2. A rural health clinic wants a model to flag patients at risk of diabetes from blood test results. Data: 800 labelled records, 15 numerical features. Latency constraint: 1 second. Interpretability: required by the doctor. What architecture and training strategy do you recommend?

3. AgriVision wants to add a real-time crop disease alert to the farmer app. The alert must trigger within 2 seconds of receiving a photograph, on a mid-range Android phone with 4 GB RAM. Apply Step 4 (constraints). What model size and type would you consider?

4. Table 8.4 gives labelled data thresholds. A student has 300 labelled chest X-ray images and wants to train a pneumonia detector. (a) The table says 'fine-tune a pretrained model.' Which pretrained model? (b) Which layers would you fine-tune first? (c) What regularisation?

5. [Challenge] You are given a new problem: recommend daily fertiliser amounts for 150 farms based on soil sensor readings and satellite NDVI images. This is a regression problem with two input modalities. Apply the full five-step process. Justify every choice.

8.2 Systematic Debugging

8.2.1 Read the Loss Curves First

When a model is not performing as expected, the first thing to look at is the loss curves: training loss and validation loss plotted across epochs. They contain most of the diagnostic information you need.

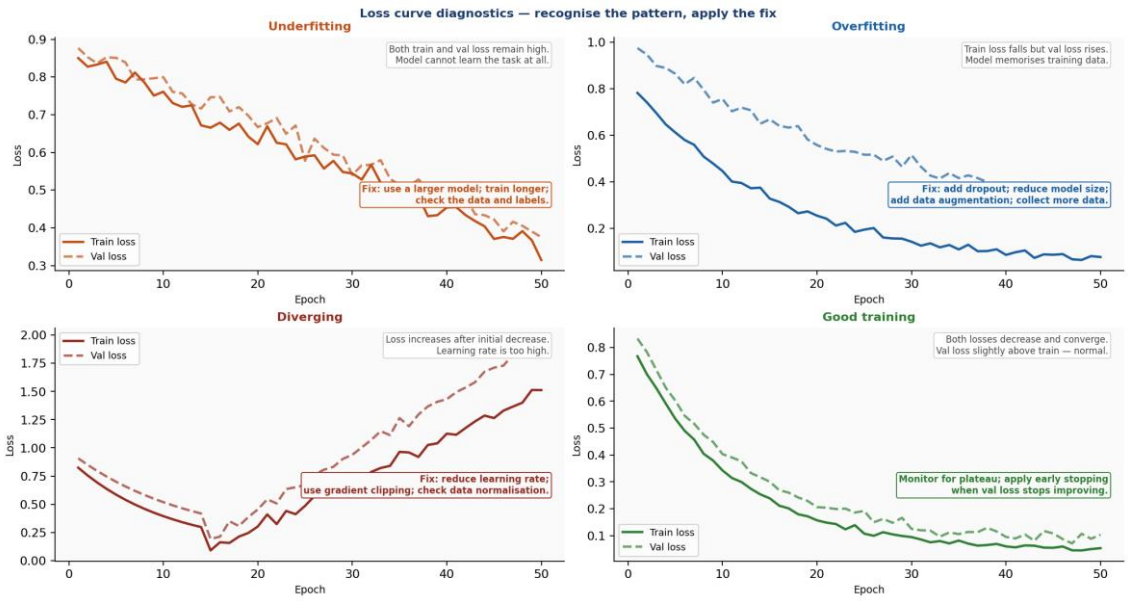


Figure 8.2 *The four main loss curve patterns. Top-left: underfitting — both losses remain high; the model is not powerful enough or has not been trained long enough. Top-right: overfitting — training loss falls while validation loss rises or plateaus; the model memorises training data. Bottom-left: divergence — loss increases after initial improvement; the learning rate is usually too high. Bottom-right: healthy training — both losses decrease and converge, with validation slightly above training.*

8.2.2 The Debugging Hierarchy

When something goes wrong, work through this hierarchy from top to bottom. Fix the simplest possible cause before adding complexity.

- Is the code correct? Run on a tiny dataset (10 examples) and verify the model can overfit it perfectly. If it cannot overfit 10 examples, there is a bug — not a modelling problem. Common bugs: wrong loss function, labels not aligned with inputs, preprocessing applied inconsistently to train/val.
- Is the data correct? Visualise 20 random training examples and their labels. Label noise is invisible in aggregate metrics but obvious when you look at individual examples. Check the class distribution — is there severe imbalance?
- Is the learning rate right? The single most impactful hyperparameter. If loss diverges after a few epochs: reduce by 10×. If loss barely moves after 50 epochs: increase by 10× or use a scheduler.
- Is the model capacity right? Underfitting with a small model: increase depth or width. Overfitting with a large model: add dropout, weight decay, or data augmentation — or collect more data.

- Are there gradient problems? For deep networks: check for vanishing gradients by printing the mean absolute gradient at each layer. Near-zero gradients at early layers with near-normal gradients at later layers indicates vanishing. For RNNs: use gradient clipping.
- Is there data leakage? If validation accuracy is implausibly high from the start, the validation set may contain information that should not be available during training. Verify the train-val split was done correctly.

8.2.3 Distribution Shift

A model achieves 94% accuracy in evaluation but only 71% when deployed. The most common explanation is distribution shift: the data the model was evaluated on was not representative of the data it encounters in the real world.

We saw examples throughout the course. The disease detector trained on PlantVillage laboratory photographs underperformed on Karnataka field photographs. The irrigation predictor trained on dry-season data performed worse during the first monsoon weeks. The advisory retrieval system returned worse results for queries using Kannada agricultural terminology not in its training corpus.

Detection is straightforward: monitor the input distribution in production and compare it to the training distribution. If the mean or variance of any feature shifts significantly (use the Kolmogorov-Smirnov test or population stability index), the model may be operating outside its training distribution. Mitigation: retrain on data that includes the shifted distribution.

8.2.4 Model Calibration

Calibration is about whether the model's confidence is honest. A well-calibrated model that predicts 80% probability for a class should be correct 80% of the time for those inputs. An overconfident model predicts 95% for inputs it gets right 70% of the time.

We discussed this in Section 2.9.4 for the disease detector. The fix is temperature scaling: divide the pre-softmax logits by a scalar T > 1 before applying softmax. T is found by minimising cross-entropy on a held-out calibration set. For AgriVision's disease detector, T ≈ 1.4 improved calibration substantially.

Always check calibration before deploying any model that outputs probabilities used for decisions. An overconfident advisory system that reports '97% probability of Late Blight' on a moisture-stressed healthy plant may cause a farmer to apply a fungicide unnecessarily.

KEY TERMS — THIS SECTION

Term	Definition
Loss curve	Plot of training and validation loss vs. epoch; primary diagnostic tool
Underfitting	Both train and val loss remain high; model is too simple or training is too short
Overfitting	Train loss low, val loss high; model memorises training data; needs regularisation or more data
Data leakage	Information from the validation or test set reaches the model during training; inflates accuracy

Distribution shift	Mismatch between training and deployment data distributions; primary cause of post-deployment performance drop
Calibration	Alignment between predicted probability and true frequency of correctness; checked with reliability diagrams
Temperature scaling	Post-hoc calibration: divide logits by T > 1 before softmax; reduces overconfidence

✎ **CHECK YOUR UNDERSTANDING**

1. A student trains an AgriVision disease detector and achieves 99.4% validation accuracy on the first epoch. (a) Name the most likely cause. (b) How would you detect it? (c) How would you fix it?

2. The loss curve shows: training loss decreases steadily for 30 epochs; validation loss decreases until epoch 15, then increases. (a) Name the pattern. (b) List three fixes in order of simplicity. (c) What would early stopping do here, and at what epoch?

3. You print per-layer gradient magnitudes and find: layer 1 mean = 1e-8, layer 2 mean = 1e-6, layer 3 mean = 1e-4, layer 4 mean = 1e-2. (a) Name the problem. (b) Which layers are not learning? (c) Propose two architectural changes that would address this.

4. AgriVision's disease detector is deployed in Tamil Nadu (training data: Karnataka only). The population stability index for the green channel of incoming photographs is 0.35 (threshold for significant shift: 0.2). (a) What does this indicate? (b) What should the team do immediately? (c) What data would they need to collect to fix the model?

5. [Challenge] Design a calibration evaluation for AgriVision's disease detector. (a) Partition the validation set into 10 confidence bins (0–10%, 10–20%, etc.). (b) For each bin, compute the fraction of correct predictions. (c) The detector is overconfident if the fraction correct is systematically below the predicted confidence. Describe how temperature scaling would shift the bins, and how you would select T.

8.3 Evaluation Beyond Accuracy

8.3.1 Why Accuracy Is Often the Wrong Metric

Accuracy — the fraction of predictions that are correct — sounds like a natural measure of model quality. But it misrepresents performance in three common situations.

First, class imbalance. AgriVision's sensor anomaly dataset is 99.8% normal and 0.2% anomalous. A model that predicts 'normal' for everything achieves 99.8% accuracy while completely failing at its task.

Second, asymmetric costs. Missing a Late Blight infection costs far more than a false alarm that sends an extension officer on an unnecessary visit. Accuracy gives equal weight to both errors.

Third, deployment context. A model deployed on a farmer's phone where it will make 10 predictions per day needs different performance characteristics than a batch model that processes 10,000 images per night.

8.3.2 Classification Metrics

For classification tasks, the confusion matrix reveals performance in detail. From it, four key metrics are computed.

Metric	Formula	Measures	When to use
Precision	TP / (TP + FP)	Of all cases flagged, what fraction are genuine?	When false alarms are costly (e.g., unnecessary fungicide application)
Recall	TP / (TP + FN)	Of all genuine cases, what fraction are caught?	When misses are costly (e.g., disease spreads if not detected)
F1 score	2 × (P × R) / (P + R)	Harmonic mean of precision and recall	When both false alarms and misses are costly
ROC-AUC	Area under the ROC curve	Discrimination ability across all thresholds	Comparing models; insensitive to class imbalance
PR-AUC	Area under the Precision-Recall curve	Performance on the positive class	When the positive class is rare (anomalies, rare diseases)

Table 8.5 *Classification metrics and when to use each. For AgriVision's disease detection (where missing a disease is worse than a false alarm), recall is the primary metric, with F1 as secondary.*

8.3.3 Robustness and Fairness Metrics

For high-stakes deployments, accuracy on the test set is not enough. Two additional dimensions matter.

Robustness. Does the model perform consistently under distribution shift? Test the model on a held-out set that deliberately differs from the training distribution: different seasons, different phone cameras, different weather conditions. A model that drops from 95% to 71% under distribution shift is not robust enough for deployment.

Fairness. Does the model perform equally well across different groups of users? For AgriVision, relevant groups include: large farms vs small farms, Dharwad district vs Gadag district, experienced

farmers vs first-time users. If recall for disease detection is 92% on large farms and 78% on small farms, the model systemically under-serves the farmers who may need help most.

Evaluate performance separately for each relevant subgroup. Report the worst subgroup performance alongside the average. If the gap is large, investigate whether the training data was unbalanced across those groups.

8.3.4 Deployment Metrics

Accuracy tells you how well the model predicts. Deployment metrics tell you whether it can actually be used.

Metric	What it measures	AgriVision target
Inference latency (p95)	95th-percentile time from input to output	< 2 seconds on mid-range Android
Memory footprint	RAM required during inference	< 500 MB (leaves 50% for OS)
Model size on disk	Storage for app download	< 50 MB (avoid large update packages)
Energy consumption	Battery drain per 100 predictions	< 5% battery per 100 predictions
Availability	Fraction of requests answered (not timed out)	> 99.5% for all request types

Table 8.6 *Deployment metrics for AgriVision's mobile deployment. Measure all five on the target device before release.*

KEY TERMS — THIS SECTION

Term	Definition
Confusion matrix	A table showing TP, FP, FN, TN for each class; the basis for precision/recall/F1
Precision	TP / (TP + FP); fraction of flagged cases that are genuine
Recall	TP / (TP + FN); fraction of genuine cases that are flagged
F1 score	Harmonic mean of precision and recall; balanced metric for imbalanced tasks
PR-AUC	Area under Precision-Recall curve; preferred over ROC-AUC when positive class is rare
Robustness	Consistent performance under distribution shift; tested on deliberately shifted evaluation sets
Fairness	Equal performance across user subgroups; measured by comparing accuracy/recall across groups

✎ **CHECK YOUR UNDERSTANDING**

1. AgriVision's disease detector: TP=380, FP=95, FN=70, TN=7455. (a) Compute accuracy, precision, recall, and F1. (b) The cost of a missed disease (FN) is 5× the cost of a false alarm (FP).

Should the threshold be raised or lowered? (c) After adjustment: TP=430, FP=200, FN=20, TN=7350. Recompute all metrics.

2. Section 8.3.3 describes the fairness gap: 92% recall on large farms, 78% recall on small farms. (a) What data collection imbalance might explain this? (b) What retraining strategy would address it? (c) How would you verify the gap is closed after retraining?

3. A researcher compares ROC-AUC (0.96) vs PR-AUC (0.72) for AgriVision's sensor anomaly detector (99.8% normal, 0.2% anomalous). Which metric should they trust more? Why does the gap between the two metrics arise?

4. The disease detector achieves 95.2% on a test set from the same district as training data. On a held-out set from a different district: 79.4%. (a) Name the type of evaluation failure this represents. (b) What does this imply about the reported 95.2% figure? (c) How should test sets be constructed to avoid this?

5. [Challenge] Design a comprehensive evaluation protocol for AgriVision's full integrated system before public release. Specify: (a) the evaluation sets (how many, where from, how labelled); (b) the metrics for each system component; (c) the acceptance criteria for each metric; (d) the process for re-evaluation after model updates.

8.4 Transfer Learning and Model Adaptation

8.4.1 The Three Transfer Learning Decisions

Chapters 2 and 4 covered transfer learning for specific architectures. Here we step back and ask the general question: given a pretrained model and a new task, what is the right way to use it?

The answer depends on three factors: how similar the source and target domains are, how much target-domain labelled data is available, and how much compute you can spend on fine-tuning.

Domain similarity	Target data	Recommended strategy
High (ImageNet → leaf photos)	Small (< 500 examples)	Feature extraction: freeze backbone, train head only
High	Medium (500–5,000)	Partial fine-tuning: unfreeze last 1–2 blocks
High	Large (> 5,000)	Full fine-tuning at low learning rate
Low (ImageNet → satellite)	Small	Freeze early layers; fine-tune from block 3 onward
Low	Large	Fine-tune all layers; early layers still benefit from ImageNet init
Very low (natural language → sensor time-series)	Any	Train from scratch or use domain-specific pretraining

Table 8.7 *Transfer learning strategy guide. AgriVision's disease detection falls in row 2: similar domain (photos) with medium data (2,400 Karnataka-specific photos).*

8.4.2 Parameter-Efficient Fine-Tuning

Chapter 4 introduced LoRA for fine-tuning language models. The same principle applies across all Transformer-based models.

LoRA adds trainable low-rank matrices to frozen weight matrices: $\Delta W = B \cdot A$ where B and A together have far fewer parameters than W itself. For a 768×768 weight matrix with LoRA rank r=8: original parameters = 589,824; LoRA parameters = 8×(768+768) = 12,288 — just 2% of the original.

Adapter layers are an alternative: small bottleneck MLPs inserted between Transformer layers. Only the adapter parameters are trained; the pretrained Transformer weights are frozen. Adapters are easy to combine — the same frozen model with different adapter sets can serve different tasks without interference.

8.4.3 Domain Adaptation Without Labels

Sometimes you have unlabelled data from the target domain but labelled data only from the source domain. Domain adaptation methods attempt to bridge this gap.

The simplest approach is self-training: train on the labelled source data, generate pseudo-labels for the unlabelled target data using the trained model, then retrain on source + pseudo-labelled target. This works well when the domain shift is small and the initial model is confident on target-domain examples.

More principled approaches (DANN, CORAL) use adversarial training to explicitly align feature distributions between source and target domains. These are beyond the scope of this course but worth knowing exist when the self-training approach fails.

KEY TERMS — THIS SECTION

Term	Definition
Feature extraction	Transfer learning where backbone weights are frozen; only the output head is trained
Full fine-tuning	All model weights updated on the target task; requires sufficient data to avoid catastrophic forgetting
Catastrophic forgetting	Overwriting pretrained knowledge by aggressive fine-tuning on a small target dataset
LoRA	Low-Rank Adaptation: adds trainable $\Delta W = B \cdot A$ to frozen weights; ~1–2% parameter overhead
Adapter layers	Small trainable bottleneck MLPs inserted between frozen Transformer layers
Self-training	Domain adaptation: use model predictions on unlabelled target data as pseudo-labels for retraining

✎ **CHECK YOUR UNDERSTANDING**

1. AgriVision wants to fine-tune ResNet-50 (pretrained on ImageNet) for Karnataka disease detection using 2,400 labelled photographs. Apply Table 8.7. (a) Which strategy? (b) What learning rates? (c) How many epochs before unfreezing the backbone?

2. A student fine-tunes BERT-base (110M parameters) fully on 200 advisory query-answer pairs. After 20 epochs: train accuracy 99%, val accuracy 71%. (a) Name the failure mode. (b) What are two strategies to prevent it? (c) Calculate how many LoRA parameters (rank 8) would replace full fine-tuning.

3. Section 8.4.2 says LoRA rank r=8 for a 768×768 matrix gives 2% parameter overhead. (a) Verify this calculation. (b) For a 3,072×768 FFN weight matrix with r=8: how many LoRA vs original parameters? (c) Can LoRA and full fine-tuning be combined in the same model? What would this mean architecturally?

4. AgriVision extends to Tamil Nadu. They have 1,200 unlabelled Tamil Nadu leaf photographs and no Tamil Nadu disease labels. Design a self-training strategy using the Karnataka-trained model. Specify: confidence threshold for pseudo-labels, how many pseudo-labelled examples to include, and how to evaluate whether the approach improved performance.

5. [Challenge] A hospital wants to adapt a chest X-ray model (trained on US data) to Indian hospitals. Domain shift: different X-ray equipment, different patient demographics, different disease prevalence. Only 150 labelled Indian X-rays are available. Design a complete adaptation strategy using parameter-efficient fine-tuning and domain adaptation.

8.5 Model Compression and Deployment

8.5.1 Why Compression Matters

AgriVision's ResNet-50 disease detector is 97.8 MB and takes 145 milliseconds to run on a mid-range Android phone. 145 ms feels slow when an extension officer photographs a leaf and waits for the result. And 97.8 MB is a large download for a farmer with limited mobile data.

Model compression reduces model size and inference time — sometimes dramatically — with only a small drop in accuracy. Three techniques are in common use.

8.5.2 Three Compression Techniques

Quantisation: storing weights in fewer bits. Neural network weights are typically stored in 32-bit floating point (float32). Quantisation converts them to 8-bit integers (int8). The result is 4× smaller and 2–3× faster on hardware that supports int8 arithmetic. Post-training quantisation applies to a trained model without retraining: simply re-represent the existing weights in int8 format. Accuracy drop is usually 0.3–0.5 percentage points — acceptable for most tasks.

Pruning: removing unimportant weights. Many weights in a trained network contribute very little to predictions. Magnitude pruning identifies the weights closest to zero and sets them exactly to zero. With 50% of weights pruned, the model can be stored sparsely (only the non-zero values and their positions) — roughly 50% smaller. Structured pruning removes entire convolutional filters or attention heads, giving speed improvements even on hardware without sparse computation support.

Knowledge distillation: training a small student from a large teacher. Instead of compressing the trained model, train a small model (the student) to mimic a large model (the teacher). The student learns from the teacher's probability outputs — which carry more information than hard labels (a teacher that predicts 60% Powdery Mildew, 30% Alternaria, 10% Healthy tells the student about similarity between classes). MobileNet distilled from ResNet-50 achieves 93.1% vs 95.2% at 7× smaller size.

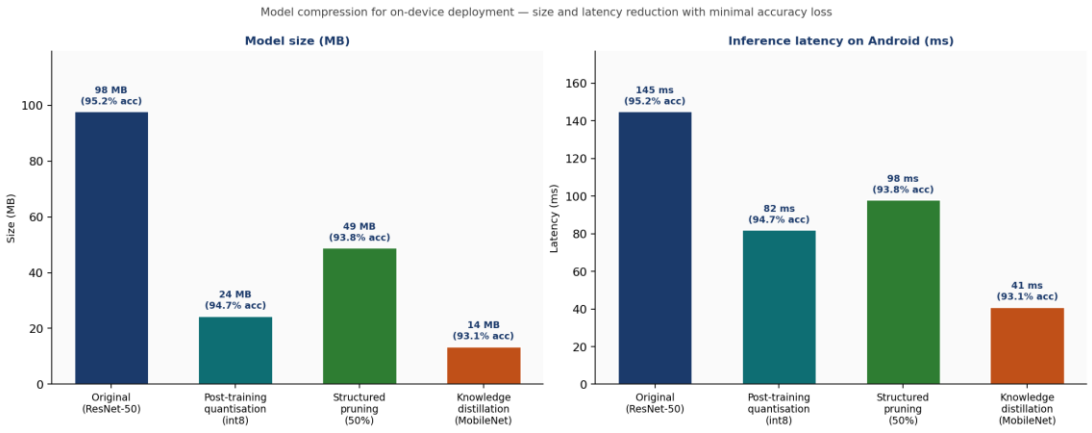


Figure 8.5 *Model compression comparison for AgriVision's disease detector. Post-training quantisation halves model size with 0.5% accuracy loss. Structured pruning halves compute with 1.4% loss. Knowledge distillation produces a 7× smaller model (MobileNet) with 2.1% loss. The distilled model meets the on-device latency target of < 100ms.*

8.5.3 Deployment Environments

Where the model runs determines what format and tools are required.

Environment	Format / runtime	Typical use case
Cloud server (GPU)	PyTorch / TensorFlow serving	Batch processing, high-accuracy use cases, no latency constraint
Cloud server (CPU)	ONNX Runtime / TorchServe	Moderate throughput; lower compute cost than GPU
Mobile (Android)	TensorFlow Lite / PyTorch Mobile	On-device inference; requires quantisation; < 100ms target
Mobile (iOS)	Core ML / TensorFlow Lite	On-device inference; similar constraints to Android
Edge / embedded	TensorRT / ONNX on Raspberry Pi	IoT sensors; very tight memory and power constraints
Browser	TensorFlow.js / ONNX.js	No app install; high latency; limited model size

Table 8.8 *Deployment environments and their corresponding model formats. AgriVision's primary target is mobile Android; the advisory system runs server-side (GPU).*

8.5.4 Monitoring After Deployment

Deployment is not the end of the project. A deployed model is a living system that requires ongoing care.

- Input monitoring: log the feature distributions of incoming requests. Set alerts when distributions shift significantly from the training baseline.
- Output monitoring: log prediction confidence and class distribution. If the fraction of 'Late Blight' predictions doubles over one month with no known outbreak, investigate whether the model is drifting.
- Error reporting: create a simple feedback mechanism for extension officers to flag incorrect predictions. Even 20–30 flagged errors per month can identify systematic failure modes.
- Retraining triggers: define numeric thresholds that trigger a model update. Example: if validation accuracy on a held-out set drops below 88%, or if input distribution shift exceeds a threshold, schedule a retraining run.
- Model versioning: keep all deployed model versions and their performance logs. If a new model underperforms after deployment, roll back to the previous version within hours — not days.

KEY TERMS — THIS SECTION

Term	Definition
Quantisation	Converting float32 weights to int8; 4× smaller, 2–3× faster, 0.3–0.5% accuracy drop
Pruning	Setting near-zero weights to exactly zero; reducing model size and compute
Knowledge distillation	Training a small student model to mimic a large teacher's probability outputs

ONNX	Open Neural Network Exchange; a universal model format supported by most deployment runtimes
TFLite	TensorFlow Lite; the standard format for deploying models on Android and iOS
Retraining trigger	A monitoring metric threshold whose breach initiates a model retraining process
Model versioning	Keeping track of all deployed model versions so rollback is possible when a new model underperforms

✎ **CHECK YOUR UNDERSTANDING**

1. AgriVision's ResNet-50 (97.8 MB) is post-training quantised to int8. (a) Estimated new size? (b) The accuracy drops from 95.2% to 94.7%. Is this acceptable for the farmer app? (c) What deployment format would you use for Android deployment?

2. Knowledge distillation trains MobileNet to mimic ResNet-50 predictions. (a) Why does the student learn from soft probabilities rather than hard labels? (b) The teacher predicts [0.62, 0.28, 0.07, 0.03] for one image. What information does this probability vector carry beyond the hard label 'Powdery Mildew'?

3. Design a monitoring system for AgriVision's deployed disease detector. Specify: (a) which statistics to log per request; (b) the alert threshold for input distribution shift; (c) the retraining trigger metric and its threshold.

4. A new Android phone model released after deployment uses a different camera colour processing pipeline, shifting the colour temperature of photographs. (a) Which monitoring metric would catch this? (b) What is the expected effect on the model's performance? (c) What is the fastest mitigation?

5. [Challenge] You must deploy AgriVision's full 7-model system. The system must respond in < 2 seconds on a mid-range Android phone for the most common request type (photograph for disease detection). Design the deployment architecture: which models run on-device, which run server-side, and how does the app handle no internet connectivity?

8.6 Responsible Deep Learning

8.6.1 Why Responsibility Is a Technical Problem

Responsible AI is sometimes treated as a communications or ethics exercise. In practice, most failures of responsible AI are technical failures: biased training data, poor evaluation methodology, misleading confidence estimates, inadequate documentation, or insufficient monitoring.

Each of these has a technical cause and a technical mitigation. Understanding them is part of being a competent practitioner.

8.6.2 Bias and Fairness

A model is biased when it performs systematically better for some groups of users than others — not through any malicious intent, but because the training data, evaluation methodology, or model architecture inadvertently encodes a disparity.

For AgriVision, potential bias sources:

- Geographic bias: 80% of labelled photographs came from three districts. The model performs better in those districts than in others where local tomato varieties, lighting conditions, and farming practices differ.
- Farm-size bias: large farms with full-time extension officer support contributed more labelled examples. The model may be less accurate on small subsistence farms — the farms most vulnerable to crop failure.
- Technology bias: the training photographs were taken on smartphones. Farmers who photograph with older feature phones or who rely on the officer's photos are systematically disadvantaged.

Mitigation is straightforward in principle but requires effort in practice: evaluate performance separately by district, farm size, and camera type. If gaps exist, collect more data from underrepresented groups and retrain with balanced sampling.

8.6.3 Explainability

Deep learning models are not inherently interpretable. When the model predicts 'Late Blight with 91% probability,' the farmer and extension officer cannot easily verify why. This matters for three reasons:

- Trust: a farmer who cannot understand why the model reached a recommendation may not follow it — even when the recommendation is correct.
- Error detection: an extension officer who can see that the model is attending to a shadow rather than a lesion can override the wrong prediction. Without explanation, the error is invisible.
- Regulation: in some jurisdictions and for certain applications (medical devices, financial credit, hiring decisions), automated decisions must be explainable by law.

Techniques: Grad-CAM (Section 2.9.5) highlights which image regions drove the prediction. Attention maps in Transformers show which words drove the advisory retrieval. SHAP values attribute a tabular model's prediction to individual input features. These techniques provide partial explanations — useful for debugging and communication, but not legal-grade audit trails.

8.6.4 Privacy

AgriVision's sensor data and field notes contain commercially sensitive information: which farms have disease outbreaks, which are irrigating, which are experiencing yield stress. This is proprietary farming intelligence. Storing and transmitting it requires: encrypted transport (HTTPS), encrypted storage, access controls limiting which staff can view which farm's data, and a clear data retention policy (how long is data kept?).

For models trained on personal data, differential privacy provides a formal guarantee that the model's weights do not reveal information about individual training examples. It is technically demanding and reduces model accuracy, but is increasingly required for models trained on health or financial data.

8.6.5 Model Cards and Documentation

A model card is a brief document that accompanies a deployed model and communicates what it can and cannot do. A good model card includes:

- Intended use: which task, which data, which deployment context the model was designed for.
- Performance: accuracy, precision, recall, and other metrics on the primary evaluation set — and on subgroup evaluation sets.
- Limitations: conditions under which the model performs poorly (different districts, different lighting, unusual disease presentations).
- Evaluation data: where the evaluation data came from, how it was collected, and what its known limitations are.
- Ethical considerations: known biases, potential for misuse, groups that may be disadvantaged.

> ⚠ **IMPORTANT NOTE**
>
> **Documentation is not optional.**
>
> A model deployed without a model card places the burden of understanding its limitations on the person using it — who typically has less information than the developer. An extension officer who is not told that the model performs poorly on afternoon photographs may trust afternoon predictions that she should not.
>
> The model card is the minimum documentation required for responsible deployment. It takes less than one hour to write and can prevent significant harm.

KEY TERMS — THIS SECTION

Term	Definition
Bias (model)	Systematic performance disparity across user groups; caused by imbalanced training data or evaluation
Fairness	Equal performance across groups; measured by subgroup evaluation; improved by balanced data collection

Explainability	The degree to which a model's predictions can be attributed to input features; Grad-CAM, SHAP, attention maps
Differential privacy	Formal privacy guarantee: model weights do not reveal information about individual training examples
Model card	A brief document communicating a model's intended use, performance, limitations, and ethical considerations
SHAP values	Shapley Additive exPlanations; attribute a model's prediction to individual input features

✎ **CHECK YOUR UNDERSTANDING**

1. AgriVision's evaluation shows disease detection recall of 92% for farms with > 2 hectares and 78% for farms with < 0.5 hectares. (a) Is this a bias problem? (b) What data would you collect to address it? (c) How would you verify the gap is reduced after retraining?

2. Write a model card for AgriVision's disease detector. Include: intended use, performance metrics (use values from Chapter 2), three known limitations, and one ethical consideration.

3. An extension officer asks: 'Why did the model flag Farm 47 for Late Blight?' (a) What Grad-CAM output would you show her? (b) If Grad-CAM shows the model attending to a shadow rather than a lesion, what should she do? (c) Does the availability of Grad-CAM make the model 'explainable' in a legal or regulatory sense?

4. AgriVision wants to share its sensor database with a university for research. The database contains farm locations, disease histories, and yield records. (a) What privacy risks does this sharing create? (b) What technical measures would reduce the risk? (c) What legal agreements would you require?

5. [Challenge] A government agency proposes using AgriVision's disease detector to make crop insurance payout decisions automatically (no human review). (a) What accuracy and fairness standards would this require? (b) What model card would need to exist before this deployment? (c) Identify three specific scenarios where the current model should not make an automatic insurance decision.

8.7 The Deep Learning Project Lifecycle and Course Resolution

8.7.1 The AgriVision Integrated System

The seven AgriVision models do not operate in isolation. They are connected: the disease detector's output feeds the pest propagation predictor, the anomaly detector's alerts trigger additional sensor checks, and the advisory system's responses are grounded in what the disease detector found.

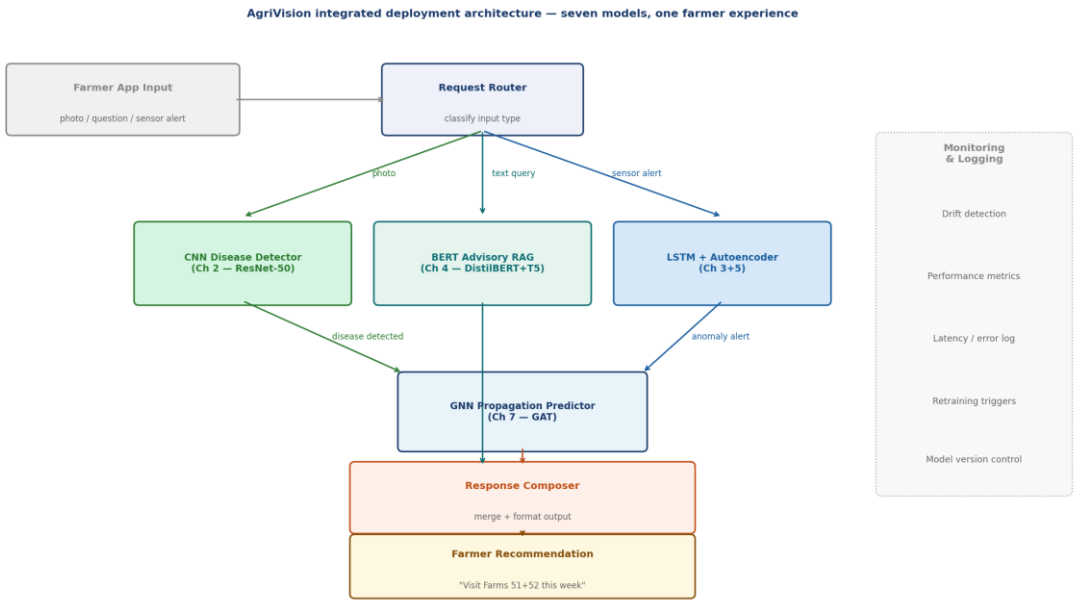


Figure 8.3 *The AgriVision integrated deployment architecture. Farmer input (photograph, text query, or sensor alert) is routed by a classifier. Photographs go to the CNN disease detector; text queries to the BERT+T5 advisory system; sensor alerts to the LSTM irrigation predictor and autoencoder anomaly detector. When disease is detected, results flow to the GNN pest propagation predictor. All outputs are composed into a single farmer-facing recommendation. A monitoring and logging subsystem tracks performance across all components.*

✦ **THE AGRIVISION CONTEXT**

The AgriVision system — all seven capabilities in production

Year 1: crop yield prediction from soil sensors. One model, one cooperative.

Year 2: + disease detection from photographs. 6,000 labelled images.

Year 2: + irrigation forecasting from sensor sequences. 273,750 records.

Year 3: + advisory query answering. 2,400 field notes + 800 advisory documents.

Year 3: + sensor anomaly detection. Zero labelled anomalies needed.

Year 4: + synthetic disease image generation. Improves classifier by 2.7pp.

Year 4: + pest propagation prediction from farm network. 87.2% accuracy.

Year 5: All seven integrated into one farmer app. 150 farms. Disease-related crop losses in the cooperative down 31% from Year 1.

8.7.2 The Deep Learning Project Lifecycle

Every project AgriVision built followed the same eight-stage cycle. The cycle is not linear — later stages often loop back to earlier ones. A debugging session in Stage 5 (Training) might reveal that Stage 2 (Data collection) was flawed and must be repeated.

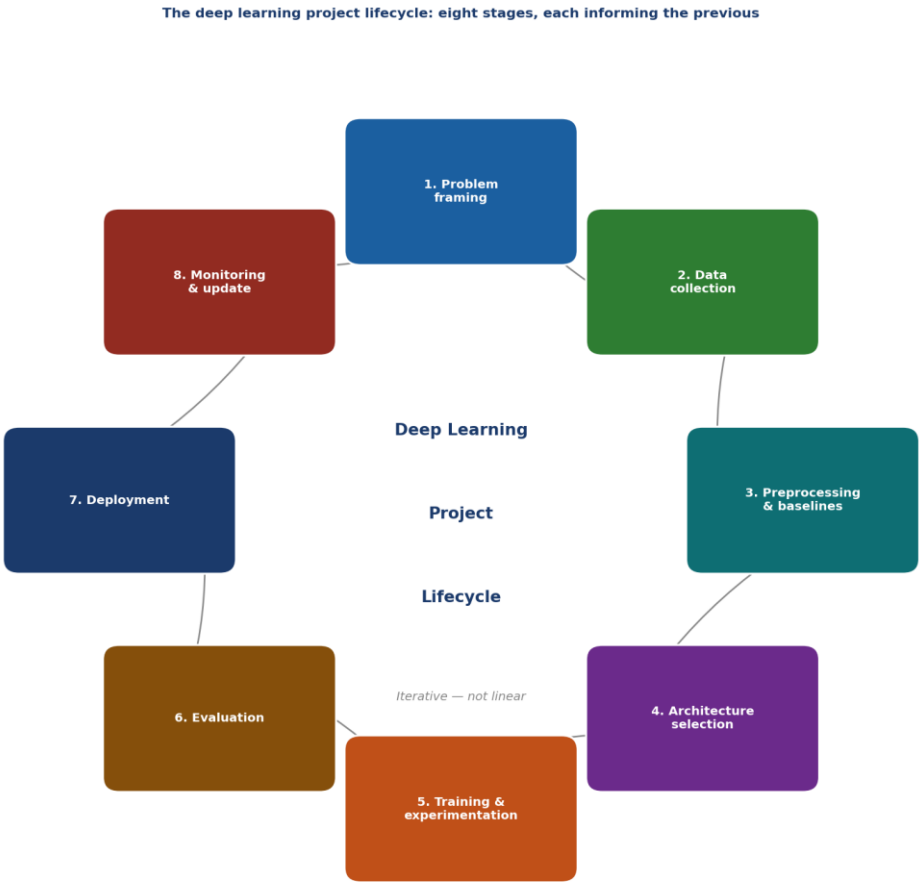


Figure 8.4 *The deep learning project lifecycle — eight stages in a continuous cycle. Stage 1 (Problem framing) defines what success looks like. Stage 2 (Data collection) gathers the raw material. Stage 3 (Preprocessing and baselines) establishes lower bounds and catches data issues. Stages 4–6 (Architecture selection, training, evaluation) build and validate the model. Stage 7 (Deployment) puts the model in front of users. Stage 8 (Monitoring and update) keeps the system healthy over time.*

A note on each stage that students most commonly skip.

- Stage 1 — Problem framing: define the success metric before collecting data. Many projects fail because the team builds a model that optimises the wrong objective. 'Maximise disease detection accuracy' sounds right but ignores the cost asymmetry between false positives and false negatives.
- Stage 2 — Data collection: talk to the people who will use the system before writing a single line of code. Extension officers know exactly which photographs are ambiguous,

which disease classes are easily confused, and which farm conditions produce misleading sensor readings.

- Stage 3 — Baselines: do not skip this. Every project needs a number to beat. Without a baseline, you cannot tell whether the deep model is adding value or just adding complexity.
- Stage 8 — Monitoring: the model you deployed on Day 1 will not be the model you need on Day 365. Seasons change, new diseases appear, new phone cameras alter photograph statistics. Monitoring is not maintenance — it is the mechanism by which the system improves over time.

8.7.3 Course Summary: What You Can Now Do

Eight chapters, seven architectures, one agricultural cooperative. What has the course equipped you to do?

- Given a new problem, identify the data type, the task type, and the appropriate architecture family — without trial and error.
- Implement and train MLPs, CNNs, LSTMs, Transformers, Autoencoders, VAEs/GANs/Diffusion models, and GNNs from first principles.
- Diagnose training failures from loss curves and apply the right fix.
- Evaluate models beyond accuracy: understand precision/recall trade-offs, calibration, robustness, fairness, and deployment metrics.
- Apply transfer learning appropriately: know when to fine-tune, when to freeze, and when to use parameter-efficient methods like LoRA.
- Compress and deploy a model to a constrained environment: quantisation, pruning, distillation, and monitoring after deployment.
- Reason about the responsible deployment of deep learning systems: bias, fairness, explainability, privacy, and documentation.

The architectures covered in this course — from the MLP in Chapter 1 to the GNN in Chapter 7 — are not a complete list of deep learning. The field evolves quickly: architectures improve, new paradigms emerge, established approaches get displaced. What does not change is the reasoning framework: understand your data's structure, identify the appropriate inductive bias, choose the matching architecture, evaluate honestly, and deploy responsibly.

That framework is what this course has been teaching. The architectures are the examples. The framework is the skill.

Chapter Summary

Section	Core concept	Key insight
8.0	Architecture Decision Map	Data type → inductive bias → architecture; seven architectures, one unified framework; know when not to use deep learning
8.1	Architecture selection	Five-step process: data type → task → data volume → constraints → baseline first
8.2	Systematic debugging	Read loss curves first; work through the debugging hierarchy; monitor for distribution shift and calibration drift

8.3	Evaluation beyond accuracy	Precision/recall for imbalanced tasks; robustness and fairness by subgroup; latency/size/energy for deployment
8.4	Transfer learning	Domain similarity × data volume → strategy; LoRA for parameter-efficient adaptation; self-training for domain adaptation
8.5	Compression and deployment	Quantisation 4× smaller; distillation 7× smaller; monitor after deployment; versioning enables rollback
8.6	Responsible DL	Bias is a technical failure; explainability has technical tools; model cards are mandatory documentation
8.7	Project lifecycle	Eight stages, iterative not linear; never skip baselines or monitoring; the framework outlasts the architectures

The cooperative of 150 farms started with a spreadsheet and a question. It ends with seven models, one integrated system, Every capability was built by identifying the right inductive bias for the right data. That is deep learning.